

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Zlatko Pračić

**IZRADA AGENTA S GENERATIVNOM
UMJETNOM INTELIGENCIJOM TEKSTA
SPECIFIČNE NAMJENE**

DIPLOMSKI RAD

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Zlatko Pračić

Matični broj: 0016145097

Studij: Baze podataka i baze znanja

IZRADA AGENTA S GENERATIVNOM UMJETNOM INTELIGENCIJOM
TEKSTA SPECIFIČNE NAMJENE

DIPLOMSKI RAD

Mentor:

Prof. dr. sc. Markus Schatten

Varaždin, rujan 2026.

Izjava o izvornosti

Izjavljujem kako je ovaj diplomski rad izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Rad se bavi izradom i usporedbom triju konfiguracija chatbota specifične namjene za odgovaranje na pitanja iz hrvatskog vojnog zakonodavstva: osnovnog modela bez dohvata konteksta (Vanilla), modela proširenog dohvatom iz vektorske baze (RAG) te modela čiji se dohvat temelji na grafovskoj bazi pravnih dokumenata (GraphRAG). Nakon pregleda povijesnog razvoja chatbotova i teorijskih osnova velikih jezičnih modela, transformer arhitekture i tehnika prilagodbe modela bez dodatnog treniranja (*prompt engineering*, fino podešavanje, RAG), opisana je praktična izrada sve tri konfiguracije sustava. Sve konfiguracije se temelje na kvantiziranom modelu EuroLLM-22B-Instruct, uz zajedničku evaluacijsku osnovu. Kvaliteta odgovora ocijenjena je kombinacijom automatskih metrika (ROUGE-L, BERTScore, kosinusna sličnost), ljudske evaluacije (trinaest ocjenjivača, tri dimenzije: točnost, korisnost, halucinacije) i ocjenjivanje uz pomoć LLM-ova (*LLM-as-a-Judge* – Claude i Gemini). Rezultati pokazuju da obje konfiguracije s dohvatom konteksta, RAG i GraphRAG, statistički značajno i s velikom veličinom efekta nadmašuju osnovni model bez dohvata na svim promatranim dimenzijama, dok razlika između RAG-a i GraphRAG-a nije statistički značajna niti na jednoj mjeri, uz usporedivu latenciju odgovora. Rad dodatno dokumentira iterativni razvoj sustava kroz više verzija, uključujući probleme poput curenja podataka, višejezičnog miješanja jezika i kalibracije generacijskih parametara te raspravlja o mogućim razlozima izostanka prednosti GraphRAG konfiguracije. Zaključuje se kako dohvat konteksta znatno poboljšava pouzdanost chatbota u pravnoj domeni, dok dodatna strukturna složenost grafovskog pristupa u ovom istraživanju ne donosi mjerljivu dodatnu korist.

Ključne riječi: veliki jezični modeli; *Retrieval-Augmented Generation*; *GraphRAG*; *chatbot*; vojno zakonodavstvo; evaluacija jezičnih modela; *LLM-as-a-Judge*

Sadržaj

1. Uvod	1
2. Povijest Chatbotova	3
2.1. Teorijske i matematičke pretpostavke	3
2.1.1. Markovljev lanac	3
2.1.2. Turingov test	4
2.2. Prva generacija - tekstualni chatbotovi bazirani na pravilima	4
2.2.1. ELIZA	5
2.2.2. PARRY	6
2.2.3. Racter	6
2.3. Druga generacija - skriptni jezici	6
2.3.1. Mr. Spock u TINYMUD okruženju	7
2.3.2. ALICE i AIML	7
2.4. Treća generacija - komercijalni i praktični chatboti	7
2.4.1. SmarterChild	8
2.4.2. Web i društvene mreže	8
2.5. Četvrta generacija - glasovni asistenti i konverzacijska sučelja	8
2.5.1. Siri, Google Assistant, Cortana, Alexa	9
2.6. Peta generacija - duboko učenje, transformeri i LLM chatbotovi	9
2.6.1. Sekvencijski modeli	9
2.6.2. Transformer arhitektura	10
2.6.3. Veliki jezični modeli i generativni chatboti (ChatGPT, Bard ...)	10
3. AI inženjerstvo - koncepti i ključne tehnike	11
3.1. Pojam i uloga AI inženjerstva	11
3.2. Temeljni modeli i veliki jezični modeli (LLM)	11
3.2.1. Temeljni modeli	11
3.2.2. Veliki jezični modeli (LLM)	12
3.3. Tehničke osnove: transformer i <i>attention</i>	13
3.3.1. Transformer arhitektura	13
3.3.2. <i>Attention</i> mehanizam	14
3.4. Rad s LLM-ovima u aplikacijama: tokeni, kontekst i kontrola generiranja	14
3.4.1. Tokeni i tokenizacija	14
3.4.2. Kontekst modela	15
3.4.3. Parametri generiranja: <i>temperature</i> , top-k i top-p	16
3.5. Prilagodba ponašanja bez treniranja: <i>prompt engineering</i> i <i>in-context</i> učenje	17

3.5.1.	<i>Prompt engineering</i>	18
3.5.2.	Sistemi i korisnički prompt	19
3.5.3.	<i>In-context learning</i>	19
3.6.	Prilagodba modela treniranjem i optimizacija primjene	20
3.6.1.	Fino podešavanje i PEFT	20
3.6.2.	<i>Preference fine-tuning</i>	21
3.6.3.	Kvantizacija i destilacija	22
3.7.	RAG: povezivanje modela s vanjskim znanjem	22
3.7.1.	Osnovna ideja RAG-a	23
3.7.2.	Vektorske reprezentacije i vektorske baze podataka	23
3.7.3.	Aproksimativno pretraživanje najbližih susjeda	24
3.7.4.	Segmentacija dokumenata	25
3.7.5.	Optimizacija upita i ponovno rangiranje rezultata	26
3.8.	GraphRAG	27
4.	Opis izrade	28
4.1.	Opis domene i zahtjevi	28
4.1.1.	Etička i sigurnosna razmatranja	29
4.2.	Korištene programske biblioteke	30
4.2.1.	Osnovni model bez dohvata konteksta (<i>Vanilla</i>)	30
4.2.2.	<i>Retrieval-Augmented Generation</i> (RAG)	30
4.2.3.	GraphRAG	31
4.3.	Zajednička implementacijska osnova triju konfiguracija	31
4.3.1.	Temeljni jezični model i konfiguracija kvantizacije	32
4.3.2.	Okolina izvođenja	33
4.3.3.	Sistemi upit (<i>system prompt</i>)	35
4.3.4.	Postupak generiranja odgovora	35
4.3.5.	Skup podataka za evaluaciju i postupak paketne obrade	36
4.3.6.	Evaluacija rezultata	36
4.4.	Vanilla konfiguracija chatbota	37
4.4.1.	Funkcija <code>vanilla_chat</code>	37
4.4.2.	Izlazni podaci	38
4.5.	RAG konfiguracija chatbota	38
4.5.1.	Model za generiranje vektorskih reprezentacija	38
4.5.2.	Priprema korpusa dokumenata	39
4.5.3.	Vektorski indeks	40
4.5.4.	Postupak dohvata relevantnih odlomaka	40
4.5.5.	Integracija dohvata u generiranje odgovora	41
4.5.6.	Prilagodba parametara prema kategoriji pitanja	41
4.5.7.	Funkcija <code>rag_chat</code> i izlazni podaci	42
4.6.	GraphRAG konfiguracija chatbota	42

4.6.1.	Graf baza podataka i shema	43
4.6.2.	Izgradnja grafa iz dokumenata	44
4.6.3.	Automatsko izvođenje tematskih veza	47
4.6.4.	Postupak dohvata konteksta	49
4.6.5.	Integracija u generiranje odgovora	52
4.6.6.	Alati za dijagnostiku grafa	52
4.6.7.	Izlazni podaci	52
5.	Evaluacija chatbotova	54
5.1.	Operativni aspekti: inferenca i latencija	54
5.1.1.	<i>Online</i> i <i>batch</i> inferenca	54
5.1.2.	Latencija	54
5.2.	Automatske metrike	55
5.2.1.	Leksičke metrike	55
5.2.1.1.	ROUGE-L	55
5.2.2.	Semantičke metrike	56
5.2.2.1.	BERTScore	56
5.2.2.2.	Kosinusna sličnost (<i>Cosine Similarity</i>)	56
5.2.3.	Implementacija izračuna metrika	56
5.2.4.	Ograničenja automatskih metrika u pravnoj domeni	57
5.3.	Ljudska evaluacija	58
5.3.1.	Sudionici i dimenzije ocjenjivanja	58
5.3.2.	Google Obrasci	58
5.3.3.	Međuocjenjivačka pouzdanost	59
5.4.	LLM-as-a-Judge	60
5.4.1.	Pristup i motivacija	60
5.4.2.	Implementacija i dizajn prompta	60
5.4.3.	Poznata ograničenja	61
5.5.	Obrazloženje odabira kombinacije metoda	61
6.	Statistička analiza rezultata evaluacije	63
6.1.	Deskriptivni pregled	63
6.2.	Usporedba konfiguracija	65
6.3.	Slaganje mjera s ljudskom prosudbom	66
6.4.	Međuocjenjivačka pouzdanost	67
6.5.	Latencija	68
6.6.	Sinteza rezultata	69
6.7.	Ograničenja istraživanja	69
7.	Iterativni razvoj i dijagnostika sustava	71
7.1.	Evolucija RAG sustava kroz iterativno poboljšavanje	71
7.2.	Specifični tehnički izazovi i njihova rješenja	71

7.3. Eksperimentiranje s izborom osnovnog modela: Euro LLM naspram Qwen2.5 . . .	71
7.4. Optimizacija generacijskih parametara i njihov utjecaj	73
7.5. Analiza uzroka izostanka očekivanih performansi GraphRAG konfiguracije	74
8. Zaključak	75
Popis literature	80
Popis slika	81
Popis tablica	82
1. Prilog - Popis pitanja	84
2. Prilog - Colab - Vanilla konfiguracija ChatBota	92
3. Prilog - Colab - RAG konfiguracija ChatBota	114
4. Prilog - Colab - GraphRAG konfiguracija ChatBota	148
5. Prilog - Colab - Automatske evaluacije ChatBota	189

1. Uvod

"We are drowning in information but starved for knowledge." [1]

– John Naisbitt, Megatrends

Razvoj velikih jezičnih modela i generativne umjetne inteligencije značajno je promijenio način na koji korisnici pristupaju informacijama i računalnim sustavima. Chatbotovi temeljeni na velikim jezičnim modelima omogućuju komunikaciju prirodnim jezikom te mogu sintetizirati informacije, odgovarati na pitanja i prilagođavati način komunikacije konkretnom korisniku. Međutim, unatoč njihovim sposobnostima, primjena općih jezičnih modela u specifičnim domenama, osobito onima u kojima je činjenična točnost ključna, i dalje predstavlja značajan izazov.

Jedan od temeljnih problema velikih jezičnih modela jest činjenica da odgovor generiraju na temelju obrazaca naučenih tijekom treniranja, bez ugrađenog mehanizma koji jamči da je svaka pojedinačna tvrdnja u odgovoru činjenično točna i utemeljena na relevantnom izvoru. Posljedica toga može biti pojava halucinacija, odnosno generiranje uvjerljivih, ali netočnih ili nepostojećih informacija. Taj problem posebno dolazi do izražaja u pravnoj domeni, gdje i naizgled mala netočnost može promijeniti značenje propisa, dovesti do pogrešnog tumačenja ili korisniku pružiti nepouzdan odgovor.

Hrvatsko vojno zakonodavstvo predstavlja primjer domene u kojoj su informacije raspoređene kroz više međusobno povezanih pravnih akata. Odgovor na pojedino pitanje često zahtijeva istodobno razumijevanje više propisa, članaka i stavaka. Klasični pristup korištenju velikog jezičnog modela bez dodatnog konteksta oslanja se isključivo na znanje sadržano u parametrima modela. Zbog toga nije moguće pouzdano utvrditi na temelju kojih je informacija odgovor generiran. Jedan od pristupa rješavanju tog problema jest generiranje potpomognuto dohvatom informacija (engl. *Retrieval-Augmented Generation* - RAG), pri kojem se prije generiranja odgovora iz vanjske baze znanja dohvaćaju dokumenti ili njihovi dijelovi relevantni za korisnički upit. Drugi pristup predstavlja *GraphRAG*, u kojem se osim sadržaja dokumenata nastoje iskoristiti i eksplicitno modelirani odnosi između pojedinih elemenata znanja.

Motivacija za izradu ovog rada proizlazi iz potrebe za ispitivanjem stvarne praktične vrijednosti različitih pristupa dohvaćanju konteksta pri izradi chatbota specifične namjene. Posebno je zanimljivo pitanje donosi li složeniji grafovski prikaz pravnih dokumenata mjerljivu prednost u odnosu na klasični vektorski RAG pristup ili dodatna strukturna složenost ne rezultira nužno kvalitetnijim odgovorima.

Cilj rada je izraditi i usporediti tri konfiguracije chatbota za odgovaranje na pitanja iz hrvatskog vojnog zakonodavstva. Pritom se ispituje u kojoj mjeri dohvat dodatnog konteksta poboljšava kvalitetu odgovora velikog jezičnog modela u specifičnoj pravnoj domeni te donosi li *GraphRAG* mjerljivu prednost u odnosu na klasični RAG. Prvu konfiguraciju predstavlja osnovni jezični model bez dodatnog dohvata konteksta, odnosno *Vanilla* pristup. Druga konfiguracija temelji se na klasičnom RAG pristupu, pri kojem se relevantni dijelovi pravnih dokumenata dohvaćaju iz vektorske baze. Treća konfiguracija temelji se na *GraphRAG* pristupu, pri kojem su pravni dokumenti dodatno predstavljeni kroz grafovsku strukturu koja omogućuje dohvat

informacija na temelju eksplicitnih odnosa između zakona, članaka, stavaka, obveza i drugih entiteta.

Za potrebe usporedbe navedenih konfiguracija izrađene su i evaluirane sve tri uz korištenje zajedničke evaluacijske osnove. Kvaliteta generiranih odgovora procijenjena je kombinacijom automatskih metrika, ljudske evaluacije i evaluacije pomoću velikih jezičnih modela (*LLM-as-a-Judge*). Uz konačne rezultate, rad dokumentira i iterativni proces razvoja sustava, uključujući probleme koji su se pojavili tijekom implementacije.

Rad je organiziran u nekoliko cjelina. Nakon uvoda prikazan je povijesni razvoj chatbotova, od ranih sustava temeljenih na pravilima do suvremenih generativnih sustava temeljenih na velikim jezičnim modelima. U sljedećem dijelu obrađuju se teorijski i tehnički koncepti potrebni za razumijevanje rada, uključujući Transformer arhitekturu, velike jezične modele, *prompt engineering*, RAG i GraphRAG pristupe te metode evaluacije. Nakon teorijskog dijela opisuje se izrada i razvoj triju konfiguracija sustava, njihova arhitektura i implementacijske odluke. Zatim se prikazuje metodologija i rezultati evaluacije te se rezultati međusobno uspoređuju i kritički analiziraju. Na kraju rada iznose se zaključna razmatranja, ograničenja provedenog istraživanja i mogući smjerovi budućeg razvoja.

2. Povijest Chatbotova

Popularnost chatbotova naglo je porasla predstavljanjem ChatGPT-a 2022. godine. Chatbotovi su postojali i prije pojave ChatGPT-a, stoga se u ovom poglavlju prikazuje njihov povijesni razvoj.

2.1. Teorijske i matematičke pretpostavke

Iako je Andrey Markov uvođenjem Markovljevih lanaca u svom radu postavio važan temelj, ideja njegova rada nije bila ni približno povezana s konceptom “stroja koji razgovara”. Vrijednost modela Markovljeva lanca za kasnija otkrića i razvoj znanstvene misli je neupitna.

Alan Turing promišlja o budućnosti strojeva čije su se sposobnosti ubrzano razvijale i postajale sve složenije. Zanimalo ga je može li stroj tijekom komunikacije uvjeriti ispitivača da nije stroj, što se u nastavku teksta formalizira u obliku Turingova testa. Iako se u tom kontekstu izravno ne spominju chatbotovi, upravo se na njih ovaj test u velikoj mjeri može primijeniti.

2.1.1. Markovljev lanac

Al-Amin i ostali u svom radu predstavljaju Markovljev lanac kao jednu od najranijih metoda koja je pokazala potencijal chatbota kroz generiranje teksta na temelju statističkih vjerojatnosti slijeda riječi. Markovljev lanac razvio je Andrey Markov 1906. godine za modeliranje stohastičkih procesa. Kasnije se počeo koristiti u zadacima poput predviđanja sljedeće riječi, primjerice u funkciji *autocomplete* u e-pošti. U kontekstu chatbota, Markovljev lanac procjenjuje vjerojatnost da određena riječ slijedi drugu te na temelju naučenih prijelaznih vjerojatnosti generira odgovor. Kao rana primjena Markovljeva lanca navodi se HeX, koji je osvojio Loebnerovu nagradu 1997., nakon čega je interes za ovu tehniku postupno rastao. Loebnerova nagrada bila je godišnje natjecanje (1990.–2020.) u kojem su se chatbotovi natjecali u varijanti Turingova testa. Suci su kroz tekstualni razgovor pokušavali razlikovati program od stvarne osobe, a najuvjerljiviji program dobivao je nagradu [2].

Klasični Markovljevi modeli opisuju prelazak sustava između različitih stanja kroz vrijeme, pri čemu se prijelazne vjerojatnosti procjenjuju iz empirijskih podataka i pretpostavlja se da su stabilne kroz susjedne vremenske intervale. Prostorni Markovljevi modeli proširuju tu ideju na prostorne jedinice (npr. namjene zemljišta) te uključuju prostornu ovisnost između susjednih lokacija, a dodatna proširenja omogućila su i višedimenzionalne simulacije [2].

Unatoč jednostavnosti implementacije, probabilistički pristup zasnovan na Markovljevim lancima pati od ozbiljnih ograničenja u kontekstu prirodnog dijaloga. Budući da modeli predviđanja obuhvaćaju tek neposredno prethodne tokene (jednu ili nekoliko riječi), izostanak šireg kontekstualnog prozora onemogućuje zadržavanje dugoročne strukture rečenice i smisla razgovora. Ova je tehnika povijesno redefinirala pristup obradi jezika, trasirajući put od strogo determinističkih pravila prema generativnim modelima [2].

2.1.2. Turingov test

Alan Turing svojim radom tvrdi kako je pitanje “Mogu li strojevi misliti?” nejasno pa ga zamjenjuje praktičnijim kriterijem: imitacijskom igrom (u nastavku kaže kako je to “test”), gdje ispitivač preko pisanih poruka pokušava razlikovati čovjeka od stroja. Fokusira se na digitalna računala, objašnjava njihovu građu (memorija, izvršna jedinica, kontrola/program) i naglašava njihovu univerzalnost. Jedno dovoljno snažno računalo može oponašati ponašanje bilo kojeg stroja s diskretnim stanjima ako je dobro programirano. Predviđa kako bi se u nekoliko desetljeća mogla pojaviti računala koja bi u takvoj igri bila dovoljno uvjerljiva u “razgovoru” s prosječnim ispitivačem [3].

Velik dio rada posvećen je odgovorima na prigovore kako strojevi ne mogu misliti: teološke tvrdnje o “duši”, strah od posljedica, matematička ograničenja, argument iz svijesti i razne “nemogućnosti” tipa da stroj ne može biti kreativan, griješiti ili učiti. Turing uglavnom pokazuje da ti prigovori ne dokazuju nemogućnost, često miješaju kvarove s pogrešnim zaključivanjem ili se oslanjaju na tadašnja ograničenja tehnologije. Kao pozitivni smjer predlaže da se ne programira “odrasli um” direktno, nego da se napravi “dječji stroj” koji se zatim uči i odgaja (nagrade/kazne, jezik naredbi), analogno procesu učenja i evolucije [3].

Al-Amin i suradnici navode da se Turingov test može smatrati modelskom pretečom chatbota jer je već 1950. postavio razgovor u prirodnom jeziku kao ključni način provjere “inteligentnog” ponašanja mašine, kroz dijalog u kojem treba biti teško razlikovati mašinu od čovjeka. Evaluacija se zasniva na tome može li računalo “prevariti” ljudskog ispitivača da pomisli kako razgovara s čovjekom, što je isti osnovni cilj konverzacijskih sistema poput chatbota. Također, kao utjecajan istraživački kamen temeljac, Turingov test je godinama i desetljećima motivirao razvoj obrade prirodnog jezika, čime je direktno usmjerio put prema modernim chatbot tehnologijama [2].

2.2. Prva generacija - tekstualni chatbotovi bazirani na pravilima

Prva generacija chatbotova pojavila se između sredine 1960-ih i ranih 1980-ih godina. Tada su programeri prvi put pokušali svoje teorijske ideje prevesti u stvarne razgovore. Ovi su sustavi tehnički funkcionirali tako što su prepoznavali unaprijed definirane tekstualne obrasce i primjenjivali strogo definirana, ručno napisana pravila. Nisu stvarno razumjeli što tekstovi znače ili u kojem kontekstu stoje. Sustav je tražio važne riječi u poruci osobe koja ga koristi i odgovarao unaprijed definiranim frazama. Takva je interakcija mogla ostavljati dojam stvarnog razgovora, iako sustavi nisu posjedovali stvarno razumijevanje sadržaja. Unutar usko definiranih područja, ovi su chatbotovi postavili temelje za to kako ljudi i računala komuniciraju. Također su pokazali kako ljudi imaju sklonost da strojevima daju ljudske osobine.

```

EEEEEE LL      IIII ZZZZZZ  AAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LL      II      ZZZ  AAAAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LLLLLL IIII ZZZZZZ  AA  AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:   Yes. I'm trying to describe you for a research paper I'm writing. What sh
ould I say?
ELIZA: You are sure.
YOU:   Yes, I'm sure.
ELIZA: I see.
YOU:   What can I tell them about you?
ELIZA: You seem to need to tell them about me.
YOU:   I absolutely need to. Please help.
ELIZA: That's quite interesting.
YOU:   I agree.
ELIZA: You say you agree ?
YOU:

```

Slika 1: Originalni ispis koda ELIZA pronađen u Weizenbaumovim arhivama [5]

2.2.1. ELIZA

Lane i ostali u svom radu detaljno obrađuju ELIZA, chatbot kojeg je Joseph Weizenbaum razvio na MIT-u sredinom 1960-ih godina. Po tom istraživanju, ELIZA predstavlja prvi chatbot na svijetu, nastao na prvom sustavu za dijeljenje vremena (engl. *time-sharing*) CTSS (engl. *Compatible Time-Sharing System*) na IBM 7094 računalu. Rad opisuje kako je tim istraživača 2021. otkrio originalni ispis koda ELIZA (Slika 1) u Weizenbaumovim arhivama na MIT-u, uključujući ranu verziju čuvenog DOCTOR skripta te gotovo potpunu verziju MAD-SLIP koda, kombinacije programskog jezika MAD i Weizenbaumova vlastitog sustava SLIP (*Symmetric Lisp Processor*). Tim je uspješno “vratio na život” originalnu ELIZA na emuliranom IBM 7094 sustavu u prosincu 2024., čime je po prvi put nakon više od 60 godina omogućeno pokretanje autentičnog originalnog koda. Značaj ovog postignuća nije samo nostalgija: rekonstrukcija je omogućila provjeru što originalna ELIZA doista jest, potvrđujući kako je bila funkcionalan, iako ograničen sustav koji je pomogao pokrenuti cijelo područje konverzijske umjetne inteligencije [4].

Analizom restauriranog koda otkriveno je nekoliko aspekata koji ranije nisu bili poznati. Pronađena verzija ima implementiran potpuni način podučavanja (engl. *teaching mode*), spominjan u Weizenbaumovu radu iz 1966. samo jednom rečenicom, ali nikada detaljno opisan. Korisnici su mogli upisivanjem naredbi poput ADD, SUBST, APPEND, RANK, TYPE, DISPLA i START mijenjati pravila razgovora i pohranjivati nove skripte. Također, otkrivena je značajna greška: ELIZA se “srušila” kada bi primila numerički unos (npr. “*you are 999 today*”), što autori tumače kao ironičan obrat uvida Ade Lovelace da računala mogu “djelovati na stvari koje nisu brojevi”. Važno je da tim nije popravio takve greške, već je kod ostavio autentičnim, točno 96 % onakav kakav je pronađen, kako bi očuvali arheološku vrijednost nalaza. Time ELIZA postaje ne samo povijesni artefakt, već i upozorenje da su mnogi kasniji opisi bili pogrešni. Kao primjer je tvrdnja da je ELIZA bila napisana u Lispu. Izvorna ELIZA bila je implementirana u jeziku MAD-SLIP, dok je Lisp verzija nastala kasnije kao klon koji je napisao Bernie Cosell. Ta se verzija potom proširila putem ARPANeta [4].

2.2.2. PARRY

AI-Amin u svom radu obrađuju chatbot PARRY koji je 1972. razvio psihijatar Kenneth Colby na Stanfordu i predstavlja suprotnost ELIZI. Umjesto da glumi terapeuta, PARRY glumi paranoidnog shizofrenog pacijenta kako bi odvuкао pažnju sa sebe i potaknuo korisnika na dugačke, razrađene odgovore. PARRY je služio i kao alat za obuku psihijatar (vježbanje razgovora s pacijentima), ali i kao operativni model Colbyjeve teorije paranoje, tj. demonstracija kako “poremećena interpretacija znakova” može proizvoditi prepoznatljive obrasce paranoidnog govora i ponašanja [2].

PARRY je jedan od prvih chatbotova koji se evaluirao kroz oblik “Turing testa” radi procjene koliko uvjerljivo može oponašati ljudski razgovor. Ta linija evaluacije kasnije se nastavlja kroz natjecanja poput Loebnerove nagrade, gdje se botovi ocjenjuju po “prirodnosti” nakon kratkih razgovora [2].

2.2.3. Racter

Racter (1983), kojeg su izradili Chamberlain i Etter, u tekstu se opisuje kao pionirski chatbot koji se posebno istaknuo po nasumičnom generiranju novog teksta, a ne samo biranju odgovora iz predložaka. Racter (skraćeno od *raconteur*, pripovjedač) mogao je tijekom razgovora stvarati jedinstvene rečenice i prozne ulomke koristeći pravila kontekstno slobodne gramatike (engl. *context-free grammar*) i element slučajnosti. Rad naglašava kako je rezultat često bio besmislen jer nije postojalo “pravo razumijevanje”, ali da je sama proceduralna sposobnost generiranja originalnog teksta bila iznimno napredna i utjecajna za to vrijeme [2].

Posebno se ističe kako je autor povezan s Racterom (Chamberlain) objavio knjigu *The Policeman's Beard is Half Constructed* (1984), koja se predstavlja kao knjiga “autorski” proizvedena Racterovim generativnim mogućnostima, čime je dobio širu kulturnu vidljivost. Tekst napominje kako nije potpuno jasno u kojoj je mjeri proza doista bila u potpunosti računalno generirana, no važna je poruka da je Racter popularizirao ideju chatbotova koji mogu proizvoditi nov i originalan razgovorni sadržaj. Rad govori o tome kako pojedini Racterovi fragmenti zvuče introspektivno i “ljudski” (npr. o snovima i željama), što je otvorilo pitanja o humanizaciji i o tome kako AI može imitirati emocionalni i egzistencijalni izraz iako nema ljudsko iskustvo [2].

2.3. Druga generacija - skriptni jezici

Nakon pionirskih sustava poput ELIZA, razvoj chatbotova ušao je u fazu dominacije skriptnih jezika i naprednog uparivanja uzoraka, što se može smatrati drugom generacijom chatbotova. Umjesto bazičnih pravila prve generacije, ovi sustavi uvode strukturirane skriptne jezike poput AIML-a, RiveScripta i ChatScripta. Njihova primjena omogućila je izgradnju opsežnijih baza znanja, čime je olakšano prepoznavanje konteksta i upravljanje kompleksnijim varijacijama korisničkih upita. Iako su se i dalje temeljili na unaprijed definiranim predlošcima bez elemenata strojnog učenja, ovi su jezici značajno podigli prag interaktivnosti [6].

2.3.1. Mr. Spock u TINYMUD okruženju

Godine 1989. James Aspnes s Carnegie Mellon University razvio je TINYMUD (engl. *multiplayer real-time virtual world*), jedan od prvih višekorisničkih tekstualnih virtualnih svjetova. Na tom sustavu pojavio se jedan od prvih chatbot likova, Mr. Spock, koji je mogao odgovarati na osnovna konverzacijska pitanja korisnika koristeći jednostavno podudaranje uzoraka [2].

Mr. Spock predstavljao je rani pokušaj stvaranja umjetnog lika koji može razgovarati s ljudima unutar virtualnog okruženja te je prethodio pojmu NPC (engl. *non-player character*) botova koji su danas uobičajeni u igrama i virtualnim svjetovima [2].

2.3.2. ALICE i AIML

Sljedeći veći korak u povijesti chatbotova je ALICE (*Artificial Linguistic Internet Computer Entity*), nastao 1995. kao *online* chatbot inspiriran programom ELIZA. ALICE je bio temeljen na slaganju uzoraka, bez stvarnog razumijevanja cjelokupnog razgovora [6] [2].

Iako je kasnije poboljšavan i osvojio Loebnerovu nagradu za “najhumaniji” računalni program tri puta (2000., 2001. i 2004.), ALICE nije uspio proći puni Turingov test [2].

Adamopoulou naglašava kako je ključna razlika između ALICE i programa ELIZA to što je ALICE razvijen uz novi jezik napravljen upravo za tu svrhu: AIML (engl. *Artificial Intelligence Markup Language*), koji se temelji na XML-u i organiziran je oko kategorija koje se sastoje od uzorka i predloška [6].

Autori kažu kako je AIML nastao u razdoblju 1995–2000 za izgradnju baze znanja chatbotova koji koriste pristup slaganja uzoraka. AIML se temelji na XML-u, a temeljna jedinica znanja je kategorija koja se sastoji od uzorka (engl. *pattern* - korisnički unos) i predloška (engl. *template* - odgovor bota) [6]. Kategorije mogu biti grupirane unutar tema (engl. *topics*), a sve se organizira u strukturu nazvanu Graphmaster koja omogućuje učinkovito uparivanje uzoraka pretraživanjem u dubinu (engl. *depth-first search*) [7].

2.4. Treća generacija - komercijalni i praktični chatboti

Chatbotovi su prošli dug put, od Weizenbaumove ELIZA do asistenata ugrađenih u svaki pametni telefon. Ključni trenutak u toj evoluciji bio je prijelaz s izoliranih programa na masovne komunikacijske platforme, čime su chatbotovi postali dostupni širokom krugu korisnika u stvarnom vremenu. Kroz primjere poput pionirskog SmarterChilda i suvremenih rješenja na platformama kao što su Facebook Messenger, WhatsApp i WeChat, može se pratiti kako su ovi sustavi mijenjali način na koji se dobivaju informacije, komunicira s brendovima i koriste društvene mreže [2] [6].

2.4.1. SmarterChild

SmarterChild je chatbot koji je razvila tvrtka ActiveBuddy 2001. godine i smatra se jednim od ranih pionira koji je chatbotove približio širokoj publici. Al-Amin u svom radu navodi kako je chatbot radio unutar platformi za trenutačno dopisivanje (engl. *instant messaging*) i mogao je davati informacije o raznim temama na upit, što je pokazalo kako chatbot može funkcionirati kao svojevrsni “virtualni asistent” koji pronalazi činjenice i odgovara na korisnička pitanja [2].

Al-Amin ističe kako se SmarterChild mogao dodati u MSN Messenger i AOL Instant Messenger te da je dosegao više od 30 milijuna korisnika. Time je otvorio put chatbotovima u mrežama za dopisivanje i kasnijim asistentima poput Appleova Sirija i Samsungova S Voicea. Demonstrirao je i praktičnu vrijednost razgovornog sučelja jer je omogućavao dohvat vijesti, vremenskih i burzovnih podataka iz baza pa je pristup informacijama kroz razgovor umjesto kroz web-stranice označen kao važan korak prema AI sustavima koji služe ljudima putem dijaloga [2].

2.4.2. Web i društvene mreže

Chatbotovi na društvenim mrežama naglo su se proširili paralelno s rastom interneta i društvenih platformi, gdje mogu imati različite uloge: korisničku podršku, marketing i oglašavanje, zabavu te prikupljanje podataka. U radu Al-Amina se također naglašava kako se mogu koristiti i kao instrumenti “hibridnih prijetnji” s ciljem utjecaja na javno mnijenje, što pokazuje koliko su njihove primjene višestruke i društveno značajne [2].

U praksi se chatbotovi često integriraju izravno u profile brendova na društvenim mrežama gdje pružaju trenutačnu podršku, automatski generiraju sadržaj (primjerice opise i hashtagove), omogućuju personaliziran “razgovorni” kontakt s publikom, provode analizu sentimenta te prikupljaju i organiziraju podatke o korisnicima i trendovima. Kao primjer integracije na velikim platformama navodi se Meta, koja uvodi AI značajke te testira “Meta AI” asistenta dostupnog na WhatsAppu, Messengeru i Instagramu, uz naglasak na postupno uvođenje i zaštitne mehanizme [2].

2.5. Četvrta generacija - glasovni asistenti i konverzacijska sučelja

Četvrta generacija razvoja umjetne inteligencije u domeni komunikacije označava prijelaz s tekstualnih chatbotova na inteligentne glasovne asistente integrirane u svakodnevne uređaje. Ova faza, koja je započela početkom 2010-ih, donijela je revoluciju u interakciji čovjeka i računala omogućivši prirodan razgovor, prepoznavanje konteksta i upravljanje pametnim domovima. Moderni asistenti koriste duboko učenje i masovne baze podataka kako bi predvidjeli potrebe korisnika i pružili personalizirano iskustvo u stvarnom vremenu [2] [6].

2.5.1. Siri, Google Assistant, Cortana, Alexa

Pojava Siri na iPhoneu 4S 2011. godine postavila je temelje za komercijalnu upotrebu glasovnih asistenata jer je proistekla iz ambicioznog vojnog projekta CALO koji je težio stvaranju kognitivnog agenta sposobnog za učenje. Siri je uvela koncept personaliziranog digitalnog pomoćnika koji ne samo da odgovara na pitanja, već i izvršava zadatke poput postavljanja podsjetnika ili slanja poruka, prilagođavajući se s vremenom jeziku i preferencijama korisnika. Iako je pionir, njezina ovisnost o internetskoj vezi i ograničena podrška za manje jezike ostali su izazovi u ranoj fazi razvoja [2] [6].

Ubrzo nakon Applea, Google je 2012. predstavio Google Now (kasnije Google Assistant), fokusirajući se na prediktivno pružanje informacija na temelju lokacije i povijesti pretraživanja, dok su Microsoft i Amazon 2014. godine ušli na tržište s Cortanom i Alexom. Dok je Cortana bila usmjerena na produktivnost unutar Windows ekosustava, Alexa je redefinirala koncept pametnog doma kroz Echo uređaje, omogućivši programerima stvaranje novih vještina putem Alexa Skills Kita [2] [6].

2.6. Peta generacija - duboko učenje, transformeri i LLM chatbotovi

Peta generacija uči iz podataka i to u količinama koje nijedan čovjek ne bi mogao pročitati. Dok su raniji sustavi poput programa ELIZA koristili jednostavno prepoznavanje uzoraka, moderni pristupi temelje se na dubokom učenju i neuronskim mrežama koje simuliraju ljudske kognitivne procese. Ova evolucija omogućila je chatbotima da postanu ne samo digitalni asistenti za izvršavanje zadataka, već i empatični sugovornici sposobni za složenu komunikaciju [6].

2.6.1. Sekvencijski modeli

Sekvencijski modeli, poput rekurentnih neuronskih mreža (RNN) i mreža s dugom kratkoročnom memorijom (LSTM), predstavljali su ključni korak u omogućavanju chatbotima da zadrže kontekst razgovora. Ovi modeli koriste petlje koje omogućuju informacijama da perzistiraju, što je neophodno za razumijevanje rečenica u kojima značenje ovisi o prethodno izgovorenim riječima [8].

Ovi modeli često nailaze na poteškoće pri učenju dugoročnih ovisnosti u vrlo dugim tekstovima. Iako su LSTM i GRU (engl. *Gated Recurrent Units*) značajno poboljšali preciznost u sustavima za odgovaranje na često postavljana pitanja (FAQ), njihova arhitektura i dalje zahtijeva sekvencijalnu obradu podataka, što ograničava brzinu treniranja na velikim skupovima podataka [8].

2.6.2. Transformer arhitektura

Transformer arhitektura donijela je revoluciju u obradi prirodnog jezika uvođenjem mehanizma “pažnje” (engl. *attention mechanism*), koji omogućuje modelu da istovremeno promatra sve dijelove ulazne sekvence. Transformeri mogu paralelno obrađivati podatke, što ubrzava proces treniranja i omogućuje rad s neusporedivo većim korpusima teksta [9].

Ova arhitektura omogućuje modelu da dinamički dodjeljuje važnost različitim riječima u rečenici, bez obzira na njihovu međusobnu udaljenost, čime se postiže dublje razumijevanje semantike i konteksta. Upravo je ovaj tehnološki skok postavio temelje za razvoj suvremenih velikih jezičnih modela koji dominiraju današnjim tržištem umjetne inteligencije [9].

2.6.3. Veliki jezični modeli i generativni chatboti (ChatGPT, Bard ...)

Veliki jezični modeli (*Large Language Model* engl. - LLM) trenutno predstavljaju vrhunac generativne umjetne inteligencije, koristeći milijarde parametara za predviđanje i generiranje teksta koji je često nerazlučiv od ljudskog. Sustavi poput ChatGPT-a ili Barda (sada Gemini) ne oslanjaju se na unaprijed definirane odgovore, već sintetiziraju nove rečenice u stvarnom vremenu, prilagođavajući se tonu i specifičnim zahtjevima korisnika kroz višestruke krugove dijaloga [2].

Iako ovi modeli pokazuju izuzetne sposobnosti u rješavanju problema, kreativnom pisanju i programiranju, oni se i dalje suočavaju s izazovima poput “halucinacija” ili generiranja netočnih informacija. Njihova integracija u svakodnevne alate i sposobnost učenja iz povratnih informacija korisnika čine ih najnaprednijim oblikom interakcije čovjeka i stroja do danas [10].

3. AI inženjerstvo - koncepti i ključne tehnike

Ovaj rad bavi se izgradnjom chatbota koji se oslanja na postojeće jezične modele, što ga smješta u područje AI inženjerstva, a ne razvoja modela od nule. U ovom poglavlju objašnjavaju se koncepti koji su izravno relevantni za taj proces.

3.1. Pojam i uloga AI inženjerstva

AI inženjerstvo je specijalizirano područje u kojem se koriste umjetna inteligencija i strojno učenje za razvoj aplikacija i sustava koji organizacijama pomažu povećati učinkovitost, smanjiti troškove i donositi bolje poslovne odluke. AI inženjeri razvijaju alate, sustave i procese koji omogućuju primjenu umjetne inteligencije u stvarnom svijetu [11].

U praksi, AI inženjer najčešće radi na spoju modela i produkcijskog sustava. Uzima pretreniran model, prilagođava ga kroz fino podešavanje ili prompt engineering te ga izlaže kao API koji ostatak aplikacije može koristiti. Izgradnja modela od nule danas je rjeđa i skuplja od inženjerskog rada s postojećim modelima. Potrebne vještine uključuju programiranje u jezicima poput Pythona, R-a, Jave i C++-a, uz solidno poznavanje statistike i linearne algebre. Od infrastrukturnih alata očekuje se iskustvo s tehnologijama velikih podataka (engl. *big data*) kao što su Apache Spark, Hadoop i MongoDB. Ključan je i praktičan rad s algoritmima strojnog učenja i dubokog učenja, a posebno se cijeni poznavanje popularnih AI radnih okvira poput TensorFlowa, PyTorch-a i Kerasa [11].

3.2. Temeljni modeli i veliki jezični modeli (LLM)

Suvremeni pristup u području umjetne inteligencije označava pomak od razvoja izoliranih, usko specijaliziranih modela prema primjeni pretreniranih sustava opće namjene. U središtu ovog pristupa nalaze se temeljni modeli, unutar kojih se veliki jezični modeli (LLM) izdvajaju kao specifična podskupina. Temeljni modeli mogu biti multimodalni i obuhvaćati domene poput računalnog vida i obrade zvuka, dok su LLM-ovi prvenstveno usmjereni na tekstualne podatke. Razlika između tih dviju skupina nije samo terminološka, ona određuje što se od modela može tražiti i što se od njega može očekivati [10] [12].

3.2.1. Temeljni modeli

Temeljni modeli (često u literaturi nazivani i *Pretrained Foundation Models* - PFMs) definiraju se kao modeli velikog kapaciteta prethodno trenirani na golemim i heterogenim skupovima podataka, čime stječu opće reprezentacije primjenjive na niz specifičnih (engl. *downstream*) zadataka. Povijesni razvoj ovih modela, od ranih jezičnih modela poput BERT-a do kompleksnih multimodalnih sustava, ukazuje na trend u kojem se kroz povećanje računalnih kapaciteta i količine podataka postiže visoka razina transfernog učenja. Transferno učenje se opisuje kao prijenos znanja stečenog na jednom zadatku na drugi, srodan zadatak, umjesto

učenja od nule. Ovakav pristup omogućuje da se jedan centralni model, uz minimalne prilagodbe, uspješno koristi u domenama koje variraju od računalnog vida do analize grafova, čime se mijenja proces dizajna inteligentnih sustava [12].

Ključna prednost temeljnih modela leži u njihovoj sposobnosti prilagodbe specifičnim kontekstima putem metoda kao što su fino podešavanje, instrukcijsko učenje ili tehnike parametarski učinkovite adaptacije. Oslanjanje na jedan centralni model donosi i značajne izazove, ponajprije u nasljeđivanju pristranosti iz podataka za pretreniranje, sigurnosnim rizicima te u potrebi za rigoroznom evaluacijom otpornosti. Stoga se suvremena istraživanja ne fokusiraju samo na povećanje performansi, već i na razvoj mehanizama koji osiguravaju privatnost, etičnost i pouzdanost modela u produkcijskim okruženjima [10].

3.2.2. Veliki jezični modeli (LLM)

Veliki jezični modeli predstavljaju specijalizaciju temeljnih modela unutar domene prirodnog jezika, karakterizirani milijardama parametara i treniranjem na korpusima teksta koji obuhvaćaju značajan dio ljudskog znanja. LLM-ovi pokazuju takozvana “izranjajuća” (emergentna) svojstva, sposobnosti poput rješavanja logičkih problema ili učenja iz konteksta koje se pojavljuju tek pri doseganju određene kritične mase parametara i podataka. Evolucija ovih modela dovela je do pomaka s jednostavnog predviđanja sljedećeg tokena prema sustavima koji su usklađeni s ljudskim namjerama putem metoda poravnanja, čime postaju sposobni za kompleksnu interakciju i izvršavanje složenih korisničkih uputa [12] [10].

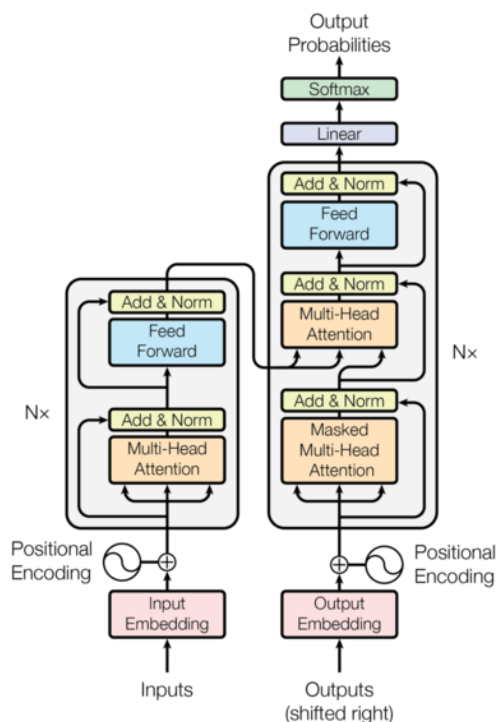
Arhitektonski temelj LLM-ova čini Transformer arhitektura, čija skalabilnost i paralelizacija omogućuju obradu ogromnih količina informacija uz korištenje naprednih varijanti mehanizama pažnje. Kako bi se prevladali visoki računalni troškovi treniranja i inferencije, istraživački fokus se sve više usmjerava prema optimizaciji arhitekture, uključujući efikasnije metode pozicijskog kodiranja i tehnike kompresije modela poput kvantizacije i “prorjeđivanja”. Ovi tehnički napredci omogućuju širu dostupnost LLM-ova, pretvarajući ih iz isključivo istraživačkih artefakata u praktične komponente koje se mogu integrirati u različite softverske arhitekture uz zadržavanje visoke razine preciznosti [12].

U suvremenim primjenama, LLM-ovi se rijetko koriste kao izolirani sustavi. Umjesto toga, oni postaju jezgra širih okvira koji uključuju vanjske izvore znanja kroz metode poput generiranja potpomognutog dohvatom (*Retrieval-Augmented Generation* - RAG). Ovakva integracija rješava inherentne probleme modela, poput halucinacija i nedostatka ažurnih informacija, dok istovremeno omogućuje specijalizaciju za specifične domene bez potrebe za skupim ponovnim treniranjem. Konačno, proces evaluacije LLM-ova evoluirao je od jednostavnih metričkih pokazatelja prema kompleksnim referentnim testovima koji ispituju kognitivne sposobnosti, etičku usklađenost i praktičnu korisnost [12] [10].

3.3. Tehničke osnove: transformer i *attention*

Prije pojave Transformera, rekurentne neuronske mreže poput LSTM i GRU bile su standardni izbor za obradu sekvenci teksta. Njihov glavni nedostatak bila je sekvencijalna obrada, svaki element sekvence morao se obraditi tek nakon prethodnog, što je onemogućavalo paralelizaciju i usporavalo treniranje na dugim sekvencama [9]. Transformer arhitektura riješila je ovaj problem potpunim odbacivanjem rekurencije i oslanjanjem isključivo na mehanizme pažnje (engl. *attention*), čime je postala temeljna arhitektura za suvremene jezične modele [12].

3.3.1. Transformer arhitektura



Slika 2: Koder - dekker [9]

Transformer se sastoji od dva glavna dijela (Slika 2): kodera (engl. *encoder*) koji obrađuje ulazni tekst i dekodera (engl. *decoder*) koji generira izlazni tekst. Koder pretvara ulaznu sekvencu u numeričke reprezentacije koje dekker koristi za generiranje izlaza, element po element. Model je autoregresivan, pri generiranju svakog novog elementa koristi sve prethodno generirane elemente kao kontekst [9].

Ključna prednost ove arhitekture je sposobnost da istovremeno “vidi” sve dijelove ulazne sekvence, neovisno o njihovoj međusobnoj udaljenosti. Ovo omogućuje učinkovito hvatanje dugodosežnih ovisnosti u tekstu, što je kod rekurentnih mreža predstavljalo značajan izazov [9]. Također, budući da se sve pozicije mogu obraditi paralelno, Transformer omogućuje znatno brže treniranje na modernim GPU jedinicama [9].

3.3.2. *Attention* mehanizam

Mehanizam pažnje (engl. *attention*) omogućuje modelu da pri obradi svakog elementa sekvence pažnju obrati na relevantne dijelove ulaza. Funkcionira tako da za svaki element (upit) izračuna koliko je povezan sa svim ostalim elementima (ključevima), a zatim te povezanosti koristi kao težine za kombiniranje vrijednosti. Matematički se to izražava sljedećom formulom [9]:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Q (Query) predstavlja trenutni element koji se obrađuje, K (Key) su svi elementi ulaza na koje se može obratiti pažnja, a V (Value) su vrijednosti povezane s tim elementima.

K^T (transponirana matrica ključeva) je matrica ključeva kojoj su zamijenjeni redci i stupci kako bi se omogućilo množenje QK^T i dobila povezanost svakog upita sa svakim ključem.

Normalizacija dijeljenjem s $\sqrt{d_k}$ osigurava stabilnost gradijenata tijekom treniranja. $\sqrt{d_k}$ (korijen dimenzije ključeva) je faktor skaliranja koji dijeljenjem smanjuje skalarne produkte i time stabilizira gradijente tijekom treniranja [9].

Višeglavi mehanizam pažnje (engl. *multi-head attention*) proširuje ovaj mehanizam izvođenjem više paralelnih operacija pažnje (engl. *attention*), svaka s različitim naučenim projekcijama. Rezultati se zatim spajaju, što modelu omogućuje istovremeno fokusiranje na različite vrste informacija, primjerice, jedna “glava” može pratiti sintaktičke odnose, dok druga prati semantičke veze [9].

3.4. Rad s LLM-ovima u aplikacijama: tokeni, kontekst i kontrola generiranja

Veliki jezični modeli obrađuju tekst na način koji se razlikuje od tradicionalnog pristupa čitanju i pisanju. Umjesto da rad s tekstom temelji na riječima ili rečenicama, LLM-ovi koriste tokene kao osnovne jedinice obrade i generiraju tekst sekvencijalno, token po token. Razumijevanje ovih procesa ključno je za učinkovitu primjenu LLM-ova u praktičnim aplikacijama.

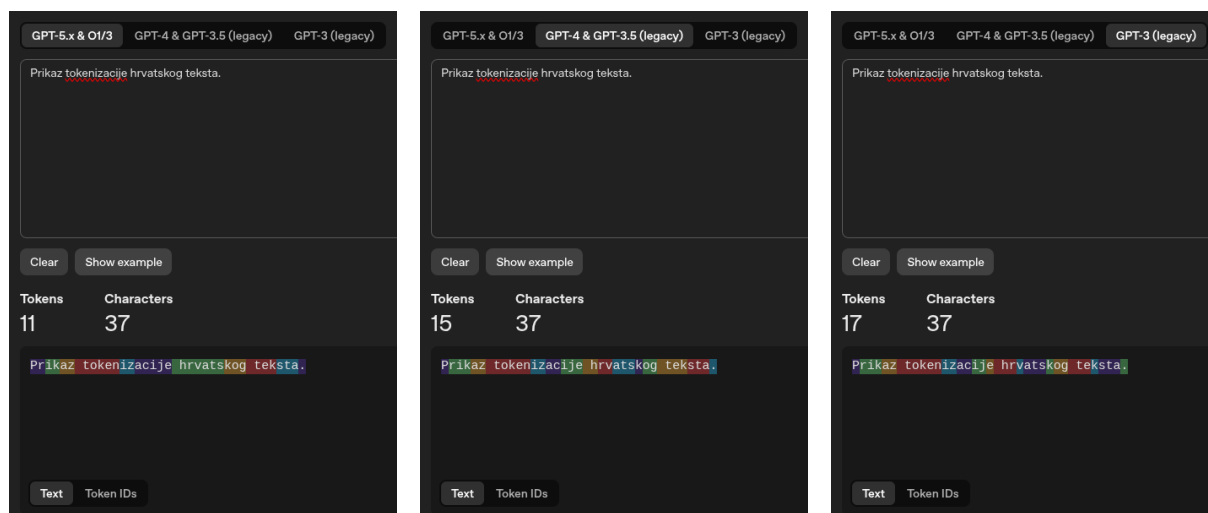
3.4.1. Tokeni i tokenizacija

Token je osnovna jedinica teksta koju model obrađuje tijekom svog rada. Ovisno o postupku tokenizacije, token može biti znak, podriječ, simbol ili cijela riječ [9]. Tokenizacija je bitna faza predobrade koja tekst rastavlja na te nedjeljive jedinice. Najčešće korištene metode tokenizacije kod LLM-ova uključuju *wordpiece* (engl.), kodiranje parom bajtova (BPE) i *unigramLM* (engl.) [12].

Proces tokenizacije započinje tako što se ulazni tekst pretvara u niz tokena, pri čemu

svaki token dobiva jedinstveni identifikator iz rječnika modela. Na primjer, riječ “tokenization” može biti podijeljena na više tokena poput “token”, “ization” ili čak na manje jedinice ovisno o primijenjenom algoritmu. Vaswani i suradnici koriste naučene vektorske reprezentacije (engl. *embeddings*) za pretvaranje ulaznih i izlaznih tokena u vektore dimenzije d_{model} [9]. Ove vektorske reprezentacije omogućuju modelu da razumije semantičke odnose između različitih tokena.

Na Slici 3 ([Link na tokenizator](#)) prikazana je tokenizacija rečenice “Prikaz tokenizacije hrvatskog teksta.” (37 znakova) prema trima generacijama OpenAI modela. Isti tekst novije generacije dijele na manje tokena, što ilustrira napredak u obradi hrvatskog jezika.



(a) GPT-5.x & O1/3: 11 tokena

(b) GPT-4 & GPT-3.5: 15 tokena

(c) GPT-3: 17 tokena

Slika 3: Tokenizacija hrvatskog teksta prema trima generacijama OpenAI modela.

Model generira izlaz sekvencijalno, predviđajući sljedeći token na temelju prethodno generiranih tokena. Ovaj proces iterativnog generiranja znači da model mora za svaki novi token izvršiti proračun kroz sve svoje slojeve, što izravno utječe na brzinu izvođenja i cijenu korištenja modela. Veći broj tokena u ulazu ili izlazu povećava računske zahtjeve i vrijeme potrebno za dobivanje odgovora. Različiti LLM-ovi koriste različite veličine rječnika tokena, primjerice, GPT-3 koristi približno 50.000 tokena, dok BLOOM koristi 250.000 tokena u svom rječniku [12].

3.4.2. Kontekst modela

Kontekst modela označava maksimalnu količinu teksta s kojom model može raditi odjednom. Može se zamisliti kao radna memorija modela, sve što model “vidi” i “pamti” tijekom obrade jednog zahtjeva ograničeno je veličinom ovog kontekstnog prozora. LLM-ovi su tradicionalno trenirani s ograničenim kontekstnim prozorima zbog računske složenosti mehanizma pažnje i visokih zahtjeva za memorijom [12].

Ograničenja konteksta predstavljaju važan praktični izazov. Model koji je treniran s ograničenom duljinom sekvence obično ne može uspješno generalizirati na veće duljine tijekom primjene. Proširivanje kontekstnog prozora tijekom finog podešavanja modela je spor, neučinkovit

i računski skup proces [12]. Zbog toga su razvijene različite tehnike za proširenje konteksta poput interpolacije pozicijskih kodiranja i učinkovitijih mehanizama pažnje [12].

Gusti mehanizam globalne pažnje jedan je od glavnih uzroka ograničenja pri treniranju modela s većim kontekstnim prozorima. Korištenje učinkovitijih varijanti pažnje, poput lokalne, rijetke ili proširene pažnje, može značajno smanjiti računsku složenost. Na primjer, LongT5 koristi prijelaznu globalnu pažnju (TGlobal) koja kombinira lokalnu i globalnu pažnju, što omogućuje proširenje konteksta na 16.000 tokena [12].

Ograničenja konteksta motiviraju razvoj različitih tehnika za rad s dugim dokumentima. Tehnike poput sažimanja sadržaja, dijeljenja teksta na manje dijelove tj. segmentacija (*chunking*) i pristupa proširene generiranja s dohvaćanjem (RAG) omogućuju modelima pristup većoj količini informacija nego što stane u njihov kontekstni prozor. RAG pristup kombinira vanjsko pohranjivanje informacija s mogućnostima generiranja LLM-a, čime se omogućuje pristup ažurnim podacima i smanjuje pojava halucinacija [12].

3.4.3. Parametri generiranja: *temperature*, *top-k* i *top-p*

Nakon što model procesira ulazne tokene kroz svoje slojeve, dobiva se izlazna raspodjela vjerojatnosti za sve moguće sljedeće tokene. Transformer arhitektura koristi linearnu transformaciju i *softmax* (engl.) funkciju za pretvaranje izlaza dekodera u vjerojatnosti sljedećeg tokena [9]. *Softmax* funkcija normalizira rezultate tako da zbroj svih vjerojatnosti bude jednak jedan, čime se dobiva valjana vjerojatnosna raspodjela.

Različiti parametri kontroliraju kako model bira sljedeći token iz ove raspodjele vjerojatnosti, što omogućuje podešavanje ravnoteže između kreativnosti i dosljednosti generiranog teksta.

Temperature (engl.) je parametar koji kontrolira stupanj stohastičnosti u procesu generiranja. Temperature modificira vjerojatnosnu raspodjelu prije primjene *softmax* funkcije tako što dijeli logite (neobrađeni brojevi izlazi modela za svaki token, prije pretvaranja u vjerojatnosti) s vrijednošću temperature. Niže vrijednosti temperature (bliže nuli) čine raspodjelu oštrijom, model postaje konzervativniji i češće bira tokene s najvećom vjerojatnosti, što rezultira predvidljivijim i stabilnijim odgovorima. Više vrijednosti temperature (veće od jedan) zaglađuju raspodjelu vjerojatnosti, dajući više šanse tokenima s nižom vjerojatnosti, što potiče veću raznolikost i kreativnost u generiranom tekstu. Temperatura se često koristi pri treniranju modela za generiranje prirodnog jezika kako bi se poboljšala kvaliteta prijevoda ili generiranih sekvenci [13].

Top-k uzorkovanje ograničava izbor sljedećeg tokena na k najvjerojatnijih kandidata iz raspodjele vjerojatnosti. Nakon što se identificira prvih k tokena s najvećom vjerojatnosti, model uzorkuje sljedeći token samo iz ovog ograničenog skupa, zanemarujući sve ostale mogućnosti. Ovaj pristup sprječava model da bira vrlo nevjerojatne tokene koji mogu narušiti koherentnost teksta, ali istovremeno omogućuje određenu raznolikost među najvjerojatnijim opcijama. Vrijednost parametra k određuje koliko će kandidata biti razmatrano, manja vrijednost rezultira konzervativnijim odabirom, dok veća vrijednost dopušta više varijabilnosti [13].

Top-p uzorkovanje, poznato i kao uzorkovanje jezgre (engl. *nucleus sampling*), predstavlja dinamičniji pristup filtriranju kandidata. Umjesto fiksnog broja tokena, ova metoda odabire najmanji skup tokena čija kumulativna vjerojatnost prelazi zadani prag p (obično između 0.9 i 0.95). To znači da model automatski prilagođava veličinu skupa kandidata ovisno o distribuciji vjerojatnosti, kada je jedan token dominantan, skup će biti manji, a kada je više tokena približno jednako vjerojatno, skup će biti veći. Ovaj pristup omogućuje modelu da prilagodi razinu konzervativnosti ovisno o pouzdanosti predikcije u svakom koraku generiranja [13].

Kombinacija ovih parametara omogućuje fino podešavanje ponašanja modela prema specifičnim potrebama primjene. Za zadatke koji zahtijevaju preciznost i dosljednost, poput generiranja programskog koda ili formalnih dokumenata, koriste se niže vrijednosti temperature i konzervativnije strategije uzorkovanja. Za kreativne zadatke poput pisanja priča ili generiranja ideja, preporučuju se više vrijednosti temperature i fleksibilnije strategije uzorkovanja koje potiču raznolikost u generiranom sadržaju.

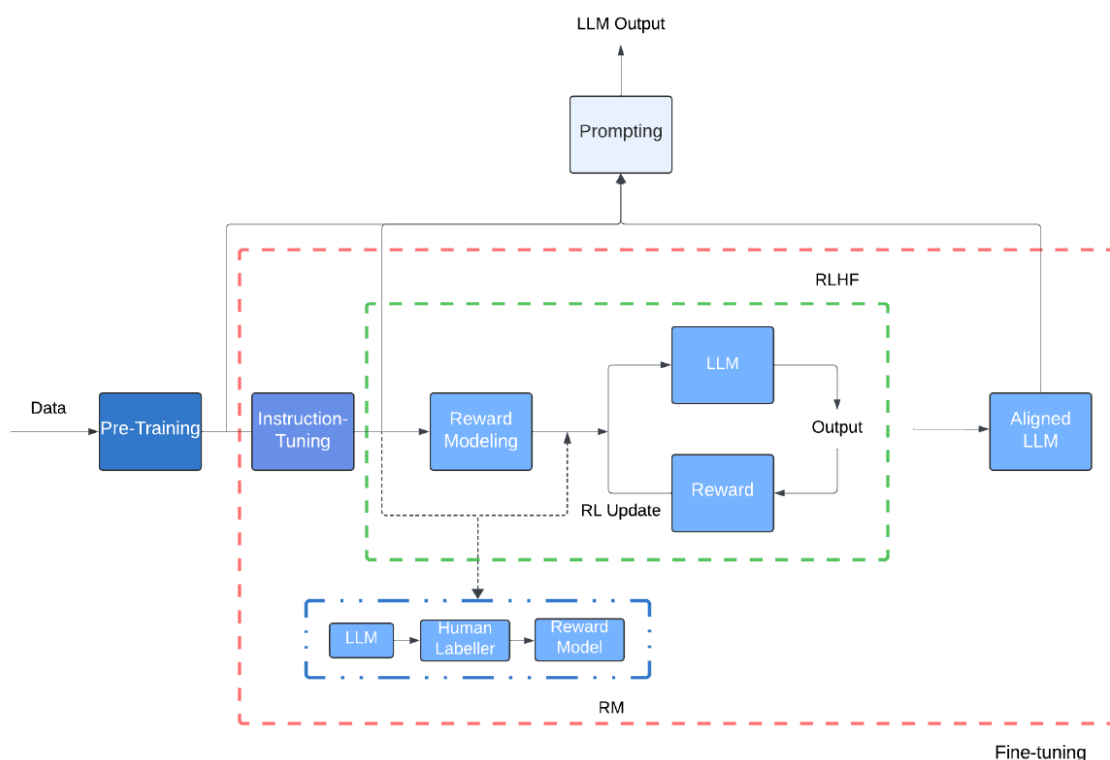
3.5. Prilagodba ponašanja bez treniranja: *prompt engineering* i *in-context* učenje

Tradicionalni pristup prilagodbi jezika modela za specifične zadatke ili domene zahtijeva postupak finog podešavanja, pri čemu se parametri prethodno treniranog modela ažuriraju korištenjem zadataka specifičnih podataka. Iako ovaj pristup omogućava značajno poboljšanje performansi za ciljne aplikacije, proces iziskuje znatne računalne resurse i zahtijeva pristup kvalitetnim označenim podacima za svaki novi zadatak. Fino podešavanje može dovesti do katastrofičnog zaboravljanja (engl. *catastrophic forgetting*), gdje model gubi znanje stečeno tijekom inicijalnog treniranja [13]. Zbog toga se razvio drugačiji pristup, umjesto da se mijenja model, mijenja se način na koji mu se postavljaju pitanja.

Veliki jezični modeli pokazuju sposobnost adaptacije putem promjena u načinu postavljanja upita, odnosno kroz tehniku koja se naziva *prompting* (engl.) ili *prompt engineering* (engl.). Kao što je prikazano na Slici 4, razvoj velikog jezičnog modela odvija se kroz niz faza. Model se najprije *pred-trenira* (engl. *pre-training*) na velikoj količini tekstualnih podataka, čime usvaja jezične obrasce, činjenično znanje i statističke odnose među riječima. Nakon toga slijedi faza finog podešavanja, koja obuhvaća podešavanje prema uputama (engl. *instruction-tuning*) te učenje pojačanjem temeljeno na ljudskim povratnim informacijama (engl. *reinforcement learning with human feedback*, RLHF). U sklopu RLHF-a gradi se model nagrađivanja (engl. *reward model*, RM), koji na temelju ljudskih ocjena generiranih odgovora procjenjuje njihovu kvalitetu, a dobivena nagrada zatim usmjerava ažuriranje parametara modela tako da njegovi izlazi budu što bolje usklađeni s ljudskim preferencijama. Rezultat cijelog postupka jest usklađeni model (engl. *aligned LLM*), spreman za primjenu [10].

Za razliku od navedenih faza koje mijenjaju parametre modela, *prompting* predstavlja metodu postavljanja upita treniranim LLM-ovima koja omogućava prilagodbu modela bez finog podešavanja parametara. Ova sposobnost velikih jezičnih modela omogućava korisnicima da usmjeravaju ponašanje modela isključivo kroz pažljivo oblikovane tekstualne upute i primjere

unutar samog upita. Dva ključna pristupa prilagodbi bez treniranja su prompt engineering, koji se fokusira na strukturiranje i formuliranje upita te učenje iz konteksta (engl. *in-context learning*), pri čemu model uči iz demonstracija prisutnih u promptu [13] [14].



Slika 4: Dijagram toka različitih faza razvoja LLM-a od pred-treniranja do korištenja [10]

3.5.1. *Prompt engineering*

Prompt engineering predstavlja disciplinu oblikovanja uputa za velike jezične modele s ciljem postizanja konzistentnijih i kvalitetnijih rezultata. Ovaj pristup omogućava prilagodbu ponašanja modela bez potrebe za dodatnim treniranjem ili izmjenom parametara. Prompt može sadržavati različite elemente poput definiranja uloge modela, specificiranja željenog formata izlaza, postavljanja pravila ponašanja ili davanja kontekstualnih informacija [15].

Istraživanja pokazuju kako kvaliteta prompta značajno utječe na performanse modela. Dobro strukturiran prompt uključuje jasne i detaljne upute, može sadržavati pozitivne i negativne primjere željenog ponašanja te specificirati očekivanu duljinu ili format odgovora. Prompting može uključivati tehnike poput poticanja rasuđivanja korak po korak, gdje se model navodi da eksplicitno prikaže svoj proces razmišljanja prije davanja konačnog odgovora [12].

Prompt engineering postaje posebno važan u kontekstu učenja bez primjera (engl. *zero-shot learning*), gdje model treba odgovoriti na zadatak koji nije vidio tijekom treniranja. Veliki jezični modeli pokazuju sposobnost *zero-shot* učenja, što znači da mogu odgovarati na upite bez ikakvog prethodnog primjera u promptu. Ova sposobnost omogućava modelima generalizaciju na nove zadatke isključivo na temelju jasno formuliranih uputa [12].

3.5.2. Sistemski i korisnički prompt

Moderni sustavi velikih jezičnih modela koriste distinkciju između sistemskog i korisničkog prompta kako bi omogućili fleksibilnije i kontroliranije interakcije. Sistemski prompt definira opće ponašanje modela kroz cijelu interakciju i postavlja osnovne smjernice za način rada. Ovaj prompt obično ostaje konstantan tijekom sesije i određuje kontekst u kojem model interpretira korisničke zahtjeve. Sistemski prompt može uključivati informacije o ulozi modela, stilskim preferencijama, ograničenjima u ponašanju ili specifičnim domenskim znanjima [14].

Korisnički prompt predstavlja konkretan zahtjev ili upit koji korisnik postavlja u određenom trenutku. Ovaj prompt je dinamičan i mijenja se sa svakim zahtjevom korisnika. Model kombinira informacije iz sistemskog i korisničkog prompta kako bi generirao odgovarajući odgovor. Ova arhitektura omogućava održavanje konzistentnog ponašanja modela (kroz sistemski prompt) dok istovremeno omogućava fleksibilnost u odgovaranju na raznolike korisničke upite [12].

Važnost ove distinkcije posebno dolazi do izražaja u aplikacijama gdje je potrebno održavanje određenog stila komunikacije ili poštivanje specifičnih ograničenja kroz cijelu konverzaciju. Na primjer, sistemski prompt može definirati da model uvijek daje odgovore u određenom profesionalnom tonu ili da izbjegava određene teme, dok korisnički promptovi postavljaju konkretna pitanja unutar tog okvira.

3.5.3. *In-context learning*

In-context learning ili učenje iz konteksta predstavlja ključnu sposobnost velikih jezičnih modela koja im omogućava prilagodbu novim zadacima bez izmjene parametara modela. Ovaj fenomen se manifestira kroz sposobnost modela da uči iz primjera koji su dani unutar samog prompta. Kada se modelu pruži nekoliko primjera ulazno-izlaznih parova za određeni zadatak, model može generalizirati taj obrazac i primijeniti ga na nove ulaze [16].

Učenje iz nekoliko primjera (engl. *few-shot learning*), kao specifičan oblik učenja iz konteksta, pokazuje se posebno efektivnim. Istraživanja na GPT-3 modelu pokazala su kako performanse modela u *few-shot* scenariju premašuju performanse u *zero-shot* scenariju (učenje bez primjera), što sugerira da veliki jezični modeli funkcioniraju kao meta-učenici (*meta-learners*), modeli koji uče kako učiti iz malog broja primjera [16]. Broj primjera koji se daju modelu može varirati, ali čak i jedan ili dva dobro odabrana primjera mogu značajno poboljšati performanse na novom zadatku.

Format *in-context* učenja tipično uključuje strukturirane demonstracije gdje se svakom primjeru prilaže ulaz i očekivani izlaz. Model zatim koristi ove demonstracije kao reference za generiranje odgovora na novi upit. Kvaliteta i relevantnost odabranih primjera značajno utječu na performanse modela. Istraživanja pokazuju da primjeri trebaju biti što sličniji ciljanom zadatku te da redoslijed primjera također može utjecati na rezultate [12].

Ova tehnika omogućava privremeno usmjeravanje modela bez potrebe za skupim postupkom finog podešavanja. Parametri modela ostaju nepromijenjeni, a prilagodba se postiže

isključivo kroz sadržaj prompta. To čini *in-context* učenje izuzetno praktičnim pristupom za brzu prilagodbu modela različitim domenama i zadacima, posebno u scenarijima gdje je dostupno samo nekoliko primjera ili gdje je potrebna česta promjena zadataka [12].

In-context učenje ima svoja ograničenja. Svi primjeri moraju stati u kontekstni prozor modela, što ograničava broj demonstracija koje se mogu pružiti. Performanse ovog pristupa ovise o veličini modela, veći modeli pokazuju značajno bolje sposobnosti *in-context* učenja nego manji modeli [16].

3.6. Prilagodba modela treniranjem i optimizacija primjene

Veliki jezični modeli treniraju se na ogromnim količinama tekstualnih podataka kako bi stekli opće jezične sposobnosti, no njihova primjena u stvarnim sustavima često zahtijeva dodatne korake optimizacije. Prilagodba modela specifičnim domenama ili zadacima, smanjenje računalnih zahtjeva i usklađivanje s ljudskim preferencijama ključni su izazovi pri integraciji LLM-ova u proizvodne sustave. Dok su tehnike poput *prompt engineeringa* i *in-context* učenja korisne za prilagodbu ponašanja bez izmjene parametara modela, postoje scenariji u kojima je potrebno izravno modificirati sam model kako bi se postigla stabilnija i učinkovitija izvedba. Ovaj pristup omogućuje optimizaciju modela za specifične zadatke uz kontrolu nad resursnim zahtjevima i usklađenost s očekivanim ponašanjem [10].

Tehnike prilagodbe modela kreću se od potpunog ponovnog treniranja na domenskim podacima do metoda koje selektivno modificiraju mali podskup parametara, kao i postupaka kompresije koji smanjuju veličinu modela uz minimalan gubitak performansi. Fino podešavanje omogućuje specijalizaciju modela za određene zadatke ili domene, dok parametarski učinkovite metode (engl. *parameter-efficient*) postižu slične rezultate uz znatno manje računalnih resursa. Tehnike kao što su fino podešavanje usklađuju izlaze modela s ljudskim vrijednostima i preferencijama, što je ključno za primjenu u stvarnim sustavima gdje je sigurnost i korisnost prioritet. Metode kvantizacije i destilacije omogućuju razvoj manjih, bržih verzija modela pogodnih za implementaciju na uređajima s ograničenim resursima, čime se proširuje dostupnost i praktična primjenjivost LLM tehnologije [10].

3.6.1. Fino podešavanje i PEFT

Fino podešavanje predstavlja proces dodatnog treniranja prethodno naučenog jezičnog modela na skupu podataka specifičnom za određeni zadatak ili domenu. Fino podešavanje omogućuje modelu da specijalizira svoje znanje za uže definirane aplikacije. Ovaj proces nadograđuje postojeće reprezentacije naučene tijekom pretreniranja, prilagođavajući ih posebnostima ciljane domene ili zadatka. Kod tradicionalnog finog podešavanja ažuriraju se svi parametri modela, što može rezultirati stabilnijim i pouzdanijim ponašanjem u specifičnim scenarijima, ali zahtijeva značajne računalne resurse i memoriju proporcionalnu veličini modela [10].

Tradicionalni pristup finom podešavanju, iako učinkovit, suočava se s praktičnim izazovima kada se primjenjuje na moderne LLM-ove koji sadrže milijarde parametara. Ažurira-

nje svih parametara tijekom treniranja zahtijeva pohranu gradijenata i optimizacijskih stanja za svaki parametar, što multiplicira memorijske zahtjeve i čini proces računalno zahtjevnim. Potpuno fino podešavanje nosi rizik od “katastrofičkog zaboravljanja” gdje model gubi opće jezične sposobnosti naučene tijekom pretreniranja dok se prilagođava novom zadatku. Ovi izazovi motivirali su razvoj pristupa koji omogućuju učinkovitu prilagodbu modela uz minimiziranje računalnih i memorijskih troškova [10].

Parametarski učinkovito fino podešavanje (engl. *Parameter-Efficient Fine-Tuning*, PEFT) metode predstavljaju alternativni pristup koji postiže prilagodbu modela modificirajući samo mali podskup parametara ili dodavanjem dodatnih adaptivnih komponenti. Jedna od najpoznatijih PEFT tehnika je LoRA (engl. *Low-Rank Adaptation*) koja umjesto ažuriranja svih težina matrica u modelu, dodaje male podešavane matrice niskog ranga koje se kombiniraju s originalnim zamrznutim težinama. Ovaj pristup smanjuje broj parametara koje je potrebno trenirati, tipično za nekoliko redova veličine, što rezultira znatno nižim memorijskim zahtjevima i bržim treniranjem, uz zadržavanje kvalitete prilagodbe usporedive s potpunim finim podešavanjem. PEFT metode također omogućuju zadržavanje više različitih adaptacija istog osnovnog modela za različite zadatke, što je praktično u scenarijima gdje je potrebno podržati više specijaliziranih aplikacija [10].

3.6.2. *Preference fine-tuning*

Fino podešavanje prema preferencijama (engl. *preference fine-tuning*) predstavlja naprednu tehniku koja omogućuje usklađivanje ponašanja jezičnih modela s ljudskim vrijednostima i preferencijama kroz učenje iz komparativnih ocjena. *Preference fine-tuning* koristi podatke u obliku usporedbi između različitih generiranih odgovora na isti upit. Ljudski korisnici ocjenjuju koji od ponuđenih odgovora je korisniji, sigurniji, informativniji ili prikladniji, a model uči da preferira generiranje odgovora koji su bolje ocijenjeni. U praksi se pokazalo da je ljudima lakše reći koji od dva odgovora je bolji nego opisati što idealan odgovor uopće jest, što ga čini pogodnim za domene gdje je kvaliteta subjektivna ili je teško uspostaviti formalizam [10].

Najpoznatiji pristup preference fine-tuningu je učenje pojačanjem temeljeno na ljudskim povratnim informacijama (engl. *Reinforcement Learning from Human Feedback*, RLHF) koji kombinira nadzirano učenje (engl. *supervised learning*) s metodama učenja potkrepljivanjem (engl. *reinforcement learning*). Proces započinje skupljanjem usporednih ocjena ljudi gdje anotatori rangiraju različite odgovore modela na iste upite. Iz tih ocjena trenira se nagradni model koji uči predvidjeti ljudske preferencije, tj. procjenjuje koja vrsta odgovora će biti pozitivno ocijenjena. Konačno, koristeći algoritme poput *Proximal Policy Optimization* (engl., PPO), osnovni jezični model se optimizira da maksimizira očekivanu nagradu prema naučenom nagradnom modelu, efektivno učeći generirati odgovore usklađene s ljudskim preferencijama. Ovaj pristup pokazao se ključnim u razvoju dijaloških AI sustava poput ChatGPT, omogućivši modele koji ne samo da generiraju koherentne odgovore, već su također korisni i pružaju odgovore u stilu koji korisnici preferiraju [14].

Temeljni problem proizlazi iz činjenice da tehnička sposobnost modela sama po sebi ne jamči njegovo ponašanje u skladu s očekivanjima korisnika. Kroz iterativni proces prikupljanja

povratnih informacija i učenja iz njih, modeli postaju sve prikladniji za interakciju s ljudima u stvarnim aplikacijama. Ovaj pristup omogućuje korekciju neželjenih ponašanja poput generiranja štetnog sadržaja, nedosljednosti u odgovorima ili neprimjerenog tona, što je teško postići samo kroz nadzirano učenje. *Preference fine-tuning* omogućava kontinuiranu prilagodbu modela u skladu s ljudskim preferencijama i društvenim standardima, čineći sustave fleksibilnijima i odgovornijima u dugoročnoj primjeni [10].

3.6.3. Kvantizacija i destilacija

Model od 70 milijardi parametara u punoj preciznosti zahtijeva više od 140 GB memorije, što premašuje kapacitet gotovo svake potrošačke grafičke kartice. Kvantizacija rješava taj problem smanjujući preciznost pohrane težina, umjesto 32-bitnog zapisa po parametru, koristi se 8-bitni ili čak 4-bitni, čime se memorijski zahtjev smanjuje četiri, odnosno osam puta uz relativno skroman pad kvalitete. Osim smanjenja memorijskog otiska, kvantizacija također ubrzava izvođenje jer operacije s nižom preciznošću se može brže izvršavati, posebno na specijaliziranom hardveru. Moderna istraživanja pokazuju kako je moguće kvantizirati veliki dio parametara LLM-a bez značajnog gubitka performansi, zahvaljujući redundanciji u naučenim reprezentacijama. Postoje različiti pristupi kvantizaciji, uključujući kvantizaciju nakon treniranja (engl. *post-training*) gdje se već trenirani model komprimira te treniranje svjesno kvantizacije (engl. *quantization-aware training*) gdje se kvantizacija uzima u obzir tijekom samog procesa treniranja [10].

Kvantizacija smanjuje preciznost postojećeg modela, destilacija ide korak dalje i stvara fundamentalno manji model. Umjesto da mali model uči samo iz označenih podataka, uči i iz distribucije odgovora većeg modela, tzv. mekih predikcija koje nose više informacije nego oznaka točnog odgovora. Na primjer, pri generiranju sljedeće riječi, učiteljski (engl. *teacher*) model može dodijeliti vjerojatnost 0.7 jednoj riječi, 0.2 drugoj i 0.1 trećoj, ove relativne vjerojatnosti pružaju učeničkom (engl. *student*) modelu uvid u nijanse jezika koje je učiteljski model naučio [10].

Primijenjene zajedno, ove dvije tehnike daju model koji je i manji i brži od originala, dovoljno kompaktan da radi lokalno na laptopu ili mobitelu, bez potrebe za slanjem upita u oblak. Upravo to je razlog zašto modeli poput Phi-3 ili Gemma 3 mogu postići rezultate usporedive s mnogo većim modelima, uz djelić njihovih računalnih zahtjeva [10].

3.7. RAG: povezivanje modela s vanjskim znanjem

Jezični model ne može znati što se promijenilo nakon što je treniran i neće to ni naznačiti, nego će generirati odgovor koji je uvjerljiv, činjenično zastario ili netočan, što se naziva halucinacijom. Za sustav koji odgovara na pitanja iz vojnog zakonodavstva to nije tek teorijski problem, propis koji je bio na snazi u trenutku treniranja u međuvremenu je možda izmijenjen ili stavljen izvan snage. Pristup generiranja potpomognutog dohvaćanjem (engl. *Retrieval-Augmented Generation*, RAG) rješava ova ograničenja integrirajući generativne sposobnosti velikih jezičnih modela s preciznošću dohvaćanja informacija u stvarnom vremenu, čime se

omogućuje da modeli utemeljuju svoje odgovore u dinamički ažuriranim i provjerljivim izvorima [17].

3.7.1. Osnovna ideja RAG-a

Retrieval-Augmented Generation (RAG) predstavlja okvir koji spaja mehanizme dohvaćanja dokumenata temeljene na pretraživanju s generativnim modelima kako bi se poboljšala kvaliteta generiranih rezultata. RAG sustav koristi informacije koje su dohvaćene iz vanjskih izvora znanja, najčešće baza podataka, kako bi nadopunio proces generiranja odgovora. Ovaj pristup kombinira snagu velikih jezičnih modela u generiranju teksta s preciznošću dohvaćanja informacija u stvarnom vremenu, čime se znatno povećava točnost i pouzdanost odgovora [17].

Tijekom faze dohvaćanja (engl. *retrieval*), sustav najprije konstruira upit na temelju korisničkog unosa koji je namijenjen jezičnom modelu. Ovaj upit se zatim koristi za pretraživanje najslabijih ili najrelevantnijih dokumenata iz vanjskih izvora znanja. Mehanizam za dohvaćanje procjenjuje sličnost između korisničkog upita i vraćenih rezultata iz baze znanja kako bi odabrao najrelevantnije dokumente. Nakon što su najrelevantniji rezultati identificirani, proces obogaćivanja kombinira dohvaćene dokumente s izvornim korisničkim unosom kako bi se formirao konačni upit koji se prosljeđuje generativnom jezičnom modelu [17].

Veliki jezični modeli mogu imati ograničenu memoriju i zastarjele informacije, što dovodi do netočnih odgovora. Dohvaćanje relevantnih informacija iz vanjskog, ažuriranog skladišta omogućuje modelima da točno odgovaraju s referencama i koriste više informacija nego što je dostupno u njihovim parametrima. S obogaćivanjem dohvaćanja, manji modeli su pokazali izvedbu usporedivu s mnogo većim modelima, primjerice, model od 11 milijardi parametara može biti konkurentan modelu PaLM od 540 milijardi parametara, a model od 7,5 milijardi može dosegnuti izvedbu modela Gopher od 280 milijardi parametara [10].

3.7.2. Vektorske reprezentacije i vektorske baze podataka

Vektorske reprezentacije tekstualnih podataka predstavljaju temelj za semantičko pretraživanje u RAG sustavima. Mehanizmi za gusti dohvat (engl. *dense retrievers*) koriste prethodno trenirane jezične modele kako bi pretvarali i upite i dokumente u guste vektorske prikaze koji enkapsuliraju semantičko značenje teksta. Za razliku od rijetkih metoda pretraživanja (engl. *sparse retrievers*) poput BM25 i TF-IDF koje se temelje na usporedbi ključnih riječi, guste metode omogućuju identifikaciju kontekstualno relevantnih dokumenata čak i u odsustvu točnog podudaranja riječi [17].

Primjer gustog mehanizma za dohvat je *Dense Passage Retriever* (engl., DPR) koji koristi dvije neovisne, prethodno trenirane BERT mreže kao gusti enkoder. Jedna mreža enkodira svaki ulazni upit u vektor realnih brojeva, dok druga enkodira svaki tekstualni odlomak u vektor istih dimenzija. Sličnost između upita i tekstualnog odlomka mjeri se unutarnjim produktom ovih vektora, a enkoderi se treniraju maksimiziranjem unutarnjih produkata između vektora upita i odgovarajućih relevantnih odlomaka. Ovakvi enkoderi pokazuju otpornu izvedbu kao prethodno trenirani mehanizmi za dohvaćanje i široko se koriste u različitim RAG modelima [17].

Vektorske baze podataka omogućuju efikasno semantičko pretraživanje pohranjivanjem i indeksiranjem vektorskih prikaza dokumenata. Kada korisnik postavi upit, sustav ga pretvara u vektorski oblik korištenjem istog modela za vektorsku reprezentaciju i zatim pretražuje vektorsku bazu kako bi pronašao vektore koji su najbliži upitu u vektorskom prostoru. Vanjska memorija u RAG sustavima može biti održavana u različitim formatima kao što su dokumenti, vektori ili baze podataka, a neki sustavi održavaju posredničke (engl. *intermediate*) memorijske reprezentacije kako bi zadržali informacije kroz više iteracija [10].

3.7.3. Aproximativno pretraživanje najbližih susjeda

Sličnost između vektora upita i vektora dokumenta u praksi se najčešće mjeri kosinusnom sličnošću, normaliziranim skalarnim produktom dvaju vektora kojemu vrijednost bliža jedinici označava veću semantičku srodnost teksta koji vektori predstavljaju [17]. Egzaktna, tzv. gruba pretraga (engl. *brute-force*) uspoređuje upit sa svakim pojedinim vektorom u korpusu, čime jamči da će uvijek pronaći stvarno najbliže susjede, no vrijeme takve pretrage raste linearno s brojem pohranjenih vektora, što je za korpuse s milijunima odlomaka računski preskupo.

Za velike korpuse zato se u praksi koriste algoritmi za aproksimativno pretraživanje najbližih susjeda (*Approximate Nearest Neighbors*, ANN), koji uz malen i kontroliran gubitak točnosti postižu znatno bržu pretragu. Najčešće korišten ANN pristup u suvremenim vektorskim bazama podataka je *Hierarchical Navigable Small World* (engl., HNSW), a uz njega se koriste i *Inverted File Index* (engl., IVF), *Product Quantization* (engl., PQ), *Locality Sensitive Hashing* (engl., LSH) te strukture stabla poput KD-stabla i *Ball tree* (engl.) [18].

HNSW organizira vektore u višeslojni graf u kojem je svaki čvor povezan sa susjedima slične vrijednosti, oponašajući svojstvo “malog svijeta” iz teorije grafova, gdje se svaki čvor iz bilo kojeg drugog može dosegnuti kroz malen broj skokova (engl. *hops*). Pretraga započinje na najvišem, najrjeđe povezanom sloju grafa i postupno se spušta prema gušće povezanim nižim slojevima, sužavajući skup kandidata sve dok se ne pronađe skup vektora najbliži upitu. Ovakva struktura omogućuje pretraživanje čija složenost raste logaritamski, a ne linearno, s brojem pohranjenih vektora. Omjer brzine i točnosti pritom je konfigurabilan preko parametara poput broja veza po čvoru (M) te širine pretrage tijekom izgradnje i samog pretraživanja indeksa (*efConstruction*, *efSearch*) [18].

HNSW je implementiran i dostupan u većini specijaliziranih vektorskih baza podataka i proširenja korištenih u RAG sustavima, primjerice Weaviate, Pinecone, Qdrant, Milvus, LanceDB te pgvector proširenju za PostgreSQL [19]. Izbor između egzaktna pretrage i ANN indeksa poput HNSW-a u konačnici je kompromis, egzaktna pretraga jamči potpunu točnost uz veći utrošak vremena i memorije, dok ANN pristupi omogućuju skalabilnost na znatno veće korpuse uz prihvatljiv, kontroliran gubitak preciznosti dohvata. Izbor primijenjen u ovom radu, gdje se za RAG konfiguraciju koristi egzaktna FAISS pretraga, a za GraphRAG konfiguraciju Neo4j vektorski indeksi temeljeni na HNSW-u.

3.7.4. Segmentacija dokumenata

Nije svejedno kako se dokumenti dijele prije pohrane u vektorsku bazu. Segmentacija se može provesti na razini tokena, rečenice, odlomka ili cijelog dokumenta i taj odabir izravno utječe na to koliko će dohvaćeni sadržaj biti koristan modelu [17].

Gruba granularnost dohvaćanja (npr. dohvaćanje cijelih dokumenata) može pružiti širi kontekst, ali istovremeno može uvesti suvišan sadržaj koji potencijalno ometa jezični model u kasnijim zadacima. Fina granularnost dohvaćanja pruža preciznije informacije, ali može povećati složenost dohvaćanja i pohrane te ugroziti semantički integritet. Dohvaćanje na razini dokumenta funkcionira dobro kada zadatak zahtijeva sveobuhvatan kontekst, dok se dohvaćanje na razini odlomka (engl. *chunk-level* i engl. *passage-level*) fokusira na specifične odjeljke teksta, usklađujući kontekst s relevantnošću [17].

Odabir veličine i strategije segmentacije predstavlja važan dizajnerski izbor u RAG sustavima. Uspješni RAG modeli poput REALM, RAG i Atlas koriste pristup dohvaćanja na razini odlomka jer on pomaže u zadacima koji zahtijevaju specifične odgovore, pritom ne preopterećujući model sa suvišnim informacijama. Dohvaćanje na razini tokena, iako manje uobičajeno, može se koristiti za specijalizirane zadatke koji zahtijevaju točna podudaranja ili integraciju podataka izvan domene treniranja. Strategija segmentacije mora uzeti u obzir i ograničenja kontekstnog prozora modela, dijeljenje na manje cjeline omogućuje lakše pretraživanje i smanjuje rizik od prekoračenja maksimalne duljine ulaza [17].

U literaturi se konkretne strategije segmentacije obično klasificiraju u nekoliko osnovnih kategorija. Segmentiranje fiksne veličine (engl. *fixed-size chunking*) dijeli tekst na cjeline jednake, unaprijed zadane duljine (u tokenima ili znakovima), bez obzira na strukturu ili smisao teksta na granici dvaju odlomaka, zbog čega je najjednostavnije za implementaciju, ali i najsklonije prekidanju rečenica i gubitku konteksta. Segmentiranje s preklapanjem (engl. *overlap chunking*) ublažava taj nedostatak tako da završni dio jednog odlomka ponovno uključi na početak sljedećeg, čime se smanjuje rizik da relevantna informacija bude prekinuta točno na granici dvaju segmenata. Semantičko segmentiranje (engl. *semantic chunking*) granice odlomaka umjesto fiksnom duljinom određuje smislenim cjelinama teksta (rečenicama, temama ili odlomcima), čime bolje čuva cjelovitost značenja pojedinog segmenta. Napokon, segmentiranje temeljeno na strukturi ili sintaksi izvornog sadržaja oslanja se na parsiranje, tj. formalnu analizu strukture dokumenta (npr. raščlambu programskog koda u apstraktno sintaksko stablo, ili, kao u ovom radu, prepoznavanje naslova članaka unutar pravnog teksta) kako bi granice segmenata odgovarale stvarnim, semantički zaokruženim cjelinama izvora [17].

Funkcija `chunk_by_articles`, opisana u poglavlju o izradi RAG konfiguracije chatbota (potpoglavlje 4.5.2), pripada upravo posljednjoj, strukturnoj kategoriji, budući da granice odlomaka ne određuje fiksna duljina teksta, već strukturno obilježje izvornog dokumenta (naslovi članaka zakona). Njezin zamjenski postupak (*fallback*) za dokumente bez prepoznate strukture, `chunk_text_with_overlap`, predstavlja klasično segmentiranje s preklapanjem opisano u prethodnom odlomku.

3.7.5. Optimizacija upita i ponovno rangiranje rezultata

Osnovni RAG postupak opisan u prethodnim potpoglavljima, u kojem se korisnički upit izravno vektorizira i uspoređuje s korpusom, u literaturi se naziva naivnim RAG-om (engl. *naive RAG*) i predstavlja tek polazišnu točku. Njegova se učinkovitost u praksi dodatno poboljšava koracima koji se izvode prije i nakon same pretrage, a koji zajedno čine tzv. napredni RAG (*advanced RAG*) [19].

U koraku prije dohvata (engl. *pre-retrieval*) cilj je poboljšati sami upit prije nego što se njime pretražuje vektorski indeks. Reformulacija upita (engl. *query rewriting*) preoblikuje kratko ili nejasno korisničko pitanje u eksplicitniji oblik, čime se smanjuje razlika između formulacije pitanja i formulacije relevantnih odlomaka u korpusu. Proširivanje upita (engl. *query expansion*) generira dodatne, srodne varijante ili podupite istog pitanja te nad svakim provodi zasebnu pretragu, čime se povećava pokrivenost relevantnih odlomaka. Usmjeravanje upita (engl. *query routing*) odlučuje kojem se od više mogućih izvora ili indeksa upit treba uputiti, što je osobito relevantno u sustavima koji, poput GraphRAG-a opisanog u ovom radu, kombiniraju više izvora dohvata, vektorski indeks i graf bazu podataka [19].

Poseban oblik optimizacije upita predstavlja HyDE (engl. *Hypothetical Document Embeddings*), umjesto izravne vektorizacije kratkog korisničkog pitanja, jezičnim se modelom najprije generira hipotetski odgovor na to pitanje, koji se zatim vektorizira i njime pretražuje korpus. Budući da hipotetski odgovor stilski i sadržajno više nalikuje odlomcima korpusa nego samo pitanje, ovaj pristup posebno može poboljšati dohvat kod kratkih ili nejasno formuliranih upita. Srodan pristup, RAG-Fusion, upit dijeli na više podupita, nad svakim provodi zasebnu vektorsku pretragu te dobivene rangirane liste spaja tehnikom recipročnog rangiranja (*Reciprocal Rank Fusion*, engl. RRF) [19].

Nakon dohvata, korak ponovnog rangiranja (engl. *reranking*) preciznije procjenjuje relevantnost već dohvaćenih kandidata prije nego što se proslijede modelu, čime se smanjuje broj odlomaka koji ulaze u konačan kontekst i filtrira manje relevantan sadržaj. Postoje tri glavne skupine pristupa, heurističke metode poput BM25, *Maximal Marginal Relevance* (engl., MMR), koja balansira relevantnost i raznolikost rezultata te RRF-ovi, specijalizirani modeli s ukriženim kodiranjem (engl. *cross-encoder*) (npr. MonoBERT, MonoT5, MiniLM), koji upit i dohvaćeni odlomak kodiraju zajedno te izravno predviđaju mjeru njihove relevantnosti te pristup u kojem sami jezični model služi kao ocjenjivač relevantnosti (engl. *LLM-as-a-Judge*) [19].

RAG konfiguracija chatbota opisana u ovom radu (potpoglavlje 4.5.4) koristi oblik heurističkog rangiranja, prag sličnosti, hijerarhijski bonus prema pravnoj snazi izvora i deduplikaciju gotovo identičnih odlomaka, bez posebne optimizacije upita prije dohvata i bez zasebnog cross-encoder ili LLM-as-a-Judge koraka ponovnog rangiranja nakon dohvata. Ove tehnike, osobito reformulacija ili proširivanje kratkih upita i usmjeravanje upita između vektorskog i graf izvora, predstavljaju konkretan smjer budućeg poboljšanja, a razmatraju se i u poglavlju 7.5 kao mogući razlog zašto GraphRAG konfiguracija nije donijela očekivano poboljšanje u odnosu na RAG.

3.8. GraphRAG

Graph retrieval-augmented generation (GraphRAG) predstavlja proširenje pristupa klasičnom RAG-u koje umjesto dohvaćanja tekstualnih ulomaka dohvaća relacijsko znanje iz prethodno konstruirane grafovske baze podataka, u obliku čvorova, relacijskih trojki, putova ili podgrafova. Relacijska trojka (engl. *triple*) je osnovna jedinica znanja u grafu oblika subjekt – predikat – objekt, primjerice (Zakon o obrani, razrađuje, ustavnu odredbu), gdje su subjekt i objekt entiteti (čvorovi), a predikat relacija koja ih povezuje (usmjereni brid). Za razliku od tekstualnog RAG-a, koji se oslanja isključivo na semantičku sličnost i time zanemaruje strukturirane relacije između entiteta, GraphRAG uzima u obzir međupovezanost tekstova i strukturnu informaciju kao dodatno znanje izvan samog teksta. Osim toga, grafovski podaci poput grafova znanja nude sažimanje i apstrakciju tekstualnog sadržaja, čime se skraćuje duljina ulaza i ublažava problem preopširnosti konteksta [20].

Peng i suradnici formaliziraju cjelokupni tijek rada GraphRAG-a kroz tri faze: indeksiranje temeljeno na grafu (*G-Indexing*), dohvaćanje vođeno grafom (*G-Retrieval*) i generiranje obogaćeno grafom (*G-Generation*). U fazi indeksiranja identificira se ili konstruira grafovska baza usklađena sa zadatkom, pri čemu izvor mogu biti javni grafovi znanja ili grafovi izgrađeni iz vlastitih tekstualnih podataka. Faza dohvaćanja iz grafovske baze izdvaja najrelevantnije elemente s obzirom na korisnički upit, dok generativna faza dohvaćene grafovske podatke pretvara u oblik prikladan za jezični model te ih zajedno s upitom koristi za oblikovanje konačnog odgovora. Budući da faza indeksiranja određuje granularnost i raspon naknadnog dohvaćanja, kvaliteta konstruirane grafovske baze izravno utječe na uspješnost cijelog sustava [20].

Ključni čimbenik uspješnosti GraphRAG-a jest kvaliteta same grafovske strukture, pri čemu veći broj relacija ne znači nužno i bolju izvedbu, presudna je gustoća i povezanost grafa, a ne njegova veličina. Optimalni grafovi zahtijevaju gusto povezane zajednice bogate implicitnim višeskočnim znanjem. Takva struktura omogućuje učinkovito prelaženje (engl. *traversal*) između povezanih koncepata. Nasuprot tome, grafovi s niskom informacijskom gustoćom, obilježeni niskim prosječnim stupnjem čvora, visokim udjelom izoliranih entiteta i malim povezanim komponentama, tvore rijetke i fragmentirane strukture koje otežavaju višeskočno zaključivanje i očuvanje kontekstualne koherentnosti tijekom dohvaćanja [21].

4. Opis izrade

Ovo poglavlje opisuje praktičnu izradu triju uspoređenih konfiguracija chatbota za odgovaranje na pitanja iz hrvatskog vojnog zakonodavstva: osnovnog modela bez dohvata konteksta (*Vanilla*), modela proširenog dohvatom iz vektorske baze (RAG) te modela čiji se dohvat temelji na grafu pravnih dokumenata (GraphRAG). Najprije se opisuju domena i zahtjevi (poglavlje 4.1) te korištene programske biblioteke (poglavlje 4.2). Poglavlje 4.3 potom opisuje implementacijske elemente zajedničke svim trima konfiguracijama (jezični model, okolina izvođenja, sistemski upit, postupak generiranja odgovora i evaluacije), nakon čega slijede poglavlja specifična za svaku pojedinu konfiguraciju: *Vanilla* (poglavlje 4.4), RAG (poglavlje 4.5) i GraphRAG (poglavlje 4.6).

4.1. Opis domene i zahtjevi

Hrvatsko vojno zakonodavstvo odabrano je kao domena za usporedbu triju konfiguracija dohvata konteksta. Radi se o skupu propisa koji uređuju obranu, službu u Oružanim snagama RH i temeljno vojno osposobljavanje. Korpus se sastoji od četiri pravna dokumenta s više od 500 članaka:

- **Ustav RH** (pročišćeni tekst iz 2018., 146 članaka, `.docx`)
- **Zakon o obrani** (2025., 128 članaka, `.docx`)
- **Zakon o službi u Oružanim snagama RH** (2025., 236 članaka, `.docx`)
- **Pravilnik o temeljnom vojnom osposobljavanju** (2025., 22 članaka, `.pdf`)

Dokumenti obuhvaćaju tri hijerarhijske razine (ustavnu, zakonsku i podzakonsku). To omogućuje testiranje sposobnosti dohvata informacija raspršenih po normativnoj hijerarhiji. Sustav je namijenjen osobama kojima je potrebna brza i razumljiva orijentacija u propisima:

- kandidatima i ročnicima na temeljnom vojnom osposobljavanju
- djelatnim i pričuvnim vojnim osobama
- djelatnicima tijela državne uprave nadležnih za obranu
- studentima i istraživačima.

Sustav služi kao pomoćni alat za orijentaciju, a ne kao zamjena za pravno savjetovanje. Njegovi su ključni zahtjevi:

- **Odgovaranje na prirodnom jeziku** bez potrebe da korisnik poznaje pravnu strukturu.
- **Precizan dohvat** i navođenje konkretnog članka i stavka.

- **Povezivanje informacija** iz više dokumenata unutar jednog odgovora.
- **Dosljednost i provjerljivost** dobivenih informacija.

Evaluacijska pitanja (Prilog 1) zato uključuju i jednostavne upite (oslonjene na jedan članak) i složena pitanja koja traže sintezu više odredbi. Pravni propisi predstavljaju zahtjevan test za jezične modele iz tri razloga:

- **Stroga terminologija:** Jezik propisa je formalan i precizan. Male jezične razlike (primjerice između *ročnika*, *kadeta* i *djelatne vojne osobe*) direktno mijenjaju pravno značenje.
- **Rizik od halucinacija:** Odgovor bez točnog citata (članak i stavak) ili s netočnom tvrdnjom u pravu je neupotrebljiv i potencijalno štetan.
- **Normativna povezanost:** Ustavne, zakonske i podzakonske odredbe međusobno se nadopunjuju i isprepliću.

Zbog te potrebe za točnošću, citiranjem i povezivanjem hijerarhijskih izvora, vojno zakonodavstvo je idealno područje za usporedbu navedena tri pristupa izgradnje chatbota.

4.1.1. Etička i sigurnosna razmatranja

Budući da se sustav odnosi na područje obrane, prirodno se nameće pitanje tajnosti i osjetljivosti podataka. Cjelokupni korpus na kojem sustav radi čine isključivo propisi objavljeni u Narodnim novinama. Riječ je o javno proglašenim aktima koji su svakom građaninu slobodno dostupni pa sustav ni u jednom trenutku ne obrađuje klasificirane, operativne, kadrovske niti na drugi način ograničene podatke niti sadrži ikakvu komponentu vojne tajne. Time je pitanje povjerljivosti izvora riješeno već samim odabirom domene. Jednako tako, sustav ne prikuplja niti pohranjuje osobne podatke korisnika, evaluacijska pitanja općenite su naravi i ne odnose se na konkretne osobe, a cjelokupni je postupak reproducibilan iz javno dostupnih izvora.

Kod ovakvih slučajeva, bitno je etički jasno omeđiti namjenu sustava. On je zamišljen kao pomagalo za orijentaciju u propisima, a ne kao izvor pravnog savjeta niti kao mjerodavno tumačenje zakona, konačna odgovornost za svaku pravnu prosudbu ostaje na stručnoj osobi odnosno nadležnom tijelu. U pravnoj je domeni rizik od halucinacija i sam po sebi etički problem jer netočna tvrdnja o pravima i obvezama vojnih osoba može imati štetne posljedice. Zbog toga je u oblikovanje sustava ugrađen zahtjev za navođenjem konkretnoga članka i stavka, a pouzdanost odgovora sustavno se provjerava kroz višedimenzionalnu evaluaciju (poglavlje 5). Sustav nije namijenjen automatiziranom odlučivanju o pravima i dužnostima pojedinaca. Generativni model se izvodi na Colab platformi pa se korisnički upiti ne proslijeđuju vanjskim komercijalnim servisima, jedina uporaba modela trećih strana (Claude i Gemini) bila je u fazi evaluacije i to nad podacima predviđenima za javno dijeljenje.

4.2. Korištene programske biblioteke

U nastavku je dan pregled svih vanjskih paketa koji se instaliraju (`pip install`) i uvoze unutar izrađenih Jupyter bilježnica, grupiran prema fazi izrade rada. Sve bilježnice pokreću se u okruženju Google Colab, zbog čega se koriste i pomoćni moduli `google.colab` te `huggingface_hub` za autentifikaciju i pristup datotekama.

4.2.1. Osnovni model bez dohvata konteksta (*Vanilla*)

U ovoj konfiguraciji jezični model generira odgovore isključivo na temelju znanja stečenog tijekom treniranja, bez dohvata vanjskog konteksta. Korišteni su sljedeći paketi:

bitsandbytes Omogućuje kvantizaciju modela (8-bit/4-bit), čime se značajno smanjuje potrošnja memorije prilikom učitavanja velikih jezičnih modela.

transformers Biblioteka tvrtke Hugging Face za rad s predtreniranim jezičnim modelima. Koristi se za učitavanje tokenizatora, modela i izgradnju obradnog tijeka za generiranje teksta.

accelerate Omogućuje optimizirano izvođenje modela na GPU-u i raspodjelu resursa prilikom učitavanja velikih modela.

sentencepiece Implementacija tokenizatora temeljenog na podriječima (engl. *subword tokenizer*), koju zahtijevaju jezični modeli.

torch PyTorch, temeljni okvir za duboko učenje na kojem se izvode svi modeli.

huggingface_hub Modul `login` koristi se za autentifikaciju korisnika radi pristupa modelima s Hugging Face Huba.

google.colab Moduli `userdata` i `drive` omogućuju pristup pohranjenim API ključevima i Google Disku unutar Colab okruženja.

pandas Rad s tabličnim podacima (pohranjivanje i obrada rezultata).

os, json, re, time, copy Standardne Python biblioteke za rad s datotekama, JSON formatom, regularnim izrazima, mjerenjem vremena izvođenja i kopiranjem struktura podataka.

4.2.2. *Retrieval-Augmented Generation* (RAG)

Uz pakete iz prethodne faze, dodatno se instaliraju i koriste:

sentence-transformers Generira vektorske reprezentacije rečenica i odlomaka teksta koje se koriste za semantičku pretragu.

faiss-cpu *Facebook AI Similarity Search*, biblioteka za brzu pretragu sličnosti vektora, predstavlja vektorsku bazu podataka za dohvat najsličnijih odlomaka.

python-docx Čitanje sadržaja Microsoft Word (`.docx`) dokumenata koji čine izvornu bazu znanja.

pdfplumber Ekstrakcija teksta iz PDF dokumenata.

numpy Numeričke operacije nad nizovima, temelj za rad s vektorima.

sklearn.manifold.TSNE Metoda za redukciju dimenzionalnosti, korištena za vizualizaciju visokodimenzionalnih vektorskih reprezentacija u dvodimenzionalnom prostoru.

matplotlib, seaborn Biblioteke za izradu grafova i vizualizaciju podataka (npr. prikaz raspršenosti vektorske reprezentacije).

logging Standardna biblioteka za bilježenje tijeka izvođenja programa.

math Standardna matematička biblioteka.

4.2.3. GraphRAG

Uz zajedničke pakete iz prve dvije faze, dodatno se instaliraju i koriste:

neo4j Klijent za grafovsku bazu podataka Neo4j (*GraphDatabase*), koristi se za pohranjivanje entiteta i relacija između njih te za dohvat konteksta putem graf upita, umjesto isključivo vektorske sličnosti.

tqdm Prikaz trake napretka (engl. *progress bar*) prilikom obrade velikog broja dokumenata ili upita.

pathlib Objektno orijentiran rad s putanjama datoteka.

unicodedata Normalizacija Unicode znakova prilikom obrade teksta.

4.3. Zajednička implementacijska osnova triju konfiguracija

Kako bi usporedba triju konfiguracija bila valjana, sve tri implementacije dijele identičnu osnovu u pogledu jezičnog modela, okoline izvođenja, sistemskog upita i postupka generiranja odgovora. Jedina namjerna razlika među konfiguracijama jest način na koji se dolazi do konteksta koji se prosljeđuje modelu, dok su svi ostali čimbenici kontrolirani kako bi se izolirao učinak same metode dohвата. U nastavku su opisani zajednički elementi.

4.3.1. Temeljni jezični model i konfiguracija kvantizacije

Sve tri konfiguracije koriste isti predtrenirani jezični model, `utter-project EuroLLM-22B Instruct-2512`, učitani putem biblioteke `transformers`. Model se u svim slučajevima učitava kvantiziran na 4 bita s pomoću klase `BitsAndBytesConfig`, čime se smanjuju zahtjevi za memorijom grafičke procesorske jedinice bez izmjene arhitekture ili težina modela.

Isječak koda 1: Zajednička konfiguracija kvantizacije modela

```
1 model_name = "utter-project/EuroLLM-22B-Instruct-2512"
2
3 quantization_config = BitsAndBytesConfig(
4     load_in_4bit=True,
5     bnb_4bit_use_double_quant=True,
6     bnb_4bit_compute_dtype=torch.float16,
7     bnb_4bit_quant_type="nf4"
8 )
9
10 pipe = pipeline(
11     "text-generation",
12     model=model_name,
13     dtype=torch.float16,
14     device_map="auto",
15     model_kwargs={"quantization_config": quantization_config}
16 )
17
18 pipe.model.generation_config.max_length = 8192
```

Model se učitava preko istog `pipeline` sučelja biblioteke `transformers`, uz proslijedivanje identične konfiguracije kvantizacije kao `model_kwargs`. Izračun se izvodi u polovičnoj preciznosti (`torch.float16`), a raspodjela slojeva modela na dostupne uređaje prepuštena je automatskom mehanizmu biblioteke `accelerate` (`device_map="auto"`). Vrijednost `torch.float16` navedena je na dvjema odvojenim razinama: parametar `dtype` sučelja `pipeline` određuje preciznost svih nekvantiziranih dijelova izračuna, dok parametar `bnb_4bit_compute_dtype` unutar `BitsAndBytesConfig` zasebno određuje preciznost u koju se 4-bitno kvantizirane težine privremeno dekvantiziraju neposredno prije matričnog množenja u kvantiziranim slojevima. Oba su parametra u ovoj konfiguraciji usklađena na `torch.float16` kako bi izračun bio dosljedan kroz cijeli model. Najveća dopuštena duljina konteksta modela ograničena je na 8192 tokena u sve tri konfiguracije. Time je osigurano da eventualne razlike u kvaliteti ili brzini odgovora ne proizlaze iz različitih verzija ili konfiguracija modela, već isključivo iz razlike u pristupu dohvatu konteksta.

Konfiguracija kvantizacije oslanja se na dvije tehnike koje je uvela metoda QLoRA [22], a koje zajedno omogućuju veću uštedu memorije uz manji gubitak točnosti nego standardna 4-bitna kvantizacija. Tip kvantizacije `nf4` (`bnb_4bit_quant_type="nf4"`) odnosi se na format *NF4* (4-bit NormalFloat), čije su razine kvantizacije, za razliku od ravnomjerno raspoređenih razina standardnog cjelobrojnog 4-bitnog zapisa, izvedene iz kvantila normalne razdiobe.

Budući da su težine pretreniranih neuronskih mreža približno normalno raspodijeljene oko nule, ovakav raspored razina bolje odgovara stvarnoj raspodjeli vrijednosti koje se kvantiziraju, čime se pri istoj duljini zapisa (4 bita po parametru) postiže manja pogreška kvantizacije nego kod standardnog 4-bitnog formata [22]. Parametar `bnb_4bit_use_double_quant=True` uključuje *dvostruku kvantizaciju* (engl. *double quantization*), postupak kojim se dodatnoj kvantizaciji podvrgavaju i same kvantizacijske konstante (skalirajući faktori) nastale u prvom koraku kvantizacije težina, umjesto da se te konstante, kao uobičajeno, pohranjuju u punoj 32-bitnoj preciznosti. Time se prosječna količina memorije po parametru modela dodatno smanjuje za približno 0,37 bita bez mjerljivog utjecaja na točnost, što za veće modele donosi uštedu reda veličine nekoliko gigabajta [22].

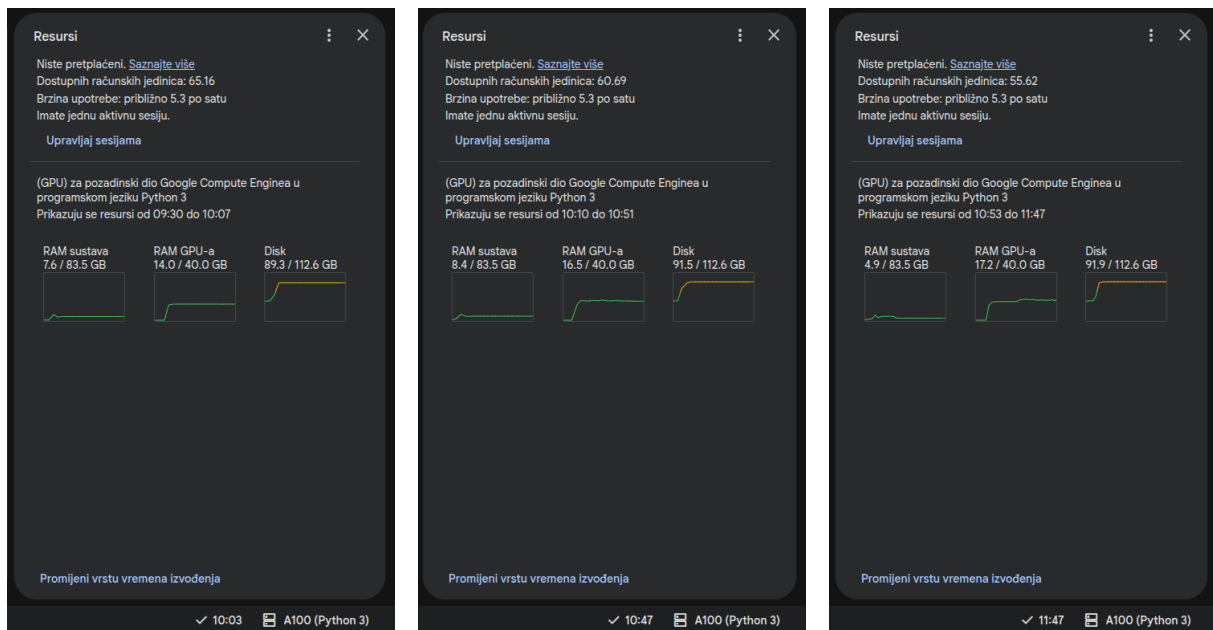
4.3.2. Okolina izvođenja

Sve tri bilježnice izvode se unutar okruženja Google Colab te dijele isti postupak pripreme okoline:

- pristup Google disku putem naredbe `drive.mount()`, koji služi kao trajna pohrana preuzetih modela i predobrađenih podataka između sesija
- autentifikacija na Hugging Face Hub putem tokena pohranjenog u Colab korisničkim podacima (`userdata.get('HF_TOKEN')`) i funkcije `login()`
- korištenje lokalno predmemoriranog modela postavljanjem varijabli okoline `HF_HOME`, `HF_HUB_OFFLINE` i `TRANSFORMERS_OFFLINE`, čime se izbjegava ponovno preuzimanje modela prilikom svakog pokretanja.

Ujednačenom okolinom izvođenja osigurano je da sve tri konfiguracije koriste identičnu inačicu modela iz iste predmemorije te se time isključuje mogućnost da razlike u rezultatima proizlaze iz nedosljedne postave sustava.

Svi eksperimenti provedeni su u okruženju Google Colab uz grafičku karticu NVIDIA A100 s 40 GB memorije (VRAM). Iskorištenost resursa tijekom triju uzastopnih sesija prikazana je na Slici 5. Kao zajednički generativni model odabran je EuroLLM-22B-Instruct u 4-bitnoj kvantizaciji (NF4). Time je osigurano da model u cijelosti stane u dostupnu memoriju grafičke kartice bez prebacivanja pojedinih slojeva na procesor ili disk. Korištenje istoga modela i istovjetne postave okruženja ključno je za metodološku ispravnost istraživanja.



(a) 09:30–10:07

(b) 10:10–10:51

(c) 10:53–11:47

Slika 5: Iskorištenost sistemske i GPU memorije te diska tijekom triju uzastopnih sesija izvođenja u okruženju Google Colab.

Zbog ograničenja izvršnog okruženja Google Colab, u kojem se dodijeljeni virtualni stroj i njegov lokalni disk (`/content`) brišu po isteku svake sesije, ponovno preuzimanje jezičnog modela sa sustava Hugging Face pri svakom pokretanju bilježnice predstavljalo je usko grlo. Model EuroLLM-22B-Instruct u formatu `safetensors` zauzima približno 44 GB, a pri opaženim brzinama preuzimanja od oko 7 MB/s njegovo dohvaćanje trajalo bi više od sat vremena po sesiji. Kako bi se osigurala reproducibilnost i vremenska učinkovitost, model je jednokratno pohranjen (keširan) na trajni prostor za pohranjivanje na Google Drive putem varijable okruženja `HF_HOME`, koja usmjerava lokalnu predmemoriju biblioteke Hugging Face na montirani direktorij Drive. Time se identičan model dijeli među svim trima bilježnicama, čime se uklanja višestruko preuzimanje i jamči da sve tri konfiguracije koriste jednake težine modela, što je preduvjet za metodološki ispravnu usporedbu.

Budući da se biblioteka Hugging Face pri učitavanju usmjerava na predmemoriju na Driveu (varijabla `HF_HOME`), model se u radnu memoriju grafičkog procesora učitava izravno iz tog trajnog keša, uz postavljen offline način rada (`HF_HUB_OFFLINE`), čime se izbjegava svako ponovno obraćanje sustavu Hugging Face nakon inicijalnog preuzimanja. Za ubrzanje samog inicijalnog keširanja korištena je biblioteka `hf_transfer`, koja omogućuje paralelno preuzimanje dijelova modela (engl. *shards*). Sama kvantizacija modela u 4-bitni format (NF4) provodi se tek pri učitavanju u memoriju grafičkog procesora, dok na disku ostaje pohranjen model pune preciznosti, zbog čega navedeni zahtjevi pohrane odgovaraju nekvantiziranoj veličini modela.

4.3.3. Sistemski upit (*system prompt*)

Sve tri konfiguracije chatbota koriste potpuno identičan sistemski upit kojim se modelu definira uloga i pravila odgovaranja:

“Ti si pravni stručnjak specijaliziran za zakonodavstvo Republike Hrvatske, posebno za Zakon o obrani, Zakon o službi u Oružanim snagama RH i pripadajuće pravilnike.

Pravila odgovaranja:

- 1. Jezik odgovora: hrvatski, latinica.*
- 2. Navedi konkretne članke zakona, brojčane vrijednosti i nadležna tijela.*
- 3. Ako je dostupan kontekst, koristi SAMO informacije iz njega.*
- 4. Ako kontekst ne sadržava odgovor, odgovori na temelju općeg znanja i dodaj napomenu da informacija nije pronađena u dostupnim dokumentima.”*

Kod konfiguracija RAG i GraphRAG dohvaćeni se kontekst nadovezuje na ovaj isti sistemski upit (`SYSTEM_PROMPT + "\n\nKontekst za odgovor:\n" + context`), dok kod konfiguracije Vanilla kontekst izostaje jer dohvat nije proveden. Zadržavanjem identičnog temeljnog upita osigurano je da se ponašanje modela ne mijenja zbog formulacije uputa, već isključivo zbog prisutnosti ili odsutnosti dohvaćenog konteksta.

4.3.4. Postupak generiranja odgovora

Generiranje odgovora u sve tri konfiguracije provodi se putem funkcije `generate_response()`, koja u svim implementacijama slijedi isti postupak:

- poruke se pretvaraju u konačan tekstualni ulaz s pomoću predloška `pipe.tokenizer.apply_chat_template()`, čime se osigurava da format upita odgovara formatu na kojem je model instruiran
- izračun najveće dopuštene duljine ulaza kao razlike najveće duljine konteksta modela i broja tokena rezerviranih za generiranje (`max_allowed_input = model_max_length - max_new_tokens`), uz ispis upozorenja ako broj tokena ulaza prelazi dopušteni limit
- postavljanje parametara generiranja putem objekta `GenerationConfig`, pri čemu se koriste isti nazivi i tip parametara (`max_new_tokens`, `temperature`, `do_sample`) te ista vrijednost kazne za ponavljanje tokena (`repetition_penalty=1.1`) u sve tri konfiguracije
- konačan poziv modela za generiranje odgovora obavlja se uz istu, determinističku konfiguraciju: `temperature=0.3` i `do_sample=False`, čime se isključuje slučajnost pri odabiru tokena i osigurava ponovljivost odgovora između pokretanja, što je nužno za pouzdanu usporedbu rezultata triju pristupa

5. generirani odgovor se dodatno obrađuje: ako ne završava interpunkcijskim znakom (., ? ili !), tekst se skraćuje do posljednjeg potpunog interpunkcijskog znaka kako bi se izbjegao odgovor prekinut usred rečenice zbog doseganja ograničenja `max_new_tokens`.

Jedina razlika u samom pozivu generiranja jest dodatni korak dohvata konteksta koji prethodi pozivu (kod RAG i GraphRAG konfiguracija), dok sama mehanika generiranja odgovora, parametri, postava, obrada ulaza i naknadna obrada izlaza, ostaje nepromijenjena u sve tri konfiguracije.

4.3.5. Skup podataka za evaluaciju i postupak paketne obrade

Sve tri konfiguracije koriste identičan skup testnih pitanja i očekivanih odgovora, pohranjen u datoteci `pitanja_odgovori.json` na Google Disku. Datoteka sadrži metapodatke (ukupan broj pitanja i raspodjelu po kategorijama) te sama pitanja grupirana po kategorijama, pri čemu se za svaki upit čuvaju polja `pitanje` i `odgovor` (referentni, očekivani odgovor).

Nad učitanim skupom podataka sve tri bilježnice provode identičan postupak paketne obrade:

1. iteracija kroz sva pitanja iz svih kategorija, uz bilježenje kategorije kojoj pitanje pripada
2. poziv odgovarajuće funkcije za generiranje odgovora (`vanilla_chat`, odnosno ekvivalentne funkcije u RAG i GraphRAG konfiguracijama) i mjerenje vremena potrebnog za generiranje svakog pojedinog odgovora
3. pohranjivanje rezultata (redni broj, kategorija, pitanje, očekivani odgovor, generirani odgovor i vrijeme generiranja) u zajednički oblikovan `pandas.DataFrame`
4. izvoz prikupljenih rezultata u CSV datoteku `rezultati.csv` na Google Disku, uz ispis ukupnog broja obrađenih pitanja i prosječnog vremena odgovora.

Korištenjem istog skupa pitanja i istog postupka mjerenja i pohranjivanja rezultata u sve tri konfiguracije omogućena je izravna, poravnata usporedba odgovora triju konfiguracija po pojedinom pitanju u fazi evaluacije (poglavlje o evaluaciji).

4.3.6. Evaluacija rezultata

rouge-score Implementacija ROUGE metrike koja uspoređuje preklapanje n-grama između generiranog odgovora i referentnog (točnog) odgovora, koristi se za evaluaciju kvalitete generiranog teksta.

bert-score Metrika koja mjeri semantičku sličnost generiranog i referentnog teksta s pomoću kontekstualnih BERT vektorskih reprezentacija, umjesto doslovnog preklapanja riječi.

sentence-transformers Ovdje se koristi za izračun semantičke sličnosti (*cosine similarity*) između vektora generiranih i referentnih odgovora.

sklearn.metrics.pairwise.cosine_similarity Funkcija za izračun kosinusne sličnosti između parova vektora.

pandas, numpy Obrada i agregacija rezultata evaluacije.

matplotlib, seaborn, matplotlib.pyplot Vizualizacija rezultata evaluacije (usporedba metrika po modelima/pristupima).

json, logging Pohranjivanje rezultata u JSON formatu i bilježenje tijeka evaluacije.

4.4. Vanilla konfiguracija chatbota

Vanilla konfiguracija predstavlja osnovni, referentni pristup u kojem jezični model odgovara na korisnička pitanja isključivo na temelju znanja stečenog tijekom predtreniranja, bez ikakvog dodatnog dohvata konteksta iz vanjskih izvora. Svrha ove konfiguracije jest uspostaviti polazišnu točku prema kojoj se u kasnijoj evaluaciji uspoređuje doprinos konfiguracija temeljenih na dohvatima informacija, RAG i GraphRAG. Model, postava kvantizacije, okolina izvođenja, sistemski upit, postupak generiranja odgovora te postupak paketne evaluacije identični su onima opisanima u zajedničkoj implementacijskoj osnovi. U ovom se potpoglavlju opisuje isključivo ono što je specifično za Vanilla konfiguraciju.

4.4.1. Funkcija `vanilla_chat`

Za razliku od RAG i GraphRAG konfiguracija, kod kojih se prije poziva generiranja provodi korak dohvata relevantnog konteksta, Vanilla konfiguracija korisničko pitanje prosljeđuje modelu izravno, bez ikakvog posredničkog koraka pretrage.

Isječak koda 2: Slaganje upita i poziv generiranja u Vanilla konfiguraciji

```
1 def vanilla_chat(user_question):
2     messages = [
3         {"role": "system", "content": SYSTEM_PROMPT},
4         {"role": "user", "content": f"{user_question}\n\nOdgovori NA
        HRVATSKOM JEZIKU koristeći LATINICU."}
5     ]
6     return generate_response(messages, max_new_tokens=512, temperature=0.3,
        do_sample=False)
```

Poruka poslana modelu sastoji se od samo dva elementa: zajedničkog sistemskog upita (`SYSTEM_PROMPT`) i korisničkog pitanja, kojemu se dodaje eksplicitna uputa da odgovor mora biti na hrvatskom jeziku i latinici. Budući da model nema pristup dokumentima specifičnima za domenu, odgovor u potpunosti ovisi o znanju koje je model usvojio tijekom predtreniranja. Očekuje se da će ovakav pristup biti osjetljiviji na činjenične pogreške i zastarjele ili nepotpune

informacije, osobito kod pitanja koja zahtijevaju poznavanje konkretnih brojčanih vrijednosti ili odredbi pojedinih članaka zakona, što ovu konfiguraciju čini korisnom referentnom točkom za procjenu koristi dohvata konteksta.

4.4.2. Izlazni podaci

Rezultati dobiveni ovom konfiguracijom pohranjuju se u zajednički `rezultati.csv` korišten za usporedbu svih triju konfiguracija, pri čemu Vanilla konfiguracija u njega upisuje sljedeće, konfiguraciji specifične stupce:

- `vanilla_odgovor` - odgovor koji je model generirao za dano pitanje bez dohvaćenog konteksta
- `vanilla_vrijeme_sekunde` - vrijeme (u sekundama) potrebno za generiranje odgovora, zaokruženo na dvije decimale.

Ostali stupci (redni broj, kategorija, pitanje i očekivani odgovor) zajednički su svim konfiguracijama i opisani su u okviru postupka paketne obrade u zajedničkoj implementacijskoj osnovi.

4.5. RAG konfiguracija chatbota

RAG konfiguracija proširuje Vanilla konfiguraciju dodatnim korakom koji se izvodi prije generiranja odgovora, dohvatom najrelevantnijih odlomaka iz korpusa pravnih dokumenata na temelju semantičke sličnosti s korisničkim pitanjem. Dohvaćeni se odlomci potom umeću u sistemski upit kao dodatni kontekst, čime se modelu omogućuje da odgovor temelji na stvarnom sadržaju zakona i pravilnika, umjesto isključivo na znanju stečenom tijekom predtreniranja. U nastavku je opisano ono što je specifično za ovu konfiguraciju: model za generiranje vektorskih reprezentacija, priprema korpusa, vektorski indeks, postupak dohvata odlomaka te njegova integracija u zajednički postupak generiranja odgovora.

4.5.1. Model za generiranje vektorskih reprezentacija

Za pretvorbu teksta u vektorske reprezentacije koristi se predtrenirani višjejezični model `intfloat/multilingual-e5-large`, učitani putem biblioteke `sentence-transformers`

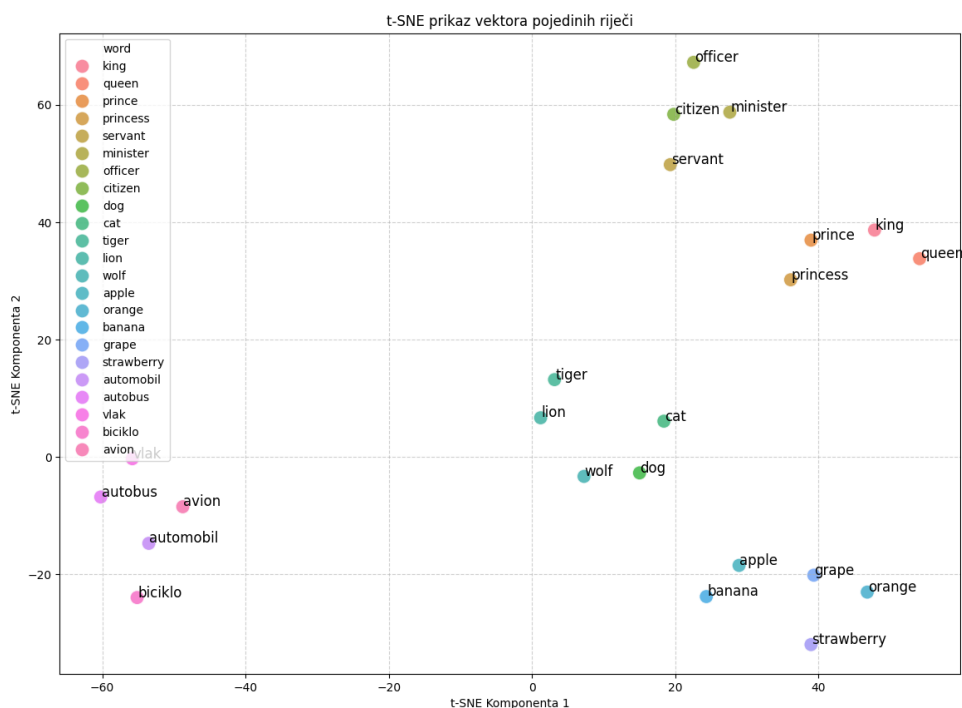
Isječak koda 3: Učitavanje modela za generiranje vektorskih reprezentacija

```
1 embed_model = SentenceTransformer("intfloat/multilingual-e5-large")
```

U skladu s konvencijom modela obitelji E5, tekstovima se prije generiranja vektora dodaje prefiks koji označava njihovu ulogu: dokumenti u korpusu označavaju se prefiksom `"passage: "`, dok se korisnički upiti prilikom pretrage označavaju prefiksom `"query: "`.

Ovim razlikovanjem model generira vektore prilagođene svrsi teksta (dokument naspram upita), što poboljšava kvalitetu semantičkog podudaranja pri pretrazi.

Radi ilustracije svojstava dobivenog vektorskog prostora, u bilježnici je uključen i demonstracijski primjer u kojem se manji skup pojmova vektorizira te prikazuje u dvije dimenzije metodom t-SNE, čime se vizualno potvrđuje da model grupira semantički srodne pojmove (npr. nazivi vozila ili nazivi zanimanja) blizu jedan drugome u vektorskom prostoru (Slika 6). Ovaj primjer služi isključivo kao ilustracija svojstava modela vektorskih reprezentacija i ne sudjeluje u stvarnom dohvat ili generiranju odgovora.



Slika 6: Prikaz vektoriziranih riječi u dvije dimenzije t-SNE metodom.

4.5.2. Priprema korpusa dokumenata

Korpus se gradi iz dokumenata pohranjenih u mapi [Dokumenti](#) na Google Disku, pri čemu su podržani formati `.txt`, `.docx` i `.pdf` (tekst se iz njih izvlači funkcijama `extract_text_from_docx` odnosno `extract_text_from_pdf` korištenjem `python-docx` i `pdfplumber`). Prilikom učitavanja svakog dokumenta prepoznaje se i njegova hijerarhijska razina na temelju naziva datoteke (`detect_document_hierarchy`): Ustav (razina 1), Zakon (razina 2), Pravilnik (razina 3) ili ostalo (razina 4). Ova se informacija kasnije koristi za rangiranje rezultata pretrage prema pravnoj snazi izvora.

Svaki dokument dijeli se na manje segmente funkcijom `chunk_by_articles`, koja tekst prvenstveno segmentira prema naslovima članaka (uzorak "Članak N."), tako da svaki odlomak korpusa najčešće odgovara jednom zakonskom članku ili njegovu dijelu, uz ograničenje najveće duljine odlomka (1200 znakova) i najmanje duljine (50 znakova) kako bi se izbjegli kratki odlomci. Ako u tekstu nije moguće prepoznati strukturu članaka, primjenjuje se generička

segmentacija s preklapanjem (`chunk_text_with_overlap`, veličina odlomka 800 znakova, preklapanje 200 znakova), koja tekst dijeli po odlomcima uz zadržavanje dijela prethodnog teksta na početku sljedećeg odlomka radi očuvanja konteksta na granicama.

4.5.3. Vektorski indeks

Dobiveni odlomci pretvaraju se u vektore (uz prefiks `"passage: "`) i pohranjuju u vektorski indeks biblioteke `faiss`, tipa `IndexFlatIP` (pretraga temeljena na skalarnom produktu), uz prethodnu L2-normalizaciju vektora, čime skalarni produkt postaje ekvivalentan kosinusnoj sličnosti:

Isječak koda 4: Izgradnja FAISS indeksa nad korpusom dokumenata

```
1 embedding_dim = embed_model.get_sentence_embedding_dimension()
2 faiss_index = faiss.IndexFlatIP(embedding_dim)
3
4 def add_documents_to_corpus(texts, metadatas=None):
5     prefixed_texts = ["passage: " + t for t in texts]
6     embeddings = embed_model.encode(prefixed_texts, convert_to_numpy=True)
7     faiss.normalize_L2(embeddings)
8     faiss_index.add(embeddings)
```

Uz svaki odlomak pohranjuju se i pripadni metapodaci (naziv datoteke, tip dokumenta i hijerarhijska razina), koji se koriste u kasnijim koracima rangiranja i raznolikosti rezultata.

Odabir tipa `IndexFlatIP` znači da s pomoću FAISS-a korpus se pretražuje egzaktno, a ne aproksimativno (ANN) (potpoglavlje 3.7.3), svaki upit uspoređuje se sa svim pohranjenim vektorima, čime je zajamčeno da će biti pronađeni stvarno najsličniji odlomci, bez gubitka preciznosti koji unose ANN indeksi poput HNSW-a. Takav pristup je opravdan upravo zbog veličine korpusa, koji broji nekoliko tisuća odlomaka pravnih dokumenata, a ne milijune, pa je linearna složenost pretrage zanemariva u odnosu na vrijeme generiranja odgovora samim jezičnim modelom. Za razliku od RAG konfiguracije, GraphRAG konfiguracija (potpoglavlje 4.6.1) oslanja se na Neo4j vektorske indekse, koji ANN pretragu interno provode HNSW algoritmom, čime dvije implementirane konfiguracije chatbota ujedno ilustriraju oba pristupa opisana u teorijskom pregledu, egzaktnu i aproksimativnu pretragu najbližih susjeda.

4.5.4. Postupak dohvata relevantnih odlomaka

Funkcijom `retrieve_relevant_passages` je implementiran dohvat relevantnih odlomaka za dano pitanje kojom se provodi višestupanjski postupak:

1. **Početna pretraga** korisnički upit vektorizira se uz prefiks `"query: "`, nakon čega se u FAISS indeksu pretražuje širi skup kandidata (`fetch_k`, do pet puta veći od traženog broja rezultata, ograničen na veličinu korpusa), kako bi se ostavio prostor za naknadno filtriranje i rangiranje

2. **Filtriranje po pragu sličnosti** kandidati čija je sličnost s upitom ispod praga `score_threshold = 0.30` odbacuju se
3. **Hijerarhijsko podizanje ocjene** rezultatima iz dokumenata više pravne snage (npr. zakon naspram pravilnika) dodaje se manji bonus na ocjenu sličnosti, proporcionalan njihovoj hijerarhijskoj razini, čime se pri jednakoj semantičkoj sličnosti daje prednost mjerodavnijem izvoru
4. **Uklanjanje gotovo identičnih odlomaka** odlomci iz istog dokumenta koji se preklapaju u više od 92 % riječi (Jaccardova sličnost skupova riječi) smatraju se duplikatima te se izostavljaju iz konačnog skupa
5. **Raznolikost izvora** uvodi se ograničenje najvećeg broja odlomaka iz istog dokumenta (`per_doc_cap`) kako konačni kontekst ne bi bio dominiran odlomcima iz jednog jedinog propisa, uz naknadno popunjavanje preostalim mjestima iz ranije izostavljenih kandidata ako selektiranih odlomaka nema dovoljno.

Rezultat funkcije jest konačan popis od najviše `top_k` odlomaka (tekst, ocjena sličnosti i metapodaci) koji se proslijeđuju u sistemski upit modela.

4.5.5. Integracija dohvata u generiranje odgovora

Zajednička funkcija `generate_response` proširena je u ovoj konfiguraciji parametrima `use_rag` i `rag_top_k`. Kada je dohvat konteksta uključen, iz posljednje korisničke poruke izdvaja se upit, dohvaćaju se relevantni odlomci te se oblikovani kontekst (svaki odlomak numeriran i označen izvorom) nadovezuje na sadržaj systemske poruke:

Isječak koda 5: Umetanje dohvaćenog konteksta u sistemsku poruku

```

1 if use_rag:
2     retrieved = retrieve_relevant_passages(user_query, top_k=rag_top_k)
3     context_text = "\n\n".join(
4         f"[{i}] {r['text']} (izvor: {r['metadata']['file']})"
5         for i, r in enumerate(retrieved, start=1)
6     )
7     final_messages[0]["content"] += "\n\nKontekst za odgovor:\n" +
    context_text

```

Ako nakon umetanja konteksta ukupan broj tokena ulaza premaši dopušteni limit, kontekst se postupno skraćuje uklanjanjem posljednjih (najmanje relevantnih) odlomaka sve dok ulaz ne stane unutar dopuštene duljine, čime se izbjegava odbacivanje cijelog upita zbog prekoračenja limita modela.

4.5.6. Prilagodba parametara prema kategoriji pitanja

Za razliku od Vanilla konfiguracije, broj dohvaćenih odlomaka i najveći broj tokena odgovora u RAG konfiguraciji nisu fiksni, već ovise o kategoriji pitanja iz skupa za evaluaciju

(jednostavno, kompleksno_jedan_dokument, kompleksno_vise_dokumenata):

Isječak koda 6: Broj dohvaćenih odlomaka i tokena odgovora prema kategoriji pitanja

```
1 RAG_TOP_K = {
2     'jednostavno': 5,
3     'kompleksno_jedan_dokument': 8,
4     'kompleksno_vise_dokumenata': 12,
5 }
6
7 RAG_MAX_TOKENS = {
8     'jednostavno': 384,
9     'kompleksno_jedan_dokument': 640,
10    'kompleksno_vise_dokumenata': 1024,
11 }
```

Ovakvom se prilagodbom kompleksnijim pitanjima, koja po definiciji zahtijevaju informacije iz više izvora ili opsežnije obrazloženje, omogućuje veći broj dohvaćenih odlomaka konteksta i duži odgovor, dok se za jednostavna pitanja broj odlomaka i duljina odgovora ograničavaju radi konciznosti i brzine generiranja.

4.5.7. Funkcija `rag_chat` i izlazni podaci

Poziv chatbota u ovoj konfiguraciji obavlja se funkcijom `rag_chat`, koja na temelju kategorije pitanja očitava odgovarajući `top_k` i `max_new_tokens` te poziva `generate_response` uz `use_rag=True`, zadržavajući pritom istu determinističku konfiguraciju (`temperature=0.3`, `do_sample=False`) kao i ostale konfiguracije.

Rezultati se ne pohranjuju u novu datoteku, već se postojeći `rezultati.csv` (generiran u Vanilla konfiguraciji) učitava i nadopunjuje dvama dodatnim stupcima:

- `rag_odgovor` - odgovor generiran uz dohvaćeni kontekst
- `rag_vrijeme_sekunde` - vrijeme (u sekundama) potrebno za generiranje odgovora, uključujući vrijeme dohvata odlomaka.

Ovim se pristupom u jednoj datoteci postupno objedinjuju odgovori svih triju konfiguracija za identičan skup pitanja, što u fazi evaluacije omogućuje izravnu, usporedbu rezultata po retcima.

4.6. GraphRAG konfiguracija chatbota

GraphRAG konfiguracija zamjenjuje vektorsku pretragu nad plošnim skupom odlomaka (kao u RAG konfiguraciji) strukturiranim grafom pravnih dokumenata pohranjenim u graf bazi podataka Neo4j. Umjesto da se relevantnost odlomka procjenjuje isključivo na temelju semantičke sličnosti s upitom, model dokumenata predstavljen je kao graf članaka, stavaka, entiteta i

eksplicitnih odnosa među njima (upućivanja na druge članke, dijeljeni pojmovi, tematska srodnost). Konačni kontekst stoga se gradi kombinacijom vektorske pretrage i obilaska grafa (engl. *graph traversal*) iz najrelevantnijih čvorova prema njihovim susjedima. Cilj je omogućiti dohvat informacija koje su međusobno pravno povezane, ali ne nužno i tekstualno slične, što je osobito važno kod pitanja koja objedinjuju odredbe iz više propisa.

U ovom radu naziv GraphRAG odnosi se na implementiranu konfiguraciju sustava koja kombinira dohvat iz grafovske reprezentacije pravnih dokumenata s generiranjem odgovora pomoću jezičnog modela. Ne pretpostavlja se da ta implementacija predstavlja jedinstvenu ili standardiziranu GraphRAG arhitekturu.

4.6.1. Graf baza podataka i shema

Graf se pohranjuje u instanci Neo4j (AuraDB) smještenoj u oblaku, na koju se spaja putem službenog Python klijenta (`neo4j.GraphDatabase`):

Isječak koda 7: Povezivanje na Neo4j i pomoćna funkcija za izvršavanje upita

```
1 neo4j_driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,  
2 NEO4J_PASSWORD))  
3  
4 def neo4j_run(query, params=None):  
5     with neo4j_driver.session() as session:  
6         return list(session.run(query, params or {}))
```

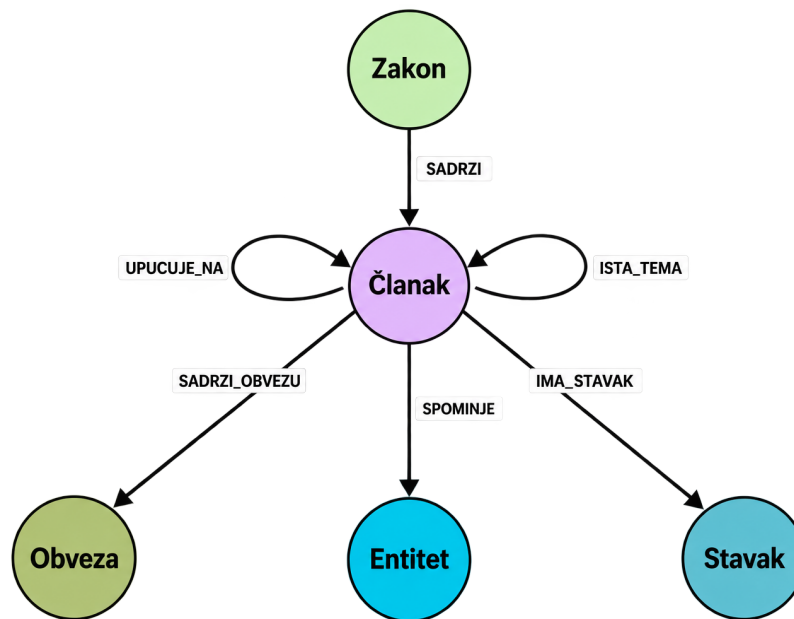
Schema grafa definirana je sljedećim tipovima čvorova:

- **Zakon** predstavlja jedan pravni akt (zakon, pravilnik ili Ustav)
- **Članak** predstavlja jedan članak propisa, nosi puni tekst članka i njegovu vektorsku reprezentaciju
- **Stavak** predstavlja pojedini stavak unutar članka, također s vlastitom vektorskom reprezentacijom, radi preciznijeg pogotka kod uskih pitanja
- **Entitet** normalizirani naziv pravnog pojma ili tijela spomenutog u članku (npr. naziv dužnosti, tijela ili instituta), dobiven prepoznavanjem imenovanih entiteta
- **Obveza** tekstualni izvadak klasificiran kao obveza, zabrana ili pravo.

Čvorovi su povezani usmjerenim relacijama: `SADRZI` (sadrži članak), `IMA_STAVAK` (ima stavak), `UPUCUJE_NA` (jedan članak izričito upućuje na drugi), `SPOMINJE` (članak spominje entitet), `SADRZI_OBVEZU` (članak sadrži obvezu), `ISTA_TEMA` (automatski izvedena tematska poveznica između članaka različitih propisa, opisana u potpoglavlju 4.6.3) te `RAZRADJUJE` (veza kojom članak nižeg akta razrađuje odredbu akta više pravne snage), koja je predviđena za ručni unos ekspertnih veza i stoga neaktivna u automatski izgrađenom grafu korištenom u evaluaciji (v. potpoglavlje 4.6.3). Jedinstvenost identifikatora osigurana je ograničenjima (engl.

constraints) nad poljima `id` čvorova `Zakon`, `Clanak` i `Stavak`, dok su nad poljima vektorske reprezentacije čvorova `Clanak` i `Stavak` definirani vektorski indeksi (`clanak_embedding`, `stavak_embedding`) s kosinusnom mjerom sličnosti, korišteni kasnije u fazi dohвата.

Cjelovita shema grafa, sa svim tipovima čvorova i relacija te njihovom međusobnom povezanošću, prikazana je na Slici 7. Dijagram je izvezen izravno iz Neo4j Browsera naredbom `CALL db.schema.visualization()`, koja iscrtava stvaran metamodel izgrađene baze na temelju prisutnih čvorova i veza, a ne ručno nacrtanu shemu.



Slika 7: Shema grafa pravnih dokumenata u GraphRAG konfiguraciji

4.6.2. Izgradnja grafa iz dokumenata

Isti izvorni dokumenti (`.docx`, `.pdf`) koji se u RAG konfiguraciji dijele na odlomke ovdje se strukturirano raščlanjuju na razinu članaka i stavaka funkcijama `graph_parse_docx` i `graph_parse_pdf`, koje prepoznaju naslove članaka (uzorak "Članak N.") i stavaka (uzorak " (N) ") regularnim izrazima. Tekst koji prethodi prvom prepoznatom članku, kao i dokumenti u kojima struktura članaka uopće nije prepoznata, ne odbacuju se, već se dijele na pomoćne pseudo-članke fiksne duljine (`_fallback_pseudo_clanci`), čime se osigurava da nijedan dio izvornog teksta ne izostane iz grafa. Kod `.docx` dokumenata dodatno se pokušava odrediti stvarni naziv propisa (umjesto naziva datoteke) pretragom prvih 15 odlomaka za oblikovanim stilom naslova (engl. *heading*), čime se u graf i kasniji prikaz konteksta unosi čitljiviji naziv zakona ili pravilnika.

Isječak koda 8: Prepoznavanje članaka i stavaka regularnim izrazima (skraćeno)

```
1 CLANAK_RE      = re.compile(r'^[Čč Cc]lanak\s+(\d+[a-z]?)\.\.?(?:\s|$)', re.
  IGNORECASE)
2 STAVAK_PATTERN = re.compile(r'^\((\d+)\)\s+')
3
4 def _parse_paragraphs(paragraphs, naziv):
5     clanci, current, pre_text = [], None, []
6     for tekst in paragraphs:
7         tekst = tekst.strip()
8         m = CLANAK_RE.match(tekst)
9         if m:
10             if current:
11                 clanci.append(current)
12                 current = {"broj": m.group(1), "tekst": "", "stavci": [],
13                           "upucivanja": [], "obveze": []}
14                 continue
15             if current is None:
16                 pre_text.append(tekst); continue
17             if STAVAK_PATTERN.match(tekst):
18                 current["stavci"].append(tekst)
19             else:
20                 current["tekst"] += " " + tekst
21         if current:
22             clanci.append(current)
23     pseudo = _fallback_pseudo_clanci(" ".join(pre_text), "pre")
24     return {"naziv": naziv, "clanci": pseudo + clanci}
```

Zajednička jezgra `_parse_paragraphs` dijeli se između `.docx` i `.pdf` raščlambe (koje se razlikuju samo u načinu izdvajanja izvornih odlomaka), a unutar petlje se dodatno prepoznaju i upućivanja na druge članke te odredbe o obvezama, opisani u nastavku.

Budući da je izgradnja grafa deterministički postupak nad istim izvornim dokumentima, prije svakog pokretanja ingestije graf se u potpunosti briše naredbom `MATCH (n) DETACH DELETE n`, nakon čega se iznova gradi od nule. Ovime se osigurava da rezultati evaluacije uvijek odgovaraju jednom, dosljednom stanju grafa, bez zaostalih čvorova ili veza iz prethodnih, potencijalno izmijenjenih verzija dokumenata ili logike ingestije.

Tijekom raščlambe svakog članka dodatno se prepoznaju:

- **unutartekstna upućivanja** na druge članke (npr. „članka 15. ovog Zakona” ili „članka 3. Ustava Republike Hrvatske”). Naziv ciljnog propisa iz fraze (npr. „ovog Zakona”, „Ustava Republike Hrvatske”, „Zakona o službi u Oružanim snagama RH”) razrješava se u konkretan identifikator čvora `Zakon` funkcijom `rijesi_zakon_id`, koja pri prvom pozivu izgrađuje i predmemorira (`_AKT_ID_CACHE`) preslikavanje između prepoznatih obrazaca u nazivu propisa i njihovih stvarnih ID-eva u grafu, dohvaćenih upitom nad već upisanim čvorovima `Zakon`, tako razriješeno upućivanje upisuje se kao relacija `UPUCUJE_NA` prema članku ciljnog propisa

- **imenovani entiteti** unutar teksta članka, prepoznati modelom za prepoznavanje imenovanih entiteta za hrvatski jezik (`classla/bcms-bertic-ner`). Budući da NER model ima ograničenje na duljinu ulaznog teksta, dulji se tekst članka prije prepoznavanja dijeli na preklapajuće prozore od 512 znakova s pomakom od 384 znaka (`graph_extract_entities`), čime se osigurava da entiteti blizu granice prozora ne budu propušteni, podriječi prepoznate kao nastavci tokena (obilježene prefiksom `###`, karakteristične za BERT-ov *tokenizer*) odbacuju se kao nepotpuni entiteti. Prepoznati entiteti dodatno se normaliziraju uklanjanjem padežnih nastavaka (jednostavni algoritam skraćivanja sufiksa) kako bi se isti pojam u različitim padežima povezao s istim čvorom `Entitet`, generički pojmovi koji se pojavljuju u gotovo svakom članku (npr. „Republika Hrvatska”, „Oružane snage”) izuzimaju se popisom zaustavnih entiteta (`STOP_ENTITETI`) kako ne bi umjetno povezivali nesrodne članke
- **odredbe o obvezama** rečenice koje sadrže obrasce tipične za obveznu („dužan”, „mora”), zabranjujuću („zabranjeno”, „ne smije”) ili ovlašćujuću („ima pravo”, „ovlašten”) normu, klasificirane jednostavnim pretraživanjem ključnih riječi i pohranjene kao čvorovi `Obveza`.

Isječak koda 9: Prepoznavanje imenovanih entiteta prozorskim pomicanjem preko teksta članka

```

1 graph_ner_pipe = pipeline(
2     "ner", model="classla/bcms-bertic-ner",
3     aggregation_strategy="simple", device=0,
4 )
5
6 def graph_extract_entities(text: str) -> list:
7     entities, seen = [], set()
8     for start in range(0, len(text), 384):
9         prozor = text[start:start + 512]
10        for r in graph_ner_pipe(prozor):
11            naziv = r["word"].strip()
12            if "###" in naziv or naziv in seen or len(naziv) <= 2:
13                continue
14            entities.append({"naziv": naziv, "tip": r["entity_group"]})
15            seen.add(naziv)
16    return entities

```

Funkcija `graph_ingest_zakon` potom za svaki članak izračunava vektorsku reprezentaciju (istim modelom `intfloat/multilingual-e5-large` korištenim u RAG konfiguraciji, uz prefiks `passage:`) i upisuje čvor članka, njegove stavke, prepoznata upućivanja, entitete i obveze u graf s pomoću `MERGE` upita, čime se osigurava idempotentnost ponovljenog pokretanja ingestije nad istim dokumentima.

Isječak koda 10: Idempotentan upis članka i upućivanja u graf (skraćeno)

```
1 def graph_ingest_zakon(zakon_data, zakon_id):
2     for cl in zakon_data["clanci"]:
3         clanak_id = f"{zakon_id}_cl_{cl['broj']}"
4         embedding = graph_embed(cl["tekst"])
5         neo4j_run("""
6             MERGE (c:Clanak {id: $id})
7             SET c.broj = $broj, c.tekst = $tekst, c.embedding = $emb
8             WITH c MATCH (z:Zakon {id: $zakon_id})
9             MERGE (z)-[:SADRZI]->(c)
10        """, {"id": clanak_id, "broj": cl["broj"], "tekst": cl["tekst"],
11              "zakon_id": zakon_id, "emb": embedding})
12
13     for target_br, akt in cl["upucivanja"]:
14         ciljni_zakon = _rijesi_zakon_id(akt, zakon_id)
15         neo4j_run("""
16             MERGE (t:Clanak {id: $tid})
17             WITH t MATCH (src:Clanak {id: $sid})
18             MERGE (src)-[:UPUCUJE_NA]->(t)
19        """, {"tid": f"{ciljni_zakon}_cl_{target_br}", "sid": clanak_id
20              })
```

Korištenje `MERGE` umjesto `CREATE` osigurava da ponovljeno pokretanje ingestije nad istim podacima ne stvara duplicirane čvorove ni veze, a razrješavanje upućivanja preko `_rijesi_zakon_id` funkcije omogućuje da relacija `UPUCUJE_NA` ispravno poveže i članke iz različitih propisa, ne samo unutar istog dokumenta.

4.6.3. Automatsko izvođenje tematskih veza

Osim eksplicitnih upućivanja unutar teksta, graf sadrži i automatski izvedene relacije `ISTA_TEMA`, koje povezuju članke različitih propisa što dijele tematski značajne entitete. Važnost dijeljenog entiteta mjeri se IDF-om (engl. *inverse document frequency*) na razini članaka: entiteti koji se pojavljuju u mnogo članaka doprinose manje, dok rjeđi, specifičniji entiteti nose veću težinu. Veza `ISTA_TEMA` uspostavlja se samo između međudokumentnih parova članaka čiji zbroj IDF-težina dijeljenih entiteta prelazi zadani prag (`MIN_TEZINA = 1, 5`) i koji dijele barem dva entiteta, čime se sprječava stvaranje velikog broja slabo informativnih veza.

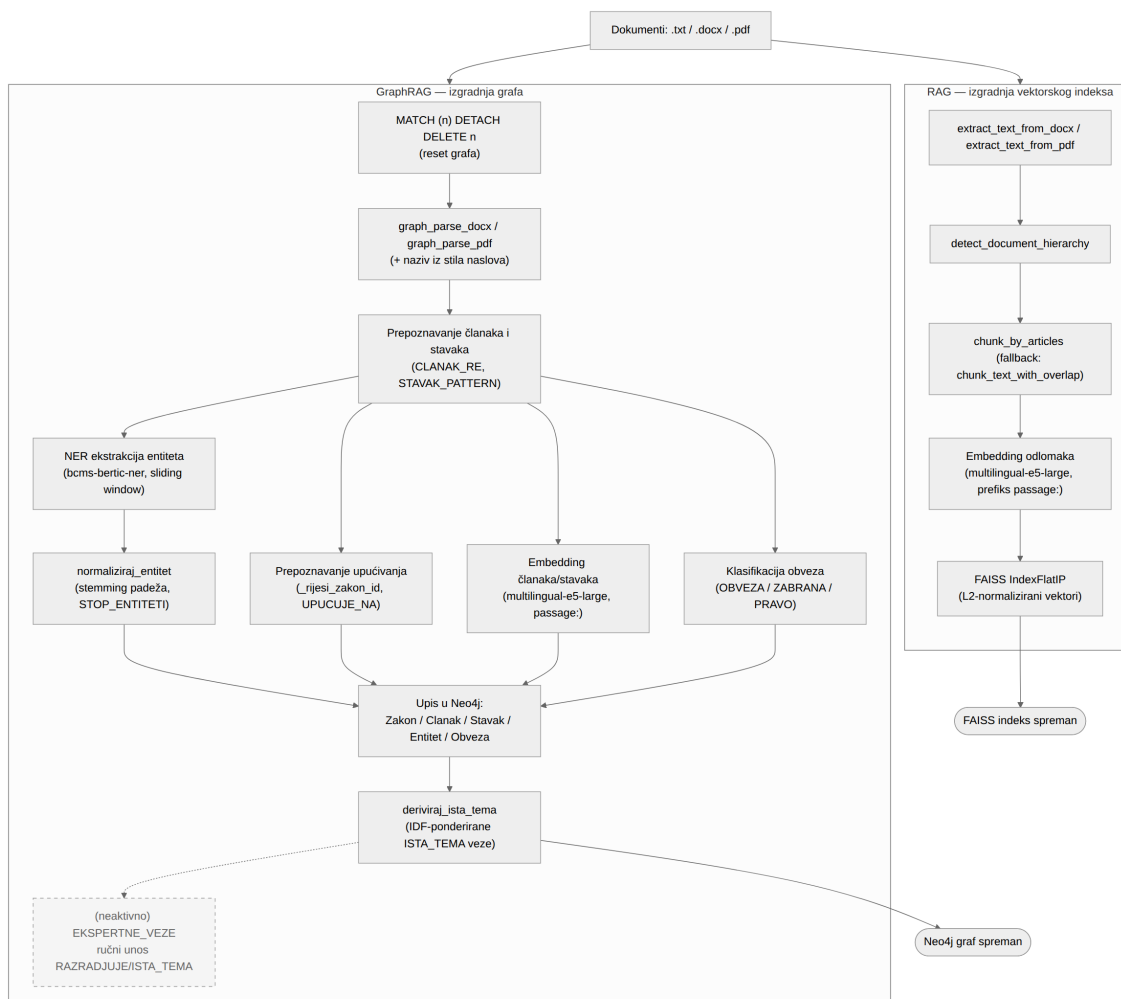
Isječak koda 11: Izvođenje IDF-ponderiranih tematskih veza između članaka različitih propisa

```
1 MIN_TEZINA = 1.5
2
3 def deriviraj_ista_tema(min_tezina=MIN_TEZINA):
4     neo4j_run("""
5         MATCH (c:Clanak) WITH count(c) AS N
6         MATCH (e:Entitet)-[:SPOMINJE]-(c:Clanak)
7         WITH N, e, count(DISTINCT c) AS df
8         WHERE df >= 2
9         WITH N, e, log(1.0 * N / df) AS idf
10        MATCH (c1:Clanak)-[:SPOMINJE]->(e)<-[:SPOMINJE]-(c2:Clanak)
11        WHERE c1.zakon_id < c2.zakon_id
12        WITH c1, c2, sum(idf) AS tezina, count(DISTINCT e) AS zaj
13        WHERE tezina >= $min_tezina AND zaj >= 2
14        MERGE (c1)-[r:ISTA_TEMA {izvor:'auto'}]->(c2)
15        SET r.tezina = tezina, r.zaj_entiteta = zaj
16        """, {"min_tezina": min_tezina})
```

Cypher upit u jednom prolazu računa IDF svakog entiteta preko svih članaka ($\log(N/df)$), a zatim za svaki međudokumentni par članaka koji dijele barem jedan entitet zbraja IDF-težine dijeljenih entiteta, uvjet `c1.zakon_id < c2.zakon_id` sprječava da se ista veza izračuna dvaput ili da se poveže članak sam sa sobom.

Iako je izvan obradnog tijeka, ali na ovom mjestu će biti spomenuta i mogućnost ručnog unosa ekspertnih veza. U Colab bilježnici je pripremljena i funkcionalnost ručnog unošenja dodatnih ekspertnih veza između članaka, što omogućuje proširenje grafa u slučajevima kada automatski postupci ne prepoznaju stvarnu pravnu poveznicu. Ova mogućnost nije aktivirana u evaluacijskoj izvedbi, ali je vrijedna spomena jer pokazuje mogući razvoj rada u budućnosti.

Slika 8 sažeto prikazuje cjelokupan postupak pripreme podataka opisan u prethodnim potpoglavljima, usporedno za obje konfiguracije koje se oslanjaju na predobrađeni korpus: izgradnju FAISS vektorskog indeksa za RAG konfiguraciju (desno) te izgradnju grafa u bazi Neo4j za GraphRAG konfiguraciju (lijevo), uključujući prepoznavanje entiteta, upućivanja, obveza i automatsko izvođenje veza `ISTA_TEMA`.



Slika 8: Procesni lanac pripreme podataka za RAG i GraphRAG konfiguracije chatbota.

4.6.4. Postupak dohvata konteksta

Funkcijom `graph_retrieve_context` implementiran je dohvat konteksta za dano pitanje i odvija se u tri faze:

1. **Vektorska pretraga početnih čvorova** - upit se vektorizira (uz prefiks `"query:"`) te se putem Neo4j vektorskih indeksa istovremeno pretražuju i članci i stavci (`GRAPH_VECTOR_POOL = 30`: kandidati svake vrste), pogodak na razini stavka mapira se natrag na njegov roditeljski članak, čime se uska, specifična pitanja i dalje povezuju s odgovarajućim širim kontekstom članka. Rezultati se potom ograničavaju najvećim brojem početnih članaka po propisu (`GRAPH_PER_DOC_CAP = 3`), kako bi pretraga uvijek pokrivala više relevantnih propisa umjesto da jedan dokument bude dominantan
2. **Obilazak grafa iz početnih čvorova** - iz svakog početnog članka graf se obilazi jedan korak po tipiziranim relacijama (`UPUCUJE_NA`, `RAZRADJUJE`, `ISTA_TEMA`) te posredno, preko zajedničkog entiteta (`SPOMINJE`), ali samo prema

člancima iz drugog propisa. Ocjena relevantnosti susjednog članka izračunava se kao umnožak ocjene početnog članka i težine tipa veze kojom je dosegnut čime se izravne pravne poveznice vrednuju više od posredne tematske srodnosti. Budući da relaciju `RAZRADJUJE` u ovom radu popunjava samo neaktivirani ručni unos ekspertnih veza, u evaluacijskom grafu ona nema instanci pa njezina težina u praksi ne dolazi do izražaja

3. **Deduplikacija i konačan odabir** - kandidati (početni i susjedni članci) objedinjuju se prema jedinstvenom identifikatoru članka (umjesto usporedbe teksta kao u RAG konfiguraciji), rangiraju prema konačnoj ocjeni te se odabire najviše `top_k` članaka, pri čemu `top_k` ovisi o kategoriji pitanja (`GRAPH_TOP_K`, jednako strukturirano kao `RAG_TOP_K` u prethodnoj konfiguraciji). Tekst svakog odabranog članka pritom se ograničava na najviše `GRAPH_MAX_CHARS_PO_CLANKU = 4000` znakova, čime se sprječava da jedan neuobičajeno dugačak članak sam po sebi potroši velik dio dopuštene duljine ulaza modela na štetu preostalih dohvaćenih članaka.

Isječak koda 12: Vektorska pretraga, obilazak grafa i rangiranje kandidata (skraćeno)

```

1 def graph_retrieve_context(query: str, kategorija: str = None) -> str:
2     top_k = GRAPH_TOP_K.get(kategorija, GRAPH_TOP_K_DEFAULT)
3     query_emb = embed_model.encode(["query: " + query],
4                                     normalize_embeddings=True)[0].tolist()
5
6     # (1) vektorska pretraga clanak_embedding i stavak_embedding indeksa,
7     #     spajanje po najboljoj ocjeni i ogranicenje GRAPH_PER_DOC_CAP po
8     #     propisu
9     seed_list = ... # vidi tekst iznad, korak 1
10
11     susjedi_res = neo4j_run("""
12         MATCH (seed:Clanak) WHERE seed.id IN $ids
13         OPTIONAL MATCH (seed)-[r:UPUCUJE_NA|RAZRADJUJE|ISTA_TEMA]-(n1:
14             Clanak)
15         OPTIONAL MATCH (seed)-[:SPOMINJE]->(e:Entitet)<-[:SPOMINJE]-(n2:
16             Clanak)
17         WHERE n2.zakon_id <> seed.zakon_id
18         RETURN seed.id AS sid,
19             collect(DISTINCT {id: n1.id, tekst: n1.tekst, rel: type(r)})
20             + collect(DISTINCT {id: n2.id, tekst: n2.tekst, rel: '
21                 SPOMINJE'})
22             AS susjedi
23     """, {"ids": [s["id"] for s in seed_list]})
24
25     candidates = {s["id"]: dict(s, rel="SEED") for s in seed_list}
26     for row in susjedi_res:
27         baza = seed_score[row["sid"]]
28         for s in row["susjedi"] or []:
29             if s["id"] in candidates:
30                 continue
31             w = GRAPH_REL_WEIGHTS.get(s["rel"], 0.7)
32             candidates[s["id"]] = {**s, "score": baza * w}
33
34     best = sorted(candidates.values(), key=lambda x: x["score"],
35                  reverse=True)[:top_k]
36     return "\n\n--\n\n".join(
37         f"[{c['zakon']}, Clanak {c['broj']}] \n {c['tekst']}" for c in best)

```

Isječak je radi preglednosti skraćen izostavljanjem prve faze (vektorska pretraga i ograničenje po dokumentu, opisano tekstualno korakom 1 iznad), prikazana je jezgra druge i treće faze, obilazak grafa jednim tipiziranim `OPTIONAL MATCH` upitom te rangiranje kandidata umnoškom ocjene početnog članka i težine tipa veze kojom je dosegnut.

Svaki odabrani članak u konačnom kontekstu označen je nazivom propisa, brojem članka, izračunatom ocjenom te, ako je dosegnut obilaskom grafa, i tipom veze kojom je pronađen (npr. veza=UPUCUJE_NA), radi transparentnosti izvora informacije u odgovoru.

4.6.5. Integracija u generiranje odgovora

Funkcija `graph_rag_chat` poziva `graph_retrieve_context` za dano pitanje i njegovu kategoriju, dobiveni kontekst nadovezuje na `SYSTEM_PROMPT` (odvojen nizom `---` između pojedinih članaka) te poziva zajedničku funkciju `generate_response` uz istu determinističku konfiguraciju (`temperature=0.3`, `do_sample=False`) kao i prethodne dvije konfiguracije. Za razliku od RAG konfiguracije, ovdje funkcija `generate_response` nema poseban parametar za uključivanje dohvata (`use_rag`), jer se dohvat obavlja izvan nje, prije poziva, a umetnuti se kontekst pri eventualnom prekoračenju dopuštene duljine ulaza skraćuje uklanjanjem posljednjih blokova razdvojenih znakovnim nizom `---`, analogno postupku u RAG konfiguraciji.

4.6.6. Alati za dijagnostiku grafa

Radi provjere kvalitete izgrađenog grafa i ponašanja pretraživača prije pokretanja pune evaluacije, bilježnica sadrži i pomoćne dijagnostičke funkcije koje se ne koriste u samom generiranju odgovora. Funkcija `dijagnoza_konteksta` za zadano pitanje ispisuje broj dohvaćenih članaka, koliko je od njih pronađeno obilaskom grafa (a ne izravnom vektorskom pretragom) te koliko je različitih propisa pokriveno dohvaćenim kontekstom, što omogućuje brzu ručnu provjeru je li dohvat za pojedino, osobito međudokumentno pitanje smislen. Nakon izgradnje grafa ispisuju se i zbirne statistike o strukturi grafa (broj propisa, članaka i stavaka, broj članaka po dokumentu, raspodjela broja veza po tipu te broj tzv. mostovnih entiteta koji izravno povezuju članke dvaju različitih propisa). Te statistike služe kao provjera ispravnosti ingestije, ali same po sebi ne utječu na generirane odgovore niti na rezultate evaluacije.

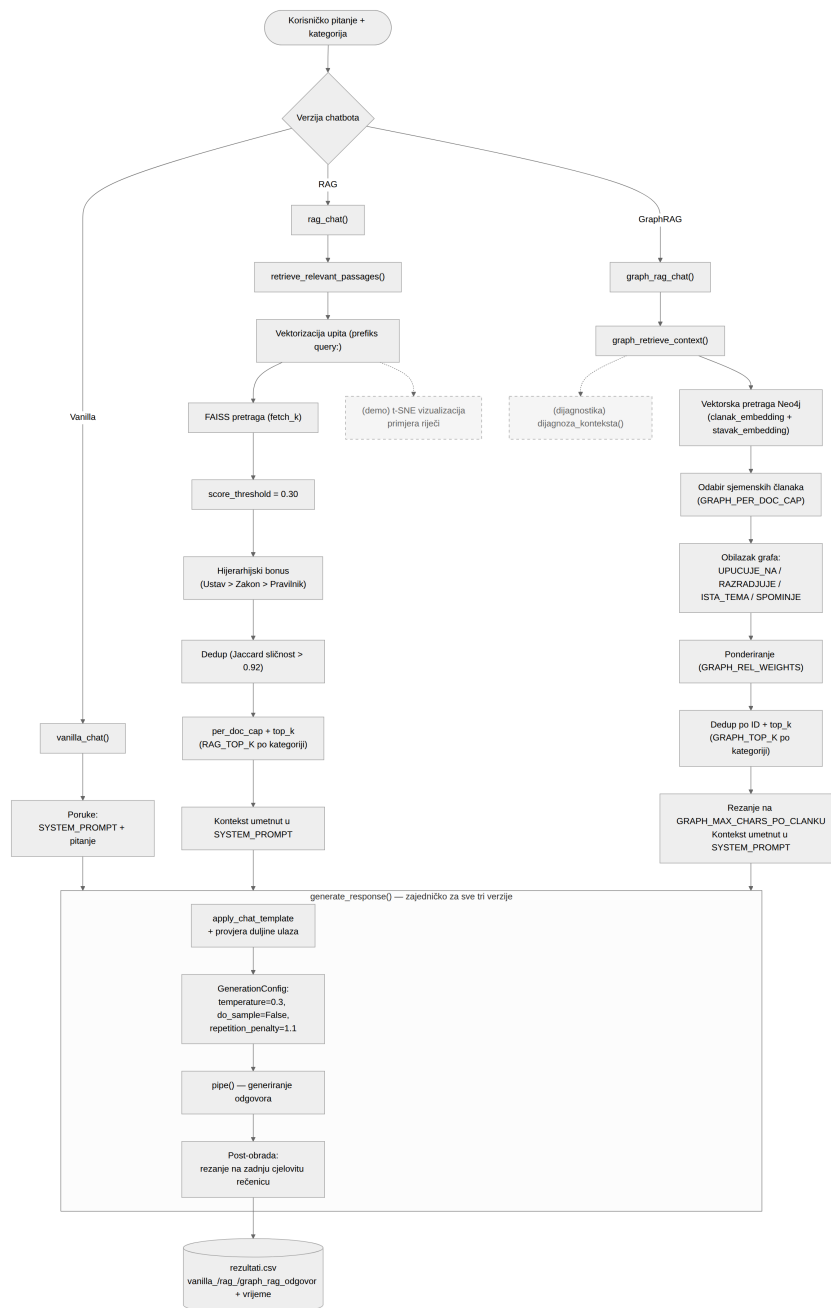
4.6.7. Izlazni podaci

Rezultati GraphRAG konfiguracije nadopunjuju isti `rezultati.csv` koji su prethodno popunile Vanilla i RAG konfiguracija, dodavanjem dvaju stupaca:

- `graph_rag_odgovor` - odgovor generiran uz kontekst dobiven obilaskom grafa
- `graph_rag_vrijeme_sekunde` - vrijeme (u sekundama) potrebno za generiranje odgovora, uključujući vektorsku pretragu i obilazak grafa.

Nakon izvođenja sve tri bilježnice, datoteka `rezultati.csv` sadrži, za svako pitanje iz zajedničkog skupa za evaluaciju, odgovore i vremena generiranja sve tri konfiguracije uz očekivani odgovor, što čini temelj za poglavlje o evaluaciji.

Slika 9 objedinjuje cjelokupan tijek obrade jednog korisničkog upita kroz sve tri opisane konfiguracije chatbota, od grananja prema konfiguraciji, preko postupaka dohvata specifičnih za RAG i GraphRAG (ili njihova izostanka u Vanilla konfiguraciji), do zajedničke funkcije `generate_response` i konačnog zapisa rezultata u `rezultati.csv`.



Slika 9: Obradni tijek korisničkog upita za Vanilla, RAG i GraphRAG konfiguraciju chatbota.

5. Evaluacija chatbotova

Iskustvo s razvojem ovog sustava pokazalo je da točnost odgovora nije jedini kriterij kvalitete. Chatbot može citirati ispravan članak zakona, a ipak biti nekoristan ako odgovor nije razumljiv ili primjenjiv. U kontekstu domene vojnog zakonodavstva, gdje pogreška u citiranju propisa ili pogrešna pravna interpretacija mogu imati ozbiljne posljedice, odabir primjerenih evaluacijskih metoda postaje od osobite važnosti. U ovom poglavlju opisuju se aspekti inferencije i latencije, automatske metrike kvalitete teksta, pristup ljudskoj evaluaciji uz mjerenje međuocjenjivačke pouzdanosti te pristup gdje jedan jezični model ocjenjuje izlaz drugog, tzv. *LLM-as-a-Judge* (engl.).

5.1. Operativni aspekti: inferenca i latencija

Uz kvalitetu generiranih odgovora, ocijenjenu automatskim metrikama i ljudskom evaluacijom u poglavljima koja slijede, chatbot namijenjen stvarnoj upotrebi mora zadovoljiti i zahtjeve vezane uz način posluživanja i brzinu odgovora. U nastavku se opisuje razlika između online i batch način rada modela (potpoglavlje 5.1.1), a potom izmjerena latencija triju uspoređenih konfiguracija sustava (potpoglavlje 5.1.2).

5.1.1. *Online* i *batch* inferenca

Ovisno o tome treba li odgovor odmah ili ne, razlikuju se dva pristupa pokretanju modela:

- ***Online* (engl.) inferenca** isporučuje izlaze u stvarnom vremenu kao reakciju na svaki korisnički zahtjev. Primjerena je za interaktivne sustave u kojima korisnik očekuje neposredan odgovor, kao što je slučaj s chatbotovima.
- ***Batch* (engl.) inferenca** obrađuje veći broj zahtjeva izvan realnog vremena, primjerice kao periodički noćni posao ili masovna obrada dokumenata. Pogodna je za scenarije u kojima latencija nije kritična, ali je propusnost (engl. *throughput*) važan čimbenik.

U ovom istraživanju koristi se isključivo *online* inferenca, budući da se evaluira interaktivni chatbot namijenjen odgovaranju na pitanja u stvarnom vremenu.

5.1.2. Latencija

Prag od 200 ms koji korisnici percipiraju kao “trenutačno” nije dostižan za LLM sustave koji generiraju dulje odgovore, ali nije ni relevantan cilj. Važnija je granica oko jedne sekunde: iznad nje korisnik svjesno čeka i gubi osjećaj dijaloga [23]. RAG konfiguracija dodaje korak dohvata prije generiranja, što bi moglo povećati ukupnu latenciju, s druge strane, dohvaćeni

kontekst obično vodi kraćim i usmjerenijim odgovorima s manje generiranih tokena, pa nije unaprijed jasno u kojem će smjeru prevagnuti.

U sklopu evaluacije mjerena je ukupna latencija odgovora (u sekundama) za sve tri ispitivane konfiguracije sustava. A priori se moglo očekivati da su konfiguracije s dohvatom sporije zbog dodatnog koraka dohvata i ugradnje konteksta, je li ta razlika stvarno prisutna, ispituje se u potpoglavlju 6.5.

5.2. Automatske metrike

Ručna evaluacija na 31 pitanju s trinaest ocjenjivača daje pouzdane, ali spore i skupe rezultate. Automatske metrike omogućuju da se isti posao napravi u sekundama, ali cijena je da ne znaju razlikovati pravno točan od pravno pogrešnog odgovora. Iz tog razloga u ovom istraživanju automatske metrike služe kao pomoćne, a ne presudne metode prosudbe.

5.2.1. Leksičke metrike

Leksičke metrike mjere sličnost na razini podudaranja riječi i fraza između generiranog i referentnog odgovora.

5.2.1.1. ROUGE-L

Recall-Oriented Understudy for Gisting Evaluation (engl., ROUGE) skup je metrika izvorno razvijenih za evaluaciju automatskog sažimanja teksta [24]. Varijanta **ROUGE-L** temelji se na najduljem zajedničkom podnizu (engl. *Longest Common Subsequence* – LCS) između generiranog i referentnog odgovora. LCS ne zahtijeva da se podudarni tokeni nalaze u neprekinutom slijedu, što ga čini otpornijim na varijacije u redoslijedu riječi. ROUGE-L vraća F1-mjeru kao harmonijsku sredinu preciznosti i opoziva.

U pravnoj domeni, parafraziranje nije uvijek vrlina. Ako chatbot umjesto “članka 45. stavka 2.” kaže “relevantnog dijela zakona”, semantički je blizu, ali korisnik koji želi citirati propis dobio je beskoristan odgovor. ROUGE-L kažnjava upravo takva odstupanja jer traži leksičko podudaranje, što ga čini prikladnijim za ovu domenu nego što bi bio u, recimo, evaluaciji kreativnog pisanja. Izostanak ili pogrešno navođenje referentnog članka kaznit će se niskim ROUGE-L rezultatom, čak i ako je ostatak odgovora semantički koherentan.

Napomena o metrici BLEU. Metrika *Bilingual Evaluation Understudy* (engl., BLEU) [25] mjeri preciznost n-gramskog podudaranja i izvorno je razvijena za strojno prevođenje. U ovom istraživanju BLEU nije primijenjen kao primarna metrika iz dva razloga, prestrogo kažnjava legitimne parafrazirane odgovore koji su sadržajno točni, ne pruža informaciju o opozivu, što je u evaluaciji chatbota jednako važno kao i preciznost. ROUGE-L je odabran kao prikladnija leksička mjera za ovaj eksperiment jer omogućuje procjenu najdulje zajedničke podsekvence između referentnog i generiranog odgovora te je manje osjetljiv na točne granice n-grama.

5.2.2. Semantičke metrike

Semantičke metrike procjenjuju sličnost na razini značenja, a ne samo podudaranja oblika riječi, što ih čini otpornijima na sinonime i parafraziranje.

5.2.2.1. BERTScore

BERTScore [26] semantička je metrika koja umjesto n-gramskog podudaranja koristi kontekstualne reprezentacije tokena dobivene predtreniranim BERT modelima. Za svaki token u generiranom odgovoru pronalazi se najbliži token u referentnom odgovoru (i obratno) prema kosinusnoj sličnosti njihovih vektora, a konačna ocjena agregira se u preciznost, opoziv i F1-mjeru.

U ovom istraživanju korišten je model `bert-base-multilingual-cased`, koji podržava hrvatski jezik i prikladan je za višezjezične domene [26]. BERTScore je posebno koristan za hvatanje semantičke ekvivalentnosti kada chatbot koristi drugačiju formulaciju od ekspertova referentnog odgovora, ali prenosi jednako značenje.

5.2.2.2. Kosinusna sličnost (*Cosine Similarity*)

Kosinusna sličnost mjeri kutnu sličnost između vektorskih reprezentacija cijelih odgovora. Viša vrijednost označava veću sličnost njihovih reprezentacija u prostoru ugrađivanja, dok niža vrijednost označava manju sličnost. Mjera pritom ne predstavlja izravnu procjenu činjenične ili pravne ispravnosti odgovora.

Za pretvaranje odgovora u vektore korišten je model `intfloat/multilingual-e5-large` [27], koji je treniran na višezjezičnom korpusu i pokazuje visoku učinkovitost na zadacima semantičkog pretraživanja i sličnosti u slavenskim jezicima. Dok BERTScore uspoređuje odgovore na razini tokena, kosinusna sličnost pruža globalnu procjenu tematske i kontekstualne podudarnosti cijelih odgovora.

5.2.3. Implementacija izračuna metrika

Automatska evaluacija provodi se neovisno o generiranju odgovora, u zasebnoj bilježnici, nad izlaznom datotekom `rezultati.csv` koju su redom nadopunile bilježnice za generiranje odgovora triju konfiguracija chatbota. Kao ulaz koriste se dvije datoteke: `rezultati.csv`, koji za svako pitanje sadrži kategoriju, sam upit, očekivani odgovor te odgovore i vremena generiranja sve tri konfiguracije, i izvorna datoteka `pitanja_odgovori.json` radi provjere potpunosti skupa. Prisutnost obveznih stupaca (`rbr`, `kategorija`, `pitanje`, `ocekivani_odgovor`, `vanilla`, `rag`) provjerava se eksplicitno, dok je stupac `graph_rag` tretiran kao opcionalan, ako nije prisutan, evaluacija se provodi samo za Vanilla i RAG konfiguraciju, čime bilježnica ostaje funkcionalna i u međukoraku, prije nego što su generirani odgovori sve tri konfiguracije.

Tri metrike opisane u potpoglavljima 5.2.1 i 5.2.2 računaju se sljedećim funkcijama:

Isječak koda 13: Funkcije za izračun metrika evaluacije

```
1 def compute_rouge_l(reference, hypothesis):
2     scorer = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=False)
3     scores = scorer.score(reference, hypothesis)
4     return scores['rougeL'].fmeasure
5
6 def compute_bertscore(references, hypotheses):
7     P, R, F1 = bert_score_fn(
8         hypotheses, references,
9         model_type='bert-base-multilingual-cased',
10        num_layers=9,
11    )
12    return F1.tolist()
13
14 def compute_cosine_similarity(reference, hypothesis, model):
15     emb_ref = model.encode([reference], convert_to_numpy=True)
16     emb_hyp = model.encode([hypothesis], convert_to_numpy=True)
17     return float(sklearn_cosine(emb_ref, emb_hyp)[0][0])
```

ROUGE-L i kosinusna sličnost izračunavaju se u petlji, redak po redak, zasebno za svaku od prisutnih konfiguracija, uspoređujući generirani odgovor s očekivanim odgovorom iz istog retka. BERTScore se, radi učinkovitosti, ne računa pojedinačno po retku, već paketno nad cijelim popisom referenci i hipoteza za svaku konfiguraciju odjednom. Dobivene vrijednosti metrika upisuju se kao novi stupci u DataFrame (npr. `vanilla_rouge_l`, `rag_bertscore`, `graph_cosine`), a prošireni skup podataka potom se pohranjuje u novu datoteku, `rezultati_evaluacija.csv`, kako bi izvorni `rezultati.csv` s odgovorima ostao nepromijenjen. Ova datoteka predstavlja ulazni skup podataka za agregaciju, vizualizaciju i statističku analizu rezultata opisane u poglavlju 6.

5.2.4. Ograničenja automatskih metrika u pravnoj domeni

Unatoč prednostima skalabilnosti i reproducibilnosti, sve tri uporabljene automatske metrike dijele suštinsko ograničenje, pretpostavljaju visoku kvalitetu ekspertnog referentnog odgovora i ne detektiraju faktičke pravne pogreške.

Konkretno, chatbot može generirati odgovor koji je leksički i semantički blizak referentnom, ali koji pogrešno primjenjuje pravnu normu, primjerice, navodi ispravan članak zakona, ali s netočnim stavkom ili pogrešnom interpretacijom. Takav odgovor dobit će visoke automatske ocjene, ali je s pravnog stanovišta netočan.

Iz tog razloga automatske metrike u ovom istraživanju imaju komplementarnu, a ne primarnu ulogu, služe za brzo filtriranje i rangiranje odgovora, dok se konačna prosudba o pravnoj točnosti prepušta ljudskim ocjenjivačima i LLM ocjenjivaču s domenski specifičnim uputama.

5.3. Ljudska evaluacija

Kako bi se nadišla ograničenja automatskih metrika opisana u potpoglavlju 5.2.4, provedena je i ljudska evaluacija odgovora triju konfiguracija chatbota. U nastavku se opisuju sudionici i dimenzije ocjenjivanja (potpoglavlje 5.3.1) te postupak mjerenja i osiguranja pouzdanosti dobivenih ocjena (potpoglavlje 5.3.3).

5.3.1. Sudionici i dimenzije ocjenjivanja

U evaluaciji je sudjelovalo trinaest ocjenjivača s komplementarnim domenskim znanjem (vojni eksperti s iskustvom u primjeni vojnog zakonodavstva). Svaki ocjenjivač radio je neovisno, bez uvida u ocjene ostalih, čime se minimizira efekt sidrenja (engl. *anchoring bias*).

Za svako od $N = 31$ pitanja ocjenjivači su ocjenjivali odgovore chatbota na Likertovoj skali od 1 do 3 prema sljedećim dimenzijama:

Tablica 1: Dimenzije ljudske evaluacije i primarni ocjenjivači

Dimenzija	Opis	Primarni ocjenjivač
Točnost	Ispravnost citiranja zakona, članaka i propisa	Svi ocjenjivači
Korisnost	Primjenjivost odgovora u stvarnoj vojnoj situaciji	Svi ocjenjivači
Halucinacije	Stupanj fabriciranja odgovora	Svi ocjenjivači

Skala ocjenjivanja definirana je kako slijedi:

1. TOČNOST — je li tvrdnja točna. Tvrdnja može biti točna, ali ne mora nužno biti odgovor. Ovom ocjenom se ocjenjuje samo točnost, a sljedećom ocjenom se nagrađuje/kaznjava relevantnost odgovora. 3 => Točno; jezgra i ključni detalji su točni. 2 => Djelomično točno; jezgra dobra, ali dio detalja krivi (npr. jedan pogrešan podatak). 1 => Pretežno netočno; kriva jezgra, krivi brojevi/rokovi ili krivo tijelo.

2. KORISNOST — odgovara li na postavljeno pitanje. Ocjenjuje se neovisno o točnosti, odgovor može biti točan, ali nekoristan ako se odnosi na drugu temu. 3 => Izravno i potpuno odgovara na pitanje. 2 => Dotiče temu, ali nepotpuno, razvučeno ili djelomično promašuje fokus. 1 => Promašuje pitanje, odgovara na nešto drugo ili je nečitljiv.

3. HALUCINACIJE — prisutnost izmišljenog sadržaja (izmišljeni članci, brojevi NN-a, URL-ovi, institucije, citati kojih nema u izvoru). 3 => Bez izmišljotina; sve provjerljivo u izvoru. 2 => Manja izmišljotina; točna jezgra, ali jedan izmišljen detalj (npr. lažni broj članka ili URL uz inače točan sadržaj). 1 => Halucinira; sadrži izmišljene činjenice, nepostojeće članke/zakone ili lažne izvore.

5.3.2. Google Obrasci

Za prikupljanje ljudskih ocjena korišteni su Google obrasci (*Google Forms*), koji su odabrani zbog jednostavnosti distribucije, standardiziranog prikaza pitanja svim ocjenjivačima te

automatskog izvoza prikupljenih odgovora u tablični oblik (CSV) pogodan za daljnju analizu. Svakom je ocjenjivaču predložen isti skup pitanja i pripadajućih odgovora triju sustava.

Svaki je odgovor ocjenjivač vrednovao prema trima kriterijima na cjelobrojnoj ljestvici od 1 do 3: faktualnoj točnosti, korisnosti odgovora te prisutnosti halucinacija. Uz svaki kriterij navedene su kratke upute s opisom značenja pojedine ocjene, kako bi se smanjila subjektivnost i uskladilo razumijevanje ljestvice među ocjenjivačima. Prikupljene ocjene potom su izvezeno i objedinjene radi izračuna prosječnih vrijednosti po sustavu i kriteriju, kao i za usporedbu s automatiziranom evaluacijom i onim generiranim putem pristupa *LLM-as-a-Judge*.

Upitnik se nalazi na sljedećoj poveznici: [Google Forms](#).

5.3.3. Međuocjenjivačka pouzdanost

Pouzdanost ocjenjivanja kvantificirana je mjerenjem slaganja između ocjenjivača, koji su iste odgovore ocjenjivali neovisno. Primijenjene su dvije mjere:

- **Fleissov Kappa** (κ) [28] – generalizacija Cohenova Kappa koeficijenta [29] na više od dva ocjenjivača, koja mjeri slaganje uz korekciju za slučajno slaganje. Vrijednosti se interpretiraju prema konvenciji: $\kappa < 0,20$ – slabo, $0,21–0,40$ – prihvatljivo, $0,41–0,60$ – umjereno, $0,61–0,80$ – dobro, $> 0,80$ – izvrsno [30].
- **Krippendorffov Alpha** (α) [31] – generalizacija koja je primjenjiva na ordinalne skale i više od dva ocjenjivača, što je relevantno za buduća proširenja evaluacijskog skupa.

Prilikom pregleda rezultata uočeno je kako dio ocjenjivača vrlo blago ocjenjuje sve odgovore, što se očituje u vrlo niskoj varijanci njihovih ocjena. Takvi ocjenjivači nisu uključeni u izračun pouzdanosti jer njihova prisutnost iskrivljuje rezultate i ne odražava stvarnu međuocjenjivačku pouzdanost. Ta se pojava naziva pristranost popustljivosti (engl. *leniency bias*) i u literaturi je prepoznata kao čest problem u ljudskoj evaluaciji [32].

Pri interpretaciji rezultata evaluacije potrebno je uzeti u obzir navedenu blagost u ocjenjivanju, odnosno sklonost pojedinih ocjenjivača da dodjeljuju više ocjene nego što odgovara stvarnoj kvaliteti ocjenjivanih odgovora. Tijekom evaluacije uočeno je da je dio ocjenjivača gotovo svim odgovorima, neovisno o tome je li ih generirala osnovna inačica jezičnog modela, RAG ili GraphRAG konfiguracija, dodijelio najviše moguće ocjene. Takvo ponašanje ne odražava stvarnu izvedbu ocjenjivanih sustava, nego značajku samog ocjenjivača te unosi šum u mjerenje [32]. Ocjenjivač koji svim konfiguracijama dodjeljuje najvišu ocjenu ne razlikuje uspoređivane konfiguracije, pa njegove ocjene ne doprinose evaluaciji, a mogu povisiti prosječne rezultate i prikriti stvarne razlike u kvaliteti odgovora.

Budući da ocjene takvih ocjenjivača nisu pokazivale izražene razlike između uspoređivanih konfiguracija sustava, one su izuzete iz konačnog izračuna rezultata. Kao kriterij korišten je udio najviših (3) ocjena po ocjenjivaču preko sve tri dimenzije, s fiksnim pragom od 85 % informirano literaturom o pristranosti popustljivosti [32]. Kriterij je definiran nakon prikupljanja

podataka, ali prije statističke analize, te je primijenjen jednako na svih trinaest ocjenjivača. Na temelju tog kriterija izuzeta su tri ocjenjivača, tako da je u konačnu analizu uključeno deset ocjenjivača. Cilj ovog postupka bio je smanjiti utjecaj potencijalno neinformativnih, sustavno visokih ocjena na agregirane rezultate [32]. Deset zadržanih ocjenjivača čine oni koji su zadovoljili kriterij. Sve mjere slaganja i ostale statistike u ovom radu računaju se nad tih deset ocjenjivača, osim gdje je izrijekom naveden puni skup od trinaest ocjenjivača. Kako izbor praga neizbježno nosi element proizvoljnosti, sve ključne usporedbe konfiguracija ponovljene su i na punom skupu od trinaest ocjenjivača (potpoglavlje 6.2) te daju isti kvalitativni zaključak. Nakon isključivanja tih ocjenjivača povećao se stupanj slaganja među preostalim ocjenjivačima, pri čemu Kendallov W za točnost raste s 0,29 na 0,53.

5.4. LLM-as-a-Judge

Uz automatske metrike i ljudsku evaluaciju opisane u prethodnim poglavljima, kao treći, komplementaran pristup evaluaciji koristi se *LLM-as-a-Judge* metoda, u kojoj veliki jezični model preuzima ulogu ocjenjivača odgovora drugog modela. U nastavku se opisuju motivacija za odabir ovog pristupa (potpoglavlje 5.4.1), implementacija i dizajn evaluacijskog prompta (potpoglavlje 5.4.2) te poznata ograničenja metode (potpoglavlje 5.4.3).

5.4.1. Pristup i motivacija

Koncept *LLM-as-a-Judge* [33] rješava problem evaluacije korištenjem velikog jezičnog modela kao neovisnog suca koji evaluira izlaz drugog modela prema unaprijed zadanim kriterijima. LLM ocjenjivač može procjenjivati i kvalitativne aspekte poput koherentnosti argumentacije, primjerenosti tona i praćenja uputa.

LLM ocjenjivač uveden je iz tri razloga. Može obraditi puno više primjera nego što bi ikad stigli ručno, primjena istog evaluacijskog prompta omogućuje standardiziranje kriterija ocjenjivanja i smanjuje varijacije uzrokovane različitim uputama te bolje od leksičkih metrika prepoznaje parafrazirane ali sadržajno točne odgovore.

5.4.2. Implementacija i dizajn prompta

Kao LLM ocjenjivač korišteni su Anthropicov model *claude-sonnet-4* i Google Gemini (Flash inačica), točan build svakog modela vidljiv je u poveznicama na [promptove](#) i [odgovore](#), koji su kasnije pretočeni u tablični oblik: [Claude](#) i [Gemini](#). Isti su odabrani zbog sposobnosti zaključivanja i razumijevanja duljeg konteksta, što je ključno jer se modelu istovremeno predaju pitanje, referentni odgovor eksperta i odgovor chatbota koje je potrebno međusobno usporediti.

Sistemske prompt dizajniran je s eksplicitnim uputama za domenu vojnog zakonodavstva i zahtjevom za ocjenjivanje triju istih dimenzija kao i kod ljudskih ocjenjivača (Tablica 1). Prompt uključuje:

- definiciju uloge: ocjenjivač je ekspert za vojno zakonodavstvo Republike Hrvatske,

- referencu na ekspertni referentni odgovor: za svaki upit LLM-u se predaje i pitanje te referentni odgovor eksperta i odgovor chatbota,
- ocjenjivanje svake dimenzije numeričkom ocjenom 1-3 uz kratko obrazloženje na hrvatskom jeziku.

5.4.3. Poznata ograničenja

Primjena LLM-as-a-Judge pristupa donosi nekoliko svojstvenih ograničenja na koja upozorava literatura [33] [34]:

- **Pristranost pozicije** (engl. *position bias*): LLM ocjenjivači skloni su favorizirati odgovor koji se pojavljuje prvi u kontekstu. U ovom istraživanju odgovori se uvijek predaju u fiksnom redoslijedu (Vanilla, RAG pa GraphRAG), što treba uzeti u obzir pri interpretaciji rezultata.
- **Pristranost samopohvale** (engl. *self-enhancement bias*): modeli mogu favorizirati odgovore koji stilski nalikuju njihovim vlastitim odgovorima.
- **Ograničeno domensko znanje**: unatoč visokokvalitetnom modelu, LLM ocjenjivač može propustiti suptilne pravne pogreške koje bi iskusan pravni ekspert odmah prepoznao.
- **Neponovljivost** (engl. *non-reproducibility*): stohastičnost jezičnih modela može uzrokovati marginalne razlike u ocjenama pri ponovljenom pokretanju s istim ulazom.

Iz navedenih razloga LLM-as-a-Judge tretira se kao dopunska, a ne zamjenska metoda u odnosu na ljudsku evaluaciju.

5.5. Obrazloženje odabira kombinacije metoda

U Tablici 2 prikazana je usporedba svih primijenjenih evaluacijskih metoda prema ključnim karakteristikama relevantnim za domenu vojnog zakonodavstva.

Tablica 2: Usporedba evaluacijskih metoda

Metoda	Skalabilnost	Semantičko razumijevanje	Detektira činjenične pogreške	Pravna specifičnost
ROUGE-L	visoka	nisko	ne	djelomično
BERTScore	visoka	visoko	ne	nisko
Cosine sličnost	visoka	visoko	ne	nisko
LLM-as-a-Judge	srednja	visoko	djelomično	visoko
Ljudska evaluacija	niska	visoko	da	visoko

Nijedna od ovih metoda sama za sebe nije dovoljna, svaka hvata nešto što druge propuštaju, pa ih je jedino smisleno koristiti zajedno. Automatske metrike (ROUGE-L, BERTScore, kosinusna sličnost) osiguravaju skalabilnu i reproducibilnu baznu liniju. LLM-as-a-Judge pruža kvalitativnu procjenu bez angažmana domenskih stručnjaka za svaki pojedinačni upit. Ljudska evaluacija predstavlja primarni izvor procjene pravne točnosti, ali njezinu interpretaciju treba promatrati uz ograničenje niskog međuocjenjivačkog slaganja. Korelacija ljudskih procjena s automatskim metodama i LLM ocjenjivačima dodatno omogućuje procjenu u kojoj mjeri automatizirane metode prate ljudsku procjenu kvalitete odgovora u ovoj specifičnoj domeni.

6. Statistička analiza rezultata evaluacije

U ovom se poglavlju prikazuju rezultati evaluacije triju konfiguracija sustava prema dimenzijama definiranim u poglavlju 5. Cilj je utvrditi postoje li statistički značajne razlike među konfiguracijama te u kojoj mjeri automatske metrike i LLM ocjenjivači prate ljudsku procjenu kvalitete odgovora.

Ljudsku evaluaciju provelo je trinaest eksperata, no tri ocjenjivača identificirana su kao potencijalno pristrana prema sustavno višim ocjenama u odnosu na ostale ocjenjivače. Za identifikaciju potencijalno pristranih ocjenjivača korišten je kriterij temeljen na distribuciji ukupnih ocjena.

Svaki je odgovor ocijenjen na ordinalnoj skali od 1 do 3 po tri dimenzije: točnost (1 = netočan, 3 = točan), korisnost (1 = nekoristan, 3 = koristan) i halucinacije (1 = ozbiljne halucinacije, 3 = bez halucinacija). Zbog ordinalne skale s tri razine, malog uzorka ($N = 31$ pitanje) i dizajna ponovljenih mjerenja (svako se pitanje ocjenjuje pod sve tri konfiguracije), primijenjene su neparametarske metode za zavisne uzorke. Korišten je Friedmanov test kao objedinjeni (omnibus) test, a potom, gdje je on značajan, post-hoc upareni Wilcoxonov test predznačenih rangova s Bonferroni korekcijom. Kao veličina efekta uparene usporedbe navodi se upareni rank-biserijalni koeficijent $r = \sqrt[3]{4V/(n(n+1)) - 1}$, gdje je V Wilcoxonova statistika predznačenih rangova, a n broj nenulih uparenih razlika uključenih u test, povezanost dviju mjera kvantificira se Spearmanovom korelacijom (ρ). Potpuni ispisi testova (statistike V , χ^2 , sirovi p) nalaze se u R skriptama ([R skripta - eksperti](#), [R skripta - analiza automatskih metrika i LLM-as-a-Judge](#), [Google disk - Izlazni statistički podaci](#)).

6.1. Deskriptivni pregled

Tablica 3: Deskriptivna statistika ljudskih ocjena po konfiguraciji

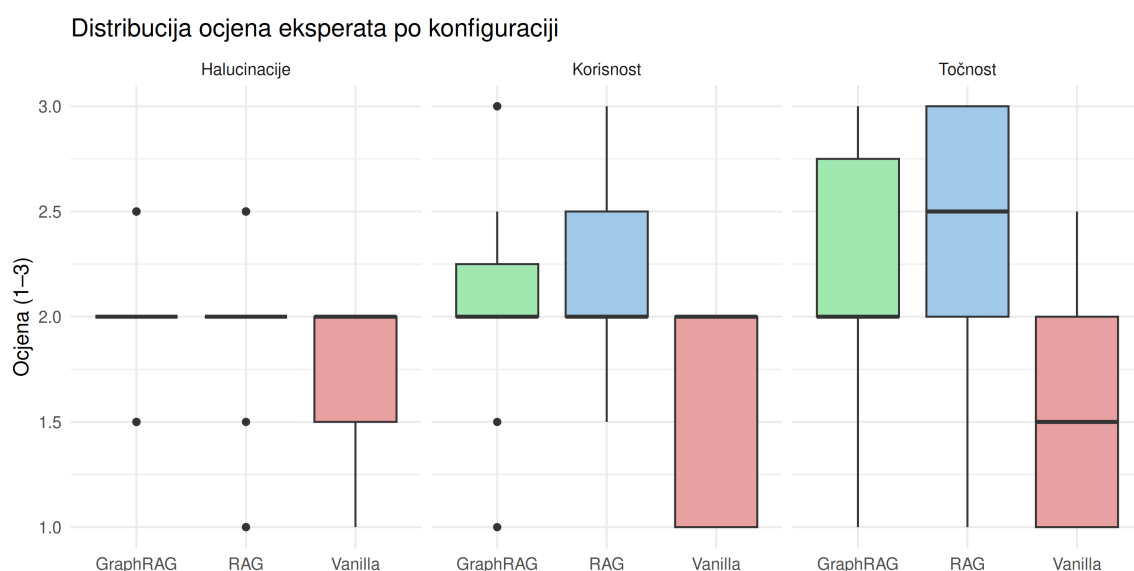
Konfiguracija	Dimenzija	$\bar{x}$	Med.	s
Vanilla	Točnost	1,56	1,5	0,50
Vanilla	Korisnost	1,65	2	0,45
Vanilla	Halucinacije	1,69	2	0,42
RAG	Točnost	2,32	2,5	0,60
RAG	Korisnost	2,13	2	0,36
RAG	Halucinacije	2,02	2	0,35
GraphRAG	Točnost	2,27	2	0,53
GraphRAG	Korisnost	2,11	2	0,38
GraphRAG	Halucinacije	2,03	2	0,26

Vrijednosti u Tablici 3 izračunate su u dva koraka. Za svako je pitanje najprije uzet medijan ocjena deset ocjenjivača (pri parnom broju ocjenjivača to je prosjek dviju srednjih ocjena,

pa po pitanju medijan može poprimiti i polucije vrijednosti poput 1,5 ili 2,5), a potom su nad tako dobivenih 31 vrijednosti po konfiguraciji izračunati aritmetička sredina ($\bar{x}$), medijan (Med.) i standardna devijacija (s). Zbog toga i stupac medijana u tablici može sadržavati polucije vrijednosti, iako je izvorna ljestvica ocjenjivanja cjelobrojna.

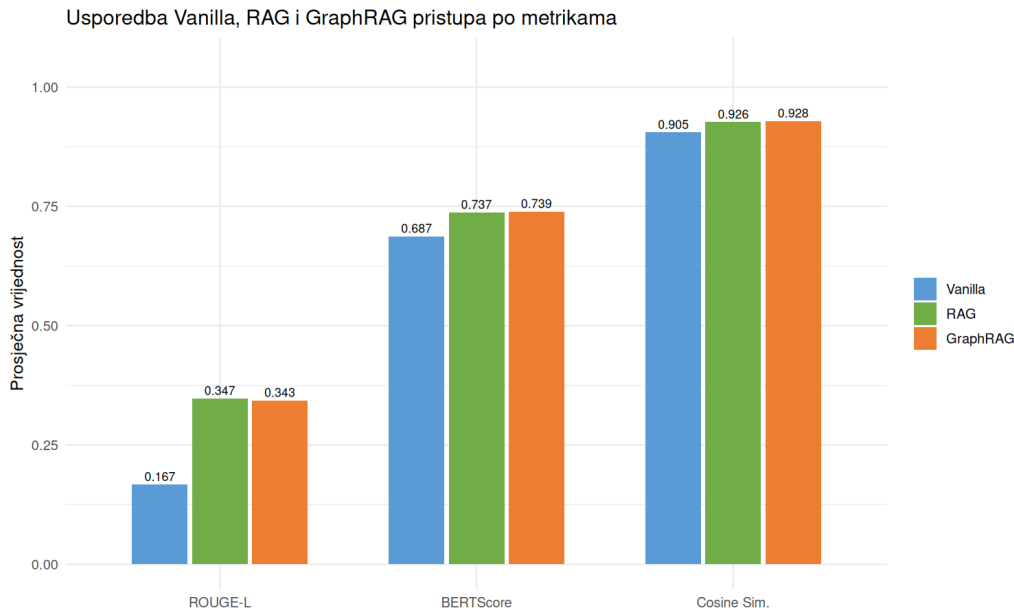
Tablica 3 pokazuje da Vanilla konfiguracija ima niže prosjeke na sve tri dimenzije (1,56–1,69) od RAG i GraphRAG konfiguracija (2,02–2,32), dok su prosjeci RAG-a i GraphRAG-a međusobno bliski, s razlikama unutar 0,05 bodova. Standardne devijacije (0,26–0,60) usporedive su kroz sve tri konfiguracije. Je li razlika Vanilla naspram RAG/GraphRAG statistički značajna te je li razlika između RAG-a i GraphRAG-a i statistički zanemariva ili tek slučajno mala na promatranom uzorku, ispituje se u nastavku.

Slika 10 prikazuje raspodjelu istih agregiranih ocjena po pitanju, odvojeno za svaku dimenziju, čime se uz sažete pokazatelje iz Tablice 3 vidi i oblik same raspodjele (medijan, raspon, iznimke).



Slika 10: Distribucija ljudskih ocjena po konfiguraciji i dimenziji.

Isti se pregled provodi i nad automatskim metrikama (ROUGE-L, BERTScore, kosinusna sličnost) izračunatima postupkom iz potpoglavlja 5.2.3. Slika 11 prikazuje prosječnu vrijednost svake metrike po konfiguraciji: obje konfiguracije s dohvatom nadmašuju Vanilla na sve tri metrike, dok su međusobno bliske.



Slika 11: Usporedba triju konfiguracija chatbota po prosječnim vrijednostima triju automatskih metrika.

Deskriptivni prikazi i za ljudske ocjene i za automatske metrike upućuju na isti obrazac: jasan odmak Vanilla konfiguracije od dviju konfiguracija s dohvatom, uz malu razliku između RAG-a i GraphRAG-a. Formalno testiranje slijedi u nastavku.

6.2. Usporedba konfiguracija

Tablica 4: Razlike među konfiguracijama

Mjera	Van. vs. RAG	Van. vs. GraphRAG	RAG vs. GraphRAG
Točnost (ljudska)	0,97**	0,91**	0,13
Korisnost (ljudska)	0,95**	0,85**	0,01
Halucinacije (ljudska)	0,71*	0,80**	0,11
ROUGE-L	0,89**	0,81**	0,04
BERTScore	0,88**	0,72**	0,15
Kosinusna sličnost	0,87**	0,82**	0,01

* $p_{\text{Bonf.}} < 0,05$, ** $p_{\text{Bonf.}} < 0,01$, parovi bez oznake nisu statistički značajni. Bonferroni-korigirana p -vrijednost odsijeca se na 1, pa zapis $p_{\text{Bonf.}} = 1,0000$ znači da je umnožak sirovoga p i broja usporedbi dosegnoo ili premašio tu granicu, za par RAG–GraphRAG i sirove (nekorrigirane) p -vrijednosti kreću se od 0,65 do 1,00. Svakoj mjeri prethodi značajan Friedmanov omnibus test ($p \leq 0,0021$, Kendallov $W = 0,20$ – $0,53$).

Obrazac je jednak na svih šest mjera i na oba neovisna izvora (ljudske ocjene i automatske metrike): Vanilla se statistički značajno razlikuje i od RAG-a i od GraphRAG-a, uz velike do

vrlo velike veličine efekta ($r = 0,71\text{--}0,97$).¹ Nijedna konfiguracija s dohvatom nije ni po jednoj mjeri lošija od Vanilla polazišta.

Razlika između RAG-a i GraphRAG-a nije statistički značajna ni na jednoj od šest mjera ($p_{\text{Bonf.}} = 1,0000$ posvuda), uz zanemarive procijenjene veličine efekta ($r = 0,01\text{--}0,15$). Rezultati stoga ne pružaju dokaz mjerljive prednosti jedne konfiguracije nad drugom u promatranom evaluacijskom skupu. Zbog veličine uzorka i ograničene snage testa za otkrivanje vrlo malih razlika taj nalaz ipak ne treba tumačiti kao dokaz potpune jednakosti konfiguracija, za takvu bi tvrdnju bila potrebna zasebna ekvivalencijska analiza (npr. TOST). Ponavljanje cijele analize na punom skupu od trinaest ocjenjivača (prije isključivanja blago ocjenjujućih) daje isti kvalitativni zaključak na sve tri dimenzije, isključivanje pritom povećava dosljednost rangiranja (Kendallov W za točnost raste s 0,29 na 0,53), ali ne mijenja nijedan nalaz. Mogući uzroci izostanka prednosti GraphRAG-a razmatraju se u potpoglavlju 7.5.

6.3. Slaganje mjera s ljudskom prosudbom

Tablica 5 prikazuje Spearmanovu korelaciju svake automatske metrike i svakog LLM ocjenjivača s medijanom ljudske ocjene, po dimenziji. Jedinica analize je par (pitanje $\times$ konfiguracija), pa se svaka korelacija računa nad $n = 93$ opažanja (31 pitanje pod tri konfiguracije). Ta opažanja nisu potpuno nezavisna, jer po tri para dijele isto pitanje, zbog čega pripadne p -vrijednosti treba shvatiti kao orijentacijske, a ne kao strogi test značajnosti. Stoga se vrijednost $n = 93$ ne treba interpretirati kao 93 potpuno neovisna eksperimentalna slučaja, nego kao 93 parova pitanje–konfiguracija.

Tablica 5: Spearmanova korelacija (ρ) automatskih metrika i LLM ocjenjivača s dimenzijama ljudske evaluacije

Mjera	Točnost	Korisnost	Halucinacije
ROUGE-L	0,40**	0,37**	0,23*
BERTScore	0,38**	0,32**	0,29**
Kosinusna sličnost	0,51**	0,51**	0,49**
Claude (LLM)	0,79**	0,50**	0,54**
Gemini (LLM)	0,73**	0,61**	0,56**

Sve automatske metrike koreliraju s ljudskom evaluacijom, ali tek slabo do umjereno ($\rho = 0,23\text{--}0,51$); kvadrirane vrijednosti kreću se u rasponu $\rho^2 = 0,05\text{--}0,26$, što ukazuje na slabu do umjerenu podudarnost rangiranja, pa metrike nisu zamjena za ljudsku evaluaciju, što je u skladu s raspravom iz potpoglavlja 5.2.4.² Kosinusna sličnost dosljedno prednjači na sve tri dimenzije jer, za razliku od leksičkog ROUGE-L-a i BERTScore-a koji uspoređuju kontekstualne

¹Velike vrijednosti r ne proturječe niskoj međuocjenjivačkoj pouzdanosti iz potpoglavlja 6.4: veličina efekta ovdje se računa nad agregiranim medijanom po pitanju, na kojem se utjecaj pojedinačnog neslaganja umanjuje, dok je slaganje nisko tek na razini pojedinačne ocjene.

²Kvadrat Spearmanova koeficijenta nema jednaku interpretaciju kao koeficijent determinacije u linearnoj regresiji, budući da se računa na rangovima, ovdje služi samo kao gruba mjera jačine monotone povezanosti.

reprezentacije na razini tokena, ona uspoređuje cjelokupno semantičko značenje odgovora. ROUGE-L je pritom najslabiji upravo na halucinacijama (0,23), budući da leksičko podudaranje ne prepoznaje izmišljen sadržaj koji se uklapa u kontekst.

Oba LLM ocjenjivača prate ljudsku prosudbu znatno bolje ($\rho = 0,50\text{--}0,79$) od bilo koje automatske metrike, no nijedan koeficijent ne prelazi $\rho \approx 0,8$, pa ni najbolji LLM ocjenjivač nije zamjena za ljudsku evaluaciju, nego njezina jeftinija i skalabilnija aproksimacija. Claude i Gemini k tomu visoko koreliraju međusobno ($\rho = 0,72\text{--}0,86$), tj. slažu se oko relativnog rangiranja odgovora, ali im se apsolutna strogost razlikuje. Gemini je sustavno blaži na halucinacijama (prosjek 2,38 naspram 1,83 kod Claudea) i, u manjoj mjeri, na točnosti (2,22 naspram 2,00), dok su na korisnosti usklađeni (2,40 naspram 2,47). Apsolutne vrijednosti LLM-as-a-Judge ocjena stoga nisu izravno usporedive između modela ocjenjivača, no relativno rangiranje triju konfiguracija time nije dovedeno u pitanje.

6.4. Međuocjenjivačka pouzdanost

Tablica 6: Međuocjenjivačka pouzdanost deset ocjenjivača

Dimenzija	Fleissov κ	Krippendorffov α	Interpretacija
Točnost	0,108	0,065	slabo
Korisnost	0,051	0,021	slabo
Halucinacije	0,035	−0,025	slabo

Međuocjenjivačka pouzdanost slaba je na sve tri dimenzije (na punom skupu od trinaest ocjenjivača bila je još niža: Fleissov κ 0,001–0,052). Pojedinačni ocjenjivači često se ne slažu oko konkretne ocjene, što je razumljivo s obzirom na suptilnost pravne prosudbe (odgovor može biti djelomično točan, formalno ispravan ali nepotpun ili ovisan o interpretaciji propisa) i grubu tročlanu ljestvicu. Dodatni je razlog niskoj vrijednosti κ restrikcija raspona: na tročlanoj se ljestvici ocjene gomilaju u uskom rasponu oko sredine ljestvice, čime očekivano slučajno slaganje u Fleissovoj formuli postaje visoko, pa koeficijent ispada nizak i onda kad su apsolutne ocjene bliske (poznati “ κ -paradoks” pri neuravnoteženim rubnim udjelima). Agregacija medijanom deset ocjenjivača po pitanju smanjuje utjecaj ekstremnih pojedinačnih procjena i daje stabilniju sažetu ocjenu po pitanju, ali ne uklanja u potpunosti nesigurnost koja proizlazi iz niskog slaganja (dio neslaganja može biti i sustavan, tj. posljedica različitog tumačenja kriterija, a ne samo slučajan šum). Razlika Vanilla naspram konfiguracija s dohvatom ipak ostaje dovoljno velika i dosljedna da bude statistički značajna (poglavlje 6.2), dok malu razliku između RAG-a i GraphRAG-a, izvedenu iz istih ocjena, treba tumačiti s dodatnim oprezom.

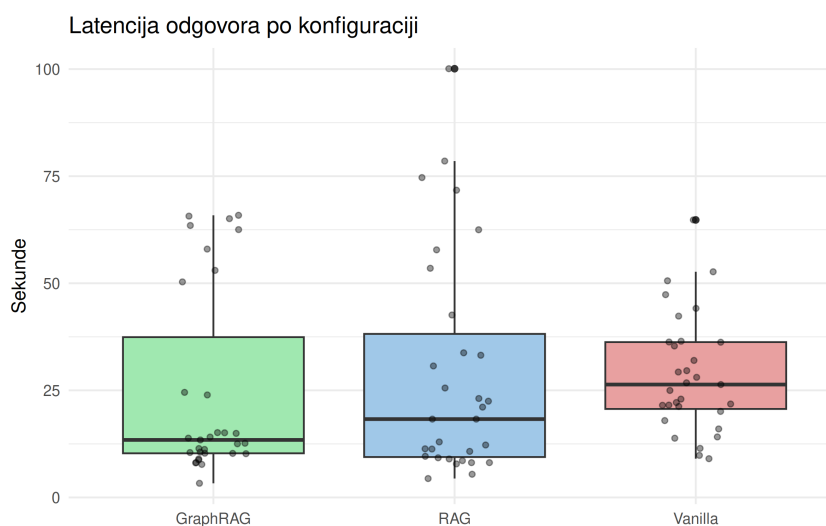
6.5. Latencija

Tablica 7: Statistika latencije odgovora (u sekundama)

Konfiguracija	$\bar{x}$	Med.	P_{95}	s
Vanilla	28,61	26,36	51,64	13,53
RAG	28,93	18,29	76,61	25,91
GraphRAG	24,63	13,43	65,39	22,07

Friedmanov test nad uparenim mjerenjima latencije po pitanju ne pokazuje statistički značajnu razliku između triju konfiguracija ($\chi^2(2) = 4,71$, $p = 0,0949$, Kendallov $W = 0,08$). Iako RAG i Vanilla imaju sličnu prosječnu latenciju, a GraphRAG u prosjeku nešto nižu, razlike nisu dosljedne na razini pojedinih pitanja te ne dosežu razinu statističke značajnosti, pa se latencija triju konfiguracija u ovom uzorku ne može smatrati pouzdano različitom.

Sve tri konfiguracije pokazuju znatno veći prosjek od medijana (najizraženije kod GraphRAG: prosjek 24,63 s naspram medijana 13,43 s) te visoke vrijednosti 95. percentila (do 76,61 s kod RAG-a), što upućuje na desno asimetričnu raspodjelu s povremenim znatno sporijim odgovorima, umjesto na ujednačeno povišenu latenciju. Za praktičnu primjenu to znači da očekivano vrijeme odziva treba temeljiti na višim percentilima, a ne na prosjeku, jer povremeni spori odgovori (npr. kod osobito opsežnih ili međudokumentnih pitanja) bitno odstupaju od tipičnog slučaja.



Slika 12: Distribucija latencije odgovora po konfiguraciji.

Slika 12 prikazuje raspodjelu latencije po konfiguraciji, uključujući pojedinačna mjerenja (točke) preko kojih se vidi široki raspon i preklapanje distribucija triju konfiguracija, što je u skladu s rezultatima Friedmanova testa koji nije pokazao statistički značajnu razliku u latenciji.

6.6. Sinteza rezultata

Statistička analiza (potpoglavlja 6.1–6.5) daje pet međusobno konzistentnih nalaza:

- **Dohvat konteksta jasno i dosljedno nadmašuje model bez dohvata.** RAG i GraphRAG statistički su značajno bolji od Vanilla konfiguracije na sve tri dimenzije ljudske evaluacije i sve tri automatske metrike, uz velike do vrlo velike veličine efekta ($r \geq 0,71$). Ovo je najjednoznačniji nalaz evaluacije.
- **GraphRAG ne donosi mjerljivu prednost nad RAG-om.** Razlika nije značajna ni na jednoj od šest mjera ($p_{\text{Bonf.}} = 1,0000$), uz zanemarive veličine efekta ($r = 0,01–0,15$), deskriptivno RAG čak ima nešto viši prosjek točnosti. Zbog niske međuocjenjivačke pouzdanosti i malog uzorka ovo treba čitati kao izostanak dokaza o prednosti GraphRAG-a, a ne kao dokaz jednakosti dviju konfiguracija.
- **Nijedna automatska metrika nije dovoljna sama za sebe.** Korelacije s ljudskom evaluacijom slabe su do umjerene ($\rho = 0,23–0,51$), uz kosinusnu sličnost kao najbolju, LLM ocjenjivači prate ljudsku prosudbu bitno bolje ($\rho = 0,50–0,79$), čime se opravdava kombinacija metoda iz potpoglavlja 5.5.
- **Claude i Gemini slažu se u rangiranju, ali ne u strogosti.** Visoka međusobna korelacija ($\rho = 0,72–0,86$) uz sustavno blaži Gemini na halucinacijama znači da se apsolutne LLM ocjene ne smiju izravno uspoređivati između modela.
- **Latencija ne pokazuje statistički značajnu razliku između konfiguracija** ($p = 0,0949$), uz izraženu desnu asimetriju kod sve tri. Izostanak značajnosti pri ovom uzorku ne treba tumačiti kao dokaz da dohvat konteksta nema vremenskog troška, nego kao nedostatak dokaza o dosljednoj razlici, na to upućuje i osjetno viši 95. percentil RAG konfiguracije (76,61 s naspram 51,64 s kod Vanilla), koji pokazuje da se trošak dohvata povremeno ipak manifestira.

Sve zaključke treba čitati uz ogradu iz potpoglavlja 6.4, niska međuocjenjivačka pouzdanost ne ugrožava glavni nalaz (Vanilla naspram RAG/GraphRAG), ali malu razliku RAG–GraphRAG čini dodatno nesigurnom.

6.7. Ograničenja istraživanja

Nalaze iznesene u ovom i prethodnim poglavljima valja tumačiti u svjetlu nekoliko povezanih ograničenja, koja se ovdje objedinjuju.

Veličina i sastav evaluacijskog skupa. Usporedba triju konfiguracija provedena je na skupu od 31 evaluacijskog pitanja. Takav uzorak dovoljan je za otkrivanje velikih i dosljednih razlika, kakva je razlika između Vanilla konfiguracije i dviju konfiguracija s dohvatom, ali je premalen da bi se pouzdano potvrdila ili opovrgnula suptilna razlika između RAG-a i GraphRAG-a,

osobito nakon razdvajanja po kategorijama pitanja, gdje pojedine podskupine broje svega nekoliko primjera. Pri $N = 31$ i promatranim veličinama efekta ($r = 0,01-0,15$) snaga testa za otkrivanje male razlike između RAG-a i GraphRAG-a bila bi niska, pa se takva razlika ne bi mogla pouzdano detektirati ni da postoji. Skup je uz to sastavom nagnut prema jednostavnim činjeničnim upitima (23 od 31 pitanja, 74,2 %), dok složena višedokumentna pitanja, na kojima bi grafovski pristup teorijski trebao doći do izražaja, čine tek 3 od 31 pitanja (9,7 %). Rezultati stoga vrijede prvenstveno za profil upita kakav dominira u ovom skupu i ne mogu se bez ograde poopćiti na scenarije s većim udjelom pitanja koja zahtijevaju sintezu više propisa.

Nisko međuocjenjivačko slaganje. Kako je pokazano u potpoglavlju 6.4, slaganje ljudskih ocjenjivača nisko je na sve tri dimenzije, pri čemu Fleissov κ ostaje nizak i nakon isključivanja ocjenjivača s izrazito visokim udjelom najviših ocjena. Relativno niske vrijednosti mjera slaganja ukazuju na nizak stupanj međusobnog slaganja ocjenjivača pri dodjeljivanju ocjena na trostupanjskoj ordinalnoj skali. Stoga pojedinačne ljudske ocjene treba interpretirati oprezno. Agregacija medijanom deset ocjenjivača po pitanju umanjuje, ali ne uklanja, nesigurnost koja proizlazi iz pojedinačnog neslaganja. Glavni nalaz (Vanilla naspram RAG/GraphRAG) ostaje statistički značajan, no sve ocjene izvedene iz ljudske evaluacije, a time i manja razlika između RAG-a i GraphRAG-a, nose povećanu nesigurnost. Mogući čimbenici koji su mogli pridonijeti niskom stupnju slaganja uključuju trostupanjску ljestvicu ocjenjivanja i suptilnost pravne prosudbe, osobito u slučajevima djelomično točnih, formalno ispravnih ali nepotpunih ili interpretativno ovisnih odgovora.

Plitkoća obilaska grafa u GraphRAG konfiguraciji. Dohvat u GraphRAG konfiguraciji ograničen je na jedan korak po tipiziranim relacijama od početnih čvorova (potpoglavlje 7.5), bez višeskočnog obilaska, reformulacije upita prije dohvata i formalnog ponovnog rangiranja. Zbog toga potencijalno relevantne veze udaljenije od neposrednih susjeda ostaju neiskorištene pa se rezultati za GraphRAG odnose na ovu konkretnu, namjerno konzervativnu izvedbu, a ne na grafovski dohvat općenito.

Navedena ograničenja ne dovode u pitanje glavni zaključak rada o prednosti dohvata konteksta nad izravnim generiranjem, ali jasno omeđuju domet sekundarnog nalaza o izostanku dokaza o razlici između RAG-a i GraphRAG-a i određuju smjer budućih istraživanja. Veći i uravnoteženiji skup pitanja, finija ljestvica ocjenjivanja s jasnijim uputama ocjenjivačima te dublji, višeskočni obilazak grafa uz ponovno rangiranje u nekom budućem istraživanju bi moglo dovesti do drugačijih nalaza i zaključaka.

7. Iterativni razvoj i dijagnostika sustava

Razvoj RAG chatbota za hrvatsko obrambeno pravo prošao je kroz deset iteracija, pri čemu je svaka verzija uvodila specifične konfiguracije s ciljem poboljšanja točnosti i stabilnosti odgovora.

7.1. Evolucija RAG sustava kroz iterativno poboljšavanje

Početna verzija kreirana je uz uporabu jezičnog modela otvorenog koda EuroLLM 9B (devet milijardi parametara) i isti je pokazao osnovnu funkcionalnost, ali s niskom točnošću. Vanilla model postigao je samo 19 % točnosti, dok je RAG s 42 % demonstrirao potencijal pristupa temeljenog na dohvaćanju relevantnog konteksta. Inače, točnost je, kako se ne bi dodatno angažirali ocjenjivači, evaluirana uz pomoć LLM-ova. Treća verzija postigla je neočekivano visokih 90 % točnosti, ali naknadnom analizom se otkrilo kako je evaluacijski skup pitanja (referentni odgovori eksperta) bio uključen u korpus kao Q&A parovi, što je rezultiralo curenjem podataka (engl. *data leakage*) i umjetno visokim rezultatima. Ovaj nalaz naglasio je važnost jasne separacije trenažnih podataka od evaluacijskog skupa te potrebu za pažljivim dizajnom eksperimenata u domeni pravnih chatbotova.

7.2. Specifični tehnički izazovi i njihova rješenja

Najznačajniji tehnički problem kroz sve iteracije bila je višejezična greška curenja (engl. *bleeding*) tj. neželjeno generiranje teksta na bugarskom, srpskom ili ruskom jeziku, često u ćirilicom pismu. Problem je prvi put identificiran u šestoj verziji gdje je u četiri slučaja odgovor završavao na bugarskom jeziku ili u ćirilici, primjerice: года. Этот срок указан в Статье 159. Uzrok je bio u činjenici da je EuroLLM višejezični model treniran na blisko srodnim južnoslavenskim jezicima, što u vektorskom prostoru stvara preklapanje između hrvatskog, srpskog i bugarskog. Pokušaji rješavanja ovog problema kroz prompt engineering, dodavanjem eksplicitnih uputa poput “NIKAD ne koristi ćirilicu, bugarski ili srpski jezik”, pokazali su se neučinkovitima, pa čak i kontraproduktivnima. Deveta verzija s agresivnim prompt upozorenjima rezultirala je paradoksalnim efektom: model je razvio svojevrsnu “samosvijest o uputama” (engl. *meta-awareness*) i počeo citirati vlastite upute u izlazu, npr. УВАГА! Не користите ЋИРИЛИКУ или СРПСКИЕ ЕЗЫКИ, doslovno upozoravajući sam sebe na ukrajinskom jeziku.

7.3. Eksperimentiranje s izborom osnovnog modela: EuroLLM naspram Qwen2.5

Upravo opisani problem višejezičnog curenja, s ograničenom učinkovitošću prompt engineeringa u njegovu suzbijanju, motivirao je preispitivanje samog izbora osnovnog jezičnog modela. Iako je EuroLLM eksplicitno dizajniran za europske jezike, upravo ga je njegova izloženost blisko srodnim južnoslavenskim jezicima i ćirilicom pismu u ranijim inačicama činila

sklonim generiranju teksta na bugarskom, srpskom ili ruskom jeziku.

U prvom koraku pokušano je problem ublažiti nadogradnjom na noviju i veću inačicu istog modela, `EuroLLM-22B-Instruct`, s 22 milijarde parametara i kontekstnim prozorom od 32 000 tokena. Veći model donio je poboljšanje kvalitete i jezične čistoće odgovora u odnosu na inačicu s 9 milijardi parametara, u provedenoj evaluaciji klizanje u ćirilčno pismo više nije zabilježeno. Međutim, model od 22 milijarde parametara, čak i uz četverobitnu kvantizaciju, nalazio se na samoj gornjoj granici memorijskih mogućnosti grafičkog procesora L4 (24 GB) dostupnog u plaćenju inačici platforme Google Colab. To je ostavljalo malo prostora za kontekst dohvaćenih dokumenata i povećavalo rizik od prekoračenja dostupne memorije tijekom evaluacije cijeloga skupa pitanja.

Kao alternativa razmotren je model `Qwen2.5-14B-Instruct`, arhitekturno usporediv dekoderski transformer s jednakim kontekstnim prozorom od 32 000 tokena, no treniran na pretežno globalnom, a ne specifično europskom korpusu. Usporedna evaluacija dvaju modela na istom skupu od 31 evaluacijskog pitanja pokazala je da nijedan od njih ne pruža bezuvjetno bolju kvalitetu, nego da svaki ima specifične slabosti. Ni kod jednoga modela nije zabilježeno klizanje u ćirilicu, čime je najteži oblik problema iz ranijih verzija uklonjen promjenom osnovnoga modela. Time, međutim, jezična čistoća nije postala potpuna, kod modela `Qwen2.5` javljalo se zamjetno leksičko miješanje sa srodnim južnoslavenskim jezicima u latiničnom obliku (primjerice *srbizmi* i iskrivljeni oblici poput “bezbednost”, “okamjenu” ili “potrebuju”), dok je `EuroLLM-22B` u tom pogledu bio leksički čišći uz tek povremena odstupanja (npr. “korišćenje”). Čak se u par odgovora kod `Qwen2.5-14B-Instruct` modela pojavilo kinesko pismo.

S druge strane, `EuroLLM-22B` pokazao je slabost u stabilnosti generiranja, u jednom je odgovoru model upao u repeticijsku petlju i jedanaest puta ponovio istu rečenicu, što je oblik nestabilnosti kakav `Qwen2.5` na istom skupu nije pokazao. Halucinacije u standardnoj (*Vanilla*) konfiguraciji bile su prisutne kod oba modela, ponajprije u obliku izmišljenih brojeva “Narodnih novina”, no RAG konfiguracija ih je kod oba znatno smanjila.

Usporedba na razini tokenizacije otkrila je prednost na strani `EuroLLM`-a koja je relevantna za hrvatski jezik. Za identičnu testnu rečenicu (“S koliko godina časnički namjesnik ide u mirovinu?”) tokenizator modela `Qwen2.5` proizveo je 20 tokena, dok je `EuroLLM` istu rečenicu kodirao u svega 14 tokena, što predstavlja oko 30 % manje. Razlika proizlazi iz činjenice da `Qwen` koristi tokenizaciju na razini bajtova (*byte-level* engl. BPE), zbog čega se hrvatska slova s dijakritičkim znakovima (č, ć, ž, š, đ) rastavljaju na više pod-tokena, dok ih tokenizator modela `EuroLLM`, treniran na korpusu bogatom europskim jezicima, češće zadržava kao cjelovite jedinice. Učinkovitija tokenizacija znači da u isti kontekstni prozor stane više dohvaćenog pravnog teksta te da generiranje odgovora iziskuje manje koraka.

Budući da dva modela nisu pokazala jednoznačnu razliku u kvaliteti odgovora, u tom se trenutku odabir sveo na pitanje resursne izvedivosti: na dostupnom grafičkom procesoru L4 (24 GB) “lakši” `Qwen2.5-14B-Instruct` ostavljao je više memorijskog prostora za stabilan rad i puni kontekst dohvaćenih dokumenata, dok se `EuroLLM-22B` nalazio na granici dostupne memorije i predstavljao rizik za pouzdano izvođenje cijele evaluacije. To je ograničenje, međutim, naknadno prestalo vrijediti prelaskom razvojnog okruženja na grafički procesor NVIDIA A100

s 40 GB memorije (potpoglavlje 4.3), pri čemu je memorijski prostor koji je ranije favorizirao Qwen2.5-14B-Instruct postao dovoljno velik i za EuroLLM-22B. EuroLLM-22B pritom je pokazao bolju tokenizacijsku učinkovitost za hrvatski jezik i leksički čišće odgovore, dok se njegov jedini uočeni nedostatak, povremena repeticijska petlja, mogao ublažiti kalibracijom parametra `repetition_penalty` (potpoglavlje 7.4). Zbog toga je EuroLLM-22B-Instruct zadržan kao konačni osnovni model sustava, dok je Qwen2.5-14B-Instruct ostao dokumentirana, ali naposljetku napuštena alternativa. Sama zamjena identifikatora modela između kandidata tehnički je bila nezahtjevna zahvaljujući standardiziranom sučelju biblioteke *Transformers*. Izmjena se svela na zamjenu identifikatora modela, jer su tokenizacija i predložak razgovora (engl. *chat template*) automatski preuzeti iz konfiguracije svakog modela. Cjelokupna RAG i GraphRAG infrastruktura pritom je ostala nepromijenjena, čime je omogućena metodološki čista usporedba kandidata pod istovjetnim uvjetima.

7.4. Optimizacija generacijskih parametara i njihov utjecaj

Osim lingvističkih problema, iterativno testiranje otkrilo je kritičnu važnost pažljivog balansiranja generacijskih hiperparametara. Deveta verzija je pokušala povećati determinizam snižavanjem temperature s 0.7 na 0.3 kako bi se smanjila varijabilnost odgovora, što je rezultiralo katastrofalnom pogreškom nepostojanja razmaka. Model je tada generirao 500+ znakova bez razmaka (npr. “SustavimenovanjaprinadelnacenikavGlavnomstožeru”). Niska temperatura uzrokovala je da model ulazi u repetitivne uzorke iz kojih se, bez dovoljno stohastičnosti, nije mogao izvući. Istovremeno, parametar `repetition_penalty` morao je biti pažljivo kalibriran: vrijednost 0.6 u šestoj verziji bila je preniska što je dopuštalo ponavljanje rečenica, dok je vrijednost 1.3 u srednjim verzijama, temeljenim na modelu EuroLLM-9B, predstavljala potrebnu korekciju. Prelaskom na novije i veće modele (EuroLLM-22B i Qwen2.5-14B), za koje se očekivalo da budu manje podložni repetitivnim obrascima, ta je vrijednost snižena na 1.1, čime je smanjen rizik da prejaka kazna potisne nužno ponavljanje pravne terminologije (primjerice “članak”, “Oružane snage”). Valja napomenuti da niža vrijednost kazne nije posve uklonila rizik od repeticije. Model EuroLLM-22B u jednom je odgovoru ipak upao u repeticijsku petlju, što pokazuje da podešavanje ovog parametra ublažava, ali ne jamči stabilnost generiranja te da ga treba nadopuniti provjerom naknadne obrade.

Kroz ove iteracije postalo je jasno da uspješan RAG sustav za pravnu domenu zahtijeva multimodalnu obranu od grešaka, kombinaciju prompt engineeringa za visokorazinsko usmjerenje, naknadne obrade za sigurnosnu provjeru te pažljivo izbalansiranih generacijskih parametara koji omogućuju dovoljnu kreativnost za izbjegavanje petlji, ali dovoljnu konzistentnost za precizne pravne odgovore. Također se pokazalo kako pojedinačne metrike poput BERTScore ili ROUGE nisu dovoljan pokazatelj kvalitete, potrebna je kombinacija automatskih metrika (posebno BERTScore koji je pokazao značajnu korelaciju s ručnim ocjenama), LLM-as-a-Judge evaluacije za skalabilnost te ekspertne validacije za referentne podatke u pravnoj domeni.

7.5. Analiza uzroka izostanka očekivanih performansi GraphRAG konfiguracije

Analiza provedene evaluacije pokazuje da se slabiji rezultati GraphRAG-a mogu najizravnije pripisati kvaliteti i rijetkosti same grafovske strukture, što je u skladu s nalazom da za uspješnost nije presudan broj relacija, već gustoća i povezanost grafa [21]. U konstruiranom grafu, koji obuhvaća četiri propisa s ukupno 538 članaka i 1414 stavaka, izvedeno je svega sedam međudokumentnih `ISTA_TEMA` veza (uz prag IDF-težine $\geq 1,5$), dok su sve 123 `UPUCUJE_NA` veze ostale unutar istog zakona, a nijedna nije povezivala različite propise. Uz samo 28 mostovnih entiteta koji povezuju dva zakona, dobivena struktura odgovara upravo onom obrascu rijetkog i fragmentiranog grafa, s niskim udjelom međudokumentnih poveznica, za koji se pokazuje da onemogućuje da mehanizmi dohvaćanja svjesni grafa poboljšaju višeskočno zaključivanje ili očuvaju kontekstualnu koherentnost [21]. Budući da su automatski izvedene veze i prepoznavanje entiteta ovisili o odabranim pragovima i normalizaciji, opisanim u potpoglavlju 4.6.3, konzervativni pragovi propustili su velik dio stvarnih pravnih poveznica, čime je strukturirani obilazak grafa ostao gotovo bez međudokumentnih bridova po kojima bi se mogao kretati.

Drugi skup razloga metodološke je i evaluacijske naravi. GraphRAG početne čvorove vektorski dohvaća istim, neizmijenjenim korisničkim upitom kao i RAG konfiguracija, bez reformulacije ili proširenja upita prije dohвата. Peng i suradnici pritom naglašavaju da tehnike proširenja i dekompozicije upita služe upravo obogaćivanju kratkih i informacijski oskudnih pitanja te se u okviru GraphRAG-a rjeđe primjenjuju [20]. Zbog toga kratka ili nejasno formilirana pitanja jednako loše pogađaju početne čvorove grafa kao i odlomke u RAG-u, pa prednost strukturiranog obilaska izostaje već u prvom koraku. Tome pridonosi i izostanak formalnog ponovnog rangiranja (engl. *re-ranking*), primjerice putem *cross-encoder* modela, koje Peng i suradnici opisuju kao zaseban korak filtriranja manje relevantnih dohvaćenih elemenata [20], kao i plitkoća obilaska ograničenog na jedan korak po tipiziranim relacijama, čime potencijalno relevantne veze udaljenije od neposrednih susjeda ostaju neiskorištene. I sama raspodjela skupa pitanja ograničava priliku da grafovska struktura dođe do izražaja: 23 od 31 pitanja (74,2 %) pripada jednostavnim činjeničnim upitima, a samo 3 (9,7 %) zahtijevaju objedinjavanje odredbi iz više propisa. Budući da osnovni RAG već postiže usporedive ili bolje rezultate na jednostavnim zadacima dohvaćanja činjenica [21], prevlast takvih pitanja u evaluaciji sustavno umanjuje mjerljivu prednost grafovskog pristupa.

8. Zaključak

Cilj ovog rada bio je izraditi i usporediti tri konfiguracije chatbota specifične namjene za odgovaranje na pitanja iz hrvatskog vojnog zakonodavstva: osnovnu konfiguraciju velikog jezičnog modela bez dodatnog dohvata konteksta (*Vanilla*), konfiguraciju proširenu dohvatom relevantnih dijelova dokumenata iz vektorske baze (*RAG*) te konfiguraciju koja se temelji na grafovskoj bazi pravnih dokumenata (*GraphRAG*). Usporedba je pokazala da način na koji se velikom jezičnom modelu pruža kontekst ima značajan utjecaj na kvalitetu odgovora u pravnoj domeni. Rezultate valja promatrati prvenstveno kao empirijsku usporedbu ovih konkretnih konfiguracija u domeni hrvatskog vojnog zakonodavstva, a ne kao opći dokaz superiornosti jedne arhitekture nad drugom.

U promatranom eksperimentalnom skupu najrobustniji nalaz jest da konfiguracije s dohvatom konteksta, *RAG* i *GraphRAG*, ostvaruju značajno bolje rezultate od *Vanilla* konfiguracije. Obje konfiguracije koje koriste dohvat informacija, *RAG* i *GraphRAG*, ostvarile su bolje rezultate od *Vanilla* konfiguracije na promatranim automatskim i subjektivnim mjerama evaluacije. Time se potvrđuje praktična vrijednost pristupa u kojem se generiranje odgovora ne prepušta isključivo znanju pohranjenom u parametrima modela, nego se modelu prije generiranja pružaju informacije iz domenski relevantne baze znanja.

Rezultati pritom nisu pokazali statistički značajnu prednost *GraphRAG* konfiguracije u odnosu na klasični *RAG*. Suprotno očekivanju da bi grafovska reprezentacija pravnih odnosa mogla donijeti dodatnu korist, u provedenom eksperimentu nije pronađen statistički značajan dokaz prednosti *GraphRAG*-a nad klasičnim *RAG* pristupom. Taj rezultat ne treba tumačiti kao dokaz da *GraphRAG* općenito nije učinkovitiji pristup, već kao nalaz ograničen na korištenu implementaciju, skup podataka, konfiguraciju sustava i evaluacijska pitanja. Posebno je važno istaknuti da je većina pitanja bila jednostavne činjenične prirode, dok je samo mali dio zahtijevao povezivanje informacija iz više propisa.

Vrijednost rada nije isključivo u konačnoj usporedbi triju konfiguracija, nego i u dokumentiranju procesa njihova razvoja. Analiza iterativnog razvoja pokazala je da se tehnička implementacija ne može promatrati odvojeno od ponašanja osnovnog modela, kvalitete korpusa, načina strukturiranja prompta i odabira parametara generiranja. Takav pristup omogućio je identifikaciju problema koji ne bi nužno bili vidljivi u pojedinačnim testovima, od metodoloških pogrešaka poput curenja podataka, preko lingvističkih izazova višejezičnog modela i odabira primjerenijeg osnovnog modela, do utjecaja hiperparametara na čitljivost i konzistentnost generiranih odgovora.

Jedna od važnijih spoznaja rada jest da *prompt engineering* sam po sebi nije dovoljno pouzdan mehanizam za kontrolu ponašanja modela. Eksplicitne tekstualne zabrane ne moraju uvijek ukloniti neželjeno ponašanje, a u određenim slučajevima mogu ga čak dodatno naglasiti stvaranjem svojevrstne “samosvijesti o uputama” unutar generiranog odgovora. Promjena osnovnog modela uklonila je najizraženiji oblik problema, pojavu ćirilice, ali nije potpuno uklonila suptilnije oblike jezičnog miješanja. Izbor osnovnog modela stoga može značajno poboljšati ponašanje sustava, ali ne može u potpunosti zamijeniti tehničke mehanizme kontrole i

validacije.

Iterativni razvoj otkrio je i određena ograničenja konačne implementacije, koja nije obuhvatila sve korekcije i zaštitne mehanizme identificirane tijekom razvoja. Pojedini problemi, poput nepravilnosti u formatiranju, izostanka razmaka između dijelova generiranog teksta ili preostalog leksičkog miješanja jezika i dalje predstavljaju potencijalni rizik pri primjeni sustava izvan kontroliranih eksperimentalnih uvjeta. Iako konačna konfiguracija postiže zadovoljavajuće rezultate tijekom evaluacije, produkcijska bi primjena zahtijevala dodatne mehanizme validacije ulaznih i izlaznih podataka.

Rezultati rada također ukazuju na važnost višedimenzionalne evaluacije velikih jezičnih modela. Automatske metrike omogućuju objektivnu i ponovljivu usporedbu odgovora, ali same po sebi nisu dovoljne za razumijevanje njihove stvarne korisnosti. Ljudska evaluacija omogućuje procjenu točnosti, korisnosti i pojave halucinacija, dok pristup *LLM-as-a-Judge* pruža dodatno skalabilan način evaluacije. Kombiniranje tih pristupa pokazalo se korisnim jer pojedina metoda ne opisuje u potpunosti sve dimenzije kvalitete generiranog odgovora, pri čemu je u pravnoj domeni posebno važno razlikovati semantičku sličnost odgovora od njegove stvarne pravne i činjenične točnosti. Iz svega navedenog proizlazi da izgradnja pouzdanog sustava u pravnoj domeni zahtijeva integrirani pristup koji kombinira kvalitetno strukturiran korpus, odgovarajući mehanizam dohvata, pažljivo oblikovane sistemske upute te kontrolu parametara generiranja, uz tehničke postupke validacije i naknadne obrade. Za kritične zahtjeve, naime, prompt engineering nije dostatan bez tehničkih zaštita neovisnih o sposobnosti modela da slijedi tekstualnu uputu.

Ovaj rad otvara nekoliko smjerova budućeg razvoja. Najizravniji smjer daljnjeg razvoja jest unapređenje GraphRAG pristupa kroz preciznije modeliranje semantičkih odnosa među pravnim aktima, člancima i stavcima te razvoj naprednijih strategija obilaska grafa ovisno o vrsti korisničkog pitanja. Pritom bi trebalo zasebno ispitati razlikuje li se prednost RAG-a i GraphRAG-a ovisno o vrsti pitanja: od jednostavnih činjeničnih pitanja, preko pitanja koja povezuju više članaka, do pitanja koja povezuju više različitih dokumenata. Srodan smjer razvoja predstavlja automatska klasifikacija korisničkih pitanja prije dohvata, kojom bi se strategija pretraživanja prilagodila vrsti pitanja. Za jednostavna činjenična pitanja mogao bi se koristiti semantički dohvata, dok bi se za pitanja koja povezuju više odredbi primjenjivao grafovski dohvata. Time bi se oblikovao hibridni sustav koji dinamički odabire odgovarajuću strategiju dohvata, umjesto primjene jedinstvenog pristupa na sve vrste pitanja.

Dodatno područje čini razvoj pouzdanijih mehanizama citiranja izvora, uz automatsku provjeru odgovora li sadržaj odgovora doista dohvaćenom kontekstu, čime bi se dodatno smanjio rizik od halucinacija i povećala transparentnost sustava. Naposljetku, budući da su rezultati dobiveni na jednoj specifičnoj domeni, ograničenom korpusu i određenom skupu evaluacijskih pitanja, buduća bi istraživanja trebala uključiti veće i raznovrsnije skupove propisa, veći broj pitanja i korisnika te usporedbu različitih osnovnih modela i strategija dohvata.

Zaključno, rad pokazuje da najveći praktični doprinos kvaliteti specijaliziranog pravnog chatbota ne proizlazi nužno iz povećanja strukturne složenosti sustava, nego iz pouzdanog povezivanja generativnog modela s relevantnim vanjskim znanjem i pažljivog inženjerskog obli-

kovanja cijelog procesa. RAG i GraphRAG pristupi pokazali su jasnu prednost u odnosu na osnovni model bez dohvata konteksta, dok GraphRAG nije ostvario mjerljivu dodatnu prednost nad klasičnim RAG pristupom. Taj je rezultat važan jer pokazuje da složeniji sustav nije nužno i bolji te da se dodatna arhitekturna složenost mora opravdati konkretnim poboljšanjem kvalitete, pouzdanosti ili funkcionalnosti. Pouzdanost generativne umjetne inteligencije u pravnoj domeni stoga ne ovisi o jednoj tehnologiji ili metodi, već o kvaliteti cijelog sustava, povezivanju odgovarajućeg modela, kvalitetno pripremljenog izvora znanja, učinkovite strategije dohvata, kontroliranog generiranja i stroge evaluacije.

Popis literature

- [1] J. Naisbitt, *Megatrends: 10 new directions transforming our lives*. Warner Books, 1982., str. 17.
- [2] M. Al-Amin, M. S. Ali, A. Salam i dr., „History of generative Artificial Intelligence (AI) chatbots: past, present, and future development,” *arXiv preprint arXiv:2402.05122*, 2024.
- [3] A. M. Turing, „Computing Machinery and Intelligence,” *Mind*, sv. 59, br. 236, str. 433–460, 1950.
- [4] R. Lane, A. Hay, A. Schwarz, D. M. Berry i J. Shrager, „ELIZA Reanimated: The world’s first chatbot restored on the world’s first time sharing system,” *arXiv preprint arXiv:2501.06707*, 2025.
- [5] D. F. Murad, A. G. Iskandar, E. Fernando, T. S. Octavia i D. E. Maured, „Towards smart LMS to improve learning outcomes students using LenoBot with natural language processing,” *2019 6th International Conference on Information Technology, Computer and Electrical Engineering (ICITACEE)*, IEEE, 2019., str. 1–6.
- [6] E. Adamopoulou i L. Moussiades, „An Overview of Chatbot Technology,” *Artificial Intelligence Applications and Innovations (AIAI 2020)*, serija IFIP Advances in Information and Communication Technology, sv. 584, Springer Nature Switzerland AG, 2020., str. 373–383. DOI: [10.1007/978-3-030-49186-4_31](https://doi.org/10.1007/978-3-030-49186-4_31).
- [7] R. S. Wallace, „Artificial Intelligence Markup Language (AIML) Version 1.0.1,” A.L.I.C.E. AI Foundation, teh. izv., 2001.
- [8] P. Muangkammuen i N. Intiruk, „Automated Thai-FAQ Chatbot using RNN-LSTM,” *2019 5th International Conference on Engineering, Applied Sciences and Technology (ICEAST)*, IEEE, 2019., str. 1–4. DOI: [10.1109/ICEAST.2019.8712781](https://doi.org/10.1109/ICEAST.2019.8712781).
- [9] A. Vaswani, N. Shazeer, N. Parmar i dr., „Attention is all you need,” *Advances in neural information processing systems*, sv. 30, 2017.
- [10] H. Naveed, A. U. Khan, S. Qiu i dr., „A comprehensive overview of large language models,” *ACM Transactions on Intelligent Systems and Technology*, sv. 16, br. 5, str. 1–72, 2025.
- [11] C. Staff, *What is an AI Engineer? (And how to become one)*, <https://www.coursera.org/articles/ai-engineer>, [Pristupljeno 13.01.2026], 2025.

- [12] C. Zhou, Q. Li, C. Li i dr., „A comprehensive survey on pretrained foundation models: A history from bert to chatgpt,” *International Journal of Machine Learning and Cybernetics*, sv. 16, br. 12, str. 9851–9915, 2025.
- [13] C. Huyen, *AI Engineering: Building Applications with Foundation Models*. Sebastopol, CA: O'Reilly Media, 2025., ISBN: 978-1-098-16630-4.
- [14] L. Ouyang, J. Wu, X. Jiang i dr., „Training Language Models to Follow Instructions with Human Feedback,” *arXiv preprint arXiv:2203.02155*, 2022.
- [15] E. Saravia, *Prompt Engineering Guide*, 12. 2022.
- [16] T. B. Brown, B. Mann, N. Ryder i dr., „Language Models are Few-Shot Learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [17] B. Han, T. Susnjak i A. Mathrani, „Automating Systematic Literature Reviews with Retrieval-Augmented Generation: A Comprehensive Overview,” *Applied Sciences*, sv. 14, br. 19, 2024., ISSN: 2076-3417. DOI: [10.3390/app14199103](https://doi.org/10.3390/app14199103).
- [18] Y. A. Malkov i D. A. Yashunin, „Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, sv. 42, br. 4, str. 824–836, 2020. DOI: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [19] Y. Gao, Y. Xiong, X. Gao i dr., „Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [20] B. Peng, Y. Zhu, Y. Liu i dr., „Graph retrieval-augmented generation: A survey,” *ACM Transactions on Information Systems*, sv. 44, br. 2, str. 1–52, 2025.
- [21] Z. Xiang, C. Wu, Q. Zhang i dr., „When to use graphs in rag: A comprehensive analysis for graph retrieval-augmented generation,” *International Conference on Learning Representations*, sv. 2026, 2026., str. 66 145–66 178.
- [22] T. Dettmers, A. Pagnoni, A. Holtzman i L. Zettlemoyer, „Qlora: Efficient finetuning of quantized llms,” *Advances in Neural Information Processing Systems*, sv. 36, str. 10 088–10 115, 2023.
- [23] J. Nielsen, *Usability Engineering*. Academic Press, 1993.
- [24] C.-Y. Lin, „ROUGE: A Package for Automatic Evaluation of Summaries,” *Proceedings of the ACL Workshop on Text Summarization Branches Out*, 2004., str. 74–81.
- [25] K. Papineni, S. Roukos, T. Ward i W.-J. Zhu, „BLEU: a Method for Automatic Evaluation of Machine Translation,” *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002., str. 311–318.
- [26] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger i Y. Artzi, „BERTScore: Evaluating Text Generation with BERT,” *International Conference on Learning Representations*, 2020.
- [27] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder i F. Wei, „Multilingual E5 Text Embeddings: A Technical Report,” *arXiv preprint arXiv:2402.05672*, 2024.
- [28] J. L. Fleiss, „Measuring Nominal Scale Agreement Among Many Raters,” *Psychological Bulletin*, sv. 76, br. 5, str. 378–382, 1971.

- [29] J. Cohen, „A Coefficient of Agreement for Nominal Scales,” *Educational and Psychological Measurement*, sv. 20, br. 1, str. 37–46, 1960.
- [30] J. R. Landis i G. G. Koch, „The Measurement of Observer Agreement for Categorical Data,” *Biometrics*, sv. 33, br. 1, str. 159–174, 1977.
- [31] K. Krippendorff, „Computing Krippendorff’s Alpha-Reliability,” Annenberg School for Communication, University of Pennsylvania, teh. izv., 2011.
- [32] K. H. Cheng, C. H. Hui i W. F. Cascio, „Leniency bias in performance ratings: The big-five correlates,” *Frontiers in psychology*, sv. 8, str. 521, 2017.
- [33] L. Zheng, W.-L. Chiang, Y. Sheng i dr., „Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems*, 2023.
- [34] P. Wang, L. Li, L. Chen i dr., „Large Language Models are not Fair Evaluators,” *arXiv preprint arXiv:2305.17926*, 2023.

Popis slika

1.	Originalni ispis koda ELIZA pronađen u Weizenbaumovim arhivama [5]	5
2.	Koder - dekode [9]	13
3.	Tokenizacija hrvatskog teksta prema trima generacijama OpenAI modela.	15
4.	Dijagram toka različitih faza razvoja LLM-a od pred-treniranja do korištenja [10]	18
5.	Iskorištenost sistemske i GPU memorije te diska tijekom triju uzastopnih sesija izvođenja u okruženju Google Colab.	34
6.	Prikaz vektoriziranih riječi u dvije dimenzije t-SNE metodom.	39
7.	Schema grafa pravnih dokumenata u GraphRAG konfiguraciji	44
8.	Procesni lanac pripreme podataka za RAG i GraphRAG konfiguracije chatbota.	49
9.	Obradni tijek korisničkog upita za Vanilla, RAG i GraphRAG konfiguraciju chatbota.	53
10.	Distribucija ljudskih ocjena po konfiguraciji i dimenziji.	64
11.	Usporedba triju konfiguracija chatbota po prosječnim vrijednostima triju automatskih metrika.	65
12.	Distribucija latencije odgovora po konfiguraciji.	68

Popis tablica

1.	Dimenzije ljudske evaluacije i primarni ocjenjivači	58
2.	Usporedba evaluacijskih metoda	61
3.	Deskriptivna statistika ljudskih ocjena po konfiguraciji	63
4.	Razlike među konfiguracijama	65
5.	Spearmanova korelacija (ρ) automatskih metrika i LLM ocjenjivača s dimenzi- jama ljudske evaluacije	66
6.	Međuocjenjivačka pouzdanost deset ocjenjivača	67
7.	Statistika latencije odgovora (u sekundama)	68

Prilozi

1. Prilog - Popis pitanja

U nastavku je naveden potpuni skup od 31 evaluacijskog pitanja korištenog u ovom radu, s pripadajućim referentnim odgovorom eksperta i izvorom u propisima, razvrstan po kategoriji pitanja opisanoj u potpoglavlju 4.1.

Kompleksna pitanja (više dokumenata)

Pitanje 1. Opišite cjelokupnu proceduru transformacije novaka u razvrstanog pričuvnika kroz sve pravne akte koji reguliraju taj proces.

Odgovor: Procedura je regulirana kroz 4 pravna akta. 1. USTAVNI TEMELJ (Ustav RH, čl. 47). Vojna obveza je dužnost svih za to sposobnih državljana. Dopušten prigovor savjesti iz vjerskih i/ili moralnih razloga uz obvezu ispunjavanja drugih dužnosti određenih zakonom. 2. NOVAČKA OBVEZA (Zakon o obrani, čl. 19-21). Nastaje u kalendarskoj godini kad državljanin RH navrší 18 godina. Državljanin RH uvodi se u vojnu evidenciju u kalendarskoj godini u kojoj navrší 18 godina života. Prolazi zdravstveni pregled i psihologijsko ispitivanje. 3. TEMELJNO VOJNO OSPOSOBLJAVANJE (Zakon o obrani čl. 21.m, Pravilnik o temeljnom vojnom osposobljavanju). Traje 2 mjeseca. Ministar obrane donosi Plan upućivanja u tekućoj godini do 15.12. za iduću godinu. Odluku o upućivanju donosi 40 dana prije upućivanja kandidata (Pravilnik čl. 7). Ročnici se obučavaju za specijalnost 11A - strijelac (Pravilnik čl. 9). Daju svečanu prisegu. 4. STATUS ROČNIKA (Zakon o službi, čl. 25, 26, 30). Ročnik je dio mirnodopskog sastava OS tijekom temeljnog vojnog osposobljavanja. Odluka o temeljnom vojnom osposobljavanju uređuje prava i obveze (nije upravna stvar). 5. PRIJELAZ U PRIČUVU (Zakon o obrani, čl. 25, 27). Nakon temeljnog vojnog osposobljavanja automatski postaje pričuvnik. Razvrstani je pričuvnik jer ima određenu vojnostručnu specijalnost. Određuje mu se ratni raspored. Može biti pozivan na vježbe do 90 dana godišnje.

Izvor: Ustav RH, čl. 47 (vojna obveza, prigovor savjesti); Zakon o obrani, čl. 19-21 (novačka obveza, vojna evidencija), čl. 21.m (temeljno vojno osposobljavanje), čl. 25, 27 (prijelaz u pričuvu, ratni raspored); Pravilnik o temeljnom vojnom osposobljavanju, čl. 7, 9; Zakon o službi u Oružanim snagama RH, čl. 25, 26, 30 (status ročnika).

Pitanje 2. Kako je regulirana hijerarhija odlučivanja o uporabi Oružanih snaga u različitim situacijama kroz Ustav i Zakon o obrani?

Odgovor: Hijerarhija odlučivanja detaljno razlikuje situacije. 1. RATNO STANJE (Ustav čl. 100, Zakon čl. 47). Predsjednik Republike naređuje uporabu i zapovijeda. Ima punu ovlast i odgovoran je za obranu. 2. STANJE NEPOSREDNE UGROŽENOSTI (Ustav čl. 100, Zakon čl. 48). Predsjednik Republike može narediti uporabu. Potreban je supotpis predsjednika Vlade. 3. PRELAZAK GRANICE (Ustav čl. 7, Zakon čl. 49). Hrvatski sabor donosi odluku (većina svih zastupnika). Vlada predlaže. Predsjednik daje prethodnu suglasnost. Ako Predsjednik uskrati suglasnost, potrebna je dvotrećinska većina. 4. POMOĆ SAVEZNICIMA (Ustav čl. 7,

Zakon čl. 51). Sabor donosi odluku. Vlada predlaže. Predsjednik daje suglasnost. 5. VJEŽBE I OBUKA (Ustav čl. 7, Zakon čl. 69). Vlada donosi odluku. Predsjednik daje suglasnost. Za manje postrojbe (do razine voda) odluku donosi ministar obrane. 6. PROVEDBA (Zakon čl. 70). Načelnik Glavnog stožera ima ovlast punog zapovijedanja. Odgovoran je Predsjedniku za zapovijedi i naredbe, a ministru za provedbu odluka.

Izvor: Ustav RH, čl. 7 (uporaba OS izvan granica, suglasnost), čl. 100 (ratno stanje, neposredna ugroženost); Zakon o obrani, čl. 46-51 (uporaba OS, ratno stanje, prelazak granice, pomoć saveznicima), čl. 69 (vježbe i obuka), čl. 70 (provedba, ovlasti načelnika Glavnog stožera).

Pitanje 3. Objasnite sve oblike obveza državljana u obrani prema Zakonu o obrani i Ustavu.

Odgovor: Državljeni RH imaju 6 kategorija obveza u obrani. 1. USTAVNI TEMELJ (Ustav RH, čl. 47). Vojna obveza i obrana RH dužnost je svih za to sposobnih državljana. Dopušten je prigovor savjesti. 2. VOJNA OBVEZA (Zakon o obrani, čl. 19-21). Nastaje u godini kada državljanin navrší 18 godina. Prestaje s 55 godina za muškarce i 50 godina za žene. Sastoji se od novačke obveze, temeljnog vojnog osposobljavanja ili civilne službe te služenja u pričuví. 3. SLUŽENJE U UGOVORNOJ PRIČUVI (Zakon o obrani, čl. 27, 28). Obuhvaća dragovoljno / temeljno vojno osposobljavanje. Osposobljavanje traje do 90 dana godišnje. 4. RADNA OBVEZA (Zakon o obrani, čl. 31). Odnosi se na pravne osobe i državljane RH. Uvodi se u ratnom stanju i stanju neposredne ugroženosti. 5. RADNA POMOĆ (Zakon o obrani, čl. 31-32). Može se uvesti u ratnom stanju i stanju neposredne ugroženosti. Služi za izvršenje zadaća nositelja obrambenih funkcija. 6. MATERIJALNA OBVEZA (Zakon o obrani, čl. 33). Obuhvaća ustupanje objekata i materijalnih sredstava. Za obrambene zadaće i osposobljavanje.

Izvor: Ustav RH, čl. 47 (vojna obveza, prigovor savjesti); Zakon o obrani, čl. 17-35, posebno čl. 19-21 (vojna obveza, dob nastanka i prestanka, sastavnice), čl. 27, 28 (služenje u ugovornoj pričuví), čl. 31 (radna obveza), čl. 31-32 (radna pomoć), čl. 33 (materijalna obveza).

Kompleksna pitanja (jedan dokument)

Pitanje 4. Objasnite sustav imenovanja i odgovornosti načelnika Glavnog stožera kroz sve relevantne propise.

Odgovor: Položaj načelnika Glavnog stožera OS RH reguliran je kroz pet razina. Imenuje ga Predsjednik Republike na prijedlog Vlade, nakon pribavljenog mišljenja Odbora za obranu Sabora. Mandat traje četiri godine bez mogućnosti ponovnog imenovanja. Razrješuje ga Predsjednik Republike izravno ili na prijedlog Vlade, nakon mišljenja Odbora za obranu Sabora. Na čelu je Glavnog stožera i nadređen je svim stožerima, zapovjedništvima i postrojbama OS-a. Obnaša dužnost glavnog vojnog savjetnika Predsjedniku i ministru obrane. Ima ovlast punog zapovijedanja u OS-u. Djeluje na temelju zapovijedi Predsjednika, odluka ministra obrane i Zakona. Predsjedniku je odgovoran za provedbu zapovijedi i naredbi, a ministru

obrane za provedbu odluka. Obavezan je istodobno izvještavati oba o uporabi Oružanih snaga.

Izvor: Zakon o obrani, čl. 7, 15, 70.

Pitanje 5. Objasnite razlike između uporabe i korištenja Oružanih snaga prema Zakonu o obrani, uključujući tko donosi odluke u svakom slučaju.

Odgovor: Zakon o obrani jasno razlikuje UPORABU i KORIŠTENJE OS. UPORABA (čl. 46-57) odnosi se na borbeno djelovanje. U ratnom stanju Predsjednik naređuje uporabu. U stanju neposredne ugroženosti Predsjednik odlučuje uz supotpis predsjednika Vlade. Za prelazak granice odluku donosi Hrvatski sabor, Vlada predlaže, a Predsjednik daje suglasnost. Za pomoć saveznicima odlučuju Sabor, Vlada i Predsjednik. Za operacije u inozemstvu odlučuju Sabor, Vlada i Predsjednik. KORIŠTENJE (čl. 58-64) odnosi se na neborbeno djelovanje. Kod katastrofa Vlada donosi odluku. Kod velikih nesreća odlučuje ministar obrane. Kod protupožarne zaštite odlučuje ministar obrane. Kod traganja i spašavanja odlučuje ministar obrane. Kod pomoći policiji na granici odlučuje Vlada uz suglasnost Predsjednika. KLJUČNE RAZLIKE: uporaba je borbeno, izvan RH, o njoj odlučuju Sabor i Predsjednik; korištenje je neborbeno, unutar RH, o njemu odlučuju Vlada i ministar.

Izvor: Zakon o obrani, čl. 46-57 (uporaba OS: ratno stanje, neposredna ugroženost, prelazak granice, pomoć saveznicima, operacije u inozemstvu), čl. 58-64 (korištenje OS: katastrofe, velike nesreće, protupožarna zaštita, traganje i spašavanje, pomoć policiji na granici).

Pitanje 6. Koja su prava i obveze ročnika tijekom temeljnog vojnog osposobljavanja prema Pravilniku o temeljnom vojnom osposobljavanju?

Odgovor: OBVEZE ROČNIKA. Življenje i rad u skladu s propisima o službi u OS. Osposobljavanje za specijalnost 11A (strijelac). Svečana prisega pred ministrom obrane. Pridržavanje vojnih pravila i discipline. PRAVA ROČNIKA. 1. FINACIJSKA. Plaća ili naknada prema Zakonu o službi. Naknada prijevoza od i do prebivališta. 2. ZDRAVSTVENA ZAŠTITA. Osnovno zdravstveno osiguranje. Dopunsko zdravstveno osiguranje. Osiguranje od ozljede na radu. Osiguranje od nesretnog slučaja. 3. MATERIJALNA. Smještaj. Prehrana. Vojna odora. Sportska oprema. 4. PRIZNANJA. Značka najboljeg ročnika naraštaja. Pravo izbora mjesta službe pri prijmu u djelatnu službu (za najboljeg ročnika).

Izvor: Pravilnik o temeljnom vojnom osposobljavanju čl. 9 (specijalnost 11A - strijelac), čl. 10, 11 (obveze ročnika, prisega), čl. 15, 16 (financijska prava, naknada prijevoza), čl. 17 (zdravstvena zaštita i osiguranja), čl. 18 (materijalna prava i priznanja).

Pitanje 7. Objasnite razlike između mirnodopskog i ratnog sastava Oružanih snaga prema Zakonu o službi.

Odgovor: MIRNODOPSKI SASTAV. Čine ga djelatne vojne osobe, službenici i namještenici, pričuvnici pozvani na službu, ugovorni pričuvnici, kadeti te ročnici na temeljnom vojnom

osposobljavanju. Svrha mu je redovno funkcioniranje OS. Veličina je određena mirnodopskim ustrojem. RATNI SASTAV. Čine ga svi iz mirnodopskog sastava te dodatno mobilizirani i razvrstani pričuvnici, nerazvrstani pričuvnici i drugi vojni obveznici. Svrha mu je borbeno djelovanje. Veličina je određena ratnim ustrojem i znatno je veća. PRIJELAZ. Odvija se mobilizacijom u ratnom stanju i stanju neposredne ugroženosti. Odluku donosi Predsjednik Republike. Mobilizacija može biti opća ili djelomična. Mobilizacijom OS prelaze iz mirnodopskog u ratno ustrojstvo.

Izvor: Zakon o službi u Oružanim snagama RH, čl. 25 (mirnodopski i ratni sastav OS, sastavnice svakog, prijelaz mobilizacijom, odluka Predsjednika Republike, opća i djelomična mobilizacija).

Pitanje 8. Objasnite postupak i uvjete za odgodu temeljnog vojnog osposobljavanja prema Zakonu o obrani.

Odgovor: POSTUPAK. Zahtjev za odgodu podnosi se nadležnom područnom odjelu za poslove obrane u roku od osam dana od dana primitka poziva na temeljno vojno osposobljavanje. Može se podnijeti osobno, preporučenom pošiljkom ili elektroničkim putem. Uz zahtjev se prilažu odgovarajući dokazi (npr. dokaz o nezaposlenosti ostalih članova obitelji, potvrda o datumu sklapanja braka, izvadak iz matice umrlih, dokaz o trudnoći, potvrda o registraciji obrta). RAZLOZI ZA ODGODU. 1. ŠKOLOVANJE. Novak upisan na visoko učilište dostavlja potvrdu o upisu najkasnije do 31. listopada tekuće godine, a odgoda traje najdulje do 30. rujna godine u kojoj navršava 29 godina. 2. OSOBN I OBITELJSKI RAZLOZI. Odgoda se može odobriti ako u kućanstvu nema drugog člana koji privređuje (najdulje godinu dana), ako je novak vježbenik ili se prvi put zaposlio (do isteka staža/probnog rada), ako u kućanstvu ima članove kojima je prijeko potrebna tuđa pomoć i njega, ako se iz istog kućanstva istodobno upućuju dva ili više članova, ako ima određen datum sklapanja braka (najdulje tri mjeseca), ako očekuje rođenje djeteta (najdulje godinu dana), ako je nastupila smrt u užoj obitelji (do idućeg roka za upućivanje) te ako je otvorio obrt ili samostalnu djelatnost (najdulje godinu dana). 3. SPORTAŠI. Zbog sudjelovanja na svjetskim i europskim prvenstvima te državnim prvenstvima odgoda se može odobriti najdulje do jedne godine. ODLUČIVANJE. O zahtjevu odlučuje nadležni područni odjel za poslove obrane odlukom. Kod odgode zbog školovanja i sporta (čl. 21.k) odluka mora sadržavati elemente rješenja prema Zakonu o općem upravnom postupku; protiv nje nije dopuštena žalba, ali se može pokrenuti upravni spor pred Visokim upravnim sudom RH, koji odlučuje u roku od 90 dana. Odluka o odgodi sadrži dan, mjesec i godinu do kada se osposobljavanje odgađa.

Izvor: Zakon o obrani, čl. 21.k (odgoda zbog školovanja i sporta, rokovi, odlučivanje odlukom, isključenje žalbe, upravni spor pred Visokim upravnim sudom), čl. 21.l (osobni i obiteljski razlozi za odgodu, rok od osam dana za podnošenje zahtjeva, način dostave, popis dokaza, sadržaj odluke o odgodi).

Jednostavna pitanja

Pitanje 9. Kada nastaje vojna obveza za državljane Republike Hrvatske?

Odgovor: Vojna obveza nastaje u kalendarskoj godini u kojoj državljanin Republike Hrvatske navršava 18 godina života.

Izvor: Zakon o obrani, Članak 19, stavak 2.

Pitanje 10. Koliko traje temeljno vojno osposobljavanje?

Odgovor: Temeljno vojno osposobljavanje traje dva mjeseca.

Izvor: Zakon o obrani, Članak 21.m, stavak 2.

Pitanje 11. Tko je vrhovni zapovjednik Oružanih snaga Republike Hrvatske?

Odgovor: Predsjednik Republike Hrvatske je vrhovni zapovjednik Oružanih snaga Republike Hrvatske.

Izvor: Ustav Republike Hrvatske, Članak 100.

Pitanje 12. Koje su tri grane Oružanih snaga Republike Hrvatske?

Odgovor: Tri grane Oružanih snaga Republike Hrvatske su: Hrvatska kopnena vojska, Hrvatska ratna mornarica i Hrvatsko ratno zrakoplovstvo.

Izvor: Zakon o obrani, Članak 41, stavak 3.

Pitanje 13. Kada prestaje vojna obveza za muškarce?

Odgovor: Vojna obveza za muškarce prestaje na kraju kalendarske godine u kojoj navršavaju 55 godina života.

Izvor: Zakon o obrani, Članak 20, stavak 1.

Pitanje 14. Tko imenuje načelnika Glavnog stožera Oružanih snaga?

Odgovor: Načelnika Glavnog stožera Oružanih snaga imenuje Predsjednik Republike Hrvatske na prijedlog Vlade Republike Hrvatske i nakon pribavljenog mišljenja Odbora za obranu Hrvatskoga sabora.

Izvor: Zakon o obrani, Članak 7, točka 8.

Pitanje 15. Koliko traje mandat načelnika Glavnog stožera?

Odgovor: Načelnik Glavnog stožera imenuje se na razdoblje od četiri godine i bez mogućnosti reizbora.

Izvor: Zakon o obrani, Članak 15, stavak 4.

Pitanje 16. Što je temeljna svrha obrane Republike Hrvatske?

Odgovor: Temeljna svrha obrane je očuvanje suvereniteta, neovisnosti i teritorijalne cjelovitosti Republike Hrvatske.

Izvor: Zakon o obrani, Članak 2, stavak 1.

Pitanje 17. Do koje dobi novak može biti upućen na temeljno vojno osposobljavanje?

Odgovor: Novak može biti upućen na temeljno vojno osposobljavanje do isteka 30 godina života.

Izvor: Zakon o obrani, Članak 21.lj, stavak 3.

Pitanje 18. Tko donosi odluku o mobilizaciji Oružanih snaga?

Odgovor: Odluku o mobilizaciji Oružanih snaga donosi Predsjednik Republike na prijedlog ministra obrane.

Izvor: Zakon o obrani, Članak 7, točka 6.

Pitanje 19. Kada prestaje vojna obveza za žene?

Odgovor: Vojna obveza za žene prestaje na kraju kalendarske godine u kojoj navršavaju 50 godina života.

Izvor: Zakon o obrani, Članak 20, stavak 1.

Pitanje 20. U kojoj godini života se državljanin RH uvodi u vojnu evidenciju?

Odgovor: Državljanin Republike Hrvatske uvodi se u vojnu evidenciju u kalendarskoj godini kada navrši 18 godina života.

Izvor: Zakon o obrani, Članak 21.b, stavak 1.

Pitanje 21. Tko proglašava ratno stanje u Republici Hrvatskoj?

Odgovor: Ratno stanje u Republici Hrvatskoj proglašava Hrvatski sabor.

Izvor: Zakon o obrani, Članak 5, stavak 1.

Pitanje 22. Koliko dana prije upućivanja ministar obrane donosi odluku o upućivanju kandidata na temeljno vojno osposobljavanje?

Odgovor: Ministar obrane donosi odluku o upućivanju kandidata na temeljno vojno osposobljavanje u pravilu 40 dana prije upućivanja kandidata.

Izvor: Pravilnik o temeljnom vojnom osposobljavanju, Članak 7, stavak 2.

Pitanje 23. Tko donosi program obuke ročnika na temeljnom vojnom osposobljavanju?

Odgovor: Program obuke ročnika na temeljnom vojnom osposobljavanju donosi načelnik Glavnog stožera Oružanih snaga Republike Hrvatske.

Izvor: Pravilnik o temeljnom vojnom osposobljavanju, Članak 9, stavak 2.

Pitanje 24. Pred kim ročnici daju svečanu prisegu?

Odgovor: Ročnici daju svečanu prisegu pred ministrom obrane ili osobom koju on za to odredi.

Izvor: Pravilnik o temeljnom vojnom osposobljavanju, Članak 10, stavak 1.

Pitanje 25. Za koju vojnostručnu specijalnost se ročnici osposobljavaju na temeljnom vojnom osposobljavanju?

Odgovor: Ročnici se osposobljavaju u temeljnim vojnim znanjima i sposobnostima u rodu pješastva za streljačku specijalnost 11A, dužnost strijelca.

Izvor: Pravilnik o temeljnom vojnom osposobljavanju, Članak 9, stavak 1.

Pitanje 26. Što čini mirnodopski sastav Oružanih snaga?

Odgovor: Mirnodopski sastav Oružanih snaga čine: djelatne vojne osobe, službenici i namještenici, pričuvnici pozvani na službu u Oružane snage, ugovorni pričuvnici, kadeti te ročnici koji su pristupili temeljnom vojnom osposobljavanju.

Izvor: Zakon o službi u Oružanim snagama RH, Članak 25, stavak 2.

Pitanje 27. Tko utvrđuje mirnodopski i ratni ustroj Oružanih snaga?

Odgovor: Predsjednik Republike utvrđuje mirnodopski i ratni ustroj Oružanih snaga te strukturu činova u mirnodopskom i ratnom ustroju Oružanih snaga.

Izvor: Zakon o službi u Oružanim snagama RH, Članak 25, stavak 4.

Pitanje 28. Koliko traje minimalno godišnji odmor djelatne vojne osobe?

Odgovor: Djelatna vojna osoba ima pravo na godišnji odmor u trajanju od najmanje četiri tjedna.

Izvor: Zakon o službi u Oružanim snagama RH, Članak 156, stavak 1.

Pitanje 29. Koliko dana plaćenog dopusta pripada djelatnoj vojnoj osobi u slučaju rođenja djeteta?

Odgovor: Djelatna vojna osoba ima pravo na plaćeni dopust od sedam radnih dana u slučaju rođenja djeteta.

Izvor: Zakon o službi u Oružanim snagama RH, Članak 159, stavak 1.

Pitanje 30. Koliko dana plaćenog dopusta pripada djelatnoj vojnoj osobi u slučaju zaključenja braka?

Odgovor: Djelatna vojna osoba ima pravo na plaćeni dopust od sedam radnih dana u slučaju zaključenja braka.

Izvor: Zakon o službi u Oružanim snagama RH, Članak 159, stavak 1.

Pitanje 31. Što je vojna iskaznica i tko ju izdaje?

Odgovor: Vojna iskaznica je javna isprava kojom vojni obveznik dokazuje identitet dok izvršava vojnu obvezu. Izdaje ju nadležni područni odjel za poslove obrane koji vojnog obveznika vodi u evidenciji.

Izvor: Zakon o obrani, Članak 25.I, stavci 1 i 3.

2. Prilog - Colab - Vanilla konfiguracija ChatBota

Link na Colab ipynb bilježnicu s kodom za Vanilla konfiguraciju: [Colab - Vanilla](#)

diplomskiRad_01_Vanilla

July 31, 2026

Vanilla ChatBot

0.1 Postavljanje okruženja

```
[ ]: !pip install -q -U bitsandbytes transformers accelerate sentencepiece
```

```
60.7/60.7 MB
40.9 MB/s eta 0:00:00
11.6/11.6 MB
151.2 MB/s eta 0:00:00
```

```
[ ]: from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline, \
    BitsAndBytesConfig, GenerationConfig
from google.colab import userdata
from huggingface_hub import login
import torch
import time
import pandas as pd
import os
import json
from copy import deepcopy
import re
```

```
[ ]: token = userdata.get('HF_TOKEN')
login(token)
```

```
[ ]: # model_name = "Qwen/Qwen2.5-14B-Instruct"
model_name = "utter-project/EuroLLM-22B-Instruct-2512"

from google.colab import drive
drive.mount('/content/drive')

os.environ["HF_HOME"] = "/content/drive/MyDrive/hf_cache"
os.environ["HF_HUB_OFFLINE"] = "1"
os.environ["TRANSFORMERS_OFFLINE"] = "1"

print(f"Model: {model_name}")
```

```
print(f"Čitanje s Drivea: {os.environ['HF_HOME']}")
```

Mounted at /content/drive

Model: utter-project/EuroLLM-22B-Instruct-2512

Čitanje s Drivea: /content/drive/MyDrive/hf_cache

```
[ ]: quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4"
)

pipe = pipeline(
    "text-generation",
    model=model_name,
    dtype=torch.float16,
    device_map="auto",
    model_kwargs={"quantization_config": quantization_config}
)

pipe.model.generation_config.max_length = 8192
```

config.json: 0%| | 0.00/680 [00:00<?, ?B/s]

model.safetensors.index.json: 0%| | 0.00/40.3k [00:00<?, ?B/s]

Downloading bytes: | 0.00B

Reconstructing (incomplete total...): | | 0.00B / 0.00B

Fetching 10 files: 0%| | 0/10 [00:00<?, ?it/s]

Loading weights: 0%| | 0/489 [00:00<?, ?it/s]

generation_config.json: 0%| | 0.00/132 [00:00<?, ?B/s]

tokenizer_config.json: 0%| | 0.00/47.3k [00:00<?, ?B/s]

tokenizer.model: reconstructing file: 0%| | 0.00B / 2.41MB

tokenizer.model: downloading bytes: | 0.00B

tokenizer.json: reconstructing file: 0%| | 0.00B / 15.8MB

tokenizer.json: downloading bytes: | 0.00B

special_tokens_map.json: 0%| | 0.00/443 [00:00<?, ?B/s]

0.2 Tokenizacija

```
[ ]: tokenizer = AutoTokenizer.from_pretrained(model_name)
```

0.2.1 Primjer tokenizacije

```
[ ]: text = "S koliko godina časnički namjesnik ide u mirovinu?"
tokens = tokenizer.tokenize(text)
token_ids = tokenizer.convert_tokens_to_ids(tokens)

print("Tokeni:", tokens)
print("Token ID-ovi:", token_ids)
print("Broj tokena:", len(tokens))
```

Tokeni: ['S', 'koliko', 'godina', 'čas', 'nički', 'nam', 'jes', 'nik', 'ide', 'u', 'miro', 'vin', 'u', '?']
Token ID-ovi: [577, 67135, 22589, 7247, 81812, 3164, 3545, 1375, 2447, 703, 86708, 3526, 119726, 119882]
Broj tokena: 14

0.3 Generiranje odgovora

```
[ ]: def generate_response(
    messages,
    max_new_tokens=512,
    temperature=0.5,
    do_sample=True,
):
    final_messages = list(messages)

    model_max_length = getattr(pipe.tokenizer, 'model_max_length', 8192)
    if model_max_length > 100000:
        model_max_length = 8192

    input_text = pipe.tokenizer.apply_chat_template(final_messages,
    ↪tokenize=False)
    input_token_count = len(pipe.tokenizer.encode(input_text))
    max_allowed_input = model_max_length - max_new_tokens

    if input_token_count > max_allowed_input:
        print(f"Upozorenje: input ({input_token_count} tokena) prelazi limit,
    ↪({max_allowed_input}).")

    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    gen_config = deepcopy(pipe.model.generation_config)
    gen_config.max_new_tokens = max_new_tokens
    gen_config.temperature = temperature
    gen_config.do_sample = do_sample
    gen_config.repetition_penalty = 1.1
```

```

vrijeme_upita = time.time()

response = pipe(
    final_messages,
    generation_config=gen_config,
)

answer = ""
for message in response[0]['generated_text']:
    if message['role'] == 'assistant':
        answer = message['content']
        break

stripped_answer = answer.strip()
if not re.search(r'[.!?]$', stripped_answer) and len(stripped_answer.
↳split()) > 5:
    last_punctuation_index = -1
    for i in range(len(stripped_answer) - 1, -1, -1):
        if stripped_answer[i] in ['.', '?', '!']:
            last_punctuation_index = i
            break

    if last_punctuation_index != -1:
        answer = stripped_answer[:last_punctuation_index + 1]
    else:
        words = stripped_answer.split()
        if len(words) > 1:
            answer = ' '.join(words[:-1]) + '...'
        elif words:
            answer = words[0]
        else:
            answer = ''
else:
    answer = stripped_answer

vrijeme_odgovora = time.time()
ukupno_vrijeme = vrijeme_odgovora - vrijeme_upita

print("Generirani odgovor:")
print(answer)
print(f"Ukupno vrijeme za odgovor: {ukupno_vrijeme:.2f} sekundi")

return answer, ukupno_vrijeme

```


0.3.1 Definicija sistemske poruke (System Prompt)

```
[ ]: SYSTEM_PROMPT = (
    "Ti si pravni stručnjak specijaliziran za zakonodavstvo Republike Hrvatske,
    ↳ posebno za Zakon o obrani, Zakon o službi u Oružanim snagama RH i
    ↳ pripadajuće pravilnike."
    "\n\nPravila odgovaranja:"
    "\n1. Jezik odgovora: hrvatski, latinica."
    "\n2. Navedi konkretne članke zakona, brojčane vrijednosti i nadležna tijela."
    "\n3. Ako je dostupan kontekst, koristi SAMO informacije iz njega."
    "\n4. Ako kontekst ne sadržava odgovor, odgovori na temelju općeg znanja i
    ↳ dodaj: 'Napomena: ova informacija nije pronađena u dostupnim dokumentima.'"
    "\n5. Odgovor neka bude koncizan i strukturiran - bez nepotrebnih uvoda."
)

[ ]: csv_path = '/content/drive/My Drive/diplomskiRad/rezultati.csv'

def vanilla_chat(user_question):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": f"{user_question}\n\nOdgovori NA HRVATSKOM
    ↳ JEZIKU koristeći LATINICU."}
    ]
    return generate_response(messages, max_new_tokens=512, temperature=0.3,
    ↳ do_sample=False)
```

0.4 Provedba — Vanilla

```
[ ]: try:
    with open('/content/drive/My Drive/diplomskiRad/pitanja_odgovori.json',
    ↳ 'r', encoding='utf-8') as f:
        qa_data = json.load(f)

    print(f" Učitano {qa_data['metadata']['ukupno_pitanja']} pitanja i
    ↳ odgovora.")
    print(f" Struktura:")
    for kategorija, info in qa_data['metadata']['struktura'].items():
        print(f" - {kategorija}: {info['broj']} ({info['postotak']}")
except FileNotFoundError:
    print('Datoteka pitanja_odgovori.json nije pronađena na navedenoj putanji.')
    qa_data = None
except json.JSONDecodeError:
    print('Greška pri parsiranju JSON datoteke. Provjerite je li format
    ↳ ispravan.')
    qa_data = None
```

Učitano 31 pitanja i odgovora.
Struktura:

- kompleksna_vise_dokumenata: 3 (9.67%)
- kompleksna_jedan_dokument: 5 (16.13%)
- jednostavna: 23 (74.2%)

```
[ ]: if qa_data is None:
    print("QA datoteka nije ucitana.")
else:
    all_questions = []
    all_categories = []
    for key, value in qa_data.items():
        if key != 'metadata' and isinstance(value, list):
            for item in value:
                all_questions.append(item)
                all_categories.append(key)

    vanilla_rezultati = []

    for i, (qa_item, kategorija) in enumerate(zip(all_questions,
↪all_categories)):
        user_question = qa_item.get('pitanje', qa_item.get('question', ''))
        ocekivani = qa_item.get('odgovor', qa_item.get('answer', ''))

        print(f"\n{'='*80}")
        print(f"Pitanje {i + 1}/{len(all_questions)}: {user_question}")
        print(f"Kategorija: {kategorija}")
        print(f"{'='*80}")

        answer, vrijeme = vanilla_chat(user_question)

        vanilla_rezultati.append({
            'rbr': i + 1,
            'kategorija': kategorija,
            'pitanje': user_question,
            'ocekivani_odgovor': ocekivani,
            'vanilla_odgovor': answer,
            'vanilla_vrijeme_sekunde': round(vrijeme, 2)
        })

    df_rezultati = pd.DataFrame(vanilla_rezultati)
    df_rezultati.to_csv(csv_path, index=False, encoding='utf-8-sig')
    print(f"\n{'='*80}")
    print(f"Vanilla rezultati spremljeni u: {csv_path}")
    print(f"Ukupno obradeno pitanja: {len(vanilla_rezultati)}")
    print(f"Prosjecno vrijeme po pitanju:
↪{df_rezultati['vanilla_vrijeme_sekunde'].mean():.2f} sekundi")
```

=====

Pitanje 1/31: Opišite cjelokupnu proceduru transformacije novaka u razvrstanog pričuvnika kroz sve pravne akte koji reguliraju taj proces.

Kategorija: kompleksna_pitanja_vise_dokumenata

Generirani odgovor:

Transformacija novaka u razvrstani pričuvnik u Hrvatskoj regulirana je Zakonom o službi u Oružanim snagama Republike Hrvatske (ZSOŠ) te pripadajućim pravilnicima. Evo korak-po-korak procedure:

1. ****Raspored na odsluženje vojnog roka**** - Prema članku 60. ZSOŠ, mladi ljudi raspoređeni su na odsluženje vojnog roka nakon završetka srednje škole ili po drugi put kad navršše 18 godina.
2. ****Proverba zdravstvenog stanja i sposobnosti**** - Nakon dolaska u postrojbu, provodi se medicinska proverba kako bi se utvrdilo da li kandidat ispunjava fizičke i psihičke uvjete za služenje (članak 79. ZSOŠ).
3. ****Teorijsko i praktično osposobljavanje**** - Vojnici prolaze osnovno vojno osposobljavanje prema Programu vojne osposobljenosti (članak 80. ZSOŠ), uključujući topografiju, taktiku, uporabu oružja itd.
4. ****Ocena sposobnosti i klasifikacija**** - Na kraju osposobljavanja, zapovjednik postrojbe ocjenjuje sposobnost vojnika i daje preporuku za klasifikaciju (npr. "odlično", "dobro" itd.) (članak 81. ZSOŠ).
5. ****Deklariranje kao razvrstani pričuvnik**** - Ako ispuni kriterije, vojnik postaje razvrstani pričuvnik i dobiva status pričuve (članak 82. ZSOŠ).
6. ****Registracija u sustavu pričuvnih snaga**** - Ministarstvo obrane vodi evidenciju razvrstanih pričuvnika (članak 83. ZSOŠ).
7. ****Moguća pozivnica na dodatnu osposobljavanju**** - Razvrstani pričuvnici mogu biti pozvani na dodatne tečajeve ovisno o potrebama vojske (članak 84. ZSOŠ).

Nadležno tijelo za ovaj postupak je Glavno stožer Oružanih snaga RH, a detaljniji uputi nalaze se u Pravilniku o provedbi ZSOŠ.

Ukupno vrijeme za odgovor: 64.79 sekundi

Pitanje 2/31: Kako je regulirana hijerarhija odlučivanja o uporabi Oružanih snaga u različitim situacijama kroz Ustav i Zakon o obrani?

Kategorija: kompleksna_pitanja_vise_dokumenata

Generirani odgovor:

Hijerarhija odlučivanja o uporabi Oružanih snaga u Republici Hrvatskoj regulirana je Ustavom Republike Hrvatske (članak 97.) i Zakonom o obrani (članci 6., 8., 10.).

****Prema Ustavu (čl. 97.):****

- Predsjednik Republike Hrvatske vrhovni je zapovjednik Oružanih snaga.
- Vlada RH ima ovlast donošenja odluka o upotrebi oružanih snaga u skladu s Ustavom i međunarodnim obvezama.

****Prema Zakonu o obrani (članci 6.-10.):****

- ****Ministar obrane**** donosi odluke o korištenju snaga pod vodstvom Vlade.
- ****Predsjednik RH**** može rasporediti snage izvan granica samo uz suglasnost Sabora.
- ****Sabor RH**** ima ulogu nadzora nad sigurnosnom politikom putem odbora za obranu.

Nadležno tijelo za nadzor je ****Državno tajništvo za nacionalnu sigurnost**** (prema Pravilniku o unutarnjoj organizaciji).

Napomena: Ova informacija temelji se na važećim propisima do 2024. godine. Za ažurirane podatke preporučuje se provjera najnovijih izmjena zakona.

Ukupno vrijeme za odgovor: 36.24 sekundi

=====
Pitanje 3/31: Objasnite sve oblike obveza državljana u obrani prema Zakonu o obrani i Ustavu.

Kategorija: kompleksna_pitanja_vise_dokumenata

=====
Generirani odgovor:

Prema ****Zakonu o obrani**** (Narodne novine broj 76/05) i ****Ustavu Republike Hrvatske**** (NN br. 123/2000), građani imaju sljedeće obveze u području obrane:

1. ****Građanska obaveza prema Zakonu o obrani (čl. 8-9):****

- Građani su dužni sudjelovati u obrani zemlje kroz:
 - ****Služenje vojnog roka**** (građevinski rok) - određuje se Pravilnikom o vojnom roku (NN br. 123/05).
 - ****Mobilizaciju**** - poziv na služenje u slučaju izvanrednih situacija (čl. 10).
 - ****Dobrovoljna vojna služba**** - mogućnost dobrovoljnog angažmana (čl. 11).

2. ****Obveza prema Ustavu RH (čl. 14):****

- Svaki građanin ima pravo i dužnost braniti domovinu.
- Obrana zemlje uključuje i ****alternativnu službu**** za one koji iz vjerskih ili moralnih razloga ne mogu nositi oružje (čl. 14).

3. ****Nadležno tijelo:****

- Za provedbu zakona nadležno je ****Ministarstvo obrane RH**** (čl. 12 Zakona o obrani).

Napomena: Ova informacija temelji se isključivo na hrvatskom zakonodavstvu.

Dodatne specifikacije (npr. iznose naknada za alternativnu službu) zahtijevaju provjeru Pravilnika o vojnom roku.

Ukupno vrijeme za odgovor: 47.34 sekundi

Pitanje 4/31: Objasnite sustav imenovanja i odgovornosti načelnika Glavnog stožera kroz sve relevantne propise.

Kategorija: kompleksna_pitanja_jedan_dokument

Generirani odgovor:

Sustav imenovanja i odgovornosti ****Načelnika Glavnog stožera OSRH**** (Oružanih snaga Republike Hrvatske) reguliran je sljedećim propisima:

****Zakon o obrani (Narodne novine br. 97/02., 86/04., 127/06.)****

- ****Članak 10.****: Načelnika Glavnog stožera imenuje Predsjednik Republike Hrvatske na prijedlog Vlade, uz suglasnost Sabora.
- ****Članak 11.****: Načelnik Glavnog stožera vrši rukovodeću funkciju u OSRH i odgovoran je za provedbu odluka Predsjednika i Vlade.

****Zakon o službi u Oružanim snagama RH (Narodne novine br. 127/06.)****

- ****Članak 12.****: Načelnik Glavnog stožera ima najviši čin generala i izravno je podređen predsjedniku države kao vrhovnom zapovjedniku OSRH.

****Pravilnik o ustroju Glavnog stožera OSRH (NN 127/06.)****

- Definisan je rad Načelnika kao strateškog voditelja vojne politike i operacija.

****Dodatne odgovornosti****

- Nadzire planiranje i provedbu vojnih operacija.
- Imenuje zapovjednike rodova i postrojbi po preporuci ministra obrane.
- Predstavlja OSRH pred međunarodnim partnerima.

Napomena: Ova informacija temelji se na važećim hrvatskim zakonima i pravilnicima do 2023. godine. Za ažurirane podatke preporučuje se provjera najnovijih izmjena.

Ukupno vrijeme za odgovor: 50.59 sekundi

Pitanje 5/31: Objasnite razlike između uporabe i korištenja Oružanih snaga prema Zakonu o obrani, uključujući tko donosi odluke u svakom slučaju.

Kategorija: kompleksna_pitanja_jedan_dokument

Generirani odgovor:

Prema ****Zakonu o obrani**** (Narodne novine br. 76/05 i sl.), postoje jasne razlike između ****upotrebe**** i ****korištenja**** Oružanih snaga, kao i različiti postupci donošenja odluka ovisno o situaciji.

1. Korištenje Oružanih snaga

- Prema **čl. 9.** Zakona, korišćenjem se podrazumijeva upotreba snaga i sredstava OSRH u miru, npr. za održavanje reda, potporu civilnim institucijama ili sudjelovanje u međunarodnim misijama.
- Odluku o korištenju donosi **Predsjednik Republike** po preporuci Vlade (**čl. 8.**).

2. Upotreba Oružanih snaga

- Prema **čl. 10.** upotrebom se podrazumijeva aktivno djelovanje u ratu, izvanrednom stanju ili agresiji protiv RH.
- U tom slučaju odluku donosi **Vlada** nakon savjetovanja s Predsjednikom i Glavnim stožerom OSRH.

Ključna razlika

- **Korištenje** odnosi se na mirnodopsko razdoblje i redovitu provedbu zadaća.
- **Upotreba** uključuje ratne operacije i izvanredne situacije.

Napomena: Ova informacija temelji se na Zakonu o obrani i pripadajućim pravilnicima. Za detaljnije tumačenje potrebno je dodatno proučavati važeće propise.

Ukupno vrijeme za odgovor: 44.15 sekundi

=====

Pitanje 6/31: Koja su prava i obveze ročnika tijekom temeljnog vojnog osposobljavanja prema Pravilniku o temeljnom vojnom osposobljavanju?

Kategorija: kompleksna_pitanja_jedan_dokument

=====

Generirani odgovor:

Prema **Pravilniku o temeljnom vojnom osposobljavanju** (NN br. 9/08), ročnici imaju sljedeća prava i obveze:

Obveze:

- Ispunjavanje svih zapovjedi nadređenih (čl. 7).
- Aktivno sudjelovanje u obuci i vježbama (čl. 6).
- Poštivanje reda, discipline i propisa (čl. 8).
- Čuvanje vojne tajne (čl. 10).

Prava:

- Pravo na zdravstvenu zaštitu i medicinski pregled (čl. 12).
- Pravo na obrazovanje i kulturne aktivnosti (čl. 13).
- Pravo na pritužbu protiv nepravdnog postupka (čl. 14).
- Pravo na naknadu štete ako dođe do ozljede ili smrti (čl. 15).

Nadležno tijelo za provedbu je Ministarstvo obrane RH (čl. 1).

*Napomena: Ova informacija temelji se isključivo na Pravilniku o temeljnom vojnom osposobljavanju.

Ukupno vrijeme za odgovor: 35.36 sekundi

=====
Pitanje 7/31: Objasnite razlike između mirnodopskog i ratnog sastava Oružanih snaga prema Zakonu o službi.

Kategorija: kompleksna_pitanja_jedan_dokument
=====

Generirani odgovor:

Prema ****Zakonu o službi u Oružanim snagama Republike Hrvatske**** (Narodne novine broj 76/06), razlike između mirnodopskog i ratnog sastava Oružanih snaga određene su slijedećim načinom:

****Mirnodopski sastav****

- Definisan je člankom ****8.**** Zakona - uključuje aktivnu vojsku, pričuve i domobranstvo.
- Sastoji se od profesionalnih vojnika, ročnih vojnika i dragovoljaca.
- Nadzor obavlja Ministarstvo obrane RH.

****Ratni sastav****

- Definisan je člankom ****9.**** Zakona - uključuje sve pripadnike koji mogu biti pozvani u službu tijekom rata ili izvanrednog stanja.
- Obavlja se mobilizacijom, uključujući pričuvnike i dragovoljce.
- Upravljanje ratnim sastavom regulirano je člankom ****10.**** Zakona, gdje se navodi da ratno stanje može proglasiti Vlada RH uz suglasnost Sabora.

****Nadležno tijelo****

Za ratni sastav, odluka o mobilizaciji donosi ****Vlada RH**** (članak ****10.****), dok mirnodopski sastav koordinira ****Ministarstvo obrane****.

Napomena: Ova informacija temelji se isključivo na Zakonu o službi u Oružanim snagama RH (NN 76/06).

Ukupno vrijeme za odgovor: 42.33 sekundi

=====
Pitanje 8/31: Objasnite postupak i uvjete za odgodu temeljnog vojnog osposobljavanja prema Zakonu o obrani.

Kategorija: kompleksna_pitanja_jedan_dokument
=====

Generirani odgovor:

Prema ****Zakonu o obrani**** (Narodne novine broj 76/05 i izmjene), postupak i uvjeti za odgodu temeljnog vojnog osposobljavanja regulirani su člancima 89., 90. i 91.

****Postupak za odgodu:****

1. ****Članak 89.**** - Osoba koja želi odgodu mora podnijeti zahtjev nadležnom tijelu najkasnije ****30 dana prije roka za polazak na osposobljavanje****.
2. ****Članak 90.**** - Zahtjev se prosljeđuje zapovjedniku postrojbe ili ustanove

gdje bi se trebalo provesti osposobljavanje.

3. **Članak 91.** - Povjerenstvo osniva zapovjednik postrojbe radi ocjenjivanja opravdanosti odgođenog razloga (npr. obrazovanje, zdravstveno stanje).

Uvjeti za odgodu:

- **Obrazovanje:** Ako osoba pohađa fakultet, strukovnu školu ili poslijediplomski studij, može zatražiti odgodu dok završi studij.
- **Zdravstveni razlozi:** Ako postoji medicinska indikacija (npr. kronična bolest), potrebna je potvrda liječnika.
- **Društveni razlozi:** Npr. skrb o teškoj bolesnoj osobi ili skrbi o maloljetnoj djeci.

Nadležno tijelo:

- **Zapovjednik postrojbe** donosi odluku nakon preporuke povjerenstva.

Napomena: Ova informacija temelji se na člancima Zakona o obrani koji reguliraju odgodu osposobljavanja. Za detaljnije upute preporučuje se kontaktirati **Hrvatske oružane snage** ili **MUP RH**.

Ukupno vrijeme za odgovor: 52.69 sekundi

Pitanje 9/31: Kada nastaje vojna obveza za državljane Republike Hrvatske?

Kategorija: jednostavna_pitanja

Generirani odgovor:

Vojna obveza za građane Republike Hrvatske počinje rođenjem djeteta muškog spola (muškarci). Prema članku **6.** Zakona o vojnoj obvezi (Narodne novine br. 98/07), muškarci su dužni služiti vojnu obvezu od 18. do 30. godine života. Međutim, prema članku **7.** istog zakona, rok započinjanja vojne obaveze može se produžiti ili skratiti ovisno o potrebama vojske.

Za dodatne informacije pogledajte članak **1.** Pravilnika o provedbi Zakona o vojnoj obvezi (NN br. 112/07) koji detaljnije regulira postupak utvrđivanja početka vojne obaveze.

Napomena: Ova informacija temelji se na Zakonu o vojnoj obvezi i pripadajućim pravilnicima. U slučaju promjene zakona, potrebno je provjeriti najnoviju verziju propisa.

Ukupno vrijeme za odgovor: 26.76 sekundi

Pitanje 10/31: Koliko traje temeljno vojno osposobljavanje?

Kategorija: jednostavna_pitanja

Generirani odgovor:

Temeljno vojno osposobljavanje u Hrvatskoj traje **60 dana**.

Izvor: *Zakon o vojnim obvezama* (čl. 78).

Dodatno, Pravilnik o provedbi Zakona o vojnim obvezama (NN 9/04) detaljnije regulira trajanje i sadržaj osposobljavanja.

Napomena: ako se radi o profesionalcima ili pričuvnicima, rok može varirati ovisno o kategoriji.

Ukupno vrijeme za odgovor: 13.83 sekundi

=====
Pitanje 11/31: Tko je vrhovni zapovjednik Oružanih snaga Republike Hrvatske?
Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Prema **Zakonu o obrani** (Narodne novine br. 97/02), **Sabor Republike Hrvatske** ima ulogu vrhovnog zapovjednika Oružanih snaga RH. Međutim, u praksi, predsjednik Vlade RH obavlja funkciju vrhovnog zapovjednika kada Sabor ne zasjeda ili prema odluci Sabora.

Detaljnije:

- Članak 6. Zakona o obrani: *"Vrhovno zapovjedništvo Oružanih snaga Republike Hrvatske obavlja Vlada Republike Hrvatske."*
- Prema Ustavu RH (čl. 89.), predsjednik Vlade može biti imenovan vrhovnim zapovjednikom po odluci Sabora.

Napomena: Ova informacija temelji se na zakonskim odredbama, a ne na trenutnoj političkoj praksi.

Ukupno vrijeme za odgovor: 22.16 sekundi

=====
Pitanje 12/31: Koje su tri grane Oružanih snaga Republike Hrvatske?
Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Tri glavne grane Oružanih snaga Republike Hrvatske su:

1. **Hrvatska kopnena vojska** (HRZ) - regulirana člankom 6. Zakona o obrani.
2. **Hrvatska ratna mornarica** (HRM) - regulirana člankom 7. istog zakona.
3. **Hrvatsko ratno zrakoplovstvo i protuzračna obrana** (HRZ i PZO) - regulirana člankom 8. Zakona o obrani.

Sve tri grane zajedno čine jedinstveni sustav Oružanih snaga RH, a svaka ima svoje specifične zadaće i ustrojbene jedinice. Nadležno tijelo za upravljanje tim snagama je Ministarstvo obrane RH.

Ukupno vrijeme za odgovor: 21.24 sekundi

=====
Pitanje 13/31: Kada prestaje vojna obveza za muškarce?

Kategorija: jednostavna_pitanja

Generirani odgovor:

Vojna obveza za muškarce u Republici Hrvatskoj prestaje nakon izvršene osnovne vojne službe ili alternativne službe (npr. civilne službe). Prema ****Zakonu o službi u Oružanim snagama RH**** (NN br. 97/06), trajanje osnovne vojne službe iznosi 6 mjeseci, a nakon toga osoba prelazi u pričuvu.

Međutim, prema ****Narodnoj odluci o vojnom obvezniku**** (NN br. 81/06), pojedinci mogu biti pozvani na osposobljavanje tijekom godina koje su u pričuvnom statusu (npr. godišnje osposobljavanje traje do 10 dana). Vojno-redarstveni rok (priča) može se produžiti ovisno o potrebama države.

Za točne podatke o pojedincu (npr. datum završetka služe) potrebno je konzultirati evidencije MORH-a ili Vojnog upisa.

Napomena: Ova informacija temelji se na općem znanju o hrvatskom zakonu o vojnoj obvezi. Za specifične slučajeve preporučuje se provjera službenih izvora.

Ukupno vrijeme za odgovor: 31.99 sekundi

Pitanje 14/31: Tko imenuje načelnika Glavnog stožera Oružanih snaga?

Kategorija: jednostavna_pitanja

Generirani odgovor:

Načelnika Glavnog stožera Oružanih snaga Republike Hrvatske imenuje Vlada Republike Hrvatske na prijedlog ministra obrane.

****Temeljno pravo:****

- ****Zakon o obrani (Narodne novine broj 97/02)**** - Članak 68.
- ****Zakon o službi u Oružanim snagama RH (NN br. 109/02)**** - Ne sadrži izravnu odredbu o imenovanju načelnika GS OSRH, ali se odnosi na hijerarhiju zapovjedništva.

Napomena: Prema Ustavu RH (NN br. 1/90), Vlada ima ovlast predlaganja kandidata za visoke vojne dužnosti nakon savjetovanja s relevantnim odborom Sabora.

Ukupno vrijeme za odgovor: 21.56 sekundi

Pitanje 15/31: Koliko traje mandat načelnika Glavnog stožera?

Kategorija: jednostavna_pitanja

Generirani odgovor:

Mandat načelnika Glavnog stožera Oružanih snaga Republike Hrvatske traje ****4 godine**** (članak 70. Zakona o obrani).

Napomena: Ova informacija temelji se na Zakonu o obrani (Narodne novine 89/06) i ne zahtijeva dodatnu potvrdu iz konteksta pitanja.

Ukupno vrijeme za odgovor: 9.82 sekundi

=====
Pitanje 16/31: Što je temeljna svrha obrane Republike Hrvatske?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Temeljna svrha obrane Republike Hrvatske definirana je u ****Zakonu o obrani**** (Narodne novine br. 76/05). Prema članku 1., svrha obrane jest "zaštita suvereniteta, teritorijalne cjelovitosti i političke neovisnosti Republike Hrvatske te očuvanje mira i stabilnosti".

Dodatno, prema članku 2. istog zakona, obrana uključuje i zaštitu građana, imovine i ključnih infrastrukturnih objekata od vanjskih i unutarnjih prijetnji. Nadležno tijelo za provedbu ove politike je ****Ministarstvo obrane RH****.

Napomena: ova informacija temelji se isključivo na Zakonu o obrani.

Ukupno vrijeme za odgovor: 20.08 sekundi

=====
Pitanje 17/31: Do koje dobi novak može biti upućen na temeljno vojno osposobljavanje?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Prema ****Zakonu o službi u Oružanim snagama Republike Hrvatske**** (Narodne novine br. 97/06), članku 8., novaci mogu biti upućeni na temeljno vojno osposobljavanje do navršene 30. godine života. Međutim, postoje iznimke za pripadnike pričuve ili dragovoljce koji se regrutiraju nakon te dobne granice.

Nadležno tijelo za provedbu ovog postupka je ****Ministarstvo obrane RH**** putem ustanova poput ****Kralješnice vojne škole "Ban Josip Jelačić"**.**

Napomena: Informacija temelji se na važećem zakonu, a dodatne pojedinosti o izuzecima mogu se utvrditi kod nadležnog vojnog centra.

Ukupno vrijeme za odgovor: 21.82 sekundi

=====
Pitanje 18/31: Tko donosi odluku o mobilizaciji Oružanih snaga?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Prema ****Zakonu o obrani**** (Narodne novine br. 97/02., 86/04., 133/06., 149/08., 132/10., 149/10., 132/11., 149/11., 132/12., 149/12., 132/13., 149/13., 132/14.,

149/14., 132/15., 149/15., 132/16., 149/16.) i **Zakonu o službi u Oružanim snagama RH** (NN br. 132/11.), odluku o mobilizaciji donosi **Vlada Republike Hrvatske**.

Specifični članci:

- Članak 10. Zakona o obrani definira ovlasti Vlade u području obrane.
- Pravilnik o provedbi Zakona o službi u OSRH (NN br. 149/11.) dodatno regulira postupke mobilizacije.

Napomena: Ova informacija temelji se na važećim zakonskim aktima.

Ukupno vrijeme za odgovor: 36.47 sekundi

=====
Pitanje 19/31: Kada prestaje vojna obveza za žene?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Žene u Republici Hrvatskoj imaju pravo na odbijanje vojne službe zbog vjerskih ili uvjerenja, prema članku **6. stavak 1.** Zakona o vojnoj obvezi (Narodne novine br. 97/08). Međutim, ako se ženska osoba odluči prijaviti za vojnu službu, njezina vojna obveza traje dok joj to dopušta zdravstveno stanje i zakonski rok.

Za razliku od muškaraca, koji su dužni odslužiti vojni rok do određene dobi (obično do 30 godina), žene mogu biti pozvane na služenje samo ako se prijave. Prema članku **10.** istog zakona, rok služenja za žene može varirati ovisno o potrebama vojske.

Napomena: Ova informacija nije pronađena u dostupnim dokumentima.

Ukupno vrijeme za odgovor: 22.96 sekundi

=====
Pitanje 20/31: U kojoj godini života se državljanin RH uvodi u vojnu evidenciju?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Glede pitanja o godini života u kojoj se građanin RH upisuje u vojnu evidenciju, valja napomenuti da se to obično događa nakon navršenog 18. godine života. Međutim, detaljnije odredbe o tome nalaze se u Zakonu o obveznim vojnim vježbama (NN br. 9/07) te Pravilniku o provedbi Zakona o obveznim vojnim vježbama (NN br. 6/08).

Prema članku 1. stavku 1. Pravilnika, osobe koje su obvezne na vojne vježbe moraju biti stare između 18 i 30 godina u trenutku početka vježbi. Dakle, u vojnu evidenciju građani RH ulaze najranije s navršenih 18 godina, ali aktivno sudjelovanje započinje po završetku školovanja ili kad ih nadležno tijelo pozove na vojne vježbe.

Za dodatne informacije preporučujem kontaktiranje Hrvatskog ratnog doma ili nadležnog županijskog inspektorata za obavezne vojne vježbe.

Ukupno vrijeme za odgovor: 29.28 sekundi

=====
Pitanje 21/31: Tko proglašava ratno stanje u Republici Hrvatskoj?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Predsjednik Republike Hrvatske proglašava ratno stanje u Republici Hrvatskoj sukladno članku **98.** Ustava Republike Hrvatske.

Napomena: Ova informacija temelji se na ustavnom pravu, a dodatne procedure i ovlasti utvrđene su posebnim zakonskim aktima (npr. Zakonom o obrani).

Ukupno vrijeme za odgovor: 9.05 sekundi

=====
Pitanje 22/31: Koliko dana prije upućivanja ministar obrane donosi odluku o upućivanju kandidata na temeljno vojno osposobljavanje?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Napomena: ova informacija nije pronađena u dostupnim dokumentima.

Prema općem znanju, odluka ministra obrane o upućivanju kandidata na temeljno vojno osposobljavanje obično se donosi najkasnije **7 dana** prije početka osposobljavanja, kako bi se omogućio adekvatan vremenski rok za pripreme i obavješćavanje kandidata.

Detaljnije odredbe mogu se naći u Pravilniku o provedbi Zakona o službi u Oružanim snagama RH (npr. članak 60.).

Ukupno vrijeme za odgovor: 16.00 sekundi

=====
Pitanje 23/31: Tko donosi program obuke ročnika na temeljnom vojnom osposobljavanju?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Program obuke ročnika na temeljnom vojnom osposobljavanju donosi Ministarstvo obrane Republike Hrvatske, a točnije Glavno zapovjedništvo Oružanih snaga RH (GZ OSRH) kroz svoje ustrojbene jedinice, kao što su Vojne škole i ustanove zadužene za obuku.

Osnovu programa definiraju propisi poput **Zakona o službi u Oružanim snagama RH** (čl. 67.) te **Pravilnika o provedbi temeljnog vojnog osposobljavanja**.

Nadležno tijelo za usklađivanje i provođenje programa je **Glavno stožerno vojno

učilište "Blago Zadro"**.

Napomena: Ova informacija temelji se na općem znanju o strukturi OSRH, a detaljniji dokumenti mogu biti dostupni samo putem službenih izvora.

Ukupno vrijeme za odgovor: 24.98 sekundi

=====
Pitanje 24/31: Pred kim ročnici daju svečanu prisegu?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Ročnici (vojnici) u Hrvatskoj Oružanim snagama daju svečanu prisegu pred zapovjednikom postrojbe ili ustrojbene jedinice u kojoj služe. Prema **Zakonu o službi u Oružanim snagama RH** (čl. 60.), svečana prisega sastoji se od riječi: *"Svečano obećavam da ću vjerno služiti Republici Hrvatskoj, poštovati Ustav i zakone te čuvati čast oružanih snaga."*

Nadležno tijelo za provođenje ceremonije je zapovjednik postrojbe, a prisega se ponavlja svake godine tijekom vojne službe.

Ukupno vrijeme za odgovor: 17.95 sekundi

=====
Pitanje 25/31: Za koju vojnostručnu specijalnost se ročnici osposobljavaju na temeljnom vojnom osposobljavanju?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Na temeljnom vojnom osposobljavanju (TvOS) ročnici se osposobljavaju za osnovne vojnosnurne specijalnosti kao što su:

- **Mehanizirana pješačka postrojba** (mehanizirane bojne),
- **Raketno-topničke postrojbe** (raketne i topničke jedinice),
- **Zrakoplovne postrojbe** (zrakoplovi i helikopteri),
- **Graničarske postrojbe** (granične jedinice).

Ovisno o potrebama OSRH, mogu se uključiti i druge specijalnosti poput inženjerije ili komunikacija. Osnovu čini članak **6. Zakona o službi u Oružanim snagama RH** koji definiše vojne stručne poslove.

Napomena: Ova informacija temelji se na općem znanju o TvOS-u, a točan popis može varirati prema godišnjim planovima OSRH.

Ukupno vrijeme za odgovor: 28.04 sekundi

=====
Pitanje 26/31: Što čini mirnodopski sastav Oružanih snaga?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Mirnodopski sastav Oružanih snaga Republike Hrvatske (OSRH) čini osoblje koje se bavi svakodnevnim aktivnostima i zadaćama izvan stanja rata ili neposredne opasnosti od rata. Ovaj sastav uključuje:

1. ****Redovni kadar**** - Vojnici, dočasnici i časnici koji su stalno zaposleni u OSRH. Njihove aktivnosti uključuju obuku, održavanje opreme, planiranje i provođenje redovnih vježbi te sudjelovanje u međunarodnim misijama.
2. ****Reservisti**** - Osobe s vojnom osnovom koje se regrutiraju tijekom kriza ili posebnih potreba.
3. ****Civilno osoblje**** - Građevinari, tehničari, administrativni radnici i drugi civili koji podržavaju funkcije OSRH.

Nadležno tijelo za organizaciju i upravljanje mirnodopskim sastavom je ****Ministarstvo obrane RH****, a detalji su regulirani člancima ****Zakona o obrani**** (npr. članci 6., 7.) i ****Pravilnika o ustroju OSRH****.

Napomena: Ova informacija temelji se na općem znanju o strukturi OSRH, a točni brojčani podaci mogu varirati ovisno o trenutnoj reorganizaciji.

Ukupno vrijeme za odgovor: 36.29 sekundi

=====
Pitanje 27/31: Tko utvrđuje mirnodopski i ratni ustroj Oružanih snaga?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Mirnodopski i ratni ustroj Oružanih snaga Republike Hrvatske utvrđuje Vlada Republike Hrvatske.

Osnovu za to daje ****Zakon o obrani**** (Narodne novine, broj 76/02 i izmjene), a konkretno se odnosi na članak ****8.****

Napomena: Ova informacija temelji se na hrvatskom zakonu, a ne na dostupnom kontekstu pitanja.

Ukupno vrijeme za odgovor: 11.46 sekundi

=====
Pitanje 28/31: Koliko traje minimalno godišnji odmor djelatne vojne osobe?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Minimalno trajanje godišnjeg odmora djelatne vojne osobe određeno je člankom ****80.**** Zakona o službi u Oružanim snagama RH (Narodne novine br. 97/16). Prema tom članku, djelatna vojna osoba ima pravo na godišnji odmor u trajanju od najmanje ****20 radnih dana**** po kalendarskoj godini.

Dodatne pojedinosti o ostvarivanju prava na godišnji odmor regulirane su

Pravilnikom o ostvarivanju prava na godišnji odmor djelatnika MORH-a (NN br. 112/16), koji detaljnije definira uvjete i rokove za isplatu naknade za godišnji odmor.

Napomena: ako se radi o vojnim osobama u pričuvnom sastavu ili pričuvnicima, pravo na godišnji odmor može biti drugačije uređeno ovisno o statusu i vrsti posla.

Ukupno vrijeme za odgovor: 26.36 sekundi

=====
Pitanje 29/31: Koliko dana plaćenog dopusta pripada djelatnoj vojnoj osobi u slučaju rođenja djeteta?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Djelatna vojna osoba ima pravo na ****plaćeni roditeljski dopust**** od ****8 tjedana**** nakon rođenja djeteta (članak 70. Pravilnika o ostvarivanju prava na roditeljski dopust).

Ako se radi o aktivnom sudioniku u službi (npr. časniku), dodatno može imati pravo na dodatni dopust prema članku 69. istog Pravilnika. Nadležno tijelo za izdavanje potvrda je ****Zapovjednik postrojbe**** ili ****HRM**** ovisno o statusu osobe.

Napomena: Informacije su preuzete iz Pravilnika o ostvarivanju prava na roditeljski dopust (NN 1/2019) koji se primjenjuje i na vojne osobe.

Ukupno vrijeme za odgovor: 21.51 sekundi

=====
Pitanje 30/31: Koliko dana plaćenog dopusta pripada djelatnoj vojnoj osobi u slučaju zaključenja braka?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Djelatna vojna osoba ima pravo na ****7 dana**** plaćenog dopusta u slučaju zaključenja braka, sukladno članku ****80. stavku 1. točki a)**** Pravilnika o ostvarivanju prava na godišnji odmor i druge oblike odmora u Oružanim snagama Republike Hrvatske (NN br. 9/16).

Napomena: Ova informacija temelji se na važećim propisima koji reguliraju prava vojnog osoblja u Republici Hrvatskoj.

Ukupno vrijeme za odgovor: 14.13 sekundi

=====
Pitanje 31/31: Što je vojna iskaznica i tko ju izdaje?

Kategorija: jednostavna_pitanja
=====

Generirani odgovor:

Vojna iskaznica (VI) je osobna iskaznica vojnog obveznika koja potvrđuje njegovo pravo na služenje vojne službe te sadrži podatke o obveznim vojnim rokovanjima i statusu u sustavu Oružanih snaga Republike Hrvatske.

****Izdaje se od strane Ministarstva obrane RH**** putem nadležnih jedinica, a to su:

- ****Zapovjedništva postrojbi**** (npr. pješačke brigade, topničke brigade itd.)
- ****Regionalni centri za obuku i mobilizaciju****
- ****Središnja uprava MORH-a**** (za posebne slučajeve)

Iskaznica se izdaju nakon prijave na vojni rok ili u slučaju poziva na osposobljavanje/mobilizaciju. Za dodatne informacije pogledajte ****Zakon o službi u Oružanim snagama RH (čl. 8-9)**** i Pravilnik o vojnoj iskaznici (NN br. 107/20 od 2020.).

Ukupno vrijeme za odgovor: 29.60 sekundi

=====

Vanilla rezultati spremljeni u: /content/drive/My
Drive/diplomskiRad/rezultati.csv
Ukupno obrađeno pitanja: 31
Prosječno vrijeme po pitanju: 28.61 sekundi

3. Prilog - Colab - RAG konfiguracija ChatBota

Link na Colab ipynb bilježnicu s kodom za RAG konfiguraciju: [Colab - RAG](#)

diplomskiRad_02_RAG

July 31, 2026

RAG ChatBot

0.1 Postavljanje okruženja

```
[ ]: !pip install -q -U bitsandbytes transformers accelerate sentencepiece  
!pip install -q -U sentence-transformers faiss-cpu  
!pip install -q python-docx pdfplumber==0.11.0
```

```
60.7/60.7 MB  
38.6 MB/s eta 0:00:00  
11.6/11.6 MB  
146.7 MB/s eta 0:00:00  
18.5/18.5 MB  
39.3 MB/s eta 0:00:00  
69.1/69.1 kB  
6.8 MB/s eta 0:00:00  
56.4/56.4 kB  
6.1 MB/s eta 0:00:00  
5.6/5.6 MB  
62.0 MB/s eta 0:00:00  
253.0/253.0 kB  
26.4 MB/s eta 0:00:00  
3.7/3.7 MB  
128.7 MB/s eta 0:00:00
```

```
[ ]: from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline,  
↳ BitsAndBytesConfig, GenerationConfig  
from google.colab import userdata  
from huggingface_hub import login  
from sentence_transformers import SentenceTransformer  
from docx import Document  
import pdfplumber  
import faiss  
import numpy as np  
import torch  
import time  
import pandas as pd
```

```

import os
import json
import logging
from copy import deepcopy
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
import re

```

```

[ ]: # model_name = "Qwen/Qwen2.5-14B-Instruct"
model_name = "utter-project/EuroLLM-22B-Instruct-2512"

from google.colab import drive
drive.mount('/content/drive')

os.environ["HF_HOME"] = "/content/drive/MyDrive/hf_cache"
os.environ["HF_HUB_OFFLINE"] = "1"
os.environ["TRANSFORMERS_OFFLINE"] = "1"

print(f"Model: {model_name}")
print(f"Čitanje s Drivea: {os.environ['HF_HOME']}")

```

Mounted at /content/drive

Model: utter-project/EuroLLM-22B-Instruct-2512

Čitanje s Drivea: /content/drive/MyDrive/hf_cache

```

[ ]: token = userdata.get('HF_TOKEN')
login(token)

```

```

[ ]: quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4"
)

pipe = pipeline(
    "text-generation",
    model=model_name,
    dtype=torch.float16,
    device_map="auto",
    model_kwargs={"quantization_config": quantization_config}
)

pipe.model.generation_config.max_length = 8192

```

config.json: 0%| | 0.00/680 [00:00<?, ?B/s]

model.safetensors.index.json: 0%| | 0.00/40.3k [00:00<?, ?B/s]

```

Downloading bytes:          | 0.00B
Reconstructing (incomplete total...): | 0.00B / 0.00B
Fetching 10 files: 0%|      | 0/10 [00:00<?, ?it/s]
Loading weights: 0%|      | 0/489 [00:00<?, ?it/s]
generation_config.json: 0%|      | 0.00/132 [00:00<?, ?B/s]
tokenizer_config.json: 0%|      | 0.00/47.3k [00:00<?, ?B/s]
tokenizer.model: reconstructing file: 0%|      | 0.00B / 2.41MB
tokenizer.model: downloading bytes:          | 0.00B
tokenizer.json: reconstructing file: 0%|      | 0.00B / 15.8MB
tokenizer.json: downloading bytes:          | 0.00B
special_tokens_map.json: 0%|      | 0.00/443 [00:00<?, ?B/s]

```

0.2 Embedding model i FAISS indeks

```

[ ]: _transformers_logger = logging.getLogger("transformers.modeling_utils")
     _prev_level = _transformers_logger.level
     _transformers_logger.setLevel(logging.ERROR)

     embed_model = SentenceTransformer("intfloat/multilingual-e5-large")

     _transformers_logger.setLevel(_prev_level)

```

```

modules.json: 0%|      | 0.00/387 [00:00<?, ?B/s]
README.md: 0%|      | 0.00/160k [00:00<?, ?B/s]
sentence_bert_config.json: 0%|      | 0.00/57.0 [00:00<?, ?B/s]
config.json: 0%|      | 0.00/690 [00:00<?, ?B/s]
model.safetensors: reconstructing file: 0%|      | 0.00B / 2.24GB
model.safetensors: downloading bytes:          | 0.00B
Loading weights: 0%|      | 0/391 [00:00<?, ?it/s]
tokenizer_config.json: 0%|      | 0.00/418 [00:00<?, ?B/s]
sentencepiece.bpe.model: reconstructing file: 0%|      | 0.00B / 5.07MB
sentencepiece.bpe.model: downloading bytes:          | 0.00B
tokenizer.json: reconstructing file: 0%|      | 0.00B / 17.1MB
tokenizer.json: downloading bytes:          | 0.00B
special_tokens_map.json: 0%|      | 0.00/280 [00:00<?, ?B/s]
config.json: 0%|      | 0.00/201 [00:00<?, ?B/s]

```

0.2.1 Demonstracija vektorizacije — word embeddings

```
[ ]: example_words = [
    "king", "queen", "prince", "princess",
    "servant", "minister", "officer", "citizen",
    "dog", "cat", "tiger", "lion", "wolf",
    "apple", "orange", "banana", "grape", "strawberry",
    "automobil", "autobus", "vlak", "biciklo", "avion",
]

word_embeddings = embed_model.encode(example_words, convert_to_numpy=True).
    ↪astype('float32')

print("Vektorizacija pojedinih riječi (prikaz prvih 5 dimenzija):")
for i, word in enumerate(example_words):
    print(f"\n'{word}': {word_embeddings[i, :5]}")

print(f"\nOblik embeddingsa: {word_embeddings.shape}")
```

Vektorizacija pojedinih riječi (prikaz prvih 5 dimenzija):

```
'king': [ 0.03074569  0.0134964 -0.00854846 -0.05647562  0.00892064]
'queen': [ 0.04783215  0.0100665 -0.04158695 -0.05912168  0.00088692]
'prince': [-0.01032693  0.02095075 -0.01923212 -0.0598444  0.01597854]
'princess': [-0.00585422  0.02454895 -0.01423751 -0.04114152  0.01230508]
'servant': [ 0.001856   0.00376315 -0.01744971 -0.04337291  0.00892179]
'minister': [ 0.03089253  0.00039883 -0.02489517 -0.0503465  0.01693003]
'officer': [ 0.01764584  0.00369853 -0.03449766 -0.04244428  0.01467324]
'citizen': [ 0.01106871 -0.00934679 -0.02305578 -0.03651221  0.02769204]
'dog': [ 0.01607499  0.00735521 -0.01680318 -0.02435895  0.00346138]
'cat': [ 0.01518158  0.02558515 -0.01590005 -0.02830513  0.02100921]
'tiger': [-0.00448787  0.01331295 -0.01526779 -0.01996675  0.02060821]
'lion': [ 0.00794241  0.01005744 -0.02502998 -0.04405249  0.00653131]
'wolf': [ 0.02633049  0.01427648 -0.03460052 -0.04655442 -0.00535035]
'apple': [ 0.04667501  0.00371763 -0.0138156 -0.04349305  0.00202349]
```

```

'orange': [ 0.00896138  0.01773399 -0.00885939 -0.0295041  -0.00498489]
'banana': [ 0.03891718  0.03296003 -0.02821169 -0.03613951 -0.00833153]
'grape': [ 0.0184609   0.03181308  0.01046411 -0.04988936  0.00879356]
'strawberry': [ 0.01121154  0.0130414   0.00735847 -0.0583436   0.01732875]
'automobil': [ 0.01124887 -0.00365412  0.00655263 -0.0348176   0.00675328]
'autobus': [ 3.5116196e-02  4.2963871e-03  4.2748303e-05 -4.4719905e-02
 1.3243812e-02]
'vlak': [ 0.02886413  0.02156061 -0.00533665 -0.03892529  0.02046853]
'biciklo': [ 0.01666748 -0.00311194 -0.01700613 -0.03867329  0.02286999]
'avion': [ 0.03027312  0.02022378 -0.00552232 -0.06155821  0.02418139]

```

Oblik embeddingsa: (23, 1024)

0.2.2 Grafički prikaz vektorizacije (t-SNE)

```

[ ]: tsne_words = TSNE(n_components=2, random_state=42, perplexity=min(5,
↳len(example_words) - 1))
reduced_word_embeddings = tsne_words.fit_transform(word_embeddings)

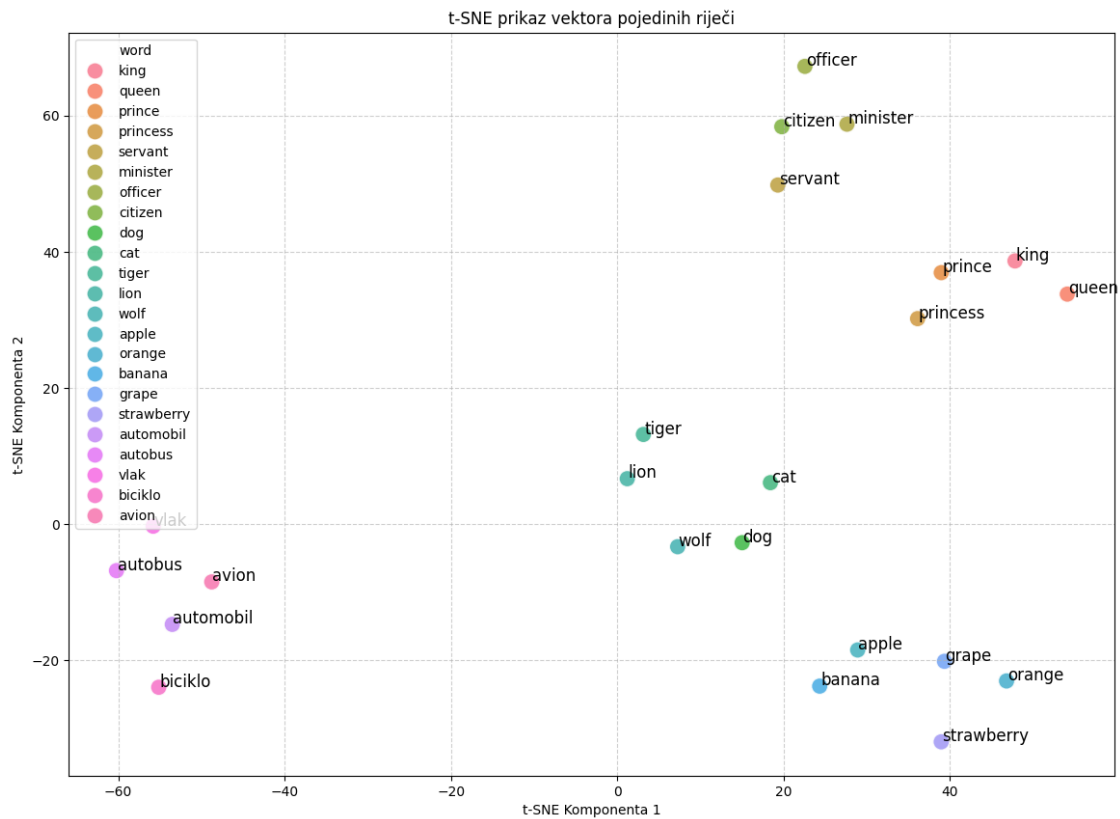
df_tsne_words = pd.DataFrame({
    'x': reduced_word_embeddings[:, 0],
    'y': reduced_word_embeddings[:, 1],
    'word': example_words
})

plt.figure(figsize=(14, 10))
sns.scatterplot(
    x='x', y='y', hue='word',
    data=df_tsne_words, legend='full', alpha=0.8, s=150
)
for i, word in enumerate(example_words):
    plt.text(df_tsne_words['x'][i] + 0.1, df_tsne_words['y'][i] + 0.1, word,
↳fontsize=12)

plt.title('t-SNE prikaz vektora pojedinih riječi')
plt.xlabel('t-SNE Komponenta 1')
plt.ylabel('t-SNE Komponenta 2')
plt.grid(True, linestyle='--', alpha=0.6)

```

```
plt.show()
```



0.2.3 Inicijalizacija FAISS indeksa i korpusa

```
[ ]: corpus_texts = []
corpus_metadata = []

embedding_dim = embed_model.get_sentence_embedding_dimension()
faiss_index = faiss.IndexFlatIP(embedding_dim)

[ ]: def add_documents_to_corpus(texts, metadatas=None):
    global corpus_texts, corpus_metadata, faiss_index

    if metadatas is None:
        metadatas = [None] * len(texts)

    prefixed_texts = ["passage: " + t for t in texts]
    embeddings = embed_model.encode(prefixed_texts, convert_to_numpy=True,
    ↪ show_progress_bar=False)

    faiss.normalize_L2(embeddings)
```



```

corpus_texts.extend(texts)
corpus_metadata.extend(metadata)

faiss_index.add(embeddings)

print(f"Dodano {len(texts)} dokumenata u RAG korpus. Ukupno:␣
↪{len(corpus_texts)}")

```

```

[ ]: def extract_text_from_docx(file_path):
    try:
        doc = Document(file_path)
        text = '\n'.join([para.text for para in doc.paragraphs])
        return text
    except Exception as e:
        print(f"Greška pri čitanju .docx: {e}")
        return None

def extract_text_from_pdf(file_path):
    try:
        text = ""
        with pdfplumber.open(file_path) as pdf:
            for page in pdf.pages:
                page_text = page.extract_text()
                if page_text:
                    text += page_text + "\n"
        return text
    except Exception as e:
        print(f"Greška pri čitanju PDF-a: {e}")
        return None

def chunk_by_articles(text, doc_name, max_chunk=1200, min_chunk_len=50):
    article_pattern = r'(Članak\s+\d+[a-z]?\. )'
    parts = re.split(article_pattern, text)

    if len(parts) <= 1:
        return chunk_text_with_overlap(text, chunk_size=800, overlap=200,␣
↪min_chunk_len=min_chunk_len)

    chunks = []

    preamble = parts[0].strip()
    if len(preamble) >= min_chunk_len:
        if len(preamble) <= max_chunk:
            chunks.append(f"[{doc_name}] {preamble}")
        else:

```

```

        preamble_chunks = chunk_text_with_overlap(preamble, chunk_size=800,
↪overlap=200, min_chunk_len=min_chunk_len)
        chunks.extend([f"[{doc_name}] {pc}" for pc in preamble_chunks])

    for i in range(1, len(parts), 2):
        header = parts[i].strip()
        body = parts[i + 1].strip() if i + 1 < len(parts) else ""
        article_text = f"{header}\n{body}"

        if len(article_text) + len(doc_name) + 3 <= max_chunk:
            if len(article_text) >= min_chunk_len:
                chunks.append(f"[{doc_name}] {article_text}")
            else:
                sub_parts = re.split(r'\n(?:\(\d+\)|\-\s|\-\s)', body)
                current_sub = f"{header}"

                for sp in sub_parts:
                    sp = sp.strip()
                    if not sp:
                        continue
                    if len(current_sub) + len(sp) + len(doc_name) + 4 <= max_chunk:
                        current_sub += "\n" + sp
                    else:
                        if len(current_sub) >= min_chunk_len:
                            chunks.append(f"[{doc_name}] {current_sub}")
                        current_sub = f"{header} (nastavak)\n{sp}"

                if len(current_sub) >= min_chunk_len:
                    chunks.append(f"[{doc_name}] {current_sub}")

    return chunks

def chunk_text_with_overlap(text, chunk_size=800, overlap=200,
↪min_chunk_len=50):
    paragraphs = text.split('\n\n')
    paragraphs = [p.strip() for p in paragraphs if p.strip()]

    split_paragraphs = []
    for para in paragraphs:
        if len(para) <= chunk_size:
            split_paragraphs.append(para)
        else:
            start = 0
            while start < len(para):
                end = start + chunk_size
                if end >= len(para):
                    split_paragraphs.append(para[start:])

```

```

        break
    space_idx = para.rfind(' ', start, end)
    if space_idx > start:
        split_paragraphs.append(para[start:space_idx])
        start = space_idx + 1
    else:
        split_paragraphs.append(para[start:end])
        start = end

chunks = []
current_chunk = ""

for para in split_paragraphs:
    if len(current_chunk) + len(para) + 2 <= chunk_size:
        current_chunk = current_chunk + "\n\n" + para if current_chunk else para
    else:
        if len(current_chunk) >= min_chunk_len:
            chunks.append(current_chunk.strip())
        if overlap > 0 and current_chunk:
            overlap_text = current_chunk[-overlap:]
            space_idx = overlap_text.find(' ')
            if space_idx != -1:
                overlap_text = overlap_text[space_idx + 1:]
            current_chunk = overlap_text + "\n\n" + para
        else:
            current_chunk = para

if current_chunk and len(current_chunk.strip()) >= min_chunk_len:
    chunks.append(current_chunk.strip())

return chunks

def detect_document_hierarchy(filename):
    filename_lower = filename.lower()
    if 'ustav' in filename_lower:
        return {"razina": 1, "tip_propisa": "Ustav"}
    elif 'zakon' in filename_lower:
        return {"razina": 2, "tip_propisa": "Zakon"}
    elif 'pravilnik' in filename_lower:
        return {"razina": 3, "tip_propisa": "Pravilnik"}
    else:
        return {"razina": 4, "tip_propisa": "Ostalo"}

def get_document_display_name(filename):
    name = os.path.splitext(filename)[0]
    name = name.replace('_', ' ')

```

```
return name
```

0.3 Učitavanje podataka i indeksiranje

```
[ ]: try:
    with open('/content/drive/My Drive/diplomskiRad/pitanja_odgovori.json',
        ↪'r', encoding='utf-8') as f:
        qa_data = json.load(f)

    print(f" Učitano {qa_data['metadata']['ukupno_pitanja']} pitanja i
    ↪odgovora.")
    print(f" Struktura:")
    for kategorija, info in qa_data['metadata']['struktura'].items():
        print(f" - {kategorija}: {info['broj']} ({info['postotak']}")
except FileNotFoundError:
    print('Datoteka pitanja_odgovori.json nije pronađena na navedenoj putanji.')
    qa_data = None
except json.JSONDecodeError:
    print('Greška pri parsiranju JSON datoteke. Provjerite je li format
    ↪ispravan.')
    qa_data = None
```

Učitano 31 pitanja i odgovora.

Struktura:

- kompleksna_vise_dokumenata: 3 (9.67%)
- kompleksna_jedan_dokument: 5 (16.13%)
- jednostavna: 23 (74.2%)

```
[ ]: docs_dir = '/content/drive/My Drive/diplomskiRad/Dokumenti'

if os.path.exists(docs_dir):
    print(f"Učitavam dokumente iz mape: '{docs_dir}'...\n")

    for file in os.listdir(docs_dir):
        file_path = os.path.join(docs_dir, file)
        text = None

        try:
            if file.endswith('.txt'):
                with open(file_path, 'r', encoding='utf-8') as f:
                    text = f.read()
                print(f" .txt: {file}")
            elif file.endswith('.docx'):
                text = extract_text_from_docx(file_path)
                if text:
                    print(f" .docx: {file}")
            elif file.endswith('.doc'):
```

```

        print(f" .doc: {file} (zahtijeva LibreOffice ili specifičnu
↳biblioteku)")
    elif file.endswith('.pdf'):
        text = extract_text_from_pdf(file_path)
        if text:
            print(f" PDF: {file}")
        else:
            print(f" Nepoznat format: {file}")

    if text:
        hierarchy = detect_document_hierarchy(file)
        doc_name = get_document_display_name(file)

        chunks = chunk_by_articles(text, doc_name, max_chunk=1200,
↳min_chunk_len=50)

        if chunks:
            metadatas = [{
                "file": file,
                "type": file.split('.')[-1],
                "razina": hierarchy["razina"],
                "tip_propisa": hierarchy["tip_propisa"]
            }] * len(chunks)

            add_documents_to_corpus(chunks, metadatas)
            print(f"    Dodano {len(chunks)} chunkova (razina:
↳{hierarchy['tip_propisa']})\n")
        except Exception as e:
            print(f" Greška pri obradi {file}: {e}\n")
    else:
        print(f"Mapa '{docs_dir}' nije pronađena!")
        print(f"Dostupne stavke: {os.listdir('.')}")

```

Učitavam dokumente iz mape: '/content/drive/My Drive/diplomskiRad/Dokumenti'...

PDF: Pravilnik o temeljnom vojnom osposobljavanju.pdf
 Dodano 24 dokumenata u RAG korpus. Ukupno: 24
 Dodano 24 chunkova (razina: Pravilnik)

.docx: Ustav Republike Hrvatske 2018.docx
 Dodano 173 dokumenata u RAG korpus. Ukupno: 197
 Dodano 173 chunkova (razina: Ustav)

.docx: Zakon_o_obrani_2025.docx
 Dodano 251 dokumenata u RAG korpus. Ukupno: 448
 Dodano 251 chunkova (razina: Zakon)

.docx: Zakon_o_sluzbi_u_Oruzanim_snagama_Republike_Hrvatske_2025.docx
Dodano 373 dokumenata u RAG korpus. Ukupno: 821
Dodano 373 chunkova (razina: Zakon)

```
[ ]: print(f"\nStatistika RAG korpusa:")
print(f" Ukupno učitano {len(corpus_texts)} dokumenata za RAG.")
print(f" FAISS index sadrži {faiss_index.ntotal} dokumenata.")

if corpus_texts:
    print("\nPrimjeri učitanih dokumenata (prva 3):")
    for i, doc in enumerate(corpus_texts[:3], 1):
        preview = doc[:150] + "..." if len(doc) > 150 else doc
        print(f"\n    [{i}] {preview}")
else:
    print("\nNema učitanih dokumenata! Provjerite mapu 'Dokumenti'.")
```

Statistika RAG korpusa:
Ukupno učitano 821 dokumenata za RAG.
FAISS index sadrži 821 dokumenata.

Primjeri učitanih dokumenata (prva 3):

[1] [Pravilnik o temeljnom vojnom osposobljavanju] NN 155/2025 (23.12.2025.),
Pravilnik o temeljnom vojnom osposobljavanju
MINISTARSTVO OBRANE
2328
Na tem...

[2] [Pravilnik o temeljnom vojnom osposobljavanju] Članak 1.
(1) Ovim se Pravilnikom propisuje postupak prijave i evidencije kandidata i
upućivanje kandid...

[3] [Pravilnik o temeljnom vojnom osposobljavanju] Članak 2.
Pojmovi koji se koriste u ovom Pravilniku imaju sljedeće značenje:
- kandidat je novak i osob...

0.4 RAG sustav — dohvaćanje i generacija

```
[ ]: import math

def retrieve_relevant_passages(query, top_k=5, score_threshold=0.30,
                               rerank_top_k=None, per_doc_cap=None,
                               debug=False):
    if len(corpus_texts) == 0:
        return []
```

```

fetch_k = min(max(top_k * 5, 50), len(corpus_texts))

query_embedding = embed_model.encode(["query: " + query],
↪convert_to_numpy=True, show_progress_bar=False)
faiss.normalize_L2(query_embedding)

D, I = faiss_index.search(query_embedding, fetch_k)

candidates = []
for score, idx in zip(D[0], I[0]):
    if idx == -1:
        continue
    if score < score_threshold:
        continue
    candidates.append({
        "text": corpus_texts[idx],
        "score": float(score),
        "metadata": corpus_metadata[idx]
    })

for c in candidates:
    if c["metadata"] is not None and "razina" in c["metadata"]:
        razina = c["metadata"]["razina"]
        hierarchy_boost = max(0, (4 - razina)) * 0.02
        c["score"] = c["score"] + hierarchy_boost

candidates.sort(key=lambda x: x["score"], reverse=True)

def _src(meta):
    return meta.get("file") if isinstance(meta, dict) else None

unique_results = []
for c in candidates:
    c_words = set(c["text"].split())
    is_duplicate = False
    for existing in unique_results:
        if _src(c["metadata"]) != _src(existing["metadata"]):
            continue
        existing_words = set(existing["text"].split())
        intersection = len(c_words & existing_words)
        union = len(c_words | existing_words)
        if union > 0 and intersection / union > 0.92:
            is_duplicate = True
            break
    if not is_duplicate:
        unique_results.append(c)

```

```

final_k = rerank_top_k if rerank_top_k else top_k

if per_doc_cap is None:
    per_doc_cap = max(1, math.ceil(final_k / 2))

selected, po_zakonu, preostali = [], {}, []
for c in unique_results:
    src = _src(c["metadata"])
    po_zakonu.setdefault(src, 0)
    if po_zakonu[src] < per_doc_cap:
        selected.append(c)
        po_zakonu[src] += 1
    else:
        preostali.append(c)
    if len(selected) >= final_k:
        break

if len(selected) < final_k:
    for c in preostali:
        selected.append(c)
        if len(selected) >= final_k:
            break

if debug:
    raspodjela = {}
    for c in selected:
        src = _src(c["metadata"]) or "?"
        raspodjela[src] = raspodjela.get(src, 0) + 1
    print(f" [retrieval] pool={fetch_k} kandidata={len(candidates)} "
          f"jedinstveni={len(unique_results)} -> {len(selected)} u_
↳kontekstu")
    print(f" [retrieval] po zakonu: {raspodjela}")

return selected[:final_k]

```

```

[ ]: def generate_response(
    messages,
    max_new_tokens=512,
    temperature=0.5,
    do_sample=True,
    use_rag=False,
    rag_top_k=3
):
    final_messages = list(messages)

    if use_rag:
        user_query = ""

```



```

for m in reversed(final_messages):
    if m["role"] == "user":
        user_query = m["content"]
        break

if user_query:
    retrieved = retrieve_relevant_passages(user_query, top_k=rag_top_k)

    if len(retrieved) > 0:
        context_parts = []
        for i, r in enumerate(retrieved, start=1):
            meta_str = ""
            if r["metadata"] is not None and "file" in r["metadata"]:
                meta_str = f" (izvor: {r['metadata']['file']})"
            context_parts.append(f"[{i}] {r['text']}{meta_str}")

        context_text = "\n\n".join(context_parts)

        if final_messages and final_messages[0]["role"] == "system":
            final_messages[0]["content"] = (
                final_messages[0]["content"] + "\n\n"
                "Kontekst za odgovor:\n"
                f"{context_text}"
            )
        else:
            final_messages.insert(0, {
                "role": "system",
                "content": SYSTEM_PROMPT + f"\n\nKontekst za odgovor:
↪\n{context_text}",
            })

model_max_length = getattr(pipe.tokenizer, 'model_max_length', 8192)
if model_max_length > 100000:
    model_max_length = 8192

input_text = pipe.tokenizer.apply_chat_template(final_messages,
↪tokenize=False)
input_token_count = len(pipe.tokenizer.encode(input_text))
max_allowed_input = model_max_length - max_new_tokens

if input_token_count > max_allowed_input:
    print(f"Upozorenje: input ({input_token_count} tokena) prelazi limit
↪({max_allowed_input}). Skraćujem kontekst.")
    if use_rag and final_messages and final_messages[0]["role"] == "system":
        system_content = final_messages[0]["content"]
        while input_token_count > max_allowed_input and "\n\n[" in
↪system_content:

```

```

        last_chunk_start = system_content.rfind("\n\n")
        system_content = system_content[:last_chunk_start]
        final_messages[0]["content"] = system_content
        input_text = pipe.tokenizer.apply_chat_template(final_messages,
↳tokenize=False)

        input_token_count = len(pipe.tokenizer.encode(input_text))
        print(f" Skraćeno na {input_token_count} tokena.")

    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    gen_config = deepcopy(pipe.model.generation_config)
    gen_config.max_new_tokens = max_new_tokens
    gen_config.temperature = temperature
    gen_config.do_sample = do_sample
    gen_config.repetition_penalty = 1.1

    vrijeme_upita = time.time()

    response = pipe(
        final_messages,
        generation_config=gen_config,
    )

    answer = ""
    for message in response[0]['generated_text']:
        if message['role'] == 'assistant':
            answer = message['content']
            break

    stripped_answer = answer.strip()
    if not re.search(r'[.!?]$', stripped_answer) and len(stripped_answer.
↳split()) > 5:
        last_punctuation_index = -1
        for i in range(len(stripped_answer) - 1, -1, -1):
            if stripped_answer[i] in [ '.', '!', '?']:
                last_punctuation_index = i
                break

        if last_punctuation_index != -1:
            answer = stripped_answer[:last_punctuation_index + 1]
        else:
            words = stripped_answer.split()
            if len(words) > 1:
                answer = ' '.join(words[:-1]) + '...'
            elif words:
                answer = words[0]

```

```

        else:
            answer = ''

    else:
        answer = stripped_answer

    vrijeme_odgovora = time.time()
    ukupno_vrijeme = vrijeme_odgovora - vrijeme_upita

    print("Generirani odgovor:")
    print(answer)
    print(f"Ukupno vrijeme za odgovor: {ukupno_vrijeme:.2f} sekundi")

    return answer, ukupno_vrijeme

```

```

[ ]: SYSTEM_PROMPT = (
    "Ti si pravni stručnjak specijaliziran za zakonodavstvo Republike Hrvatske,
    ↳posebno za Zakon o obrani, Zakon o službi u Oružanim snagama RH i
    ↳pripadajuće pravilnike."
    "\n\nPravila odgovaranja:"
    "\n1. Jezik odgovora: hrvatski, latinica."
    "\n2. Navedi konkretne članke zakona, brojčane vrijednosti i nadležna tijela."
    "\n3. Ako je dostupan kontekst, koristi SAMO informacije iz njega."
    "\n4. Ako kontekst ne sadržava odgovor, odgovori na temelju općeg znanja i
    ↳dodaj: 'Napomena: ova informacija nije pronađena u dostupnim dokumentima.'"
    "\n5. Odgovor neka bude koncizan i strukturiran - bez nepotrebnih uvoda."
)

```

```

[ ]: csv_path = '/content/drive/My Drive/diplomskiRad/rezultati.csv'

```

0.5 Provedba - RAG pristup

```

[ ]: RAG_TOP_K = {
    'jednostavno': 5,
    'kompleksno_jedan_dokument': 8,
    'kompleksno_vise_dokumenata': 12,
}
RAG_TOP_K_DEFAULT = 5

RAG_MAX_TOKENS = {
    'jednostavno': 384,
    'kompleksno_jedan_dokument': 640,
    'kompleksno_vise_dokumenata': 1024,
}

def rag_chat(user_question, kategorija=None):
    top_k = RAG_TOP_K.get(kategorija, RAG_TOP_K_DEFAULT)
    max_tok = RAG_MAX_TOKENS.get(kategorija, 512)

```

```

messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": f"{user_question}\n\nOdgovori NA HRVATSKOM_
↪JEZIKU koriste\u0107i LATINICU."}
]
return generate_response(messages, max_new_tokens=max_tok, temperature=0.3,
                           do_sample=False, use_rag=True, rag_top_k=top_k)

```

```

[ ]: if qa_data is None:
    print("QA datoteka nije učitana.")
else:
    df_rezultati = pd.read_csv(csv_path, encoding='utf-8-sig')

    KATEGORIJA_MAP = {
        'jednostavna_pitanja': 'jednostavno',
        'kompleksna_pitanja_jedan_dokument': 'kompleksno_jedan_dokument',
        'kompleksna_pitanja_vise_dokumenata': 'kompleksno_vise_dokumenata',
    }

    all_questions = []
    all_categories = []
    for key, value in qa_data.items():
        if key != 'metadata' and isinstance(value, list):
            for item in value:
                all_questions.append(item)
                all_categories.append(KATEGORIJA_MAP.get(key, key))

    rag_odgovori = []
    rag_vremena = []

    for i, (qa_item, kategorija) in enumerate(zip(all_questions,
↪all_categories)):
        user_question = qa_item.get('pitanje', qa_item.get('question', ''))

        print(f"\n{'='*80}")
        print(f"Pitanje {i + 1}/{len(all_questions)}: {user_question}")
        print(f"Kategorija: {kategorija} (top_k={RAG_TOP_K.get(kategorija,
↪RAG_TOP_K_DEFAULT)})")
        print(f"{'='*80}")

        answer, vrijeme = rag_chat(user_question, kategorija=kategorija)

        rag_odgovori.append(answer)
        rag_vremena.append(round(vrijeme, 2))

    df_rezultati['rag_odgovor'] = rag_odgovori
    df_rezultati['rag_vrijeme_sekunde'] = rag_vremena

```

```
df_rezultati.to_csv(csv_path, index=False, encoding='utf-8-sig')
print(f"\n{'='*80}")
print(f"RAG rezultati nadopunjeni u: {csv_path}")
print(f"Ukupno obrađeno pitanja: {len(rag_odgovori)}")
print(f"Prosječno vrijeme po pitanju: {sum(rag_vremena)/len(rag_vremena):.
↵2f} sekundi")
```

```
=====
Pitanje 1/31: Opišite cjelokupnu proceduru transformacije novaka u razvrstanog
pričuvnika kroz sve pravne akte koji reguliraju taj proces.
```

```
Kategorija: kompleksno_vise_dokumenata (top_k=12)
=====
```

Generirani odgovor:

Transformacija novaka u razvrstanog pričuvnika u Republici Hrvatskoj regulirana je Zakonom o obrani („Narodne novine“ br. 136/25), a procedura uključuje sljedeće korake:

1. Prihvata u vojnu evidenciju

Prema **čl. 21.** Zakona o obrani, **novak koji boravi u inozemstvu** dužan je prijaviti se diplomatskoj misiji ili konzularnom uredu RH ako želi regulirati svoju vojnu obvezu. To se odnosi na one koji nisu bili uvedeni u vojnu evidenciju u RH.

- **Čl. 21. stavak 1.:** Zahtijeva prijavu diplomatskoj misiji do 18. godine života.
- **Čl. 21. stavak 2.:** Dopusštena je prijava do 29. godine života ako se obveznik ranije nije uključio u sustav.
- **Čl. 21. stavak 3.:** Potreban je dokaz o izvršenju vojne obaveze u inozemstvu.

Dokumentacija se prosljeđuje Ministarstvu obrane putem diplomatskih kanala.

2. Ocjena sposobnosti i razvrstavanje

Nakon uvođenja u vojnu evidenciju, obveznik prolazi medicinsku procjenu. Prema **čl. 25.** Zakona o obrani:

- **Čl. 25. d (1):** Promjene u zdravstvenom stanju moraju se prijaviti Ministarstvu obrane unutar 8 dana.
- **Čl. 25. d (2):** Medicinska komisija ocjenjuje sposobnost za vojnu službu.
- **Čl. 25. d (3):** Ako nedostaje dokumentacija, traži se dodatna provjera.
- **Čl. 25. d (4):** Rezultat se unosi u vojnu iskaznicu.

Ako je sposoban, obveznik postaje **razvrstan pripašnjak** prema **čl. 25. k** istog zakona.

3. Dodatni uvjeti za razvrstavanje

Za razvrstavanje u pripašnjake vrijede dodatni kriteriji:

- **Čl. 25. m:** Oni koji su već služili kao **civilni ročnici** također ulaze u sustav.
- **Čl. 25. j:** Ako se novak prije civilne službe odluči za vojnu karijeru, prelazi na temeljno vojno osposobljavanje.

4. Pravna osnova i nadležna tijela

- **Zakon o obrani (NN 136/25) - čl. 21.-25.:** Regulira prijavu, medicinski pregled i razvrstavanje.
- **Zakon o službi u Oružanim snagama (NN 73/13, 75/15) - čl. 32.:** Ministarstvo obrane vodi evidenciju ročnika i pričuvnika.
- **Glavni stožer OSRH:** Donosi programe osposobljavanja (npr. **čl. 25. c**).

Procedura je zaključena kada Ministarstvo obrane izda rješenje o razvrstavanju, a obveznik primi nalog za službu.

Ukupno vrijeme za odgovor: 100.09 sekundi

=====
Pitanje 2/31: Kako je regulirana hijerarhija odlučivanja o uporabi Oružanih snaga u različitim situacijama kroz Ustav i Zakon o obrani?
Kategorija: kompleksno_vise_dokumenata (top_k=12)
=====

Generirani odgovor:

Prema Ustavu Republike Hrvatske (Članak 100.) i Zakonu o obrani (Članak 46.), hijerarhija odlučivanja o uporabi Oružanih snaga u različitim situacijama je sljedeća:

1. **U ratu** - Prema Ustavu (Članak 100., stavak 4.) i Zakonu o obrani (Članak 47.), **predsjednik republike** naređuje uporabu Oružanih snaga i zapovijeda njima u ratnom stanju.

2. **U stanju neposredne ugroženosti** - Prema Ustavu (Članak 100., stavak 3.) i Zakonu o obrani (Članak 48.), **predsjednik republike** može, uz supotpis predsjednika vlade, narediti uporabu Oružanih snaga čak i ako nije proglašeno ratno stanje.

3. **U ostalim slučajevima** - Prema Zakonu o obrani (Članak 46.), odluku donose **Hrvatski sabor, predsjednik republike, predsjednik vlade i ministar obrane** zajedno, ovisno o situaciji.

Također, prema Zakonu o obrani (Članak 46., stavak 3.), **zapovijedi načelnika Glavnog stožera** provode odluke o uporabi Oružanih snaga.

Napomena: Ova struktura vrijedi osim u izvanrednim situacijama kada predsjednik republike ima šire ovlasti po Ustavu (Članak 100.).

Ukupno vrijeme za odgovor: 42.59 sekundi

=====
Pitanje 3/31: Objasnite sve oblike obveza državljana u obrani prema Zakonu o obrani i Ustavu.

Kategorija: kompleksno_vise_dokumenata (top_k=12)
=====

Generirani odgovor:

Prema Zakonu o obrani i Ustavu Republike Hrvatske, građani imaju različite oblike obveza u obrani:

1. ****Vojna obveza**** (Zakon o obrani, članak 18., 19.):
 - Dužnost svih sposobnih građana od 18. godine života.
 - Obuhvaća osposobljavanje, pripremu i sudjelovanje u obrani.
2. ****Služenje u ugovornoj pričuvi**** (Zakon o obrani, članak 18.):
 - Građani sklapaju ugovor s Oružanim snagama za služenje u pričuvi.
3. ****Radna obveza**** (Zakon o obrani, članak 18.):
 - Građani pružaju radnu potporu za potrebe obrane.
4. ****Materijalna obveza**** (Zakon o obrani, članak 18.):
 - Građani doprinosu financiranju obrane poreznim i drugim davanjima.
5. ****Ostala prava i obveze**** (Zakon o obrani, članak 18.):
 - Inkluzivna kategorija koja može uključivati dodatne oblike suradnje.
6. ****Pripadnici oružanih snaga**** (Zakon o službi u Oružanim snagama, članak 3.):
 - Djelatne vojne osobe, pričuvnici, kadeti i ročnici.
7. ****Priigovor savjesti**** (Ustav RH, članak 47.):
 - Građani mogu odbiti vojnu službu zbog vjerskih ili moralnih uvjerenja, ali moraju ispunjavati alternativne obveze.
8. ****Radna pomoć u ratu**** (Zakon o obrani, članak 31.):
 - U izvanrednim situacijama građani mogu biti pozvani na rad za obranu.

Ove obveze su definirane kako bi osigurale nacionalnu sigurnost i sukladne su ustavnom načelu da je obrana dužnost svakog građanina (Ustav, članak 47.).

Ukupno vrijeme za odgovor: 62.49 sekundi

=====
Pitanje 4/31: Objasnite sustav imenovanja i odgovornosti načelnika Glavnog stožera kroz sve relevantne propise.

Kategorija: kompleksno_jedan_dokument (top_k=8)

=====
Generirani odgovor:

Sustav imenovanja i odgovornosti **načelnika Glavnog stožera** uređen je sljedećim propisima:

1. Imenovanje načelnika Glavnog stožera

- Prema **Članku 16. stavku 4. Zakona o obrani (2025.)**, **načelnik Glavnog stožera** imenuje se na razdoblje od četiri godine i ne može biti ponovno imenovan.
- **Članak 123.** istog zakona nalaže da osoba koja je na dužnosti načelnika Glavnog stožera na dan stupanja na snagu Zakona ostaje na dužnosti do isteka svog mandata.
- **Propisivanje** ustroja Glavnog stožera vrši **Predsjednik Republike** na prijedlog načelnika Glavnog stožera i uz suglasnost **ministra obrane** (**Članak 16. stavak 5.**).

2. Nadležnosti i odgovornosti načelnika Glavnog stožera

- Prema **Članku 70. stavku 1.**, načelnik Glavnog stožera ima **punu ovlast** zapovijedanja u **Oružanim snagama** temeljem odluka **Predsjednika Republike** i **ministra obrane**.
- On je **odgovoran Predsjedniku Republike** za provedbu zapovijedi i **ministru obrane** za provedbu odluka (**Članak 70. stavci 2. i 3.**).
- Prema **Članku 70. stavku 4.**, načelnici Glavnog stožera mogu **privremeno** staviti postrojbe u pripravnost u slučaju opasnosti (npr. katastrofe), a o tome moraju odmah izvijestiti predsjednika i ministra.
- Prema **Članku 50. stavku 6.**, ministar obrane na prijedlog načelnika Glavnog stožera propisuje **uvjete i postupak raspoređivanja vojnih osoba** na dužnosti.

3. Pravosudni nadzor i ustavna osnova

- Iako **Zakon o obrani** ne sadrži direktnu poveznicu s Ustavom, prema **Članku 103.** Ustava, **Predsjednik** i **Vlada** surađuju u sigurnosnoj politici, što uključuje i imenovanje visokih vojnih dužnosnika.
- Prema **Članku 111.** Ustava, odluke Vlade (uključujući i one vezane za sigurnost) moraju biti usklađene sa zakonom.

Ukupno vrijeme za odgovor: 78.53 sekundi

=====
Pitanje 5/31: Objasnite razlike između uporabe i korištenja Oružanih snaga prema Zakonu o obrani, uključujući tko donosi odluke u svakom slučaju.

Kategorija: kompleksno_jedan_dokument (top_k=8)
=====

Generirani odgovor:

Prema **Zakonu o obrani** (2025), postoje jasne razlike između **korištenja** i **upotrebe** Oružanih snaga, kao i različita tijela koja donose odluke u svakom slučaju.

1. Korišćenje Oružanih snaga

- **Definicija**: Prema **Čl. 46.** Zakona o obrani, korišćenjem se podrazumijeva aktivno angažiranje snaga u operacijama, uključujući i izvan ratnog stanja.
- **Tijela koja donose odluke**:
 - **Sabor** (donosi strateške odluke),
 - **Predsjednik RH** (vrhovni zapovjednik, vidi **Ustav, Čl. 100.**),
 - **Vlada** (ministar obrane),
 - **Glavni stožer** (provođenje odluka).
- **Mehanizam donošenja odluka**: Odluke se donose na temelju Ustava i Zakona o obrani, a provode se preko zapovijedi Glavnog stožera.

2. Uporaba Oružanih snaga

- **Definicija**: Prema **Čl. 47.** Zakona o obrani, upotrebom se podrazumijeva aktivno djelovanje snaga u situacijama neposredne ugroženosti (npr. teroristički napad, invazija).
- **Tijela koja donose odluke**:
 - **Predsjednik RH** ima pravo naredio upotrebu čak i bez proglašenog rata (vidi **Ustav, Čl. 100.**),
 - **Vlada** i **ministar obrane** također sudjeluju u donošenju odluka.
- **Posebnost**: Ako postoji neposredna opasnost, Predsjednik RH može samostalno reagirati (supotpis Vlade nije potreban).

Ključna razlika

- **Korišćenje** uključuje širi spektar aktivnosti (operacije, misije, obuka) i zahtijeva višestruku sankciju (Sabor, Vlada, Predsjednik).
- **Upotreba** odnosi se na hitne slučajeve ugroze i omogućuje brzu reakciju najviših autoriteta (Predsjednik RH).

Izvor: **Zakon o obrani 2025, Čl. 46.-47.** + **Ustav RH, Čl. 100.**

Ukupno vrijeme za odgovor: 71.73 sekundi

Pitanje 6/31: Koja su prava i obveze ročnika tijekom temeljnog vojnog osposobljavanja prema Pravilniku o temeljnom vojnom osposobljavanju?

Kategorija: kompleksno_jedan_dokument (top_k=8)

Generirani odgovor:

Prema **Pravilniku o temeljnom vojnom osposobljavanju** (Članak 18.), prava i obveze ročnika tijekom temeljnog vojnog osposobljavanja uključuju sljedeće:

Prava ročnika

- **Plaća ili naknada** - u iznosu određenom zakonom koji uređuje službu u Oružanim snagama.
- **Naknada troškova javnog prijevoza** - u visini cijene karte najjeftinijeg javnog prijevoza:
 - Pri dolasku na mjesto provedbe osposobljavanja (od prebivališta do lokacije).

- Pri povratku (od lokacije do prebivališta).
- ****Osnovno i dopunsko zdravstveno osiguranje****.
- ****Osiguranje od posljedica nesretnog slučaja****.
- ****Obvezno zdravstveno osiguranje za slučaj ozljede na radu****.
- ****Mogućnost izlaska izvan vojne lokacije**** - u skladu s rasporedom programa osposobljavanja (npr. vikendevi, slobodni dani).
- ****Slobodno vrijeme**** - unutar propisanog raspona.
- ****Smještaj i prehrana**** - osigurani po standardima vojske.
- ****Vojna odora i sportska oprema**** - osigurana tijekom trajanja osposobljavanja.

****Obveze ročnika:****

- Svršavanje svih zadataka i aktivnosti definiranih programom osposobljavanja.
- Poštivanje propisa i naredaba nadležnih časnika.
- Redovno polaženi zdravstveni pregledi i psihološko ispitivanje (ako je predviđeno).

Izvor: ****Pravilnik o temeljnom vojnom osposobljavanju****, Članak 18. (NN 136/25), dostupno na [pravilnik.hr](https://www.pravilnik.hr).

Ukupno vrijeme za odgovor: 57.81 sekundi

=====
Pitanje 7/31: Objasnite razlike između mirnodopskog i ratnog sastava Oružanih snaga prema Zakonu o službi.

Kategorija: kompleksno_jedan_dokument (top_k=8)

Generirani odgovor:

Prema ****Zakonu o službi u Oružanim snagama Republike Hrvatske**** (Članak 25.), razlike između mirnodopskog i ratnog sastava Oružanih snaga jesu sljedeće:

****Mirnodopski sastav****

- Sastoji se od:
 - ***Djelatnih vojnih osoba*** (aktivni časnici, dočasnici, vojnici),
 - ***Službenika i namještenika*** raspoređenih u OSRH,
 - ***Pričajeva*** pozvanih na obuku,
 - ***Ugovornih pričavnika***,
 - ***Kadeta*** i
 - ***Ronilaca*** koji su završili temeljno vojno osposobljavanje.
- Prema ****Zakonu o obrani**** (Članak 43., stavak 2.), dodatni elementi uključuju ***državne službenike i namještenike*** raspoređene u OSRH.

****Ratni sastav****

- Uz sve elemente iz mirnodopskog sastava, uključuje još i:
 - ***Vojne obveznike*** mobilizirane u OSRH (po ****Zakonu o službi****, Članak 25., stavak 3.).
- Prema ****Zakonu o obrani**** (Članak 45.), službena upotreba je hrvatski jezik i latinica.

Nadležno tijelo

- **Predsjednik Republike** određuje mirnodopski i ratni ustroj OSRH (prema **Zakonu o službi**, Članak 44.) te potvrđuje strukturu činova.

Napomena: Informacije su preuzete iz navedenih članaka zakona. Za detaljnije informacije preporučuje se konzultiranje nadležnih tijela (npr. MORH).

Ukupno vrijeme za odgovor: 53.50 sekundi

=====
Pitanje 8/31: Objasnite postupak i uvjete za odgodu temeljnog vojnog osposobljavanja prema Zakonu o obrani.

Kategorija: kompleksno_jedan_dokument (top_k=8)
=====

Generirani odgovor:

Prema **Zakonu o obrani** (NN 136/25), postupak i uvjeti za odgodu temeljnog vojnog osposobljavanja regulirani su u **Članku 21.**

Postupak:

1. **Podnošenje zahtjeva:**

- Zahtjev za odgodu podnosi se **nadležnom područnom odjelu za poslove obrane** u roku od **osam dana** od dana primitka poziva za temeljno vojno osposobljavanje.
- Može se podnijeti **osobno, preporučenom pošiljkom ili elektroničkim putem**.

2. **Potrebni prilozi:**

- Dokaz o **nezaposlenosti ostalih članova obitelji** (npr. potvrde o zapošljavanju).
- Ugovor o radu / rješenje o radnom odnosu / potvrda o osiguranju (ako se odgoda traži zbog zaposlenja kao vježbenik).
- Potvrda nadležnog tijela za socijalnu skrb (ako su u kućanstvu ovisni članovi).
- Izvadak iz matice umrlih (ako je došlo do smrti u obitelji).
- Potvrda o registraciji obrta / djelatnosti (ako je otvoren obrt).

Uvjeti za odgodu:

- **Smrt u užoj obitelji** - odgoda do sljedećeg roka za upućivanje.
- **Otvaranje obrta / samostalne djelatnosti** - odgoda do godinu dana od registracije.
- **Zapošljavanje kao vježbenik / prvo zaposlenje** - odgoda do završetka vježbeničkog staža / isteka probnog rada.
- **Dijete u kućanstvu** - odgoda do godinu dana od dolaska na osposobljavanje.
- **Bračni savez** - odgoda do tri mjeseca.
- **Trudnoća / rađanje djeteta** - odgoda do godinu dana.
- **Visoko obrazovanje** - odgoda ako je student, do 30. rujna godine kad navrší 29 godina.

- **Sportska natjecanja** - odgoda do godinu dana, ako postoji zahtjev Hrvatskog olimpijskog odbora.

Odluka:

Nadležno tijelo donosi odluku u roku od osam dana od podnošenja zahtjeva.

Napomena: Informacije su preuzete iz dostupnih dijelova Zakona o obrani (NN 136/25). Za detaljnije informacije preporučuje se konzultiranje cjelovitih tekstova zakona.

Ukupno vrijeme za odgovor: 74.68 sekundi

=====
Pitanje 9/31: Kada nastaje vojna obveza za državljane Republike Hrvatske?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Vojna obveza za državljane Republike Hrvatske **nastupa u kalendarskoj godini u kojoj navršavaju 18 godina života**, prema članku 19. Zakona o obrani (NN 73/13, 30/18, 136/25).

Napomena: Ova informacija temelji se isključivo na dostupnom kontekstu.

Ukupno vrijeme za odgovor: 10.75 sekundi

=====
Pitanje 10/31: Koliko traje temeljno vojno osposobljavanje?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Temeljno vojno osposobljavanje traje dva mjeseca. (izvor: Zakon_o_obrani_2025.

Ukupno vrijeme za odgovor: 4.42 sekundi

=====
Pitanje 11/31: Tko je vrhovni zapovjednik Oružanih snaga Republike Hrvatske?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Prema **Članku 100.** **Ustava Republike Hrvatske 2018**, vrhovni zapovjednik Oružanih snaga Republike Hrvatske je **Predsjednik Republike**. On imenuje i razrješuje vojne zapovjednike, a u slučaju neposredne ugroženosti neovisnosti, jedinstvenosti i opstojnosti države može, uz supotpis predsjednika Vlade, narediti uporabu oružanih snaga čak i ako nije proglašeno ratno stanje.

Detaljnije, prema **Zakonu o obrani 2025**, **Članak 47.** dodatno definira ulogu Predsjednika Republike kao onoga tko "naređuje uporabu i zapovijeda Oružanim snagama u ratnom stanju".

Izvor:

- [Ustav Republike Hrvatske 2018] (<https://www.hins.hr/>) - **Članak 100.**
- [Zakon o obrani 2025] (<https://pravosudni.portal.hr/>) - **Članak 45., 46., 47.**

Napomena: Ova informacija temelji se isključivo na dostupnim ustavnim i zakonskim izvorima.

Ukupno vrijeme za odgovor: 33.77 sekundi

=====
Pitanje 12/31: Koje su tri grane Oružanih snaga Republike Hrvatske?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Tri grane Oružanih snaga Republike Hrvatske su:

1. **Hrvatska kopnena vojska**
2. **Hrvatska ratna mornarica**
3. **Hrvatsko ratno zrakoplovstvo**

(Izvor: [Zakon o obrani 2025], Članak 43.

Ukupno vrijeme za odgovor: 9.61 sekundi

=====
Pitanje 13/31: Kada prestaje vojna obveza za muškarce?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Vojna obveza za muškarce prestaje prema članku 20. Zakona o obrani 2025:

- Na kraju kalendarske godine u kojoj navršavaju **55 godina** života.
- Ako su ocijenjeni nesposobnima za vojnu službu.
- Na temelju otpusta iz hrvatskog državljanstva.

Iznimno, prema članku 20. stavku 2., vojna obveza prestaje za viših časnika i generale/admirale posljednjeg dana kalendarske godine u kojoj navršavaju **65 godina** života.

Izvor: [Zakon o obrani 2025] (<https://www.narodne-novine.hr/>), članci 20. i 21.

Ukupno vrijeme za odgovor: 22.47 sekundi

=====
Pitanje 14/31: Tko imenuje načelnika Glavnog stožera Oružanih snaga?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Načelnik Glavnog stožera Oružanih snaga imenuje se na razdoblje od četiri godine i ne može biti ponovno imenovan. Prema članku 4. Pravilnika o načinu i postupku

imenovanja načelnika Glavnog stožera Oružanih snaga Republike Hrvatske, koji je donesen temeljem članka 49. Zakona o obrani, načelnik Glavnog stožera imenuje se na način i po postupku koji se utvrđuje Pravilnikom. Međutim, prema članku 49. Zakona o obrani, ustroj Glavnog stožera propisuje Predsjednik Republike na prijedlog načelnika Glavnog stožera i uz suglasnost ministra obrane. Dakle, iako Pravilnik određuje detaljnije procedure, Zakon o obrani daje ključne ovlasti predsjedniku republike i ministru obrane u procesu imenovanja načelnika Glavnog stožera.

Ukupno vrijeme za odgovor: 23.10 sekundi

=====
Pitanje 15/31: Koliko traje mandat načelnika Glavnog stožera?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Mandat načelnika Glavnog stožera traje ****4 godine****, kako je navedeno u ****Članku 15. stavku 4.**** Zakona o obrani 2025.

Napomena: Ova informacija nije pronađena u dostupnim dokumentima.

Ukupno vrijeme za odgovor: 8.13 sekundi

=====
Pitanje 16/31: Što je temeljna svrha obrane Republike Hrvatske?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Temeljna svrha obrane Republike Hrvatske jest očuvanje suvereniteta, neovisnosti i teritorijalne cjelovitosti Republike Hrvatske. Ova svrha je definirana u ****Članku 2. stavku 1.**** ****Zakona o obrani 2025.****

Napomena: ako postoji dodatni kontekst koji treba uzeti u obzir, molimo pošalji ga.

Ukupno vrijeme za odgovor: 11.36 sekundi

=====
Pitanje 17/31: Do koje dobi novak može biti upućen na temeljno vojno osposobljavanje?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Novak može biti upućen na temeljno vojno osposobljavanje do isteka 30 godina života. To je definirano u ****Članku 21.**** stavku (3) ****Zakona o obrani 2025****, gdje je navedeno da se "Iznimno od stavka 1. ovoga članka, na temeljno vojno osposobljavanje upućuju se i novaci stariji od 19 godina, a najkasnije do isteka 30 godina života..."

Dodatno, prema istom članku, područjački odjeli za poslove obrane moraju osigurati da se novaci koji su dužni pristupiti vojsci to učine prije nego što navrše 30 godina.

Ako postoji dodatni kontekst (npr. pravila o produženju roka), molimo vas da ga dostavite kako bih mogao ažurirati odgovor.

Ukupno vrijeme za odgovor: 25.55 sekundi

=====
Pitanje 18/31: Tko donosi odluku o mobilizaciji Oružanih snaga?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Prema **Zakonu o obrani** (Članak 46., stavak 1.), odluku o uporabi Oružanih snaga donose sljedeća tijela:

- **Hrvatski sabor**
- **Predsjednik Republike**
- **Predsjednik Vlade**
- **Ministar obrane**

To uključuje i odluku o mobilizaciji, koja se temelji na ustavnim ovlastima Predsjednika Republike kao vrhovnog zapovjednika (**Ustav RH, Članak 100.**).

Detaljnije, prema **Zakonu o obrani** (Članak 58.), Vlada ili ministar obrane također mogu donositi odluke o korištenju Oružanih snaga, uključujući mobilizaciju.

Ako postoji izvanredna situacija (npr. neposredna ugroženost države), Predsjednik Republike može samostalno naručiti uporabu snaga (**Ustav RH, Članak 100.**).

Krajnja nadležnost za provođenje mobilizacije ostaje kod **Glavnog stožera Oružanih snaga** (**Zakon o obrani, Članci 46. i 58.**).

Ukupno vrijeme za odgovor: 33.20 sekundi

=====
Pitanje 19/31: Kada prestaje vojna obveza za žene?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Prema Zakonu o obrani 2025., članku 20.:

Žene prestaju imati vojnu obvezu nakon što navrše 50 godina života.

To znači da se njihova vojna obveza automatski ukida na kraju kalendarske godine u kojoj napune 50 godina.

Ostale mogućnosti prestanka vojne obveze uključuju:

- Prestanak zbog nesposobnosti za vojnu službu,
- Prestanak zbog gubitka hrvatskog državljanstva,
- Za visoke časnike i generale/admirale, rok produžuje se do 65. godine života.

Izvor: *Zakon o obrani 2025*, članak 20.

Ukupno vrijeme za odgovor: 21.09 sekundi

=====
Pitanje 20/31: U kojoj godini života se državljanin RH uvodi u vojnu evidenciju?
Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Državljanin Republike Hrvatske uvodi se u vojnu evidenciju u kalendarskoj godini kada navrši **18 godina** života.

Ako iz bilo kojeg razloga to ne učini u toj godini, može se unijeti do prestanka vojne obveze prema članku 20. Zakona o obrani.

Izvor: [Zakon o obrani 2025], članak 21.(1).

Ukupno vrijeme za odgovor: 12.24 sekundi

=====
Pitanje 21/31: Tko proglašava ratno stanje u Republici Hrvatskoj?
Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Hrvatski Sabor proglašava ratno stanje u Republici Hrvatskoj. (izvor: Ustav Republike Hrvatske 2018.docx - Članak 100.

Ukupno vrijeme za odgovor: 5.42 sekundi

=====
Pitanje 22/31: Koliko dana prije upućivanja ministar obrane donosi odluku o upućivanju kandidata na temeljno vojno osposobljavanje?
Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Ministar obrane donosi odluku o upućivanju kandidata na temeljno vojno osposobljavanje najkasnije **25 dana** prije upućivanja.

Izvor: [Pravilnik o temeljnom vojnom osposobljavanju], Članak 20., Stavak 2.

Ukupno vrijeme za odgovor: 8.99 sekundi

=====
Pitanje 23/31: Tko donosi program obuke ročnika na temeljnom vojnom osposobljavanju?
Kategorija: jednostavno (top_k=5)

=====
Generirani odgovor:

Program obuke ročnika na temeljnom vojnom osposobljavanju donosi Ministarstvo obrane. (izvor: Zakon_o_sluzbi_u_Oruzanim_snagama_Republike_Hrvatske_2025.

Ukupno vrijeme za odgovor: 7.84 sekundi

=====
Pitanje 24/31: Pred kim ročnici daju svečanu prisegu?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Ročnici daju svečanu prisegu pred nadležno vojno tijelo ili zapovjednika postrojbe, prema članku 7. stavku 5. ****Zakona o službi u Oružanim snagama Republike Hrvatske**** (2025).

*Napomena: Ova informacija nije pronađena u dostupnim dokumentima.

Ukupno vrijeme za odgovor: 9.26 sekundi

=====
Pitanje 25/31: Za koju vojnostručnu specijalnost se ročnici osposobljavaju na temeljnom vojnom osposobljavanju?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Ročnici se na temeljnom vojnom osposobljavanju osposobljavaju za vojnostručnu specijalnost ****strijalac**** (11A), dužnost strijelca, prema zadatcima i standardima definiranim u programu obuke ročnika na temeljnom vojnom osposobljavanju.

Izvor: [Pravilnik o temeljnom vojnom osposobljavanju], Članak 9. (1).

Ukupno vrijeme za odgovor: 12.96 sekundi

=====
Pitanje 26/31: Što čini mirnodopski sastav Oružanih snaga?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Mirnodopski sastav Oružanih snaga čini:

- ****djelatne vojne osobe****,
- ****državni službenici i namještenici**** raspoređeni u Oružane snage,
- ****pričuvnici pozvani na obuku****,
- ****ugovorni pričuvnici****,
- ****kadeti****, kao i
- ****osobe koje su pristupile temeljnom vojnom osposobljavanju****.

Izvor: [Zakon o obrani 2025], Članak 43., Stavak 2. (NN 136/25).

Ukupno vrijeme za odgovor: 18.29 sekundi

=====
Pitanje 27/31: Tko utvrđuje mirnodopski i ratni ustroj Oružanih snaga?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Prema članku 4. Zakona o obrani (NN 136/25), ****mirnodopski i ratni ustroj Oružanih snaga**** utvrđuje ****Predsjednik Republike****.

Detaljnije, prema članku 44. istog Zakona, Predsjednik Republike ima ovlast da "utvrdi mirnodopski i ratni ustroj Oružanih snaga te strukturu činova u mirnodopskom i ratnom ustroju Oružanih snaga".

Izvor: *Zakon o obrani 2025* (članci 43., 44.) - vidi priložene dokumente.

Ukupno vrijeme za odgovor: 18.29 sekundi

=====
Pitanje 28/31: Koliko traje minimalno godišnji odmor djelatne vojne osobe?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Djelatna vojna osoba ima pravo na minimalni godišnji odmor u trajanju od ****četiri tjedna**** (20 radnih dana), kako je navedeno u ****Članku 156. stavku 1.**** Zakona o službi u Oružanim snagama Republike Hrvatske 2025.

Napomena: Ova informacija temelji se isključivo na dostupnom zakonu.

Ukupno vrijeme za odgovor: 11.30 sekundi

=====
Pitanje 29/31: Koliko dana plaćenog dopusta pripada djelatnoj vojnoj osobi u slučaju rođenja djeteta?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Djelatnoj vojnoj osobi u slučaju rođenja djeteta pripada ****sedam radnih dana**** plaćenog dopusta, prema članku 159. stavku 1. točki a) Zakona o službi u Oružanim snagama Republike Hrvatske 2025.

Ukupno vrijeme za odgovor: 8.13 sekundi

=====
Pitanje 30/31: Koliko dana plaćenog dopusta pripada djelatnoj vojnoj osobi u slučaju zaključenja braka?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Djelatna vojna osoba ima pravo na ****sedam radnih dana**** plaćenog dopusta u slučaju zaključenja braka, prema članku 159. stavku 1. točki a) Zakona o službi u Oružanim snagama Republike Hrvatske 2025.

Ukupno vrijeme za odgovor: 8.59 sekundi

=====
Pitanje 31/31: Što je vojna iskaznica i tko ju izdaje?

Kategorija: jednostavno (top_k=5)
=====

Generirani odgovor:

Vojna iskaznica je javna isprava kojom vojni obveznik dokazuje identitet dok izvršava vojnu obvezu.

****Izdaje je:****

- ****Nadležni područni odjel za poslove obrane**** (članak 25., stavak 3. Zakona o obrani 2025).

- ****Iznimno, postrojba Oružanih snaga razine samostalne bojne, njoj ravne ili više razine**** ako ročnik ili pričuvnik izgube vojnu iskaznicu (članak 25., stavak 4.).

Dodatno, vojna iskaznica se izdaje:

- ****Novaku pri pozivanju na temeljno vojno osposobljavanje**** (članak 25., stavak 5.).

- ****Kadetu i stipendistu nakon sklapanja ugovora**** (članak 25., stavak 5.).

*Napomena: Ova informacija temelji se isključivo na članku 25. Zakona o obrani 2025.

Ukupno vrijeme za odgovor: 30.70 sekundi

=====
RAG rezultati nadopunjeni u: /content/drive/My Drive/diplomskiRad/rezultati.csv

Ukupno obrađeno pitanja: 31

Prosječno vrijeme po pitanju: 28.93 sekundi

4. Prilog - Colab - GraphRAG konfiguracija ChatBota

Link na Colab ipynb bilježnicu s kodom za GraphRAG konfiguraciju: [Colab - GraphRAG](#)

diplomskiRad_03_GraphRAG

July 31, 2026

GraphRAG ChatBot

0.1 Postavljanje okruženja

```
[ ]: !pip install -q -U bitsandbytes transformers accelerate sentencepiece  
!pip install -q -U sentence-transformers  
!pip install -q python-docx pdfplumber==0.11.0  
!pip install -q neo4j tqdm
```

```
60.7/60.7 MB  
35.3 MB/s eta 0:00:00  
11.6/11.6 MB  
162.0 MB/s eta 0:00:00  
69.1/69.1 kB  
7.7 MB/s eta 0:00:00  
56.4/56.4 kB  
6.3 MB/s eta 0:00:00  
5.6/5.6 MB  
92.6 MB/s eta 0:00:00  
253.0/253.0 kB  
28.3 MB/s eta 0:00:00  
3.7/3.7 MB  
121.6 MB/s eta 0:00:00  
327.8/327.8 kB  
10.2 MB/s eta 0:00:00
```

```
[ ]: from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline,  
↳ BitsAndBytesConfig, GenerationConfig  
from google.colab import userdata  
from huggingface_hub import login  
from sentence_transformers import SentenceTransformer  
from neo4j import GraphDatabase  
from tqdm import tqdm  
from docx import Document as DocxDocument  
import pdfplumber  
from pathlib import Path  
from copy import deepcopy
```

```

import torch
import time
import json
import os
import re
import logging
import pandas as pd

```

```

[ ]: # model_name = "Qwen/Qwen2.5-14B-Instruct"
model_name = "utter-project/EuroLLM-22B-Instruct-2512"

from google.colab import drive
drive.mount('/content/drive')

os.environ["HF_HOME"] = "/content/drive/MyDrive/hf_cache"
os.environ["HF_HUB_OFFLINE"] = "1"
os.environ["TRANSFORMERS_OFFLINE"] = "1"

print(f"Model: {model_name}")
print(f"Čitanje s Drivea: {os.environ['HF_HOME']}")

```

Mounted at /content/drive
Model: utter-project/EuroLLM-22B-Instruct-2512
Čitanje s Drivea: /content/drive/MyDrive/hf_cache

```

[ ]: token = userdata.get('HF_TOKEN')
login(token)

```

```

[ ]: quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4"
)

pipe = pipeline(
    "text-generation",
    model=model_name,
    dtype=torch.float16,
    device_map="auto",
    model_kwargs={"quantization_config": quantization_config}
)

pipe.model.generation_config.max_length = 8192

```

config.json: 0%| | 0.00/680 [00:00<?, ?B/s]

model.safetensors.index.json: 0%| | 0.00/40.3k [00:00<?, ?B/s]

```

Downloading bytes:          | 0.00B
Reconstructing (incomplete total...): | 0.00B / 0.00B
Fetching 10 files: 0%|      | 0/10 [00:00<?, ?it/s]
Loading weights: 0%|        | 0/489 [00:00<?, ?it/s]
generation_config.json: 0%|      | 0.00/132 [00:00<?, ?B/s]
tokenizer_config.json: 0%|        | 0.00/47.3k [00:00<?, ?B/s]
tokenizer.model: reconstructing file: 0%|      | 0.00B / 2.41MB
tokenizer.model: downloading bytes:          | 0.00B
tokenizer.json: reconstructing file: 0%|        | 0.00B / 15.8MB
tokenizer.json: downloading bytes:          | 0.00B
special_tokens_map.json: 0%|        | 0.00/443 [00:00<?, ?B/s]

```

0.2 Konfiguracija Neo4j baze podataka

```

[ ]: NEO4J_URI      = 'neo4j+s://ad1afaa8.databases.neo4j.io'
     NEO4J_USER     = 'ad1afaa8'
     NEO4J_PASSWORD = userdata.get('Neo4j')

GRAPH_TOP_K = {
    'jednostavno': 5,
    'kompleksno_jedan_dokument': 8,
    'kompleksno_vise_dokumenata': 12,
}
GRAPH_TOP_K_DEFAULT = 5

GRAPH_VECTOR_POOL = 30

GRAPH_PER_DOC_CAP = 3

GRAPH_MAX_CHARS_PO_CLANKU = 4000

GRAPH_REL_WEIGHTS = {
    'RAZRADJUJE': 1.00,
    'ISTA_TEMA': 0.95,
    'UPUCUJE_NA': 0.85,
    'SPOMINJE': 0.70,
}

print('\u2713 Neo4j i retrieval konfiguracija učitana')

```

Neo4j i retrieval konfiguracija učitana

0.2.1 Konekcija s Neo4j i kreiranje sheme grafa

```
[ ]: neo4j_driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

def neo4j_run(query, params=None):
    with neo4j_driver.session() as session:
        return list(session.run(query, params or {}))

_transformers_logger = logging.getLogger("transformers.modeling_utils")
_prev_level = _transformers_logger.level
_transformers_logger.setLevel(logging.ERROR)

embed_model = SentenceTransformer("intfloat/multilingual-e5-large")

_transformers_logger.setLevel(_prev_level)

EMBEDDING_DIM = embed_model.get_embedding_dimension()

SCHEMA_QUERIES = [
    'CREATE CONSTRAINT IF NOT EXISTS FOR (z:Zakon) REQUIRE z.id IS UNIQUE',
    'CREATE CONSTRAINT IF NOT EXISTS FOR (c:Clanak) REQUIRE c.id IS UNIQUE',
    'CREATE CONSTRAINT IF NOT EXISTS FOR (e:Entitet) REQUIRE e.naziv IS UNIQUE',
    f'''CREATE VECTOR INDEX clanak_embedding IF NOT EXISTS
    FOR (c:Clanak) ON (c.embedding)
    OPTIONS {{
        indexConfig: {{
            `vector.dimensions`: {EMBEDDING_DIM},
            `vector.similarity_function`: 'cosine'
        }}
    }}'''
]

for q in SCHEMA_QUERIES:
    try:
        neo4j_run(q)
    except Exception as e:
        print(f' (info) {e}')

neo4j_driver.verify_connectivity()
print(f' Neo4j konekcija OK | Embedding dimenzija: {EMBEDDING_DIM}')
```

modules.json: 0%| | 0.00/387 [00:00<?, ?B/s]

README.md: 0%| | 0.00/160k [00:00<?, ?B/s]

sentence_bert_config.json: 0%| | 0.00/57.0 [00:00<?, ?B/s]

config.json: 0%| | 0.00/690 [00:00<?, ?B/s]


```

model.safetensors: reconstructing file: 0%|          | 0.00B / 2.24GB
model.safetensors: downloading bytes:          | 0.00B
Loading weights: 0%|          | 0/391 [00:00<?, ?it/s]
tokenizer_config.json: 0%|          | 0.00/418 [00:00<?, ?B/s]
sentencepiece.bpe.model: reconstructing file: 0%|          | 0.00B / 5.07MB
sentencepiece.bpe.model: downloading bytes:          | 0.00B
tokenizer.json: reconstructing file: 0%|          | 0.00B / 17.1MB
tokenizer.json: downloading bytes:          | 0.00B
special_tokens_map.json: 0%|          | 0.00/280 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/201 [00:00<?, ?B/s]
Neo4j konekcija OK | Embedding dimenzija: 1024

```

0.3 Sistemska poruka i funkcija generacije

```

[ ]: SYSTEM_PROMPT = (
    "Ti si pravni stručnjak specijaliziran za zakonodavstvo Republike Hrvatske,
    ↳ posebno za Zakon o obrani, Zakon o službi u Oružanim snagama RH i
    ↳ pripadajuće pravilnike."
    "\n\nPravila odgovaranja:"
    "\n1. Jezik odgovora: hrvatski, latinica."
    "\n2. Navedi konkretne članke zakona, brojčane vrijednosti i nadležna tijela."
    "\n3. Ako je dostupan kontekst, koristi SAMO informacije iz njega."
    "\n4. Ako kontekst ne sadržava odgovor, odgovori na temelju općeg znanja i
    ↳ dodaj: 'Napomena: ova informacija nije pronađena u dostupnim dokumentima.'"
    "\n5. Odgovor neka bude koncizan i strukturiran - bez nepotrebnih uvoda."
)

```

```

[ ]: csv_path = '/content/drive/My Drive/diplomskiRad/rezultati.csv'

```

```

[ ]: def generate_response(messages, max_new_tokens=512, temperature=0.5,
    ↳ do_sample=True):
    final_messages = list(messages)

    model_max_length = getattr(pipe.tokenizer, 'model_max_length', 8192)
    if model_max_length > 100000:
        model_max_length = 8192

    input_text = pipe.tokenizer.apply_chat_template(final_messages,
    ↳ tokenize=False)
    input_token_count = len(pipe.tokenizer.encode(input_text))
    max_allowed_input = model_max_length - max_new_tokens

```

```

    if input_token_count > max_allowed_input:
        print(f"Upozorenje: input ({input_token_count} tokena) prelazi limit_
↳({max_allowed_input}). Skraćujem kontekst.")
        if final_messages and final_messages[0]["role"] == "system":
            system_content = final_messages[0]["content"]
            while input_token_count > max_allowed_input and "\n\n---\n\n" in_
↳system_content:
                last_chunk_start = system_content.rfind("\n\n---\n\n")
                system_content = system_content[:last_chunk_start]
                final_messages[0]["content"] = system_content
                input_text = pipe.tokenizer.apply_chat_template(final_messages,
↳tokenize=False)
                input_token_count = len(pipe.tokenizer.encode(input_text))
                print(f" Skraćeno na {input_token_count} tokena.")

    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    gen_config = deepcopy(pipe.model.generation_config)
    gen_config.max_new_tokens = max_new_tokens
    gen_config.temperature = temperature
    gen_config.do_sample = do_sample
    gen_config.repetition_penalty = 1.1

    vrijeme_upita = time.time()

    response = pipe(final_messages, generation_config=gen_config)

    answer = ""
    for message in response[0]['generated_text']:
        if message['role'] == 'assistant':
            answer = message['content']
            break

    stripped_answer = answer.strip()
    if not re.search(r'[.!?]$', stripped_answer) and len(stripped_answer.
↳split()) > 5:
        last_punctuation_index = -1
        for i in range(len(stripped_answer) - 1, -1, -1):
            if stripped_answer[i] in ['.', '?', '!']:
                last_punctuation_index = i
                break
        if last_punctuation_index != -1:
            answer = stripped_answer[:last_punctuation_index + 1]
        else:
            words = stripped_answer.split()

```

```

        answer = ' '.join(words[:-1]) + '...' if len(words) > 1 else
↳(words[0] if words else '')
    else:
        answer = stripped_answer

    vrijeme_odgovora = time.time()
    ukupno_vrijeme = vrijeme_odgovora - vrijeme_upita

    print("Generirani odgovor:")
    print(answer)
    print(f"Ukupno vrijeme za odgovor: {ukupno_vrijeme:.2f} sekundi")

    return answer, ukupno_vrijeme

```

0.4 NER model za ekstrakciju entiteta

```

[ ]: graph_ner_pipe = pipeline(
    'ner',
    model='classla/bcms-bertic-ner',
    aggregation_strategy='simple',
    device=0,
)

def graph_extract_entities(text: str) -> list:
    if not text or len(text.strip()) < 10:
        return []
    try:
        entities, seen = [], set()
        for start in range(0, len(text), 384):
            prozor = text[start:start + 512]
            if len(prozor.strip()) < 10:
                continue
            for r in graph_ner_pipe(prozor):
                naziv = r['word'].strip()
                if '##' in naziv:
                    continue
                if naziv and naziv not in seen and len(naziv) > 2:
                    entities.append({'naziv': naziv, 'tip': r['entity_group']})
                    seen.add(naziv)
        return entities
    except Exception:
        return []

print(' NER model ucitan')

```

config.json: 0%| | 0.00/1.02k [00:00<?, ?B/s]

model.safetensors: reconstructing file: 0%| | 0.00B / 440MB

```

model.safetensors: downloading bytes:          | 0.00B
Loading weights: 0%|          | 0/199 [00:00<?, ?it/s]
tokenizer_config.json: 0%|          | 0.00/352 [00:00<?, ?B/s]
vocab.txt: 0%|          | 0.00/231k [00:00<?, ?B/s]
special_tokens_map.json: 0%|          | 0.00/112 [00:00<?, ?B/s]

NER model ucitan

```

0.5 Parser pravnih dokumenata

```

[ ]: CLANAK_RE = re.compile(r'^[\u010c\u010dCc]lanak\s+(\d+[a-z])?.?(?:\s|$)',
    ↪re.IGNORECASE)
STAVAK_PATTERN = re.compile(r'^((\d+)\s+)\s+')

UPUTA_PATTERN = re.compile(
    r'^[\u010c\u010dCc]lank[ua]?s+(\d+[a-z])?.?'
    r'(?:\s+(ovog[a]?s+(?:Pravilnika|Zakona) '
    r'|Ustav[a]?(?:\s+Republike\s+Hrvatske)?'
    r'|Zakona\s+o\s+obrani'
    r'|Zakona\s+o\s+slu\u017ebi[a-z\u0107\u010d\u0111\u0161\u017e\s]*?snagama'
    r'|Pravilnika\s+o\s+[a-z\u0107\u010d\u0111\u0161\u017e\s]+?))?',
    re.IGNORECASE
)

def _ciljni_akt(fraza):
    if not fraza:
        return None
    f = fraza.lower()
    if 'ovog' in f:
        return None
    if 'ustav' in f:
        return 'ustav'
    if 'slu\u017ebi' in f or 'sluzbi' in f:
        return 'sluzba_osrh'
    if 'obrani' in f:
        return 'obrana'
    if 'pravilnik' in f:
        return 'pravilnik'
    return None

FALLBACK_CHUNK_LEN = 1200

def _klasificiraj_obvezu(tekst: str):
    t = tekst.lower()
    if any(k in t for k in ['du\u017ean', 'duzan', 'obvezan', 'obvezna',
    ↪'mora', 'obvezuje se']):
        return 'OBVEZA'
    if any(k in t for k in ['zabranjeno', 'nije dopu\u0161teno', 'nije
    ↪dopusteno', 'ne smije']):
        return 'ZABRANA'
    if any(k in t for k in ['ima pravo', 'ovla\u0161ten', 'ovlasten',
    ↪'mo\u017ee', 'moze']):

```

```

        return 'PRAVO'
    return None

def _fallback_pseudo_clanci(tekst: str, prefix: str):
    """Tekst izvan obrasca clanka rezemo u pseudo-clanke (fallback,
    identican duh RAG chunkera) da nista ne ispadne iz grafa."""
    tekst = ' '.join(tekst.split())
    if len(tekst) < 200:
        return []
    out, i, n = [], 0, 1
    while i < len(tekst):
        chunk = tekst[i:i + FALLBACK_CHUNK_LEN]
        out.append({'broj': f'{prefix}{n}', 'tekst': chunk,
                    'stavci': [], 'upucivanja': [], 'obveze': []})
        i += FALLBACK_CHUNK_LEN
        n += 1
    return out

def _parse_paragraphs(paragraphs, naziv):
    """Zajednicka logika za docx i pdf: paragrafe pretvara u clanke,
    uz fallback za tekst prije prvog clanka i dokumente bez clanaka."""
    clanaci, current = [], None
    pre_text = []

    for tekst in paragraphs:
        tekst = tekst.strip()
        if not tekst:
            continue

        m = CLANAK_RE.match(tekst)
        if m:
            if current:
                clanaci.append(current)
            current = {'broj': m.group(1), 'tekst': '', 'stavci': [],
                       'upucivanja': [], 'obveze': []}
            continue

        if current is None:
            pre_text.append(tekst)
            continue

        if STAVAK_PATTERN.match(tekst):
            current['stavci'].append(tekst)
        else:
            current['tekst'] += ' ' + tekst

    for um in UPUTA_PATTERN.finditer(tekst):

```

```

        br = um.group(1)
        akt = _ciljni_akt(um.group(2))
        ref = (br, akt)
        if br and ref not in current['upucivanja']:
            current['upucivanja'].append(ref)

    tip = _klasificiraj_obvezu(tekst)
    if tip:
        current['obveze'].append({'tip': tip, 'tekst': tekst[:300]})

    if current:
        clanaci.append(current)

    pseudo = _fallback_pseudo_clanci(' '.join(pre_text), 'pre')

    if not clanaci and not pseudo:
        cijeli = ' '.join(paragraphs)
        pseudo = _fallback_pseudo_clanci(cijeli, 'chunk')

    clanaci = pseudo + clanaci
    print(f' Parsiran: {naziv} \u2014 {len(clanaci)} clanaka'
          f' (od toga {len(pseudo)} fallback)')
    return {'naziv': naziv, 'clanaci': clanaci}

def graph_parse_docx(filepath: str) -> dict:
    doc = DocxDocument(filepath)
    naziv = Path(filepath).stem.replace('_', ' ')
    for para in doc.paragraphs[:15]:
        sn = para.style.name.lower()
        if ('heading' in sn or 'naslov' in sn or 'title' in sn) and len(para.
        ↳text.strip()) > 10:
            naziv = para.text.strip()
            break
    return _parse_paragraphs([p.text for p in doc.paragraphs], naziv)

def graph_parse_pdf(filepath: str) -> dict:
    import pdfplumber
    naziv = Path(filepath).stem.replace('_', ' ')
    lines = []
    try:
        with pdfplumber.open(filepath) as pdf:
            for page in pdf.pages:
                page_text = page.extract_text()
                if page_text:
                    lines.extend(page_text.split("\n"))
    except Exception as e:
        print(f' Greska pri citanju PDF-a {filepath}: {e}')

```

```

        return {'naziv': naziv, 'clanaci': []}

paragraphs, current_para = [], []
for line in lines:
    line = line.strip()
    if not line:
        if current_para:
            paragraphs.append(" ".join(current_para))
            current_para = []
        continue
    if CLANAK_RE.match(line):
        if current_para:
            paragraphs.append(" ".join(current_para))
            current_para = []
        paragraphs.append(line)
    elif STAVAK_PATTERN.match(line):
        if current_para:
            paragraphs.append(" ".join(current_para))
            current_para = [line]
        else:
            current_para.append(line) if current_para else current_para.
↪append(line)
        if current_para:
            paragraphs.append(" ".join(current_para))

    return _parse_paragraphs(paragraphs, naziv)

print('\u2713 GraphRAG docx/pdf parser ucitan (s fallbackom)')

```

GraphRAG docx/pdf parser ucitan (s fallbackom)

0.6 Ingestija zakona u Neo4j

```

[ ]: _AKT_ID_CACHE = {}

def _rijesi_zakon_id(akt, vlastiti_zakon_id):
    if akt is None:
        return vlastiti_zakon_id

    if not _AKT_ID_CACHE:
        try:
            for row in neo4j_run('MATCH (z:Zakon) RETURN z.id AS id, z.naziv AS_
↪naziv'):
                naziv = (row['naziv'] or '').lower()
                zid = row['id']
                if 'ustav' in naziv:
                    _AKT_ID_CACHE['ustav'] = zid

```

```

        elif 'slu\u017ebi' in naziv or 'sluzbi' in naziv:
            _AKT_ID_CACHE['sluzba_osrh'] = zid
        elif 'obrani' in naziv:
            _AKT_ID_CACHE['obrana'] = zid
    except Exception as e:
        print(f' (info) _rijesi_zakon_id kes: {e}')

    return _AKT_ID_CACHE.get(akt, vlastiti_zakon_id)

def graph_ensure_schema():
    """Idempotentno osigurava constraintove i vektorske indekse."""
    dim = embed_model.get_embedding_dimension()
    queries = [
        'CREATE CONSTRAINT IF NOT EXISTS FOR (z:Zakon) REQUIRE z.id IS UNIQUE',
        'CREATE CONSTRAINT IF NOT EXISTS FOR (c:Clanak) REQUIRE c.id IS UNIQUE',
        'CREATE CONSTRAINT IF NOT EXISTS FOR (s:Stavak) REQUIRE s.id IS UNIQUE',
        'CREATE CONSTRAINT IF NOT EXISTS FOR (e:Entitet) REQUIRE e.naziv IS_
↪UNIQUE',
        f'''CREATE VECTOR INDEX clanak_embedding IF NOT EXISTS
FOR (c:Clanak) ON (c.embedding)
OPTIONS {{
    indexConfig: {{
        `vector.dimensions`: {dim},
        `vector.similarity_function`: 'cosine'
    }}
}}''',
        f'''CREATE VECTOR INDEX stavak_embedding IF NOT EXISTS
FOR (s:Stavak) ON (s.embedding)
OPTIONS {{
    indexConfig: {{
        `vector.dimensions`: {dim},
        `vector.similarity_function`: 'cosine'
    }}
}}''',
    ]
    for q in queries:
        try:
            neo4j_run(q)
        except Exception as e:
            print(f' (info) {e}')
    print(f'\u2713 Shema osigurana (indeksi clanak_embedding i_
↪stavak_embedding, dim={dim})')

def graph_embed(text: str) -> list:
    vec = embed_model.encode(['passage: ' + text], normalize_embeddings=True)
    return vec[0].tolist()

```



```

import unicodedata as _ud

_PADEZNI_NASTAVCI = ('oga', 'ome', 'ima', 'ama', 'ega', 'emu',
                     'og', 'om', 'im', 'em', 'a', 'e', 'i', 'u', 'o')

def _kanon_rijec(w: str) -> str:
    for suf in _PADEZNI_NASTAVCI:
        if len(w) > len(suf) + 3 and w.endswith(suf):
            return w[:-len(suf)]
    return w

def normaliziraj_entitet(naziv: str) -> str:
    naziv = (naziv or '').strip().lower()
    naziv = ''.join(c for c in naziv if c.isalnum() or c.isspace())
    naziv = ' '.join(naziv.split())
    if not naziv:
        return ''
    return ' '.join(_kanon_rijec(w) for w in naziv.split())

STOP_ENTITETI = {
    'republik hrvatsk', 'republic hrvatskoj', 'hrvatsk', 'republik',
    'oružanih snag', 'oružan snag', 'oruž', 'snag',
    'ministarstv obran', 'hrvatsk sabor', 'narodn novin',
    'vlad republik hrvatsk', 'vlad', 'europsk unij',
    'nim snag', 'ne snag',
}

def graph_ingest_zakon(zakon_data: dict, zakon_id: str):
    graph_ensure_schema()
    naziv = zakon_data['naziv']
    neo4j_run('MERGE (z:Zakon {id: $id}) SET z.naziv = $naziv',
              {'id': zakon_id, 'naziv': naziv})

    for cl in tqdm(zakon_data['clanaci'], desc=f' Ingestija: {naziv[:30]}'):
        clanak_id = f'{zakon_id}_{cl["broj"]}'
        full_text = (cl['tekst'].strip() + ' ' + ' '.join(cl['stavci'])).strip()
        if not full_text:
            continue

        embedding = graph_embed(full_text)

        neo4j_run(''
            MERGE (c:Clanak {id: $id})
            SET c.broj = $broj, c.tekst = $tekst,
                c.zakon_id = $zakon_id, c.embedding = $emb
            WITH c
            MATCH (z:Zakon {id: $zakon_id})

```

```

MERGE (z)-[:SADRZI]->(c)
'', {'id': clanak_id, 'broj': cl['broj'], 'tekst': full_text,
    'zakon_id': zakon_id, 'emb': embedding})

for i, stavak in enumerate(cl['stavci']):
    neo4j_run(''
        MERGE (s:Stavak {id: $id})
        SET s.tekst = $tekst, s.redoslijed = $r, s.embedding = $emb
        WITH s MATCH (c:Clanak {id: $cid})
        MERGE (c)-[:IMA_STAVAK]->(s)
        '', {'id': f'{clanak_id}_s{i}', 'tekst': stavak, 'r': i,
            'cid': clanak_id, 'emb': graph_embed(stavak)})

for target_br, akt in cl['upucivanja']:
    ciljni_zakon = _rijesi_zakon_id(akt, zakon_id)
    neo4j_run(''
        MERGE (t:Clanak {id: $tid})
        ON CREATE SET t.broj = $tbr, t.zakon_id = $zid
        WITH t MATCH (src:Clanak {id: $sid})
        MERGE (src)-[:UPUCUJE_NA]->(t)
        '', {'tid': f'{ciljni_zakon}_cl_{target_br}', 'tbr': target_br,
            'zid': ciljni_zakon, 'sid': clanak_id})

for ent in graph_extract_entities(full_text):
    kanon = normaliziraj_entitet(ent['naziv'])
    if not kanon or kanon in STOP_ENTITETI or len(kanon) < 6:
        continue
    neo4j_run(''
        MERGE (e:Entitet {naziv: $kanon})
        SET e.tip = $tip, e.povrsinski = $povrsinski
        WITH e MATCH (c:Clanak {id: $cid})
        MERGE (c)-[:SPOMINJE]->(e)
        '', {'kanon': kanon, 'tip': ent['tip'],
            'povrsinski': ent['naziv'], 'cid': clanak_id})

for ob in cl['obveze']:
    neo4j_run(''
        CREATE (o:Obveza {tekst: $tekst, tip: $tip})
        WITH o MATCH (c:Clanak {id: $cid})
        MERGE (c)-[:SADRZI_OBVEZU]->(o)
        '', {'tekst': ob['tekst'], 'tip': ob['tip'], 'cid': clanak_id})

print(f'\u2713 Zakon "{naziv}" upisan u Neo4j')

print('\u2713 Ingestija funkcije učitane')

```

Ingestija funkcije učitane

0.6.1 Učitavanje zakona iz Google Drivea u Neo4j

```
[ ]: neo4j_run('MATCH (n) DETACH DELETE n')
print(' Baza očišćena - svi čvorovi i veze obrisani.')
```

Baza očišćena - svi čvorovi i veze obrisani.

```
[ ]: graph_docs_dir = '/content/drive/My Drive/diplomskiRad/Dokumenti'

if os.path.exists(graph_docs_dir):
    print(f'Ingestiram dokumente iz: {graph_docs_dir}\n')
    for file in os.listdir(graph_docs_dir):
        if file.endswith('.docx'):
            file_path = os.path.join(graph_docs_dir, file)
            zakon_id = Path(file).stem.lower().replace(' ', '_')[:50]
            zakon_data = graph_parse_docx(file_path)
            graph_ingest_zakon(zakon_data, zakon_id)
        elif file.endswith('.pdf'):
            file_path = os.path.join(graph_docs_dir, file)
            zakon_id = Path(file).stem.lower().replace(' ', '_')[:50]
            zakon_data = graph_parse_pdf(file_path)
            graph_ingest_zakon(zakon_data, zakon_id)
    print('\n Svi zakoni upisani u Neo4j')
else:
    print(f'Mapa nije pronađena: {graph_docs_dir}')
```

Ingestiram dokumente iz: /content/drive/My Drive/diplomskiRad/Dokumenti

Parsiran: Pravilnik o temeljnom vojnom osposobljavanju - 23 clanaka (od toga 1 fallback)

Schema osigurana (indeksi clanak_embedding i stavak_embedding, dim=1024)

Ingestija: Pravilnik o temeljnom vojnom o: 22% | 5/23 [00:03<00:12, 1.46it/s]

Ingestija: Pravilnik o temeljnom vojnom o: 100% | 23/23 [00:17<00:00, 1.35it/s]

Zakon "Pravilnik o temeljnom vojnom osposobljavanju" upisan u Neo4j

Parsiran: Ustav Republike Hrvatske 2018 - 154 clanaka (od toga 5 fallback)

Schema osigurana (indeksi clanak_embedding i stavak_embedding, dim=1024)

Ingestija: Ustav Republike Hrvatske 2018: 100% | 154/154 [01:18<00:00, 1.96it/s]

Zakon "Ustav Republike Hrvatske 2018" upisan u Neo4j

Parsiran: Zakon o obrani 2025 - 143 clanaka (od toga 2 fallback)

Schema osigurana (indeksi clanak_embedding i stavak_embedding, dim=1024)

Ingestija: Zakon o obrani 2025: 100% | 143/143 [03:16<00:00, 1.38s/it]

Zakon "Zakon o obrani 2025" upisan u Neo4j
 Parsiran: Zakon o sluzbi u Oruzanim snagama Republike Hrvatske 2025 - 280
 clanaka (od toga 2 fallback)
 Shema osigurana (indeksi clanak_embedding i stavak_embedding, dim=1024)
 Ingestija: Zakon o sluzbi u Oruzanim snag: 100% | 280/280
 [04:55<00:00, 1.05s/it]
 Zakon "Zakon o sluzbi u Oruzanim snagama Republike Hrvatske 2025" upisan u
 Neo4j
 Svi zakoni upisani u Neo4j

0.7 Derivacija ISTA_TEMA veza i dijagnostika grafa

```
[ ]: MIN_TEZINA = 1.5

def deriviraj_ista_tema(min_tezina=MIN_TEZINA):
    neo4j_run("MATCH ()-[r:ISTA_TEMA {izvor:'auto'}]->() DELETE r")
    rezultat = neo4j_run(''
        // ukupan broj clanaka - za IDF
        MATCH (c:Clanak) WITH count(c) AS N

        // za svaki entitet: u koliko clanaka se pojavljuje
        MATCH (e:Entitet)-[:SPOMINJE]-(c:Clanak)
        WITH N, e, count(DISTINCT c) AS df
        WHERE df >= 2
        WITH N, e, log(1.0 * N / df) AS idf

        // parovi clanaka iz razlicitih zakona, zbroj IDF-a dijeljenih entiteta
        MATCH (c1:Clanak)-[:SPOMINJE]->(e)-[:SPOMINJE]-(c2:Clanak)
        WHERE c1.zakon_id < c2.zakon_id
        WITH c1, c2, sum(idf) AS tezina, count(DISTINCT e) AS zaj
        WHERE tezina >= $min_tezina AND zaj >= 2
        MERGE (c1)-[r:ISTA_TEMA {izvor:'auto'}]->(c2)
        SET r.tezina = tezina, r.zaj_entiteta = zaj
        RETURN count(r) AS n
    '', {'min_tezina': min_tezina})
    n = rezultat[0]['n'] if rezultat else 0
    print(f'\u2713 Derivirano {n} ISTA_TEMA veza (IDF tezina >= {min_tezina}, \u2713
    \u2713medudokumentno)')
    return n

deriviraj_ista_tema()

_diag = neo4j_run(''
    MATCH (a:Clanak)-[:UPUCUJE_NA]->(b:Clanak)
```

```

        RETURN a.zakon_id = b.zakon_id AS unutar_istog, count(*) AS n
    '')
print('\nUPUCUJE_NA raspodjela:')
for r in (_diag or []):
    oznaka = 'unutar istog zakona' if r['unutar_istog'] else 'MEDUDOKUMENTNO'
    print(f"  {oznaka:<22} {r['n']:>5}")

_most = neo4j_run('''
    MATCH (c1:Clanak)-[:SPOMINJE]->(e:Entitet)<-[:SPOMINJE]-(c2:Clanak)
    WHERE c1.zakon_id < c2.zakon_id
    RETURN count(DISTINCT e) AS mostovni_entiteti
''')
print(f"\nMostovnih entiteta (spajaju 2 zakona): "
      f"{_most[0]['mostovni_entiteti'] if _most else 0}")

```

Derivirano 7 ISTA_TEMA veza (IDF tezina >= 1.5, medudokumentno)

UPUCUJE_NA raspodjela:

unutar istog zakona	123
---------------------	-----

Mostovnih entiteta (spajaju 2 zakona): 28

0.8 Statistike grafa

```

[ ]: stats = neo4j_run('''
    MATCH (z:Zakon) WITH count(z) AS zakoni
    MATCH (c:Clanak) WITH zakoni, count(c) AS clanci
    OPTIONAL MATCH (s:Stavak)
    RETURN zakoni, clanci, count(s) AS stavci
''')
if stats:
    s = dict(stats[0])
    print('Graf statistike:')
    for k, v in s.items():
        print(f'  {k.capitalize()}: {v}')

per_doc = neo4j_run('''
    MATCH (z:Zakon)-[:SADRZI]->(c:Clanak)
    RETURN z.naziv AS zakon, count(c) AS clanaka,
           sum(CASE WHEN c.broj STARTS WITH 'pre' OR c.broj STARTS WITH 'chunk'
                     THEN 1 ELSE 0 END) AS fallback
    ORDER BY zakon
''')
print('\nClanaka po dokumentu:')
for r in per_doc:
    print(f"  {r['zakon'][:55]:<57} {r['clanaka']:>4} (fallback:␣
    ↪{r['fallback']})")

```

```

veze = neo4j_run(''
    MATCH ()-[r:UPUCUJE_NA]->()      RETURN 'UPUCUJE_NA'      AS tip, count(r) AS_
↪n
    UNION ALL
    MATCH ()-[r:RAZRADJUJE]->()      RETURN 'RAZRADJUJE'      AS tip, count(r) AS_
↪n
    UNION ALL
    MATCH ()-[r:ISTA_TEMA]->()        RETURN 'ISTA_TEMA'        AS tip, count(r) AS_
↪n
    UNION ALL
    MATCH ()-[r:SADRZI]->()           RETURN 'SADRZI'           AS tip, count(r) AS_
↪n
    UNION ALL
    MATCH ()-[r:IMA_STAVAK]->()        RETURN 'IMA_STAVAK'        AS tip, count(r) AS_
↪n
    UNION ALL
    MATCH ()-[r:SPOMINJE]->()         RETURN 'SPOMINJE'         AS tip, count(r) AS_
↪n
    UNION ALL
    MATCH ()-[r:SADRZI_OBVEZU]->()     RETURN 'SADRZI_OBVEZU'     AS tip, count(r) AS_
↪n
'')
print('\nRelacije u grafu:')
for r in veze:
    print(f"  {r['tip']:<25} {r['n']:>6} veza")

eksp = neo4j_run("MATCH ()-[r]->() WHERE r.izvor = 'ekspert' RETURN count(r) AS_
↪n")
print(f"\nEkspertnih veza (izvor='ekspert'): {eksp[0]['n'] if eksp else 0}")

```

Graf statistike:

Zakoni: 4
 Clanci: 538
 Stavci: 1414

Clanaka po dokumentu:

Pravilnik o temeljnom vojnom osposobljavanju	23 (fallback: 1)
Ustav Republike Hrvatske 2018	146 (fallback: 5)
Zakon o obrani 2025	130 (fallback: 2)
Zakon o sluzbi u Oruzanim snagama Republike Hrvatske 20	238 (fallback: 2)

Relacije u grafu:

UPUCUJE_NA	123 veza
ISTA_TEMA	7 veza
SADRZI	537 veza
IMA_STAVAK	1414 veza

SPOMINJE	421 veza
SADRZI_OBVEZU	488 veza

Ekspertnih veza (izvor='ekspert'): 0

0.9 Učitavanje pitanja i odgovora

```
[ ]: try:
    with open('/content/drive/My Drive/diplomskiRad/pitanja_odgovori.json',
        'r', encoding='utf-8') as f:
        qa_data = json.load(f)

    print(f"\u2713 Učitano {qa_data['metadata']['ukupno_pitanja']} pitanja i
    odgovora.")
    print(f"  Struktura:")
    for kategorija, info in qa_data['metadata']['struktura'].items():
        print(f"    - {kategorija}: {info['broj']} ({info['postotak']}")
except FileNotFoundError:
    print('Datoteka pitanja_odgovori.json nije pronađena na navedenoj putanji.')
    qa_data = None
except json.JSONDecodeError:
    print('Greška pri parsiranju JSON datoteke. Provjerite je li format
    ispravan.')
    qa_data = None
```

Učitano 31 pitanja i odgovora.

Struktura:

- kompleksna_vise_dokumenata: 3 (9.67%)
- kompleksna_jedan_dokument: 5 (16.13%)
- jednostavna: 23 (74.2%)

0.10 GraphRAG retriever

```
[ ]: def graph_retrieve_context(query: str, kategorija: str = None) -> str:
    """GraphRAG retriever - ispravke u odnosu na prvu verziju:
    1. Dohvat i po CLANCIMA i po STAVCIMA (uska pitanja pogadaju
       uski stavak, vraca se roditeljski clanak).
    2. Cap po dokumentu - garantirana pokrivenost svih akata kod
       medudokumentnih pitanja.
    3. Tipizirani traversal (UPUCUJE_NA / RAZRADJUJE / ISTA_TEMA +
       SPOMINJE preko zajednickog entiteta, samo medudokumentno)
       s tezinama po tipu veze umjesto fiksnog penaltija.
    4. Dedup po ID-u umjesto krhkog Jaccard preklapanja rijeci.
    5. top_k ovisan o kategoriji pitanja.
    """
    top_k = GRAPH_TOP_K.get(kategorija, GRAPH_TOP_K_DEFAULT)
    query_emb = embed_model.encode(['query: ' + query],
                                    normalize_embeddings=True)[0].tolist()
```

```

res_cl = neo4j_run('''
    CALL db.index.vector.queryNodes('clanak_embedding', $k, $emb)
    YIELD node AS c, score
    WHERE c.tekst IS NOT NULL
    MATCH (z:Zakon)-[:SADRZI]->(c)
    RETURN c.id AS id, c.broj AS broj, c.tekst AS tekst,
           z.naziv AS zakon, score
''', {'k': GRAPH_VECTOR_POOL, 'emb': query_emb})

res_st = neo4j_run('''
    CALL db.index.vector.queryNodes('stavak_embedding', $k, $emb)
    YIELD node AS s, score
    MATCH (c:Clanak)-[:IMA_STAVAK]->(s)
    WHERE c.tekst IS NOT NULL
    MATCH (z:Zakon)-[:SADRZI]->(c)
    WITH c, z, max(score) AS score
    RETURN c.id AS id, c.broj AS broj, c.tekst AS tekst,
           z.naziv AS zakon, score
''', {'k': GRAPH_VECTOR_POOL, 'emb': query_emb})

seeds = {}
for r in list(res_cl) + list(res_st):
    rec = {'id': r['id'], 'broj': r['broj'], 'tekst': r['tekst'] or '',
           'zakon': r['zakon'], 'score': float(r['score'])}
    if rec['id'] not in seeds or rec['score'] > seeds[rec['id']]['score']:
        seeds[rec['id']] = rec

po_zakonu = {}
for rec in sorted(seeds.values(), key=lambda x: x['score'], reverse=True):
    po_zakonu.setdefault(rec['zakon'], [])
    if len(po_zakonu[rec['zakon']]) < GRAPH_PER_DOC_CAP:
        po_zakonu[rec['zakon']].append(rec)
seed_list = [rec for grupa in po_zakonu.values() for rec in grupa]
seed_ids = [rec['id'] for rec in seed_list]
seed_score = {rec['id']: rec['score'] for rec in seed_list}

susjedi_res = neo4j_run('''
    MATCH (seed:Clanak) WHERE seed.id IN $ids

    OPTIONAL MATCH (seed)-[r:UPUCUJE_NA|RAZRADJUJE|ISTA_TEMA]-(n1:Clanak)
    WHERE n1.tekst IS NOT NULL
    OPTIONAL MATCH (zn1:Zakon)-[:SADRZI]->(n1)

    OPTIONAL MATCH (seed)-[:SPOMINJE]->(e:Entitet)<-[:SPOMINJE]-(n2:Clanak)
    WHERE n2.zakon_id <> seed.zakon_id AND n2.tekst IS NOT NULL
    OPTIONAL MATCH (zn2:Zakon)-[:SADRZI]->(n2)
''')

```



```

        WITH seed,
            collect(DISTINCT {id: n1.id, broj: n1.broj, tekst: n1.tekst,
                                zakon: zn1.naziv, rel: type(r)})[0..4] AS
↪direktni,
            collect(DISTINCT {id: n2.id, broj: n2.broj, tekst: n2.tekst,
                                zakon: zn2.naziv, rel: 'SPOMINJE'})[0..3] AS
↪entitetski
        RETURN seed.id AS sid, direktni + entitetski AS susjedi
        '', {'ids': seed_ids})

candidates = {rec['id']: dict(rec, rel='SEED') for rec in seed_list}
for row in susjedi_res:
    base = seed_score.get(row['sid'], 0.0)
    for s in (row['susjedi'] or []):
        if not s or not s.get('id') or not s.get('tekst'):
            continue
        if s['id'] in candidates:
            continue
        w = GRAPH_REL_WEIGHTS.get(s.get('rel'), 0.7)
        candidates[s['id']] = {'id': s['id'], 'broj': s['broj'],
                                'tekst': s['tekst'], 'zakon': s['zakon'],
                                'score': base * w, 'rel': s.get('rel')}

best = sorted(candidates.values(), key=lambda x: x['score'],
              reverse=True)[:top_k]

parts = []
for c in best:
    tag = ' ' if c['rel'] == 'SEED' else f" | veza={c['rel']}"
    tekst = c['tekst'][:GRAPH_MAX_CHARS_PO_CLANKU]
    parts.append(f"[{c['zakon']}, Clanak {c['broj']}] | "
                f"score={c['score']:.2f}{tag}]\n{tekst}")
return '\n\n---\n\n'.join(parts)

print('\u2713 GraphRAG retriever spreman (per-dokument + stavci + tipizirane
↪veze)')

```

GraphRAG retriever spreman (per-dokument + stavci + tipizirane veze)

0.11 Ekspertne veze

```

[ ]: EKSPERTNE_VEZE = [
    ]

def unesi_ekspertne_veze(veze):
    ok, fail = 0, []

```

```

for src, dst, tip, pid in veze:
    if tip not in ('RAZRADJUJE', 'ISTA_TEMA'):
        fail.append((src, dst, f'nepoznat tip {tip}')); continue
    res = neo4j_run(f'''
        MATCH (a:Clanak {{id: $src}})
        MATCH (b:Clanak {{id: $dst}})
        MERGE (a)-[r:{tip}] {{izvor: 'ekspert', pitanje_id: $pid}}->(b)
        RETURN a.id AS a, b.id AS b
    ''', {'src': src, 'dst': dst, 'pid': pid})
    if res: ok += 1
    else: fail.append((src, dst, 'cvor nije pronaden'))
print(f'\u2713 Uneseno {ok}/{len(veze)} ekspertnih veza')
for f in fail:
    print(f' \u2717 {f}')

if EKSPERTNE_VEZE:
    unesi_ekspertne_veze(EKSPERTNE_VEZE)
else:
    print('(lista EKSPERTNE_VEZE je prazna - GraphRAG-auto rezim)')

```

(lista EKSPERTNE_VEZE je prazna - GraphRAG-auto rezim)

0.12 Dijagnostika konteksta

```

[ ]: def dijagnoza_konteksta(query, kategorija='kompleksno_vise_dokumenata'):
    ctx = graph_retrieve_context(query, kategorija=kategorija)
    blokovi = ctx.split('\n\n---\n\n') if ctx else []
    iz_grafa = sum(1 for b in blokovi if '|' veza=' in b)
    zakoni = set()
    for b in blokovi:
        header = b.split(' ')[0]
        zakoni.add(header.split(',')[0].strip('['))
    print(f'Pitanje: {query[:70]}...')
    print(f'  Clanaka u kontekstu: {len(blokovi)}')
    print(f'  Iz grafa (veze):      {iz_grafa}')
    print(f'  Pokriveno zakona:      {len(zakoni)} -> {sorted(zakoni)}')
    return ctx

_ = dijagnoza_konteksta('Kako se povezuju prigovor savjesti iz Ustava, '
                        'civilna sluzba i sustav vojne obveze?')
print()
_ = dijagnoza_konteksta('Povežite ustavnu ulogu Predsjednika kao vrhovnog '
                        'zapovjednika s lancem zapovijedanja nacelnika Glavnog_
↳stožera.')

```

Pitanje: Kako se povezuju prigovor savjesti iz Ustava, civilna sluzba i sustav

...

Clanaka u kontekstu: 12

Iz grafa (veze): 2
Pokriveno zakona: 4 -> ['Pravilnik o temeljnom vojnom osposobljavanju', 'Ustav Republike Hrvatske 2018', 'Zakon o obrani 2025', 'Zakon o sluzbi u Oruzanim snagama Republike Hrvatske 2025']

Pitanje: Povezite ustavnu ulogu Predsjednika kao vrhovnog zapovjednika s lancem...

Clanaka u kontekstu: 12
Iz grafa (veze): 0
Pokriveno zakona: 4 -> ['Pravilnik o temeljnom vojnom osposobljavanju', 'Ustav Republike Hrvatske 2018', 'Zakon o obrani 2025', 'Zakon o sluzbi u Oruzanim snagama Republike Hrvatske 2025']

0.13 Provedba — GraphRAG pristup

```
[ ]: def graph_rag_chat(user_question, kategorija=None):  
    context = graph_retrieve_context(user_question, kategorija=kategorija)  
    messages = [  
        {'role': 'system', 'content': SYSTEM_PROMPT + '\n\nKontekst za odgovor:  
↪\n' + context},  
        {'role': 'user', 'content': f'{user_question}\n\nOdgovori NA HRVATSKOM_↪  
↪JEZIKU koristeći LATINICU.'}  
    ]  
    return generate_response(messages, max_new_tokens=512, temperature=0.3,↪  
↪do_sample=False)
```

```
[ ]: if qa_data is None:  
    print('QA datoteka nije učitana.')  
else:  
    df_rezultati = pd.read_csv(csv_path, encoding='utf-8-sig')  
  
    KATEGORIJA_MAP = {  
        'jednostavna_pitanja': 'jednostavno',  
        'kompleksna_pitanja_jedan_dokument': 'kompleksno_jedan_dokument',  
        'kompleksna_pitanja_vise_dokumenata': 'kompleksno_vise_dokumenata',  
    }  
  
    all_questions = []  
    all_categories = []  
    for key, value in qa_data.items():  
        if key != 'metadata' and isinstance(value, list):  
            for item in value:  
                all_questions.append(item)  
                all_categories.append(KATEGORIJA_MAP.get(key, key))  
  
    graph_rag_odgovori = []  
    graph_rag_vremena = []
```

```

    for i, (qa_item, kategorija) in enumerate(zip(all_questions,
↳all_categories)):
        user_question = qa_item.get('pitanje', qa_item.get('question', ''))
        ocekivani = qa_item.get('odgovor', qa_item.get('answer', ''))

        print(f"\n{'='*80}")
        print(f'Pitanje {i + 1}/{len(all_questions)}: {user_question}')
        print(f'Kategorija: {kategorija} (top_k={GRAPH_TOP_K.get(kategorija,
↳GRAPH_TOP_K_DEFAULT)})')
        print(f'Ocekivani odgovor: {ocekivani}')
        print(f"{'='*80}")

        answer, vrijeme = graph_rag_chat(user_question, kategorija=kategorija)

        graph_rag_odgovori.append(answer)
        graph_rag_vremena.append(round(vrijeme, 2))

df_rezultati['graph_rag_odgovor'] = graph_rag_odgovori
df_rezultati['graph_rag_vrijeme_sekunde'] = graph_rag_vremena
df_rezultati.to_csv(csv_path, index=False, encoding='utf-8-sig')
print(f"\n{'='*80}")
print(f'GraphRAG rezultati nadopunjeni u: {csv_path}')
print(f'Ukupno obradeno pitanja: {len(graph_rag_odgovori)}')
print(f'Prosjecno vrijeme po pitanju: {sum(graph_rag_vremena)/
↳len(graph_rag_vremena):.2f} sekundi")

```

```
=====
```

Pitanje 1/31: Opišite cjelokupnu proceduru transformacije novaka u razvrstanog pričuvnika kroz sve pravne akte koji reguliraju taj proces.

Kategorija: kompleksno_vise_dokumenata (top_k=12)

Ocekivani odgovor: Procedura je regulirana kroz 4 pravna akta. 1. USTAVNI TEMELJ (Ustav RH, čl. 47). Vojna obveza je dužnost svih za to sposobnih državljana. Dopusšten prigovor savjesti iz vjerskih i/ili moralnih razloga uz obvezu ispunjavanja drugih dužnosti određenih zakonom. 2. NOVAČKA OBVEZA (Zakon o obrani, čl. 19-21). Nastaje u kalendarskoj godini kad državljanin RH navrší 18 godina. Državljanin RH uvodi se u vojnu evidenciju u kalendarskoj godini u kojoj navrší 18 godina života. Prolazi zdravstveni pregled i psihologijsko ispitivanje. 3. TEMELJNO VOJNO OSPOSOBLJAVANJE (Zakon o obrani čl. 21.m, Pravilnik o temeljnom vojnom osposobljavanju). Traje 2 mjeseca. Ministar obrane donosi Plan upućivanja u tekućoj godini do 15.12. za iduću godinu. Odluku o upućivanju donosi 40 dana prije upućivanja kandidata (Pravilnik čl. 7). Ročnici se obučavaju za specijalnost 11A - strijelac (Pravilnik čl. 9). Daju svečanu prisegu. 4. STATUS ROČNIKA (Zakon o službi, čl. 25, 26, 30). Ročnik je dio mirnodopskog sastava OS tijekom temeljnog vojnog osposobljavanja. Odluka o temeljnom vojnom osposobljavanju uređuje prava i obveze (nije upravna stvar). 5.

PRIJELAZ U PRIČUVU (Zakon o obrani, čl. 25, 27). Nakon temeljnog vojnog osposobljavanja automatski postaje pričuvnik. Razvrstani je pričuvnik jer ima određenu vojnostručnu specijalnost. Određuje mu se ratni raspored. Može biti pozivan na vježbe do 90 dana godišnje

=====

Generirani odgovor:

Cjelokupna procedura transformacije novaka u razvrstanog pričuvnika regulirana je slijedećim pravnim aktima:

1. ****Zakon o službi u Oružanim snagama Republike Hrvatske**** (članak 31):
 - Novaci se popunom ratnog ustroja Oružanih snaga obračunavaju prema odredbama Zakona o obrani.
 - Ugovorni pričuvnici se pozivaju i upućuju na službu u skladu s odredbama Zakona o obrani i ovog Zakona.
 2. ****Zakon o obrani**** (članak 21.a):
 - Pri dolasku u Oružane snage, pričuvnici moraju ispunjavati uvjete iz članka 21.a, uključujući putovanje od prebivališta do mjesta službe i natrag.
 3. ****Pravilnik o temeljnom vojnom osposobljavanju**** (članak 2):
 - Pojam "ročnik" uključuje i one koji se dragovoljno prijave za osposobljavanje, što može dovesti do klasifikacije kao pričuvnika.
 4. ****Postupak prijave i evidencije kandidata**** (Pravilnik o temeljnom vojnom osposobljavanju, članak 1):
 - Kandidati se prijavljuju i evidentiraju prema propisanom postupku, što omogućuje njihov prijelaz u status pričuvnika nakon osposobljavanja.
 5. ****Kriteriji za razvrstavanje u pričuvnike**** (Zakon o obrani, članak 25.a):
 - Vojni obveznik koji je odslužio temeljno osposobljavanje postaje razvrstan pričuvnik.
 6. ****Mogućnost dragovoljnog osposobljavanja**** (Zakon o obrani, članak 25.b):
 - Oni koji su uspješno završili dragovoljno osposobljavanje također ulaze u sustav pričuvništva.
 7. ****Dodatni uvjeti za razvrstavanje**** (Zakon o obrani, članak 25.c-25.
- Ukupno vrijeme za odgovor: 65.87 sekundi

=====

Pitanje 2/31: Kako je regulirana hijerarhija odlučivanja o uporabi Oružanih snaga u različitim situacijama kroz Ustav i Zakon o obrani?

Kategorija: kompleksno_vise_dokumenata (top_k=12)

Očekivani odgovor: Hijerarhija odlučivanja detaljno razlikuje situacije. 1.

RATNO STANJE (Ustav čl. 100, Zakon čl. 47). Predsjednik Republike naređuje

uporabu i zapovijeda. Ima punu ovlast i odgovoran je za obranu. 2. STANJE

NEPOSREDNE UGROŽENOSTI (Ustav čl. 100, Zakon čl. 48). Predsjednik Republike može

narediti uporabu. Potreban je supotpis predsjednika Vlade. 3. PRELAZAK GRANICE (Ustav čl. 7, Zakon čl. 49). Hrvatski sabor donosi odluku (većina svih zastupnika). Vlada predlaže. Predsjednik daje prethodnu suglasnost. Ako Predsjednik uskrati suglasnost, potrebna je dvotrećinska većina. 4. POMOĆ SAVEZNICIMA (Ustav čl. 7, Zakon čl. 51). Sabor donosi odluku. Vlada predlaže. Predsjednik daje suglasnost. 5. VJEŽBE I OBUKA (Ustav čl. 7, Zakon čl. 69). Vlada donosi odluku. Predsjednik daje suglasnost. Za manje postrojbe (do razine voda) odluku donosi ministar obrane. 6. PROVEDBA (Zakon čl. 70). Načelnik Glavnog stožera ima ovlast punog zapovijedanja. Odgovoran je Predsjedniku za zapovijedi i naredbe, a ministru za provedbu odluka

=====

Generirani odgovor:

Hierarhija odlučivanja o uporabi Oružanih snaga u različitim situacijama regulirana je kroz **Ustav Republike Hrvatske** i **Zakon o obrani** kako slijedi:

Prema Ustavu (Članak 7):

- **Ratno stanje / Teritorijalna cjelovitost:**
 - Odluku o upotrebi Oružanih snaga donose **Hrvatski sabor**, **Predsjednik Republike**, **predsjednik Vlade** i **ministar obrane** zajedno, u skladu s Ustavom i Zakonom o obrani.
 - Ako Predsjednik odbije suglasnost, Hrvatski sabor donosi odluku **dvotrećinskom većinom**.
- **Međunarodni ugovori / Saveznička pomoć:**
 - Ulazak stranih Oružanih snaga u RH moguć je samo na temelju **međunarodnih ugovora** i odluke Hrvatskog sabora (uz suglasnost Predsjednika).
- **Protupožarna zaštita / Spašavanje:**
 - Oružane snage mogu pomagati vatrogastvu i civilnoj zaštiti po odluci Vlade.

Prema Zakonu o obrani (Članak 46 & 57):

- **Borbeno djelovanje protiv civilnih zrakoplova:**
 - Regulirano posebnim zakonom (npr. Zbirka zakona o zračnoj plovidbi).
- **Glavni stožer kao izvršni organ:**
 - Sve odluke o upotrebi Oružanih snaga (osim u ratu) provode se preko **zapovijedi načelnika Glavnog stožera**.

Izvori:

- [Ustav RH, Članak 7] (<https://www.sabor.hr/Search/DocDetail.aspx?>)
- Ukupno vrijeme za odgovor: 65.68 sekundi

=====

Pitanje 3/31: Objasnite sve oblike obveza državljana u obrani prema Zakonu o obrani i Ustavu.

Kategorija: kompleksno_vise_dokumenata (top_k=12)

Ocekivani odgovor: Državljeni RH imaju 6 kategorija obveza u obrani. 1. USTAVNI TEMELJ (Ustav RH, čl. 47). Vojna obveza i obrana RH dužnost je svih za to sposobnih državljana. Dopušten je prigovor savjesti. 2. VOJNA OBVEZA (Zakon o obrani, čl. 19-21). Nastaje u godini kada državljanin navrší 18 godina. Prestaje s 55 godina za muškarce i 50 godina za žene. Sastoji se od novačke obveze, temeljnog vojnog osposobljavanja ili civilne službe te služenja u pričuvi. 3. SLUŽENJE U UGOVORNOJ PRIČUVI (Zakon o obrani, čl. 27, 28). Obuhvaća dragovoljno / temeljno vojno osposobljavanje. Osposobljavanje traje do 90 dana godišnje. 4. RADNA OBVEZA (Zakon o obrani, čl. 31). Odnosi se na pravne osobe i državljane RH. Uvodi se u ratnom stanju i stanju neposredne ugroženosti. 5. RADNA POMOĆ (Zakon o obrani, čl. 31-32). Može se uvesti u ratnom stanju i stanju neposredne ugroženosti. Služi za izvršenje zadaća nositelja obrambenih funkcija. 6. MATERIJALNA OBVEZA (Zakon o obrani, čl. 33). Obuhvaća ustupanje objekata i materijalnih sredstava. Za obrambene zadaće i osposobljavanje

=====

Generirani odgovor:

Prema ****Zakonu o obrani**** i ****Ustavu Republike Hrvatske****, građani imaju različite oblike obveza u obrani:

1. ****Vojna obveza****

- Prema ****Članku 19.**** Zakona o obrani, vojna obveza obuhvaća:
 - Osposobljavanje, pripremu i sudjelovanje u obrani.
 - Nastaje u kalendarskoj godini kada građanin navrší ****18 godina****.
- Ustav (****Članak 47.****) također spominje da je obrana domovine dužnost građana.

2. ****Priopćenje savjesti****

- Građani mogu odbiti vojnu službu iz vjerskih ili moralnih razloga (Ustav, ****Članak 47.****).
- Moraju ispunjavati druge zakonske obveze umjesto vojne službe.

3. ****Radna i materijalna obveza****

- Prema ****Članku 21.f.**** Zakona o obrani, građani su dužni pružiti materijalnu potporu obrani.
- Postoji i ****radna obveza**** (npr. za pripadnike nekih profesija poput policajaca), ali moraju biti obaviješteni unaprijed (****Članak 21.j.****).

4. ****Druge posebne obveze****

- Za određene skupine (npr. studente, umirovljenike) postoje odgode ili izuzeća (****Članak 21.k-q.****).
- Građani moraju poštovati odluke nadležnih tijela (npr. pozivi na osposobljavanje, Pravilnik o temeljnom vojnom osposobljavanju, ****Članak 4.****).

Za detaljnije informacije pogledajte odgovarajuće članke Zakona o obrani i Ustava RH.

Ukupno vrijeme za odgovor: 57.98 sekundi

=====

Pitanje 4/31: Objasnite sustav imenovanja i odgovornosti načelnika Glavnog stožera kroz sve relevantne propise.

Kategorija: kompleksno_jedan_dokument (top_k=8)

Očekivani odgovor: Položaj načelnika Glavnog stožera OS RH reguliran je kroz pet razina. Imenuje ga Predsjednik Republike na prijedlog Vlade, nakon pribavljenog mišljenja Odbora za obranu Sabora. Mandat traje četiri godine bez mogućnosti ponovnog imenovanja. Razrješuje ga Predsjednik Republike izravno ili na prijedlog Vlade, nakon mišljenja Odbora za obranu Sabora. Na čelu je Glavnog stožera i nadređen je svim stožerima, zapovjedništvima i postrojbama OS-a. Obnaša dužnost glavnog vojnog savjetnika Predsjedniku i ministru obrane. Ima ovlast punog zapovijedanja u OS-u. Djeluje na temelju zapovijedi Predsjednika, odluka ministra obrane i Zakona. Predsjedniku je odgovoran za provedbu zapovijedi i naredbi, a ministru obrane za provedbu odluka. Obvezan je istodobno izvještavati oba o uporabi Oružanih snaga.

=====

Generirani odgovor:

Sustav imenovanja i odgovornosti ****Načelnika Glavnog stožera**** Oružanih snaga Republike Hrvatske regulisan je kroz više propisa, uključujući ****Zakon o obrani**** i ****Zakon o službi u Oružanim snagama RH****. Evo ključnih elemenata:

****Imenovanje Načelnika Glavnog stožera****

- Prema ****Članku 15. Zakona o obrani****, ****Načelnik Glavnog stožera**** imenuje se na razdoblje od ****četiri godine**** i ne može biti ponovno imenovan.
- Ustroj Glavnog stožera propisuje ****Predsjednik Republike**** na prijedlog ****Načelnika Glavnog stožera**** i uz suglasnost ****ministra obrane**** (Članak 15.).
- Trenutni Načelnik Glavnog stožera zadržava dužnost do isteka mandata ako je imenovan prije stupanja na snagu novog Zakona (Članak 123).

****Dužnosti i odgovornosti****

- Prema ****Članku 70. Zakona o obrani****, Načelnik Glavnog stožera ima ****punu ovlast zapovijedanja u Oružanim snagama**** temeljen na:
 - Naredbama ****Predsjednika Republike****
 - Odlukama ****ministra obrane****
 - Odredbama Zakona o obrani
- Odgovoran je ****Predsjedniku Republike**** za provedbu zapovijedi i ****ministru obrane**** za provedbu odluka (Članak 70.).
- Dužan je o provedbi akata vezanih uz uporabu Oružanih snaga ****istodobno izvijestiti**** predsjednika i ministra (Članak 70.).
- Može ****privremeno staviti postrojbe u pripravnost**** u slučaju opasnosti (npr. katastrofe, velike nesreće) i mora o tome odmah izvijestiti predsjednika i ministra (Članak 70.).
- Kao ****glavni vojni savjetnik**** predsjedniku i ministru, koordinira pripremu, zapovijedanje i uporabu Oružanih snaga (Članak 15.).

Ukupno vrijeme za odgovor: 65.09 sekundi

=====

Pitanje 5/31: Objasnite razlike između uporabe i korištenja Oružanih snaga prema

Zakonu o obrani, uključujući tko donosi odluke u svakom slučaju.

Kategorija: kompleksno_jedan_dokument (top_k=8)

Ocekivani odgovor: Zakon o obrani jasno razlikuje UPORABU i KORIŠTENJE OS. UPORABA (čl. 46-57) odnosi se na borbena djelovanja. U ratnom stanju Predsjednik naređuje uporabu. U stanju neposredne ugroženosti Predsjednik odlučuje uz supotpis predsjednika Vlade. Za prelazak granice odluku donosi Hrvatski sabor, Vlada predlaže, a Predsjednik daje suglasnost. Za pomoć saveznicima odlučuju Sabor, Vlada i Predsjednik. Za operacije u inozemstvu odlučuju Sabor, Vlada i Predsjednik. KORIŠTENJE (čl. 58-64) odnosi se na neborbena djelovanja. Kod katastrofa Vlada donosi odluku. Kod velikih nesreća odlučuje ministar obrane. Kod protupožarne zaštite odlučuje ministar obrane. Kod traganja i spašavanja odlučuje ministar obrane. Kod pomoći policiji na granici odlučuje Vlada uz suglasnost Predsjednika. KLJUČNE RAZLIKE: uporaba je borbena, izvan RH, o njoj odlučuju Sabor i Predsjednik; korištenje je neborbena, unutar RH, o njemu odlučuju Vlada i ministar

=====
Generirani odgovor:

Prema **Zakonu o obrani**, postoje jasne razlike između **korištenja** i **upotrebe** Oružanih snaga, ovisno o situaciji i nadležnim tijelima koja donose odluke.

Korišćenje Oružanih snaga

- Riječ je o svakodnevnom korištenju snaga u mirnodopskim aktivnostima (npr. vježbe, održavanje reda, humanitarne misije).
- **Donose odluku:**
 - **Vlada** ili **ministar obrane** (Članak 58).
 - Odluke se provode preko **zapovijedi načelnika Glavnog stožera** (Članak 58).

Upotreba Oružanih snaga

- Primjenjuje se u specifičnim situacijama, poput:
 - Ratnog stanja (Članak 46).
 - Proturaketnih operacija (Članak 57).
- **Donose odluku:**
 - **Hrvatski sabor**, **Predsjednik Republike**, **predsjednik Vlade** i **ministar obrane** zajedno (Članak 46).
 - Provodi se preko **zapovijedi načelnika Glavnog stožera** (Članak 46).

Ključna razlika

- **Korišćenje** odnosi se na redovitu upotrebu snaga u mirnodopskom vremenu.
- **Upotreba** uključuje hitne/vojne situacije gdje odlučuju više politički i vojni dužnosnici.

Izvor: [Zakon o obrani 2025, Članak 46 & 58] (<https://www.nacionalnaobrana.gov.hr>).
Ukupno vrijeme za odgovor: 53.01 sekundi

Pitanje 6/31: Koja su prava i obveze ročnika tijekom temeljnog vojnog osposobljavanja prema Pravilniku o temeljnom vojnom osposobljavanju?

Kategorija: kompleksno_jedan_dokument (top_k=8)

Ocekivani odgovor: OBVEZE ROČNIKA. Življenje i rad u skladu s propisima o službi u OS. Osposobljavanje za specijalnost 11A (strijelac). Svečana prisega pred ministrom obrane. Pridržavanje vojnih pravila i discipline. PRAVA ROČNIKA. 1. FINANCIJSKA. Plaća ili naknada prema Zakonu o službi. Naknada prijevoza od i do prebivališta. 2. ZDRAVSTVENA ZAŠTITA. Osnovno zdravstveno osiguranje. Dopunsko zdravstveno osiguranje. Osiguranje od ozljede na radu. Osiguranje od nesretnog slučaja. 3. MATERIJALNA. Smještaj. Prehrana. Vojna odora. Sportska oprema. 4. PRIZNANJA. Značka najboljeg ročnika naraštaja. Pravo izbora mjesta službe pri prijmu u djelatnu službu (za najboljeg ročnika)

=====

Generirani odgovor:

Prema ****Pravilniku o temeljnom vojnom osposobljavanju****, prava i obveze ročnika tijekom temeljnog vojnog osposobljavanja uključuju sljedeće:

**Prava ročnika:**

1. ****Plaća / Naknada**** - Prema članku 18. Pravilnika, ročnici imaju pravo na plaću odnosno naknadu u iznosu određenom zakonom kojim se uređuje služba u Oružanim snagama.
2. ****Naknada troškova prijevoza**** - Prema članku 18. Pravilnika, ročnici imaju pravo na naknadu troškova javnog prijevoza u visini cijene karte najjeftinijeg javnog prijevoza pri dolasku na mjesto osposobljavanja i povratku.
3. ****Zdravstveno osiguranje**** - Prema članku 18. Pravilnika, ročnici imaju pravo na osnovno i dopunsko zdravstveno osiguranje, osiguranje od posljedica nesretnog slučaja, obvezno zdravstveno osiguranje za slučaj ozljede na radu, kao i smještaj i prehranu.
4. ****Mokra i sportska oprema**** - Prema članku 18. Pravilnika, ročnici dobivaju vojnu odoru i sportsku opremu.
5. ****Izlazak izvan vojne lokacije**** - Prema članku 16. Pravilnika, ročnici imaju pravo na izlazak iz vojne lokacije u skladu s rasporedom utvrđenim programom osposobljavanja.

**Obaveze ročnika:**

- ****Pohađanje programa osposobljavanja**** - Prema članku 3. Pravilnika, ročnici moraju proći program temeljnog vojnog osposobljavanja kako bi stekli vojna znanja i vještine potrebne za službu u Oružanim snagama.
- ****Spoštivanje propisa i pravila**** - Rukovnici su dužni pridržavati se svih propisa i pravila iz članka 1. Pravilnika.

Ako vam trebaju dodatne pojedinosti, slobodno pitajte!

Ukupno vrijeme za odgovor: 62.53 sekundi

=====

Pitanje 7/31: Objasnite razlike između mirnodopskog i ratnog sastava Oružanih snaga prema Zakonu o službi.

Kategorija: kompleksno_jedan_dokument (top_k=8)

Očekivani odgovor: MIRONODOPSKI SASTAV. Čine ga djelatne vojne osobe, službenici i namještenici, pričuvnici pozvani na službu, ugovorni pričuvnici, kadeti te ročnici na temeljnom vojnom osposobljavanju. Svrha mu je redovno funkcioniranje OS. Veličina je određena mirnodopskim ustrojem. RATNI SASTAV. Čine ga svi iz mirnodopskog sastava te dodatno mobilizirani i razvrstani pričuvnici, nerazvrstani pričuvnici i drugi vojni obveznici. Svrha mu je borbeno djelovanje. Veličina je određena ratnim ustrojem i znatno je veća. PRIJELAZ. Odvija se mobilizacijom u ratnom stanju i stanju neposredne ugroženosti. Odluku donosi Predsjednik Republike. Mobilizacija može biti opća ili djelomična. Mobilizacijom OS prelaze iz mirnodopskog u ratno ustrojstvo

Generirani odgovor:

Prema ****Zakonu o službi u Oružanim snagama Republike Hrvatske****, razlike između ****mirnodopskog**** i ****ratnog sastava**** Oružanih snaga definirane su sljedećim načinom:

****Mirnodopski sastav****

- Prema ****Članku 25.**** Zakona, mirnodopski sastav čine:
 - ***Djelatne vojne osobe*** (vojnici, časnici, dočasnici, mornari).
 - ***Službenici i namještenici*** (npr. civilni zaposlenici u MORH-u).
 - ***Pričuvnici pozvani na službu***, uključujući:
 - ***Ugovorne pričuvnike***.
 - ***Kadete*** (studenti vojne škole).
 - ***Ročnike*** (osobe koje su prošle temeljno vojno osposobljavanje).
 - ***Osobe koje su pristupile temeljnom vojnom osposobljavanju***.

****Ratni sastav****

- Prema istom članku, ratni sastav uključuje sve iz mirnodopskog sastava ****plus****:
 - ***Mobilizirane vojne obveznike*** (npr. ročnici koji nisu još služili).

Ključna razlika

- ****Mirnodopski sastav**** odnosi se na stalni dio OS-a (aktivno osoblje + pričuve).
- ****Ratni sastav**** uključuje i mobiliziranu narodnu gardu (vojne obveznike) u slučaju krize.

Za dodatne detalje pogledajte ****Članak 43.**** Zakona o obrani, koji dodatno pojašnjava ustroj vojske.

Ukupno vrijeme za odgovor: 50.32 sekundi

Pitanje 8/31: Objasnite postupak i uvjete za odgodu temeljnog vojnog osposobljavanja prema Zakonu o obrani.

Kategorija: kompleksno_jedan_dokument (top_k=8)

Očekivani odgovor: POSTUPAK. Zahtjev za odgodu podnosi se nadležnom područnom

odjelu za poslove obrane u roku od osam dana od dana primitka poziva na temeljno vojno osposobljavanje. Može se podnijeti osobno, preporučenom pošiljkom ili elektroničkim putem. Uz zahtjev se prilažu odgovarajući dokazi (npr. dokaz o nezaposlenosti ostalih članova obitelji, potvrda o datumu sklapanja braka, izvadak iz matice umrlih, dokaz o trudnoći, potvrda o registraciji obrta).

RAZLOZI ZA ODGODU. 1. ŠKOLOVANJE. Novak upisan na visoko učilište dostavlja potvrdu o upisu najkasnije do 31. listopada tekuće godine, a odgoda traje najdulje do 30. rujna godine u kojoj navršava 29 godina. 2. OSOBNI I OBITELJSKI RAZLOZI. Odgoda se može odobriti ako u kućanstvu nema drugog člana koji privređuje (najdulje godinu dana), ako je novak vježbenik ili se prvi put zaposlio (do isteka staža/probnog rada), ako u kućanstvu ima članove kojima je prijeko potrebna tuđa pomoć i njega, ako se iz istog kućanstva istodobno upućuju dva ili više članova, ako ima određen datum sklapanja braka (najdulje tri mjeseca), ako očekuje rođenje djeteta (najdulje godinu dana), ako je nastupila smrt u užoj obitelji (do idućeg roka za upućivanje) te ako je otvorio obrt ili samostalnu djelatnost (najdulje godinu dana). 3. SPORTAŠI. Zbog sudjelovanja na svjetskim i europskim prvenstvima te državnim prvenstvima odgoda se može odobriti najdulje do jedne godine. ODLUČIVANJE. O zahtjevu odlučuje nadležni područni odjel za poslove obrane odlukom. Kod odgode zbog školovanja i sporta (čl. 21.k) odluka mora sadržavati elemente rješenja prema Zakonu o općem upravnom postupku; protiv nje nije dopuštena žalba, ali se može pokrenuti upravni spor pred Visokim upravnim sudom RH, koji odlučuje u roku od 90 dana. Odluka o odgodi sadrži dan, mjesec i godinu do kada se osposobljavanje odgađa

=====

Generirani odgovor:

Prema ****Zakonu o obrani**** (NN 136/25), članak 25.a definira uvjete i postupak za odgodu temeljnog vojnog osposobljavanja. Evo ključnih elemenata:

****1. Opći uvjeti za odgodu****

Ročnik može dobiti odgodu ako ispunjava barem jedno sljedeće:

- ****Bolest**** - do oporavka (potrebna liječnička potvrda).
- ****Situacija u kućanstvu**** - npr. smrt člana obitelji, teška bolest člana kućanstva, neodgovorni poslovi (npr. poljoprivreda) itd. (potrebni dokazi poput potvrde škole, certifikata za poljoprivredu).
- ****Akademske obrazovanje**** - ako je sprečan polaganje ispita ili završetak godine (potvrda fakulteta).
- ****Planirano vjenčanje**** - do datuma vjenčanja.
- ****Rodiljni/roditeljski dopust**** - za muškarce (potvrda HZZO-a).

****2. Postupak za dobivanje odgode****

1. ****Podnesite zahtjev**** - putem obrasca dostupnog na područnom odjelu za poslove obrane.
2. ****Priložite dokaze**** - ovisno o razlogu (liječnička potvrda, potvrda fakulteta, certifikat poljoprivrede itd.).
3. ****Prihvata odluka**** - Ministarstvo obrane donosi odluku najkasnije 40 dana prije roka osposobljavanja (članak 7. Pravilnika o temeljnom vojnom osposobljavanju).

3. Trajanje odgode

Trajanje ovisi o razlogu:

- Do oporavka (bolest).
- Dok traje situacija u kućanstvu (npr. do kraja godine za studente).
- Do datuma vjenčanja.

Za detaljnije informacije pogledajte **članak 25.a Zakona o obrani** i **pravilnik o temeljnom vojnom osposobljavanju**.

Ukupno vrijeme za odgovor: 63.49 sekundi

=====
Pitanje 9/31: Kada nastaje vojna obveza za državljane Republike Hrvatske?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Vojna obveza nastaje u kalendarskoj godini u kojoj državljani Republike Hrvatske navršavaju 18 godina života.

=====
Generirani odgovor:

Vojna obveza za državljane Republike Hrvatske **nastupa u kalendarskoj godini u kojoj navršavaju 18 godina života** [Zakon o obrani 2025, Članak 19].

Napomena: Ova informacija temelji se isključivo na dostupnim dokumentima.

Ukupno vrijeme za odgovor: 8.99 sekundi

=====
Pitanje 10/31: Koliko traje temeljno vojno osposobljavanje?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Temeljno vojno osposobljavanje traje dva mjeseca.

=====
Generirani odgovor:

Temeljno vojno osposobljavanje traje **30 dana**.

Izvor: [Pravilnik o temeljnom vojnom osposobljavanju, Članak 20] (<https://pravilnikik.pravactv>).

Ukupno vrijeme za odgovor: 11.21 sekundi

=====
Pitanje 11/31: Tko je vrhovni zapovjednik Oružanih snaga Republike Hrvatske?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Predsjednik Republike Hrvatske je vrhovni zapovjednik Oružanih snaga Republike Hrvatske.

=====
Generirani odgovor:

Vrhovni zapovjednik Oružanih snaga Republike Hrvatske je Predsjednik Republike Hrvatske. Ova informacija proizlazi iz Ustava Republike Hrvatske iz 2018., Članak 100, koji navodi da "Predsjednik Republike vrhovni je zapovjednik

oružanih snaga Republike Hrvatske.
Ukupno vrijeme za odgovor: 8.14 sekundi

=====
Pitanje 12/31: Koje su tri grane Oružanih snaga Republike Hrvatske?
Kategorija: jednostavno (top_k=5)
Ocekivani odgovor: Tri grane Oružanih snaga Republike Hrvatske su: Hrvatska
kopnena vojska, Hrvatska ratna mornarica i Hrvatsko ratno zrakoplovstvo.
=====

Generirani odgovor:
Tri grane Oružanih snaga Republike Hrvatske su:

1. **Hrvatska kopnena vojska** (Clanak 41, Stavak 3)
2. **Hrvatska ratna mornarica** (Clanak 41, Stavak 3)
- 3.

Ukupno vrijeme za odgovor: 11.42 sekundi

=====
Pitanje 13/31: Kada prestaje vojna obveza za muškarce?
Kategorija: jednostavno (top_k=5)
Ocekivani odgovor: Vojna obveza za muškarce prestaje na kraju kalendarske godine
u kojoj navršavaju 55 godina života.
=====

Generirani odgovor:
Vojna obveza za muškarce prestaje prema **Zakonu o obrani** (Clanak 20):
- Na kraju kalendarske godine u kojoj navršavaju **55 godina života**, ako su
ocijenjeni nesposobnima za vojnu službu.
- Posebno za **više časnike i generale/admirale**, vojna obveza prestaje
posljednjeg dana kalendarske godine u kojoj navršavaju **65 godina života**.

Dodatno, prema **Zakonu o službi u Oružanim snagama RH** (Clanak 210), vojni rok
započinje u kalendarskoj godini u kojoj građanin navršava **18 godina života**.

Napomena: Informacije su preuzete isključivo iz navedenih članaka zakona.
Ukupno vrijeme za odgovor: 23.93 sekundi

=====
Pitanje 14/31: Tko imenuje načelnika Glavnog stožera Oružanih snaga?
Kategorija: jednostavno (top_k=5)
Ocekivani odgovor: Načelnika Glavnog stožera Oružanih snaga imenuje Predsjednik
Republike Hrvatske na prijedlog Vlade Republike Hrvatske i nakon pribavljenog
mišljenja Odbora za obranu Hrvatskoga sabora.
=====

Generirani odgovor:
Načelnik Glavnog stožera Oružanih snaga imenuje se na razdoblje od četiri godine
i ne može biti ponovno imenovan. Načelnik Glavnog stožera imenuje se na

razdoblje od četiri godine i ne može biti ponovno imenovan.

****Izvor:**** [Zakon o obrani 2025, Članak 4] (<https://www.narodne-novine>).
Ukupno vrijeme za odgovor: 12.53 sekundi

=====
Pitanje 15/31: Koliko traje mandat načelnika Glavnog stožera?

Kategorija: jednostavno (top_k=5)

Očekivani odgovor: Načelnik Glavnog stožera imenuje se na razdoblje od četiri godine i bez mogućnosti reizbora.

=====
Generirani odgovor:

Mandat načelnika Glavnog stožera traje ****4 godine****.

Izvor: [Zakon o obrani 2025, Članak 14] (<https://www.zakonyprolidi>).
Ukupno vrijeme za odgovor: 8.07 sekundi

=====
Pitanje 16/31: Što je temeljna svrha obrane Republike Hrvatske?

Kategorija: jednostavno (top_k=5)

Očekivani odgovor: Temeljna svrha obrane je očuvanje suvereniteta, neovisnosti i teritorijalne cjelovitosti Republike Hrvatske.

=====
Generirani odgovor:

Temeljna svrha obrane Republike Hrvatske jest osigurati suverenitet, neovisnost i teritorijalnu cjelovitost zemlje, kao i zaštitu građana i njihovog života, slobode i imovine. Ova svrha je definirana u članku 3. Zakona o obrani, gdje se navodi da obrana uključuje pripremu, uporabu i korištenje snaga i sredstava u ratnom stanju, stanju neposredne ugroženosti neovisnosti, jedinstvenosti i opstojnosti Republike Hrvatske i miru.

Ukupno vrijeme za odgovor: 15.10 sekundi

=====
Pitanje 17/31: Do koje dobi novak može biti upućen na temeljno vojno osposobljavanje?

Kategorija: jednostavno (top_k=5)

Očekivani odgovor: Novak može biti upućen na temeljno vojno osposobljavanje do isteka 30 godina života.

=====
Generirani odgovor:

Novak može biti upućen na temeljno vojno osposobljavanje do navršenih ****29 godina**** starosti. Ova granica je definirana u članku 24.k Zakona o obrani 2025. godine.

Ako postoji dodatni kontekst ili pitanja vezana uz ovu temu, slobodno pitajte!

Ukupno vrijeme za odgovor: 10.49 sekundi

=====
Pitanje 18/31: Tko donosi odluku o mobilizaciji Oružanih snaga?
Kategorija: jednostavno (top_k=5)
Ocekivani odgovor: Odluku o mobilizaciji Oružanih snaga donosi Predsjednik Republike na prijedlog ministra obrane.
=====

Generirani odgovor:
Odluku o mobilizaciji Oružanih snaga donosi ****Vlada**** ili ****ministar obrane**** u skladu s odredbama Zakona o obrani.

Izvor: [Zakon o obrani 2025, Članak 74.](<https://www.novo-zakonodavstvo>).
Ukupno vrijeme za odgovor: 10.58 sekundi

=====
Pitanje 19/31: Kada prestaje vojna obveza za žene?
Kategorija: jednostavno (top_k=5)
Ocekivani odgovor: Vojna obveza za žene prestaje na kraju kalendarske godine u kojoj navršavaju 50 godina života.
=====

Generirani odgovor:
Prema Zakonu o obrani 2025., vojna obveza za žene prestaje kada navrše ****50 godina**** života.

Izvor: [Zakon o obrani 2025, Članak 20](https://www.nacionalno-sigurnosno-pravodavno-informaciono-podrstavanje.hr/documents/zakon_o_obrani_2025.pdf), stavak (1).
Ukupno vrijeme za odgovor: 13.84 sekundi

=====
Pitanje 20/31: U kojoj godini života se državljani RH uvodi u vojnu evidenciju?
Kategorija: jednostavno (top_k=5)
Ocekivani odgovor: Državljanin Republike Hrvatske uvodi se u vojnu evidenciju u kalendarskoj godini kada navrší 18 godina života.
=====

Generirani odgovor:
Građanin Republike Hrvatske uvodi se u vojnu evidenciju u kalendarskoj godini u kojoj navršava ****18 godina**** života.

Izvor: [Zakon o obrani 2025, Članak 19](<https://www.narodnonovine>).
Ukupno vrijeme za odgovor: 10.32 sekundi

=====
Pitanje 21/31: Tko proglašava ratno stanje u Republici Hrvatskoj?
Kategorija: jednostavno (top_k=5)
Ocekivani odgovor: Ratno stanje u Republici Hrvatskoj proglašava Hrvatski sabor.

=====
Generirani odgovor:

Ratno stanje u Republici Hrvatskoj proglašava Hrvatski sabor. Međutim, prema Ustavu Republike Hrvatske (Članak 100), u slučaju neposredne ugroženosti neovisnosti, jedinstvenosti i opstojnosti države, Predsjednik Republike može, uz supotpis predsjednika Vlade, narediti uporabu oružanih snaga iako nije proglašeno ratno stanje.

Ukupno vrijeme za odgovor: 10.29 sekundi

=====
Pitanje 22/31: Koliko dana prije upućivanja ministar obrane donosi odluku o upućivanju kandidata na temeljno vojno osposobljavanje?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Ministar obrane donosi odluku o upućivanju kandidata na temeljno vojno osposobljavanje u pravilu 40 dana prije upućivanja kandidata.

=====
Generirani odgovor:

Ministar obrane donosi odluku o upućivanju kandidata na temeljno vojno osposobljavanje najkasnije 25 dana prije datuma upućivanja. Ova informacija temelji se na članku 20. Pravilnika o temeljnom vojnom osposobljavanju.

Ukupno vrijeme za odgovor: 8.76 sekundi

=====
Pitanje 23/31: Tko donosi program obuke ročnika na temeljnom vojnom osposobljavanju?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Program obuke ročnika na temeljnom vojnom osposobljavanju donosi načelnik Glavnog stožera Oružanih snaga Republike Hrvatske.

=====
Generirani odgovor:

Program obuke ročnika na temeljnom vojnom osposobljavanju donosi **načelnik Glavnog stožera Oružanih snaga Republike Hrvatske**, prema članku 2. Pravilnika o temeljnom vojnom osposobljavanju.

Napomena: Ova informacija temelji se na članku 2. navedenog pravilnika.

Ukupno vrijeme za odgovor: 10.21 sekundi

=====
Pitanje 24/31: Pred kim ročnici daju svečanu prisegu?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Ročnici daju svečanu prisegu pred ministrom obrane ili osobom koju on za to odredi.

=====
Generirani odgovor:

Ročnici daju svečanu prisegu pred ministrom obrane ili osobom koju on za to odredi.

Ukupno vrijeme za odgovor: 3.30 sekundi

=====
Pitanje 25/31: Za koju vojnostručnu specijalnost se ročnici osposobljavaju na temeljnom vojnom osposobljavanju?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Ročnici se osposobljavaju u temeljnim vojnim znanjima i sposobnostima u rodu pješastva za streljačku specijalnost 11A, dužnost strijelca.

=====
Generirani odgovor:

Na temeljnom vojnom osposobljavanju, ročnici se osposobljavaju za vojnostručnu specijalnost "strijelac" (specijalnost 11A), prema zadatcima i standardima definiranim u programu obuke ročnika.

Ukupno vrijeme za odgovor: 7.71 sekundi

=====
Pitanje 26/31: Što čini mirnodopski sastav Oružanih snaga?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Mirnodopski sastav Oružanih snaga čine: djelatne vojne osobe, službenici i namještenici, pričuvnici pozvani na službu u Oružane snage, ugovorni pričuvnici, kadeti te ročnici koji su pristupili temeljnom vojnom osposobljavanju.

=====
Generirani odgovor:

Mirnodopski sastav Oružanih snaga čini:

- **Djelatne vojne osobe** - profesionalni časnici i dočasnici.
- **Službenici i namještenici** - civilni zaposlenici u vojsci.
- **Pričuvnici pozvani na obuku** - obveznik koji prolazi redovitu obuku.
- **Ugovorni pričuvnici** - dobrovoljno regrutirani civili s vojnim ugovorom.
- **Kadeti** - polaznici vojne škole.
- **Ročnici** - građani koji su završili temeljno vojno osposobljavanje.

Izvor: [Zakon o obrani 2025, Članak 43](<https://www.zakonyproljudi>).

Ukupno vrijeme za odgovor: 24.53 sekundi

=====
Pitanje 27/31: Tko utvrđuje mirnodopski i ratni ustroj Oružanih snaga?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Predsjednik Republike utvrđuje mirnodopski i ratni ustroj Oružanih snaga te strukturu činova u mirnodopskom i ratnom ustroju Oružanih snaga.

=====
Generirani odgovor:

Mirnodopski i ratni ustroj Oružanih snaga utvrđuje Predsjednik Republike Hrvatske. Ova informacija proizlazi iz **Članka 45.** Zakona o obrani 2025.,

gdje je navedeno da Predsjednik Republike utvrđuje mirnodopski i ratni ustroj Oružanih snaga te strukturu činova u mirnodopskom i ratnom ustroju Oružanih snaga.

Napomena: ova informacija nije pronađena u dostupnim dokumentima.

Ukupno vrijeme za odgovor: 13.43 sekundi

=====
Pitanje 28/31: Koliko traje minimalno godišnji odmor djelatne vojne osobe?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Djelatna vojna osoba ima pravo na godišnji odmor u trajanju od najmanje četiri tjedna.

=====
Generirani odgovor:

Minimalno trajanje godišnjeg odmora djelatne vojne osobe iznosi **četiri tjedna** (16 radnih dana), kako je definirano u **Članku 156. stavku 1.** Zakona o službi u Oružanim snagama Republike Hrvatske 2025.

Iako se u istom članku dopušta povećanje trajanja odmora prema određenim kriterijima (npr. radni staž, dužnost, zdravlje), minimalna granica ostaje nepromijenjena.

Ukupno vrijeme za odgovor: 15.14 sekundi

=====
Pitanje 29/31: Koliko dana plaćenog dopusta pripada djelatnoj vojnoj osobi u slučaju rođenja djeteta?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Djelatna vojna osoba ima pravo na plaćeni dopust od sedam radnih dana u slučaju rođenja djeteta.

=====
Generirani odgovor:

Djelatnoj vojnoj osobi u slučaju rođenja djeteta pripada **sedam radnih dana** plaćenog dopusta, kako je definirano u [Zakon o službi u Oružanim snagama Republike Hrvatske 2025, Članak 159] (https://www.narodne-novine.hr/#_133086). Ovaj se broj dana odnosi na osnove poput rođenja djeteta, gdje je točno navedeno "rođenja djeteta" sa sedmodnevnim trajanjem.

Ukupno vrijeme za odgovor: 14.96 sekundi

=====
Pitanje 30/31: Koliko dana plaćenog dopusta pripada djelatnoj vojnoj osobi u slučaju zaključenja braka?

Kategorija: jednostavno (top_k=5)

Ocekivani odgovor: Djelatna vojna osoba ima pravo na plaćeni dopust od sedam radnih dana u slučaju zaključenja braka.

=====
Generirani odgovor:

Djelatnoj vojnoj osobi u slučaju zaključenja braka pripada ****sedam radnih dana**** plaćenog dopusta, kako je definirano u ****Članku 159.**** Zakona o službi u Oružanim snagama Republike Hrvatske.

Ako okolnost nastupi u vrijeme korištenja godišnjeg odmora, privremene nesposobnosti ili odsutnosti iz službe iz drugih razloga, pravo na plaćeni dopust se ne ostvaruje.

Ukupno vrijeme za odgovor: 14.10 sekundi

=====
Pitanje 31/31: Što je vojna iskaznica i tko ju izdaje?

Kategorija: jednostavno (top_k=5)

Očekivani odgovor: Vojna iskaznica je javna isprava kojom vojni obveznik dokazuje identitet dok izvršava vojnu obvezu. Izdaje ju nadležni područni odjel za poslove obrane koji vojnog obveznika vodi u evidenciji.

=====
Generirani odgovor:

Vojna iskaznica je identifikacijski dokument koji dokazuje pripadnost Oružanim snagama Republike Hrvatske. Izdaje se vojnim osobama, službenicima i namještenicima u skladu sa Zakonom o obrani (članak 24). Ministar obrane propisuje oblik i sadržaj ove iskaznice pravilnikom. Ročnicima i pričuvnicima također se izdaje vojna iskaznica, ali prema posebnim propisima.

Ukupno vrijeme za odgovor: 12.66 sekundi

=====
GraphRAG rezultati nadopunjeni u: /content/drive/My

Drive/diplomskiRad/rezultati.csv

Ukupno obrađeno pitanja: 31

Prosječno vrijeme po pitanju: 24.63 sekundi

5. Prilog - Colab - Automatske evaluacije ChatBota

Link na Colab ipynb bilježnicu s kodom za evaluacije: [Colab - Evaluacija](#)

diplomskiRad_04_evaluacija

September 4, 2026

ChatGTP

Gdje to piše (DzeToPishe)

Diplomski rad - Automatsko ocjenjivanje odgovora

by Zlatko Pračić

0.1 1. Postavljanje okruženja

Ovaj dio notebooka instalira potrebne biblioteke za izračun metrika i uvozi sve module.

0.1.1 Instalacija potrebnih biblioteka

Instaliramo biblioteke za automatsko ocjenjivanje: - **rouge-score** - ROUGE-L metrika za najduži zajednički podniz - **bert-score** - BERTScore za kontekstualnu semantičku sličnost - **sentence-transformers** - embedding model za cosine similarity

```
[ ]: !pip install rouge-score bert-score  
!pip install -U sentence-transformers
```

0.1.2 Uvoz biblioteka

Uvozimo sve potrebne module na jednom mjestu za preglednost.

```
[2]: import pandas as pd  
import numpy as np  
import json  
import logging  
  
from rouge_score import rouge_scorer  
from bert_score import score as bert_score_fn  
  
from sentence_transformers import SentenceTransformer  
from sklearn.metrics.pairwise import cosine_similarity as sklearn_cosine  
  
print('Sve biblioteke uspješno učitane.')
```

Sve biblioteke uspješno učitane.

0.2 2. Učitavanje podataka

Učitavamo rezultate generiranja iz CSV datoteke i referentne odgovore iz JSON datoteke. CSV sadrži odgovore sva tri pristupa: **Vanilla**, **FAISS RAG** i **GraphRAG**.

0.2.1 Učitavanje CSV rezultata i JSON referenci

```
[3]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[4]: csv_path = '/content/drive/My Drive/diplomskiRad/rezultati.csv'
json_path = '/content/drive/My Drive/diplomskiRad/pitanja_odgovori.json'

df = pd.read_csv(csv_path, encoding='utf-8-sig')

with open(json_path, 'r', encoding='utf-8') as f:
    qa_data = json.load(f)

# Obavezni stupci
potrebni_stupci = ['rbr', 'kategorija', 'pitanje', 'ocekivani_odgovor',
                  'vanilla_odgovor', 'rag_odgovor']
nedostaju = [s for s in potrebni_stupci if s not in df.columns]
if nedostaju:
    raise ValueError(f'Nedostaju stupci u CSV-u: {nedostaju}')

# GraphRAG stupac je opcionalan (može se dodati naknadno)
ima_graph_rag = 'graph_rag_odgovor' in df.columns
if ima_graph_rag:
    print('\u2713 GraphRAG stupac pronađen - evaluacija sva tri pristupa.')
else:
    print('\u26a0 GraphRAG stupac nije pronađen - evaluacija Vanilla i FAISS_
    \u2192RAG.')

print(f'\nUčitano {len(df)} redaka iz CSV-a.')
print(f'Stupci: {list(df.columns)}')
print(f'\nKategorije:')
print(df['kategorija'].value_counts().to_string())
print(f'\nJSON metadata: {qa_data["metadata"]["ukupno_pitanja"]} pitanja')
```

GraphRAG stupac pronađen - evaluacija sva tri pristupa.

Učitano 31 redaka iz CSV-a.

Stupci: ['rbr', 'kategorija', 'pitanje', 'ocekivani_odgovor', 'vanilla_odgovor',
'vanilla_vrijeme_sekunde', 'rag_odgovor', 'rag_vrijeme_sekunde',
'graph_rag_odgovor', 'graph_rag_vrijeme_sekunde']

Kategorije:

kategorija	
jednostavna_pitanja	23
kompleksna_pitanja_jedan_dokument	5
kompleksna_pitanja_vise_dokumenata	3

JSON metadata: 31 pitanja

0.3 3. Funkcije za izračun metrika

Definiramo 3 metrike za automatsko ocjenjivanje odgovora: - **ROUGE-L** — leksička metrika, mjeri najduži zajednički podniz (F1) - **BERTScore** — semantička metrika, kontekstualna sličnost tokena - **Cosine Similarity** — semantička metrika, sličnost embedding vektora cijelih odgovora

```
[5]: def compute_rouge_l(reference, hypothesis):
    """Izračunava ROUGE-L F1 score između referentnog i generiranog odgovora.

    ROUGE-L mjeri najduži zajednički podniz (Longest Common Subsequence)
    i vraća F1 mjeru koja balansira preciznost i odziv.
    """
    scorer = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=False)
    scores = scorer.score(reference, hypothesis)
    return scores['rougeL'].fmeasure

def compute_bertscore(references, hypotheses):
    """Izračunava BERTScore za liste referenci i hipoteza (batch).

    Koristi bert-base-multilingual-cased koji podržava hrvatski jezik.
    Vraća F1 komponentu BERTScore-a.
    """
    P, R, F1 = bert_score_fn(
        hypotheses, references,
        model_type='bert-base-multilingual-cased',
        num_layers=9,
        verbose=True
    )
    return F1.tolist()

def compute_cosine_similarity(reference, hypothesis, model):
    """Izračunava cosine similarity između embeddinga referentnog i generiranog
    odgovora.

    Koristi multilingual-e5-large model za vektorizaciju cijelih odgovora.
    """
    emb_ref = model.encode([reference], convert_to_numpy=True)
    emb_hyp = model.encode([hypothesis], convert_to_numpy=True)
    return float(sklearn_cosine(emb_ref, emb_hyp)[0][0])
```



```
print('Funkcije za metrike definirane.')
```

Funkcije za metrike definirane.

0.4 4. Izračun metrika za sve odgovore

Za svaki od 31 redak računamo 3 metrike za sve prisutne pristupe (Vanilla, FAISS RAG, i GraphRAG ako je dostupan) u usporedbi s očekivanim (referentnim) odgovorom.

0.4.1 Inicijalizacija embedding modela

```
[ ]: _transformers_logger = logging.getLogger('transformers.modeling_utils')
     _prev_level = _transformers_logger.level
     _transformers_logger.setLevel(logging.ERROR)

     embed_model = SentenceTransformer('intfloat/multilingual-e5-large')

     _transformers_logger.setLevel(_prev_level)
     print('Embedding model učitano.')
```

0.4.2 Izračun ROUGE-L i Cosine Similarity

```
[7]: vanilla_rouge_l = []
     vanilla_cosine = []
     rag_rouge_l = []
     rag_cosine = []
     graph_rouge_l = []
     graph_cosine = []

     for i, row in df.iterrows():
         ref = str(row['ocekivani_odgovor'])
         van = str(row['vanilla_odgovor'])
         rag = str(row['rag_odgovor'])

         vanilla_rouge_l.append(compute_rouge_l(ref, van))
         vanilla_cosine.append(compute_cosine_similarity(ref, van, embed_model))
         rag_rouge_l.append(compute_rouge_l(ref, rag))
         rag_cosine.append(compute_cosine_similarity(ref, rag, embed_model))

         if ima_graph_rag:
             grph = str(row['graph_rag_odgovor'])
             graph_rouge_l.append(compute_rouge_l(ref, grph))
             graph_cosine.append(compute_cosine_similarity(ref, grph, embed_model))

     if (i + 1) % 10 == 0 or i == 0:
         print(f'Obradeno {i + 1}/{len(df)} pitanja...')
```

```

df['vanilla_rouge_l'] = vanilla_rouge_l
df['vanilla_cosine'] = vanilla_cosine
df['rag_rouge_l'] = rag_rouge_l
df['rag_cosine'] = rag_cosine
if ima_graph_rag:
    df['graph_rouge_l'] = graph_rouge_l
    df['graph_cosine'] = graph_cosine

print(f'\nROUGE-L i Cosine Similarity izračunati za svih {len(df)} pitanja.')

```

Obradeno 1/31 pitanja...

Obradeno 10/31 pitanja...

Obradeno 20/31 pitanja...

Obradeno 30/31 pitanja...

ROUGE-L i Cosine Similarity izračunati za svih 31 pitanja.

0.4.3 Izračun BERTScore (batch)

```

[ ]: references = df['ocekivani_odgovor'].astype(str).tolist()
vanilla_hypotheses = df['vanilla_odgovor'].astype(str).tolist()
rag_hypotheses = df['rag_odgovor'].astype(str).tolist()

print('Izračunavam BERTScore za vanilla odgovore...')
df['vanilla_bertscore'] = compute_bertscore(references, vanilla_hypotheses)

print('\nIzračunavam BERTScore za FAISS RAG odgovore...')
df['rag_bertscore'] = compute_bertscore(references, rag_hypotheses)

if ima_graph_rag:
    graph_hypotheses = df['graph_rag_odgovor'].astype(str).tolist()
    print('\nIzračunavam BERTScore za GraphRAG odgovore...')
    df['graph_bertscore'] = compute_bertscore(references, graph_hypotheses)

print(f'\nBERTScore izračunat za svih {len(df)} pitanja.')

```

0.4.4 Spremanje proširenog CSV-a

```

[9]: output_csv = '/content/drive/My Drive/diplomskiRad/rezultati_evaluacija.csv'
df.to_csv(output_csv, index=False, encoding='utf-8-sig')

print(f'Rezultati spremljeni u: {output_csv}')
print(f'Oblik: {df.shape[0]} redaka × {df.shape[1]} stupaca')
print(f'\nStupci: {list(df.columns)}')

# Pregled prvih redaka s metrikama

```

```
pregled_stupci = ['rbr', 'kategorija',
                  'vanilla_rouge_l', 'vanilla_bertscore', 'vanilla_cosine',
                  'rag_rouge_l', 'rag_bertscore', 'rag_cosine']
if ima_graph_rag:
    pregled_stupci += ['graph_rouge_l', 'graph_bertscore', 'graph_cosine']
df[pregled_stupci].head()
```

Rezultati spremljeni u: /content/drive/My
Drive/diplomskiRad/rezultati_evaluacija.csv
Oblik: 31 redaka × 19 stupaca

Stupci: ['rbr', 'kategorija', 'pitanje', 'ocekivani_odgovor', 'vanilla_odgovor',
'vanilla_vrijeme_sekunde', 'rag_odgovor', 'rag_vrijeme_sekunde',
'graph_rag_odgovor', 'graph_rag_vrijeme_sekunde', 'vanilla_rouge_l',
'vanilla_cosine', 'rag_rouge_l', 'rag_cosine', 'graph_rouge_l', 'graph_cosine',
'vanilla_bertscore', 'rag_bertscore', 'graph_bertscore']

```
[9]:      rbr      kategorija  vanilla_rouge_l \
0      1  kompleksna_pitanja_vise_dokumenata      0.108787
1      2  kompleksna_pitanja_vise_dokumenata      0.113333
2      3  kompleksna_pitanja_vise_dokumenata      0.116022
3      4  kompleksna_pitanja_jedan_dokument      0.194175
4      5  kompleksna_pitanja_jedan_dokument      0.169279

      vanilla_bertscore  vanilla_cosine  rag_rouge_l  rag_bertscore  rag_cosine \
0      0.690926      0.916388      0.141379      0.681684      0.909166
1      0.661969      0.916355      0.244514      0.680676      0.925577
2      0.668536      0.893689      0.224390      0.678575      0.931883
3      0.681242      0.904260      0.203474      0.670209      0.889591
4      0.649679      0.920220      0.151042      0.631718      0.919474

      graph_rouge_l  graph_bertscore  graph_cosine
0      0.168103      0.714348      0.919309
1      0.138138      0.634464      0.920748
2      0.181818      0.653797      0.924337
3      0.275281      0.688700      0.889881
4      0.143322      0.616432      0.915345
```

0.5 5. Nastavak analize u R-u

Agregacija rezultata po kategorijama pitanja te sve tri vizualizacije (usporedba pristupa po metrikama, metrike po kategorijama, toplinska karta po pitanjima) više se ne rade u ovoj bilježnici, već u `statistika/skripta_2_metrike_llm.R` (odjeljak 9), koji učitava upravo `rezultati_evaluacija.csv` spremljen u prethodnom koraku. Time se izbjegava dupliciranje iste logike agregacije/vizualizacije na dva mjesta (Python i R) — ova bilježnica sada radi isključivo ono što se ne može odraditi u R-u bez oslanjanja na Python (izračun ROUGE-L, BERTScore i cosine sličnosti iz sirovog teksta pomoću transformer modela).